

НАСТОЛЬНАЯ КНИГА СТО СТАРТАПА

 Неофициальный русский перевод. Оригинал на английском: [StartupCTOHandbook.md](https://startuptctohandbook.md). Лицензия: [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/).

Ключевые навыки и лучшие практики для высокоэффективных инженерных команд

Автор: Zach Goldberg

Отказ от ответственности:

Издатель и автор не дают никаких заверений или гарантий какого-либо рода в отношении этой книги или её содержания и не несут ответственности за ошибки, неточности, упущения или любые иные несоответствия в тексте.

На момент публикации URL-адреса, приведённые в этой книге, ссылаются на существующие веб-сайты, принадлежащие автору и/или аффилированным с автором лицам. WorldChangers Media не несёт ответственности за эти сайты, не одобряет и не рекомендует их; равно как и не несёт ответственности за содержимое любого веб-сайта, кроме своего собственного, или за любой контент в интернете, созданный не WorldChangers Media.

2023, Zach Goldberg, zach@zachgoldberg.com

Paperback: 978-1-955811-56-9

Ebook: 978-1-955811-57-6

Library of Congress Control Number: 2023918702

Дизайн обложки: Michael Rehder /www.rehderandcompany.com/ Вёрстка и набор: Paul Baillie-Lane/www.pbpublishing.co.uk Редакторы: Stephen Nathans-Kelly и Paul Baillie-Lane

Издано WorldChangers Media PO Box 83, Foster, RI 02825 www.WorldChangers.Media

<https://cto hb.com> <https://startuptctohandbook.com>

Посвящения

Максу Минцу (Max Mintz), за то, что научил меня учиться и ценить по-настоящему важные вещи в жизни.

Каждому моему подчинённому, который у меня когда-либо был, — спасибо за ваше терпение и за то, что вы закрывали глаза на мои многочисленные, я уверен, ошибки.

Моей жене, за то, что мирилась с моими многочисленными начинаниями и поддерживала их, включая эту книгу.

Отзывы

Настольная книга СТО от Zach Goldberg — это убедительный ежедневный помощник для всех инженерных руководителей. Будь то практические фреймворки на каждый день или проницательные наблюдения, книга Goldberg моментально поможет вам справиться с самыми сложными вопросами формирования высокоэффективной инженерной команды.

Michael Lopp, randsinrepose.com

Отличные советы для начинающих инженерных руководителей сегодняшнего дня!

Matt Mochary, исполнительный коуч, автор книги "The Great CEO Within", mocharymethod.com

Zach проделал потрясающую работу, создав ресурс для СТО в стартап-организациях (и не только). Он даёт практические советы по реальным проблемам людей, процессов и технологий, с которыми мы сталкиваемся как технологические руководители в компаниях на ранних стадиях.

Tony Karrer, Ph.D., сооснователь LA CTO Forum, основатель и CEO TechEmpower, основатель и CTO Aggregate

Фундаментальное руководство для любого инженерного руководителя!

Gordon Pretorius, CTO Typeform

Лаконичные главы Zach, полные практической мудрости, искусно вобрали в себя десятилетия реального опыта руководства техническими командами на ранних стадиях. Стоит ознакомиться!

Daniel Demetri, CEO и трёхкратный руководитель стартапов

Настольная книга СТО — это вдохновляющее собрание практичных, применимых рекомендаций как для начинающих, так и для опытных технологических руководителей. Строите ли вы инженерную команду мирового уровня с нуля, мечтаете стать СТО или уже годами занимаете эту должность — это руководство станет вашим незаменимым помощником.

Eric Johanssen, CTO в Dama Financial, автор книги C# 8.0 in a Nutshell

Когда я блуждал в потёмках, пытаюсь разобраться во всём самостоятельно и тонул в куче книг по техническому менеджменту, эта книга оказалась лаконичным резюме всего, что мне было нужно.

Charlie von Metzradt, сооснователь MetricFire/Hosted Graphite

Содержание

- [Введение](#)
 - [Об авторе](#)
 - [Как пользоваться этой книгой](#)
- [Бизнес-процессы](#)
- [Люди и культура](#)
 - [Основы менеджмента](#)
 - [Профессиональное дерево навыков](#)
 - [Kaizen: непрерывное совершенствование](#)
 - [Коучинг](#)
 - [Найдите ментора по управлению](#)
 - [Встречи 1:1](#)
 - [Встречи через уровень \(skip-level\)](#)
 - [Коучинг менеджеров](#)
 - [Найм и собеседования](#)
 - [Скорость — ваш союзник!](#)
 - [Когда нанимать: планирование штата](#)
 - [Поиск кандидатов](#)
 - [Онбординг](#)
 - [Управление эффективностью](#)
 - [Состав команды](#)
 - [Обязанности руководителя](#)
 - [Какой вы СТО стартапа?](#)
 - [СТО с фокусом на технологиях он же главный архитектор](#)
 - [СТО с фокусом на людях он же VP of Engineering \(VPE\)](#)
 - [СТО с фокусом вовне он же руководитель технических продаж/маркетинга](#)
- [Управление технической командой](#)
 - [Техническая культура и общая философия](#)

- [Технический долг](#)
- [Технологическая дорожная карта](#)
- [Технический процесс](#)
 - [Рабочий процесс](#)
- [Developer Experience \(DX\)](#)
- [Техническая архитектура](#)
 - [Архитектура](#)
 - [Инструменты](#)
 - [Скучные технологии](#)
 - [DevOps](#)
 - [Тестирование](#)
 - [Контроль версий](#)
 - [Эскалации в продакшене](#)
 - [Упражнения по анализу первопричин \(RCA\)](#)
 - [IT](#)
 - [Безопасность и комплаенс](#)
- [Заключение: измерение успеха](#)
- [Список литературы](#)
 - [Цифровые источники](#)
- [Глоссарий](#)
- [Об авторе](#)
- [Об издателе](#)

Введение

Учитесь всегда

В четырнадцать лет родители отправили меня на несколько недель в выездной компьютерный лагерь. Он был ровно настолько гиковским, насколько вы себе это сейчас представляете: ряды складных столов с десятками (в основном) мальчишек, прилипших к своим серым ЭЛТ-мониторам и уделявших куда больше внимания игре Unreal Tournament, чем урокам программирования. Два года спустя, в шестнадцать, я вернулся в компьютерный лагерь уже вожатым/преподавателем программирования и наслаждался каждой минутой. Мне очень повезло, что в юном возрасте я осознал свою любовь к компьютерам и программированию, а родители её поддержали.

Перенесёмся ещё на несколько лет вперёд, в лето перед моим первым курсом в Пенсильванском университете (University of Pennsylvania). Я точно знал, что хочу изучать информатику на бакалавриате, но при этом у меня в голове засела мысль, что мне нравится бизнес. Мой отец основал собственное дело, брат только что окончил бизнес-школу, так что бизнес казался отличной идеей. Penn известен своими программами двойного диплома, которые позволяют студентам выпускаться со степенями сразу в нескольких областях, например в инженерии и бизнесе.

Восемнадцатилетнему мне эта перспектива казалась идеальной, поэтому я написал письмо своему научному руководителю, доктору Макс Минцу (Dr. Max Mintz), и послушно назначил встречу, чтобы обсудить мою заявку на программу двойного диплома. Будучи невероятно щедрым и ориентированным на студентов профессором, доктор Минц любезно согласился и пригласил меня в Филадельфию обсудить это за чашкой кофе в Tuscany Cafe.

В назначенный день, проехав три часа из Нью-Йорка в Филадельфию, я сел напротив доктора Минца, горя желанием услышать, как обыграть систему. Я полагал, что всё сводится к выбору правильных курсов и получению достаточно хороших оценок, чтобы пройти отбор. Однако у доктора Минца были совсем другие планы.

Полный предвкушения и готовый принять инструкции о том, как отполировать своё резюме, я сделал глоток кофе и спросил его: как мне попасть на программу двойного диплома? Человек, которого вскоре я буду знать просто как Макса, взял салфетку, нарисовал на ней оси X-Y, затем посмотрел мне в глаза и спросил, знаю ли я, что такое специальная теория относительности. Жаль, что у меня нет видеозаписи того момента, ведь, как мне

представляется, моё лицо скривилось в довольно забавную гримасу. Не успел я закончить ответ, как Макс уже понёсся вперёд. Следующие два часа он знакомил меня с теориями Эйнштейна. К тому времени, как мы закончили, мозг у меня плавился, и ни разу мы не обсудили хоть что-нибудь про программы двойного диплома в Репп.

В последующие месяцы мы выпили вместе ещё немало чашек кофе, и всякий раз, когда я спрашивал Макса про заявку или резюме, он снова уводил меня прямым в настоящую науку. Макс хотел, чтобы я *учился* — не просто впитывал ту тему, о которой он в тот момент рассказывал, а становился по-настоящему хорош в учёбе, причём в изучении трудных вещей. Максиму было совершенно безразлично, какую бумажку выдадут его студентам в конце их четырёх лет, лишь бы каждый из них был готов продолжать учиться всю оставшуюся жизнь.

К моменту окончания университета Макс стал моим близким другом и наперсником и фундаментально определил путь моего образования. Вместо того чтобы дать мне рыбу, Макс вручил мне удочку и научил насаживать наживку и забрасывать леску.

Ни одна книга не сможет дать вам тот опыт, который Макс дал мне как студенту-бакалавру. Я не даю таких обещаний насчёт книги, которую вы сейчас читаете. Я рассказываю эту историю, чтобы подчеркнуть ценность и значимость акцента на самих основах учёбы.

Как технологическому руководителю, вам критически важны для успеха желание, готовность и способность продолжать учиться. Технических знаний на свете слишком много, чтобы кто-то мог стать истинным экспертом во всём, что нужно для работы в современных технологиях. Мне нравится представлять человека, строящего карьеру в технологиях, как персонажа RPG-игры, который, чтобы поднять уровень, не убивает врагов, а проводит сорок часов в неделю на работе, накапливая очки навыков. Вы вольны выбирать, в какие деревья навыков вкладывать накопленные очки, но выбирать нужно с умом. Разнообразие деревьев навыков настолько велико, что разблокировать их все невозможно, поэтому приходится специализироваться.

Самое прекрасное в технологиях — то, что наша область непрерывно развивается. Люди, с которыми вы работаете, будут меняться. Инструменты, которыми вы пользуетесь, будут обновляться или устаревать, а новые методы работы будут появляться и уходить. Когда вы отправитесь в своё приключение в технологическом лидерстве, единственный способ справиться с этими переменами — ожидать их, принимать их и использовать возможность учиться и расти вместе со своей командой и самой отраслью.

Я хочу, чтобы люди серьёзно относились к учёбе. Хочу, чтобы они погружались в неё. Хочу, чтобы они приобретали — и это важнее всего. Дело не в RSA, не в [хитроумных алгоритмах]; это не важно. Важна та уверенность в себе, что они могут расти и учиться вне академической среды: а значит, я им не нужен. Всё, что им нужно, — это уметь сесть с книгами или, может быть, сегодня с интернетом и самостоятельно отправиться постигать новое.

Dr. Max Mintz, 1942–2022

Дилемма технологического руководителя стартапа

У большинства стартапов есть технический сооснователь. Этот человек пишет основную часть первоначальной кодовой базы, нанимает первых нескольких инженеров и заправляет технической стороной стартапа как минимум до первого-второго раунда финансирования.

Где-то между наймом третьего и десятого инженера этот человек перестаёт держать руки на клавиатуре и начинает тратить всё своё время на управление командой. В этот момент часто возникают проблемы: команда начинает выпускать фичи медленнее, частота дефектов растёт, стабильность системы может страдать, общие издержки увеличиваются, и остальные основатели начинают беспокоиться.

Скорее всего, технический сооснователь — да и любой технологический руководитель — всю свою карьеру до этого момента вкладывал время и силы в то, чтобы стать отличным программистом, а не в развитие лидерских навыков. Неудивительно поэтому, что с лидерскими навыками на 1-м уровне он совершает ошибки и обходится компании в потерянное время и деньги.

Независимо от вашей должности и времени прихода в компанию, если вы посвятили большую часть карьеры и опыта технологиям, а теперь берёте на себя ответственность за людей или подразделение, критически важно осознать: вы теперь на руководящей должности; одних только ваших технических знаний и талантов не хватит для

успеха. Хотя некоторые технические навыки — это базовое условие для руководства командой разработки, реальность такова, что для хорошей работы в роли руководителя вам нужно сосредоточиться на лидерстве в работе с людьми, менеджменте, архитектуре и навыках принятия решений в целом.

Лидерство в работе с людьми подходит не каждому. Уверен, вы слышали истории о технических основателях, которые отошли в сторону по мере роста своих компаний. Steve Wozniak, сооснователь Apple, — пожалуй, самый известный пример этого паттерна. Нет ничего постыдного в том, чтобы отойти в сторону; Wozniak осознал, что любит именно техническую работу и именно ей хочет посвящать своё время. Вам стоит хотя бы рассмотреть то же самое для себя: решить, является ли программирование вашей зоной гениальности и той работой, что приносит вам больше всего радости. Если это так, вас ждёт прекрасная карьера с восхождением в ряды самого старшего технического персонала.

Если же вы или ваши обстоятельства привели вас к выводу, что управление или руководство командой — это та роль, к которой вы стремитесь, тогда эта настольная книга станет хорошей отправной точкой в том, как расширить свои навыки на пути к тому, чтобы стать успешным технологическим руководителем.

Об авторе

Мой самый первый опыт работы в стартапе случился летом после первого курса колледжа. Я не помню, почему я искал стажировку в Eduware и почему они приняли мою заявку. Что я помню, так это ежедневные поездки на работу в крошечную комнату в задней части офиса на первом этаже вместе с ещё четырьмя молодыми инженерами-программистами. Старшему из нас было, наверное, двадцать пять; мне было девятнадцать. Нас было всего пятеро, мы сидели за столом в форме подковы и плечом к плечу работали над образовательным приложением на .NET. Как инженер я, вероятно, был бесполезен, но мне повезло, что самый старший и опытный инженер группы нашёл время обучить меня и помочь разобраться в инструментах, и я постепенно становился продуктивнее.

Что-то в том опыте в душной комнате в глубине офиса, должно быть, оставило во мне хорошее впечатление, ведь с тех пор я выбрал работу ещё в семи стартапах: Invite Media, WiFast (ныне Adentro), SoChat, AutoLotto, Trellis Technologies, GrowFlow и Equi. В Invite Media, компании в сфере медийной рекламы и биржевых ставок, я вместе с СТО руководил этапом стремительного роста, который завершился её приобретением компанией Google в 2010 году за 81 миллион долларов. В Google я принял на себя обязанности по надёжности сервиса (site reliability) от уходящего СТО компании Invite и курировал интеграцию компании в стек Google.

Оттуда я отправился сооснователем в WiFast — технологическую компанию, нацеленную на демократизацию и монетизацию использования Wi-Fi, где был одновременно CEO и СТО на протяжении наших первых двух крупных раундов финансирования. Я также был приглашённым предпринимателем (Entrepreneur-in-Residence) в Tencent в Гуанчжоу, Китай, и стал сооснователем SoChat, кроссплатформенного мессенджера. С тех пор я работал СТО в Lottery.com, Trellis Technologies, GrowFlow (приобретена Dama Financial) и Equi.

К каждой из этих ролей я подходил с мышлением основателя, стремясь создавать творческую среду и продвигать идею о том, что разработка ПО должна быть больше наукой, чем искусством.

Мне также повезло учиться у других на протяжении всего этого пути, в том числе у семи команд замечательных инженеров, бесчисленных клиентов по консалтингу/коучингу и множества блестящих сооснователей. Я также целенаправленно стремился повышать собственный уровень лидерства через годы управленческого коучинга у одного из лучших коучей Кремниевой долины, а также прибегая к помощи бесчисленных менторов и читая сотни релевантных книг.

Благодаря чтению мне стало очевидно: хотя существуют сотни практических книг для программистов и людей, работающих с конкретными технологиями или инструментами, и десятки полезных книг для CEO и CFO с финансовой стороны предпринимательства, нашей индустрии не хватает одного — основательного, практического ресурса для технологических руководителей стартапов. Нам нужен ресурс, который охватывает все темы, лежащие между базовыми навыками, и затрагивает весь спектр лидерских вызовов и навыков, столь критичных для нашей роли.

Существует также множество блогов о том, как писать хороший код, как проводить опросы пользователей или как найти product market fit. А это — книга о построении технических команд; она затрагивает все навыки, которые

нужны руководителю, чтобы построить компанию, и которым он не научился в традиционном техническом образовании или опыте.

Как пользоваться этой книгой

Как руководитель команды разработки ПО, вы, скорее всего, сталкивались в своей роли с некоторыми из этих проблем:

- Отслеживание, управление или погашение технического долга.
- Найм, привлечение, развитие и удержание лучших специалистов.
- Создание объективной, справедливой и прозрачной системы оценки эффективности.
- Построение, управление и поддержание здоровой и созидательной корпоративной культуры.
- Выстраивание отношений с другими руководителями вашей компании.
- Преодоление медленного принятия решений или бесконечных хождений по кругу в спорах технического персонала о том, как спроектировать и построить вашу систему.

Похоже, каждый технологический руководитель рано или поздно сталкивается с этими проблемами, и тем не менее советы о том, как с ними справляться, по неудобному стечению обстоятельств отсутствуют почти в любой бизнес- или технической программе обучения.

Моя цель — дать перспективу на эти проблемы и не только, а также предложить контекст того, как различные техники проявляют себя в реальном мире. Задача — вооружить читателя пониманием компромиссов, возможностью заглянуть за угол и фреймворками, которые подготовят вас к принятию собственных обоснованных решений.

Эта книга написана в первую очередь для всех, кто сейчас управляет командой разработки ПО или может оказаться в этой роли в будущем, особенно в качестве движущей силы стартапа, поддержанного венчурным капиталом. Она также может оказаться полезной для индивидуальных контрибьюторов (IC) — инженеров-программистов без управленческих функций — как способ получить представление о тех задачах и требованиях, которые ложатся на менеджеров и которые с первого взгляда могут быть неочевидны.

Я оформил эту книгу как собрание независимых глав, охватывающих широкий спектр тем. Она задумана как справочное руководство: читатель берётся за главу, когда она становится полезной, и не обязан читать всё последовательно от начала до конца. По этой причине некоторый материал повторяется в разных главах, чтобы каждая глава могла существовать самостоятельно, без опоры на предыдущие разделы как на контекст.

Моя цель в каждой главе — не дать исчерпывающее обсуждение или обзор темы. Вместо этого задача — представить тему, дать обзор или структуру для размышления о ней, предложить некоторые лучшие практики и порекомендовать справочные материалы для более глубокого изучения предмета. Думайте об этой книге как о собрании тем по техническому лидерству, организованном в ширину (breadth-first). Читателю предстоит самому определить, какие темы ему наиболее интересны, и, вооружившись неким контекстом и перспективой, погрузиться в то, что наиболее актуально, и применить знания на практике.

В конечном счёте эта книга — синтез моего личного опыта и тех ресурсов, которые я счёл полезными, перемежающийся советами и комментариями коллег, менторов и консультантов. Если в этой книге есть вещи, с которыми вы не согласны или считаете ошибочными и о которых хотели бы мне сообщить, или если эта книга оказалась для вас полезной и вы хотите связаться со мной напрямую, не стесняйтесь писать на zach@ctohb.com. По тому же адресу я также с радостью обсужу возможности консультирования, коучинга и менторства.

Бизнес-процессы

На протяжении этой книги вы встретите множество описаний бизнес-процессов. Моя цель в изложении этих процессов — дать отправную точку для того, как вы могли бы реализовать решение стоящей перед вами проблемы.

В зависимости от размера вашей команды и компании то, что здесь описано, может показаться чрезмерно обременительным и громоздким, а может выглядеть слишком скудным и примитивным для ваших нужд. Реальность такова, что по мере роста вашей компании и команды вам придётся заново изобретать способы ведения дел. Ваша компания из пяти человек будет функционировать совершенно иначе, когда вырастет до двадцати, пятидесяти, сотни или тысячи. Я выделил ключевые принципы, которые имеют значение, и оставил вам адаптировать их под вашу команду в её нынешнем составе, а также масштабировать ваш подход по мере того, как потребности и ограничения вашего бизнеса будут меняться в будущем.

Люди и культура

Основы менеджмента

Рекомендуемое чтение: *Managing Humans* (Michael Lopp)

Золотое правило менеджмента: делайте всё необходимое, чтобы получить от своей команды максимум. В техническом лидерстве, как и в любой другой руководящей роли, лучшая мера вашей эффективности как менеджера — это эффективность самой команды. Это значит, что вам стоит думать и тратить время на всё необходимое, чтобы помочь отдельным членам команды работать наилучшим образом — как индивидуально, так и коллективно.

Помощь вашей команде в достижении успеха требует смирения, ведь она предполагает, что вы последовательно ставите потребности своих подчинённых выше собственных. Вам придётся корректировать и подстраивать свой стиль, поведение, мышление и действия под потребности членов вашей инженерной команды. Это будет включать готовность ошибаться, открытость ума и обучение у своих подчинённых.

Если вы примете этот путь, знайте, что вы будете совершать ошибки. Признавайте эти ошибки перед своей командой, и она будет доверять вам ещё больше. Знайте также, что стать идеальным менеджером — недостижимая цель; лучшее, на что вы можете надеяться, — это постоянно улучшаться по мелочам. После карьеры, проведённой в управлении людьми, вы накопите багаж уроков о технологиях и о людях, которые сделают вас более компетентным менеджером.

В книге *Managing Humans: Biting and Humorous Tales of a Software Engineering Manager* Michael Lopp пишет:

У каждого человека, с которым вы работаете, совершенно особый набор потребностей. Удовлетворение этих потребностей — один из способов сделать его довольным и продуктивным. Ваша работа на полную ставку — слушать этих людей и мысленно фиксировать, как они устроены. Это ваша самая важная работа. Я знаю, что старший вице-президент по инженерии говорит вам, что уложиться в срок по проекту — задача номер один, но не вы будете писать код, тестировать продукт или документировать функциональность. Это будет делать команда, а ваша работа — управлять командой.

В этом одном лаконичном абзаце Lopp затрагивает все ключевые моменты менеджмента. Прежде всего, вы — слушатель, коуч по личному и карьерному развитию и щит, защищающий вашу команду от внешних сил мира, которые могут отвлекать, нагружать стрессом или иным образом мешать ей работать наилучшим образом.

Профессиональное дерево навыков

Во многих видеоиграх есть концепция дерева навыков. Для тех, кто не знаком: дерево навыков — это последовательность навыков или способностей, которые открываются по мере продвижения игрока по игре. Каждый навык открывается за счёт траты очков навыков. И вот в чём загвоздка: в любой момент времени навыков, которые можно открыть, больше, чем у вас есть очков навыков для траты. Дерево навыков вынуждает вас выбирать одни навыки прежде других. Дерево навыков служит вполне разумной моделью и для вашей карьеры. На любой работе вы, вероятно, накапливаете очки навыков в сторону одних навыков, а не других.

На вашем пути к технологическому лидерству вы уже вложили множество очков навыков в техническую/инженерную ветку дерева навыков. Мой ключевой посыл для вас в том, что управленческая ветка дерева навыков столь же обширна, и если до сих пор вы не вкладывали в неё очки, то даже будучи инженером 100-го уровня вы начнёте свою новую руководящую должность как менеджер 1-го уровня, глядя на могучий дуб ещё не открытых

критически важных навыков. Как только в вашей компании станет больше горстки инженеров, именно эти навыки определяют вашу способность масштабироваться вместе с командой.

Kaizen: непрерывное совершенствование

Kaizen — японское слово, означающее «улучшение». Эта концепция получила известность как часть производственной системы Toyota. На Toyota каждому сотруднику дают (буквальную или метафорическую) красную рукоятку, потянув за которую, можно остановить всю производственную линию. Если работник обнаруживает проблему в производстве, идея в том, чтобы он потянул за красную рукоятку, собрал коллег и ресурсы для диагностики проблемы, а затем устранил её, прежде чем работа продолжится. Давая каждому члену команды возможность улучшать процесс и быть заинтересованным в его эффективности, Toyota может экономично выпускать автомобили более высокого качества.

Я не первый, кто предполагает, что разработка ПО имеет много общего с традиционным производством (см. *The Phoenix Project* Джина Кима). В данном случае сделайте метафору реальной: дайте своей команде цифровую красную рукоятку и поощряйте её сосредоточиться на непрерывном улучшении всего, что вы делаете. Участники сильных команд понимают, что со временем команда изменится, требования клиентов изменятся, инструменты изменятся, и команде придётся пересматривать прошлые решения и вносить улучшения.

Kaizen применим не только к процессам вашей команды, но и к отдельным людям. Ваши лучшие сотрудники примут идею непрерывного обучения и непрерывного совершенствования и будут относиться к ошибкам не как к неудачам, а как к возможностям для улучшения.

Коучинг

Ваша главная роль как менеджера — раскрывать лучшее в людях вашей команды, поэтому во многих ситуациях уместнее описывать вашу роль как роль коуча, а не менеджера. Коуч — это тот, кто на вашей стороне, источник мудрости и руководства для каждого члена команды. Коуч быстро даёт критическую обратную связь, но также первым отмечает и хвалит успех.

Ваша цель во взаимодействии с подчинёнными — будь то индивидуальные контрибьюторы-инженеры или сами менеджеры — стать лучшим коучем, который у них когда-либо был.

Найдите наставника по управлению

Один из способов ускорить ваш переход к лидерству — найти себе наставника (ментора) по управлению. Существует множество коучей по управлению с разными подходами; задача в том, чтобы найти того, кто откликается именно вам.

В моей первой роли бизнес-лидера в WiFast (тогда Zenreach, теперь Adentro) мы быстро наняли команду из десяти штатных сотрудников, в основном в инженерии. Как менеджер-новичок я понимал, что мне многому нужно научиться, и был готов использовать любой доступный ресурс, чтобы стать лучшим менеджером. Единственная проблема была в том, что я терпеть не мог большую часть управленческих советов. Я находил их либо чрезмерно предписывающими, без контекста и понимания, либо вовсе лишёнными содержания — пустословием, если угодно. Так было до тех пор, пока я не встретил своего первого коуча по управлению, Джонатана.

История такова: один из наших инвесторов, First Round Capital, проводил саммит по управлению в Сан-Франциско, примерно в тридцати минутах езды от нашего офиса. К тому моменту я успел убедиться, что люди из First Round были высокого качества, поэтому, когда мне попало приглашение, их репутация временно приглушила мою вечную аллергию на пустословие, и я записался.

Когда я приехал на саммит, меня обнадёжило, что аудитория была сравнительно небольшой — всего около тридцати человек, достаточно, чтобы поместиться в чём-то вроде школьного класса. Я сел за школьную парту с откидной столешницей, открыл блокнот, достал ручку и написал дату и «First Round Capital Management Summit» в верхней части страницы. К сожалению, это было единственное, что я записал за всё утро.

В первой половине дня три или четыре спикера говорили на разные темы, и каждому из них не доставало хоть какого-то практически применимого совета или понимания — иными словами, это было пустословие. Когда мы прервались на обед, я подумывал уехать пораньше и отработать полдня в офисе. Я заглянул в программу и

заметил, что на вторую половину дня запланирован совершенно другой состав спикеров, поэтому решил хотя бы выслушать первого.

Первым спикером после обеда был Джонатан. В отличие от предыдущих докладчиков, у него не было слайдов, и он казался немного спешащим, возможно слегка неподготовленным, а может, просто нервничающим, когда шёл к передней части класса. Однако первые же слова, вылетевшие из его уст, рассказали совсем другую историю:

Позвольте мне открыто вами манипулировать.

Я никогда не забуду этот момент. Какая забавная вещь — сказать такое; это кажущееся противоречие в терминах, как фраза «Это предложение ложно». (Если интересно, это называется [парадоксом лжеца](#)). Джонатан продолжил, объяснив, что в этом-то и был смысл: он хотел сказать что-то, что захватит наше внимание как аудитории, и в этом он преуспел совершенно. Следующие тридцать минут я делал обильные записи — не о том, как манипулировать людьми, а о том, как понимать людей в целом.

Я ловил каждое слово Джонатана. Когда получасовая сессия завершилась, Джонатан сказал, что ему нужно успеть на рейс, и несколько торопливо выбежал из комнаты. Я опустил взгляд на свой блокнот, осознал, что за последние тридцать минут сделал три страницы записей по сравнению с нулём за первые четыре часа, затем встал со стула и побежал за ним.

Мне удалось догнать его как раз в тот момент, когда он садился в жёлтое такси. Несколько раздражённо Джонатан спросил, что мне нужно. Я спросил, занимается ли он частным коучингом. Он ответил: «Попросите организаторов связать нас», затем сел в такси и уехал. Хитро, что он не дал на месте никакого определённого ответа насчёт коучинга — он оставил себе возможность навести обо мне справки через организаторов, прежде чем решить, стоит ли тратить на меня время. К счастью для меня, когда я попросил организаторов нас связать, контакт сказал обо мне достаточно хорошие вещи, чтобы Джонатан согласился на вводную коуч-сессию. Мы с Джонатаном продолжили работать вместе много лет, в нескольких компаниях, и я могу уверенно сказать, что его наставничество было бесценным в моём лидерском пути.

ВСТРЕЧИ ОДИН НА ОДИН (1:1)

Встреча один на один (1:1) — это приватная встреча между вами и подчинённым. Заманчиво относиться к 1:1 как к встречам для проверки статуса и строить повестку целиком вокруг бизнес- или технических вопросов, стоящих прямо сейчас. Ничего страшного, если повестка включает эти темы, но это ваша возможность выстроить коучинговые отношения с подчинённым. Вы должны использовать это время, чтобы по-настоящему узнать и понять, как мыслит ваш подчинённый, выявить и раскрыть его сильные стороны и распознать слабые, над которыми вы можете поработать, чтобы помочь человеку выполнять свою работу наилучшим образом.

SKIP-LEVEL ВСТРЕЧИ

Хорошая практика — на полурегулярной основе (ежемесячно или ежеквартально) проводить встречи с подчинёнными тех менеджеров, которые подчиняются вам. Они называются skip-level встречами, поскольку вы «перепрыгиваете» через один уровень в организационной схеме, встречаясь с этими людьми напрямую. Skip-level встречами вы не пытаетесь подорвать авторитет своих менеджеров — наоборот, всё совсем наоборот. Собирая больше данных и выслушивая разные точки зрения, вы сможете лучше работать с менеджерами над тем, что способно помочь улучшить бизнес.

Несколько кратких мыслей насчёт повесток skip-level встреч:

- Снимите напряжение с сотрудника, убедившись, что он понимает цель встречи: вы здесь не для того, чтобы решать проблемы или принимать решения, которыми лучше заниматься его непосредственному менеджеру.
- Дайте понять, что вы хотите выстроить отношения и услышать его соображения о лидерстве, культуре, стратегии и направлении развития компании.
- Установите контакт с сотрудником; задавайте вопросы и проявляйте любопытство.

- В интернете есть много хороших готовых шаблонов/повесток для skip-level встреч. Вот один с managementcenter.org, который я рекомендую: cto.hb.com/skip.

КОУЧИНГ МЕНЕДЖЕРОВ

По мере роста вашей организации вы, вероятно, дойдёте до точки, когда у вас больше не останется подчинённых — индивидуальных контрибьюторов. Каждым непосредственным контрибьютором, который на самом деле пишет код, управляет менеджер среднего звена. Тогда должно быть очевидно, что эффективные менеджеры среднего звена критически важны для результативности вашей организации. Ваша задача — убедиться, что у ваших менеджеров есть поддержка, ресурсы, обучение и наставничество, необходимые им, чтобы выполнять свою работу по коучингу инженеров своей команды наилучшим образом.

Главный вклад в формирование качественного менеджмента среднего звена вносит, конечно же, найм правильных людей, но на втором месте — постоянное обучение и поддержка. Если вы в положении, когда курируете команду менеджеров, я призываю вас встроить в вашу организацию следующее:

- Постройте культуру непрерывного обучения.
 - Например, поощряйте своих менеджеров организовать внутренний книжный клуб, посвящённый управлению.
 - Регулярно делитесь с управленческой командой инсайтами, которые узнаёте сами, и просите их делать то же самое со своими командами. Если вы используете корпоративный мессенджер, выделенный канал #management-insights или похожий — отличное место для такого рода диалога.
- Установите высокую планку для коучинга и управления.
 - Чётко доносите до своих менеджеров ваши ожидания о том, что означает управление, об ожиданиях в части коучинга, 1:1, управления эффективностью и т. д.
 - Однозначно закрепите ваши ожидания в части управления во внутренней документации и сделайте их частью найма и онбординга менеджеров.
- Предоставляйте тщательные и доступные материалы для обучения управлению.
 - Снабжайте менеджеров ресурсами для непрерывного обучения и профессионального развития. Это может включать покупку корпоративных подписок на обучающие программы, спонсирование участия сотрудников в конференциях, найм коучей по управлению или формализацию внутренних или внешних программ наставничества.
 - Учитывайте стоимость этих обучающих материалов в вашем регулярном процессе планирования бюджета для каждого члена вашей команды.
- Развивайте внешнюю культуру лидерства мнений (thought leadership).
 - Поощряйте своих менеджеров становиться лидерами мнений в вашей отрасли. Это может принимать форму корпоративного блога, участия в качестве гостя в технических или управленческих подкастах либо выступлений на конференциях.

Встречи 1:1 с инженерами

Ваши инженеры должны регулярно выплёскивать вам своё недовольство, и если они это делают — не паникуйте: это совершенно нормально и, по правде говоря, крайне желательно. У вас должна быть встреча 1:1 с каждым членом вашей команды как минимум раз в две недели, а лучше еженедельно. Ваша цель на этих встречах — создать безопасное пространство, чтобы инженеры могли рассказать вам, что у них на уме, а вы могли активно слушать и вовлекаться в эти темы.

С сильными инженерами это будет означать, что они замечают несовершенства в окружающем мире и хотят рассказать вам о них. Ваша задача — не решать каждую поднятую ими проблему; ваша задача — слушать, задавать вопросы, чтобы прояснить своё понимание, и убедить их, что вы действительно понимаете, а затем направлять их к решениям. Время от времени может возникнуть прямая просьба или что-то, с чем вы можете напрямую помочь, но это не норма.

Ценность, которую вы здесь приносите, — это дать вашим подчинённым почувствовать, что их услышали, и научить их продуктивно справляться с проблемами самостоятельно.

Содержание и повестка 1:1

В конечном счёте ваша цель на встрече 1:1 — выстроить отношения с другим человеком и вести откровенные, основанные на доверии и критические разговоры, которые позволят вам помочь ему выполнять свою работу наилучшим образом. Если у вашего подчинённого широкая повестка — отлично, начните с неё. Однако если его повестка постоянно ограничивается тактическими рабочими задачами в процессе, а вы не доходите до этих более высокоуровневых разговоров о том, как мы работаем, то я призываю вас дополнять его повестку, чтобы он лучше понимал цель ваших встреч и приносил на будущие сессии более содержательные вопросы.

Самый простой способ соединить вашу повестку и его — иметь общий документ, возможно, с некоторой структурой/шаблоном, чтобы вызвать те темы для обсуждения, которые вы считаете важными. Наличие такого документа, доступного до встречи, также даёт вам и вашему сотруднику общее место для фиксации идей между встречами, для структурирования мыслей заранее — всё это помогает сделать время встречи более продуктивным и эффективным.

Есть также несколько SaaS-инструментов, которые помогают вести беседы 1:1. Заметные примеры — Culture Amp и 15Five. Впрочем, инструмент вам не нужен; простой документ работает столь же хорошо. Шаблон, который использую я, доступен на cto.hb.com/templates; он включает подсказки для обсуждения того, что нравится / чего хотелось бы, на личном, департаментском и общекорпоративном уровне, а также двустороннюю обратную связь между менеджером и сотрудником.

Плейбуки для 1:1

Создание плейбука для этих инженерных 1:1 — ещё один полезный способ убедиться, что эти встречи затрагивают согласованный набор тем и не уходят в сторону. Ваш плейбук должен гарантировать, что ваши 1:1 затрагивают следующее:

Конфликт: внутри вашей непосредственной команды, между инженерными командами, кросс-функциональный

Эффективность и развитие: часто это ваши инженеры, ищущие совета о том, как они могут что-то улучшить

Ясность: у инженеров могут быть общие соображения по поводу чего-то, и они ищут вашу точку зрения или хотят узнать, нет ли у вас иной информации о чём-то, чем у них

Контекст: что происходит в компании более широко и как работа контрибьютора соотносится с этими целями/задачами

Радикальная откровенность (Radical Candor)

Фраза *Radical Candor* (радикальная откровенность) была определена Ким Скотт в её книге *Radical Candor*. Книга определяет радикальную откровенность как коммуникацию, которая включает в себя как похвалу, так и критику, и обеспечивает, чтобы в подаче сочетались и личная забота, и прямой вызов. Думаю, мысль лучше всего раскрывается в контрасте с тремя другими видами коммуникации, описанными в книге Скотт:

Отвратительная агрессия (Obnoxious Aggression): иногда называемая жестокой честностью или «ударом в лицо», характеризуется прямым вызовом, но отсутствием личной заботы, что, возможно, проявляется неискренней похвалой или недоброй критикой

Губительная эмпатия (Ruinous Empathy): коммуникация, которая исходит из личной заботы, но лишена прямого вызова

Манипулятивная неискренность (Manipulative Insincerity): также известная как «удар в спину» или пассивно-агрессивное поведение, характеризуется отсутствием как личной заботы, так и прямого вызова

Я призываю вас прочитать книгу Скотт, но если вы этого не сделаете, то хотя бы будьте осведомлены об этих терминах и используйте их как инструмент коучинга, чтобы продвигать свою команду к регулярной практике радикальной откровенности.

Преимущества избыточной коммуникации

Нет ничего хуже для сотрудника, чем ощущение, что его менеджер недостаточно с ним коммуницирует. В отсутствие информации естественный инстинкт — предполагать наихудший сценарий; нехватка информации также может быть главным источником тревоги и замешательства.

Избыточная коммуникация (overcommunication), напротив, имеет очень мало негативных последствий. Худшее, что можно сказать об избыточной коммуникации, — что она может оказаться отвлекающей или избыточной, а это проблемы, легко устранимые небольшой вдумчивостью в отношении формы избыточной коммуникации. Тогда неудивительно, что большинство стартапов вкладывают значительные силы в то, чтобы встроить избыточную коммуникацию в свою культуру, часто включая саму эту фразу в число ключевых ценностей компании.

Электронная почта

Практически каждый, с кем вы взаимодействуете в наши дни, либо пользуется электронной почтой двадцать пять лет, либо со времён начальной школы, так что, конечно, это означает, что он умеет эффективно ею пользоваться, верно? К сожалению, эффективное использование электронной почты на работе — не всегда здравый смысл. Поэтому именно вам приходится помогать поощрять лучшие практики. Вот несколько общих советов по эффективному использованию электронной почты:

- Не позволяйте электронной почте стать вашей работой.
 - Вместо того чтобы держать почту открытой весь день или непрерывно её мониторить, проверяйте почту в фиксированное время несколько раз в день.
 - Отключите уведомления почты на телефоне. Хотя именно это может показаться кошмаром, я призываю вас попробовать. Это не только значительно сократит количество получаемых уведомлений, но и вы поймаете себя на формировании новой привычки — проактивно проверять почту, когда вы готовы взаимодействовать. Это делает электронную почту намеренным занятием, а не постоянной фоновой докукой.
- Достигайте «нулевого входящего» (inbox zero) каждый день.
 - Вложите время в изучение вашего почтового инструмента или используйте опциональные дополнения/плагины-помощники для почты, которые помогают сортировать и приоритизировать письма, чтобы к концу дня, каждый день, у вас было ноль непрочитанных писем.
 - Обнуление входящих не означает, что нужно действовать по каждому письму или отвечать на него. Если вы используете почту как список дел — это нормально (хотя это не идеально — см. «Встречи и управление временем», стр. 28, для лучших альтернатив списку дел); просто убедитесь, что вы выводите этот почтовый список дел из вашего основного входящего, чтобы не путать его с необработанными письмами.
- Не решайте проблемы в почте.
 - Электронная почта — неоптимальная среда для углублённого обсуждения, особенно когда вовлечены более двух человек. Групповые письма лучше всего использовать для координации и избыточной коммуникации, а не для решения проблем.
 - Понимайте, что в почте обычно недостаёт нюансов и интонации, что делает намерение легко поддающимся неверному толкованию.
 - Склонность написать нюансированную групповую переписку или поучаствовать в ней — хороший индикатор того, что синхронный разговор является лучшим форумом для рассмотрения данной темы. Пятнадцатиминутное обсуждение часто может разрешить то, что переписка из двадцати сообщений лишь поверхностно затронет.
 - Сам акт записи своих мыслей часто является весьма продуктивным упражнением, но электронная почта — не лучший способ облегчить и зафиксировать этот письменный процесс брейншторминга. Поощряйте свою команду вместо этого писать памятки в вики, чтобы способствовать глубокому осмыслению.
- Не полагайтесь на электронную почту для длинной или углублённой коммуникации.

- В целом электронная почта — плохая среда для длинных текстов. Длинные памятки лучше размещать во внутренних вики или документации, которые можно комментировать, обновлять и легко на них ссылаться в будущем.
- Держите письма относительно короткими — в идеале с маркированными списками ключевых идей.
 - Не стесняйтесь использовать базовое форматирование, такое как жирный шрифт или выделение, для запросов/задач к действию.
- Помните о своей аудитории.
 - Инженеры в целом предпочитают писать код, а не читать почту и отвечать на неё. Спросите себя, действительно ли письмо — правильный способ коммуникации с вашей аудиторией. В целом лучший метод коммуникации с человеком — *его* предпочтительный метод, а не ваш.
 - Очень легко не включить коллег в почтовую переписку — либо намеренно, в попытке не заваливать почтовые ящики, либо по невинной ошибке. Если вы сидите и думаете, кого добавить в переписку или убрать из неё, это хороший признак того, что электронная почта изначально неподходящий форум.

Синхронный чат

С высокой вероятностью ваша компания уже внедрила ту или иную форму платформы синхронного чата; в начале 2000-х это обычно был Google Chat или продукт MSN Messenger, тогда как в 2020-х это чаще Slack, Microsoft Teams или Workspace от Meta. Если вы в данный момент не пользуетесь одной из этих платформ, стоит рассмотреть их внедрение.

Подавляющее большинство компаний — от стартапов первого дня до гигантов на 100 000+ человек — внедрили их с большим успехом.

Достичь такого успеха означает помнить о некоторых присущих им недостатках и планировать с их учётом: программы синхронного чата требуют, чтобы обе стороны прекратили то, что делают, и включились во взаимодействие, и они порождают беседы, которые плохо организованы и не оставляют долговечных артефактов, на которые ваша команда могла бы ссылаться. Вы можете и должны признать эти недостатки и компенсировать их, установив базовый этикет и ожидания для вашей команды в части того, как пользоваться этими инструментами.

Собственный блог Slack содержит отличную статью с несколькими распространёнными лучшими практиками: cto.hb.com/slack.

Вот несколько рекомендаций по работе с инструментами синхронного чата:

- Старайтесь включать всю необходимую информацию в сообщение, чтобы продолжить разговор. Если вы задаёте коллеге вопрос, дайте в вопросе достаточно контекста и информации, чтобы у него был наилучший шанс ответить исчерпывающе. Это минимизирует количество отправленных уведомлений, сокращает объём переписки туда-сюда и укорачивает время до решения. Инструменты вроде loom.com очень полезны для этого.
- Используйте функции форматирования сообщений, такие как маркированные списки и заголовки, чтобы более длинные сообщения было легче просматривать, а нужную информацию — легче находить.
- Централизуйте беседы в конкретных каналах или тредах. Это непродуктивно и раздражает — пытаться следить за разговором с несколькими людьми сразу по более чем одной теме.
- Опирайтесь на расписания уведомлений и функции «не беспокоить». Вам также следует поощрять членов вашей команды настраивать расписание «не беспокоить» в любой программе синхронного чата, чтобы минимизировать прерывания фокуса/состояния потока.
- В духе избыточной коммуникации по умолчанию ведите общение в публичных каналах. Ещё лучше — установите культуру и стандартную операционную процедуру превращения бесед и вытекающих из них решений в долгоживущую, организованную документацию в корпоративной вики или другом подходящем хранилище документов/информации.

- Будьте крайне осмотрительны с сообщениями, отправляющими уведомления множеству людей, например @here или @channel в Slack. Особенно по мере роста вашей компании велика вероятность, что отправка такого сообщения пошлёт уведомление и прервёт работу потенциально десятков сотрудников.

Асинхронная коммуникация

Асинхронная коммуникация — это любая коммуникация, которая не предполагает немедленного ответа. Чтобы она была эффективной, принимающая сторона должна иметь возможность не спешить, осмыслить информацию, а затем вдумчиво ответить. Ключевой элемент асинхронной коммуникации в том, что исходное сообщение представляет собой законченную мысль и содержит необходимый контекст, позволяющий другой стороне ответить.

Банальный пример — пресловутый баг-репорт «фича сломана». Почти во всех случаях баг-репорты должны попадать в систему тикетов, а не в личное сообщение. Инженер, получивший баг-репорт в сообщении, не имеет контекста, чтобы понять, какая именно фича сломана и каким образом она не соответствует ожиданиям. Поэтому ответ инженера, скорее всего, будет состоять из горстки вопросов, требующих новых витков переписки с автором отчёта, отнимающих время и порождающих фрустрацию.

Сравните это с баг-репортом, который включает полные письменные шаги воспроизведения, а также видео того, как пользователь пытается использовать фичу и демонстрирует сбой. Скорее всего, такой подход позволит инженеру выпустить исправление, не требуя никаких дальнейших уточнений.

Суть в следующем: всякий раз, когда вы отправляете кому-то сообщение в асинхронном формате, дайте этому человеку всю информацию, которая нужна ему, чтобы понять, осмыслить и ответить так, чтобы продвинуть беседу вперёд.

Асинхронная культура

Вы удивитесь, как часто хорошо продуманная асинхронная коммуникация может заменить синхронный чат или встречу. Хорошая асинхронная коммуникация может означать не только меньше встреч и прерываний, но и оставлять после себя исчерпывающую письменную документацию, которую другие смогут осмыслить в будущем. Некоторые стартап-компании, такие как Levels Health, фактически встроили идею «асинхронности по умолчанию» в самое ядро своей корпоративной культуры с большим успехом (cto.hb.com/async).

Документация

Документация — ключевой элемент масштабирования вашей организации. Преимущества того, чтобы фиксировать вещи письменно, многочисленны: письменная документация может помогать в онбординге, обучении, избыточной коммуникации, вдумчивости, тщательности, построении культуры, избегании невынужденных ошибок и многом другом. Ваша роль — не просто верить в ценность и ROI документации, но и строить культуру документации и команду, которая её ценит.

Несколько советов по построению культуры хорошей документации:

- Сами живите этой ценностью и подавайте пример команде. Однажды я перевёл команду от написания нуля статей во внутренней вики в неделю к написанию нескольких в день за примерно восемь недель. Буквально единственное, что я сделал для поощрения этого культурного изменения, — начал писать статьи сам. Всё, что я делал и что имело смысл поделиться с командой, я оформлял в виде статьи и старался делиться ссылками на эти статьи всякий раз, когда это было уместно. Очень быстро другие менеджеры начали делать то же самое, и в течение двух месяцев каждый член команды вносил вклад каждую неделю.
- Сделайте документацию привычкой, везде. Будь то онбординг, технические спецификации, ревью pull request'ов, внутренние запросы на комментарии (RFC) или памятки — стандартной процедурой должно быть записать это и сохранить в организованном архиве в легкодоступном месте.
- Разрабатывайте процессы поддержания документации там, где это уместно. Документации легко устареть, и во многих ситуациях это совершенно нормально. В других — важно, чтобы документация оставалась актуальной, и единственный способ, которым это случится, — наличие процесса или чек-листа, включающего обновление документации. Наличие даты последнего обновления на каждом документе —

отличный способ сигнализировать читателям, что что-то свежее или потенциально устаревшее, неактуальное или утратившее силу.

- Поощряйте команду практиковать правило бойскаута (всегда оставляй стоянку чище / код лучше, чем нашёл). Если они находят неточную документацию, им следует либо обновить её самостоятельно, либо явно пометить документ как устаревший.

Одна из ключевых областей документации, которой вам стоит уделить особое внимание, — то, как разработчик начинает писать код в рамках конкретного проекта или репозитория. Я рекомендую, чтобы каждый репозиторий имел файл README.md, объясняющий как минимум четыре вещи:

Установка: как установить приложение и запустить его локально **Структура каталогов:** как ориентироваться в этой кодовой базе **Разработка:** как выглядит цикл разработка/запуск/тестирование на этой кодовой базе **Развёртывание:** как доставить ваши изменения в более высокие окружения для этого приложения

Об аббревиатурах на работе

У каждой организации своя самобытная культура и свой стиль внутренней и внешней коммуникации. Одна из ключевых обязанностей лидера — следить за тем, чтобы культура всегда поддерживала цели организации, а не препятствовала им.

Один из элементов внутренней культуры технической организации, который склонен выходить из-под контроля, — порождение выдуманных аббревиатур, которые могут со временем множиться, затемнять и чрезмерно усложнять ту самую коммуникацию, которую они были призваны упростить. Это может показаться мелким раздражителем, но это симптом плохой коммуникационной стратегии, которая способна выйти из-под контроля по спирали, в особенности потому, что она может воздвигать барьеры между «посвящёнными» и теми членами команды, которые понятия не имеют, что означают эти аббревиатуры. Как технический лидер организации, вы обязаны задавать тон и определять культуру, и хотя разрастание выдуманных аббревиатур, скорее всего, начнётся не с вас, ваша задача — распознать, когда это происходит, и пресечь это, прежде чем оно выйдет из-под контроля.

В январском меморандуме 2018 года сотрудникам SpaceX Илон Маск призвал к политике «никаких аббревиатур». Я с тех пор применяю ту же политику и всецело её поддерживаю. Приведённое ниже взято из письма под названием «Acronyms Seriously Suck» (cto.hb.com/acronyms):

*В SpaceX наблюдается ползучая тенденция использовать выдуманные аббревиатуры. Чрезмерное использование выдуманных аббревиатур — серьёзное препятствие для коммуникации, а поддержание хорошей коммуникации по мере нашего роста невероятно важно. По отдельности несколько аббревиатур тут и там могут показаться не такими уж плохими, но если их выдумывает тысяча человек, со временем результатом станет огромный глоссарий, который нам придётся выдавать новым сотрудникам. [...] Это особенно тяжело для новых сотрудников. [...] Ключевой тест для аббревиатуры — спросить, помогает она коммуникации или вредит ей. Аббревиатуру, которую большинство инженеров за пределами SpaceX уже знают, такую как GUI, использовать нормально. На практике большинство аббревиатур выступают барьером, а не благом для ясной коммуникации. Они затрудняют новым сотрудникам понимание того, что обсуждается. Они требуют усилий от команды по поддержанию где-то списка определений аббревиатур, и в целом они меньше экономят время и при письме, и при разговоре, чем может показаться на первый взгляд.**

Это может показаться кощунством или назойливым и глупым правилом, которое пытаются навязать в культуре. Я не предлагаю наказывать людей за использование аббревиатур или писать это на стенах в коридорах вашего офиса. Совсем наоборот; особенно в небольшой организации требуется лишь очень лёгкое прикосновение, чтобы «никаких новых аббревиатур» стало частью вашей культуры. Заручитесь поддержкой вашей исполнительной команды в том, чтобы не создавать аббревиатур, а затем поощрите их выпустить мягкое напоминание своим менеджерам поступать так же, и вы поразитесь, как быстро все избавятся от этой привычки. Одного-двух предложений в вашей онбординг-документации часто достаточно как подталкивания для новых сотрудников, которые благодаря мягкому замечанию в онбординге и наблюдению за отсутствием аббревиатур вокруг них будут гораздо менее склонны создавать их сами.

Встречи и управление временем

В широком смысле существует три типа встреч: регулярно планируемые информационные встречи, встречи для разрешения конфликтов и спонтанные / ad-hoc встречи. Ваша задача как менеджера — задавать ожидания для участников в зависимости от того, к какому типу относится встреча. Для информационных встреч спросите себя, действительно ли встреча — лучший способ донести информацию; иногда так и есть, но не всегда. Если так, убедитесь, что информация донесена несколькими способами, возможно, с письменными материалами, предоставленными заранее в корпоративной вики. Если это встреча для разрешения конфликта, убедитесь, что вы заранее определили точки обсуждения, чтобы участники могли прийти подготовленными к обсуждению и проработке проблемы. У ad-hoc встреч, вероятно, есть чётко определённая цель с самого начала, и они не нуждаются в дальнейшем представлении.

Независимо от типа встречи, любая назначенная вами встреча должна иметь чёткую цель, известную приглашённым заранее. В идеале у каждого будет достаточно информации до встречи, чтобы понять, будет ли для него ценным на ней присутствовать. Не менее важно, чтобы ваша культура наделяла людей правом принять решение не присутствовать, если они сочтут это не лучшим использованием своего времени.

Управление временем

Как лидер в стартапе вы быстро обнаружите, что ваше время разделено между многими видами работы и потенциально десятками часов встреч каждую неделю. Если у вас ещё нет работающей для вас системы, сейчас самое время вложиться в хорошие привычки и навести порядок. Я рекомендую в качестве отправных точек на этом пути и «7 навыков высокоэффективных людей» Стивена Кови, и «*Getting Things Done*» Дэвида Аллена.

Тайминг встреч

Один из способов повысить продуктивность вашей команды — создавать или допускать большие блоки свободного времени для ваших инженеров. Переключение контекста (наша склонность перескакивать с одной несвязанной задачи на другую) обходится дорого (см. «Миф о многозадачности» на ctohb.com/myth), поэтому чем больше времени вы можете создать для инженеров, чтобы они выполняли инженерную работу без переключения на другие задачи (почта, телефонные звонки, встречи), тем меньше суммарный штраф за переключение контекста вы платите.

Я сторонник объявления неформального окна «часов для встреч» для команды. Поощряйте инженерную команду и кросс-функциональные команды планировать встречи в течение этого двух- или трёхчасового окна каждый день и старайтесь не назначать инженерам встречи вне этого окна. Это оставляет здоровый объём времени *каждый божий день* для вашей инженерной команды, чтобы сосредоточиться на сути своей работы, а также освобождает место для необходимых информационных встреч и встреч по разрешению конфликтов. Если ваша команда проводит более трёх часов на встречах в день (пятнадцать часов в неделю — почти половина их времени!), вам стоит внимательно присмотреться к этим встречам и спросить себя, можно ли их консолидировать и сократить.

Другие команды добились успеха с днями без встреч — выделяя один или несколько дней каждую неделю, когда никто не планирует никаких регулярных встреч. Только имейте в виду, что в сорокачасовой рабочей неделе ваша цель — зарезервировать как можно больше этих часов для вашей инженерной команды в виде непрерывных блоков времени для фокуса. Один день без встреч подразумевает восьмичасовой блок времени для фокуса, но остаётся ещё тридцать два часа, о которых нужно подумать, так что это не решает проблему целиком.

Рекомендация по времени инженера

Рассмотрим эту гипотетическую неделю программиста:

- Один час: 1:1 с менеджером
- Два с половиной часа: ежедневные тридцатиминутные стендапы
- Два часа: среднее время, проводимое на других agile-церемониях (планирование спринта, ретроспективы и т. д.)
- От четырёх до восьми часов: ревью чужого кода
- Четыре часа: коммуникация по почте/в чате

Итого это около тринадцати-семнадцати часов недели, потраченных на встречи и коммуникацию. Если добавить сверху ещё несколько часов на время, потраченное на переключение контекста и незапланированные разнообразные прерывания, вы быстро придёте к тому, что в лучшем случае половина сорокачасовой рабочей недели остаётся доступной для собственно времени фокуса. Если вы не будете внимательны к тому, когда планируются встречи, то у ваших инженеров не только останется лишь двадцать часов на их ключевые задачи, но и они не будут идти непрерывными блоками, ещё больше снижая продуктивность.

Я привожу этот надуманный пример, чтобы вбить мысль: обеспечение инженеров большими блоками времени для фокуса на инженерной работе не происходит случайно. Именно вам как лидеру, определяющему, как расходуется их время, предстоит развивать культуру и процесс, которые консолидируют и минимизируют эти отвращения и максимизируют время, доступное индивидуальным контрибьюторам для настоящей инженерной работы.

HIPPO

HIPPO — это неформально используемая в индустрии аббревиатура, расшифровывающаяся как Highest-Paid Person's Opinion (мнение самого высокооплачиваемого человека). Независимо от того, являетесь ли вы самым высокооплачиваемым человеком или нет, ваша должность будет подразумевать, что это так. Особенность HIPPO в том, что большинство сотрудников неохотно оспаривают мнение HIPPO. Я настоятельно рекомендую минимизировать этот эффект в обсуждениях, регулярно открывая дверь для возражений, открыто признавая, что вы можете ошибаться, а затем действуя в соответствии с чужими идеями и продвигая идеи, отличные от ваших собственных. Вы поймёте, что делаете это достаточно часто, когда вам начнёт казаться, что вы делаете это слишком часто. К тому моменту, когда вам покажется, что вы перебарщиваете, вы, вероятно, достигнете того минимума, который большинству сотрудников действительно нужен, чтобы вам поверить.

Несмотря на все ваши усилия выглядеть доступным и открытым к убеждению в пользу других подходов, ваше присутствие на встрече часто всё равно будет оказывать подсознательное влияние на остальных участников, особенно если они находятся более чем на один уровень ниже вас в организационной структуре. Помните об этом эффекте и старайтесь посещать встречи только тогда, когда вы действительно приносите пользу и команда нуждается в вашем присутствии. Для всего остального вы можете получить заметки или запись после окончания встречи.

Списки дел

В целом списки дел (to-do lists) — это весьма примитивная форма управления задачами: им не хватает структуры, какого-либо ощущения приоритизации или временной составляющей. Вместо этого я рекомендую использовать список дел на основе календаря. Вместо того чтобы помещать рабочие задачи в общий список, размещайте их в своём реальном календаре.

Использование календаря в качестве списка дел имеет несколько преимуществ. Оно резервирует выделенное время на фактическое выполнение пунктов из вашего списка дел и гарантирует, что вы не перегружаете своё время. Оно позволяет приоритизировать задачи, перемещая их, а также упрощает прогнозирование того, когда что-либо будет сделано. В большинстве календарных систем также есть встроенные механизмы напоминаний, которые уведомят вас, когда у вас запланировано выполнение конкретной задачи.

Ретроспективы календаря и баланс времени

Время от времени — скажем, раз в месяц — я рекомендую делать исторический обзор своего календаря и измерять, как вы потратили своё время. Например, в Google Calendar есть встроенная аналитика, и для получения точных сводок о том, как было потрачено время, требуется лишь минимальная адаптация ваших календарных привычек. При анализе этих данных спросите себя, имеет ли смысл соотношение времени, потраченного на различные типы активностей, для целей, которых вы пытаетесь достичь. Также полезно сверяться и убеждаться, что вы тратите своё время теми способами, которые играют на ваших сильных сторонах и приносят вам личное удовлетворение. Часто одно лишь беспристрастное представление подобных данных может стать хорошей мотивацией для организованности и продуктивных перемен.

Мини-фреймворки управления

Многие проблемы, с которыми вы столкнётесь как руководитель, следуют общим, повторяющимся паттернам. Наличие ментального фреймворка для подхода к проблемам может ускорить принятие решений, повысить их качество и обеспечить контекст и перспективу для объяснения решений другим. Ниже приведены некоторые фреймворки, которые я нашёл полезными в своём управленческом пути.

Три стадии решения управленческих проблем

Когда передо мной стоит крупный, неоднозначный вызов — например, принять на себя управление новой командой или диагностировать и улучшить недостаточную эффективность отдельного человека — мне нравится использовать трёхэтапный процесс. Эти этапы следует выполнять последовательно, чтобы выработать план решения конкретной проблемы.

1. Сбор информации (Ingest)

- Впитывайте как можно больше информации. Читайте доступную документацию — будь то статьи в вики, оценки эффективности, код или что угодно, что вы можете найти, связанное с вашей проблемой. Назначайте встречи один на один (1:1), упражняйте свои навыки активного слушания и делайте подробные заметки по своим находкам.
- Вы поймёте, что собрали достаточный объём данных, когда начнёте видеть одно и то же по несколько раз и перестанете замечать новые паттерны.
- Пример: несколько человек комментируют эффективность одного из членов вашей команды, и после нескольких сессий активного слушания и проявления любопытства вы перестаёте получать дальнейшую новую информацию по этой проблеме с эффективностью.

2. Синтез (Synthesize)

- После того как вы собрали достаточный массив данных, связанных с вашей проблемой, сделайте шаг назад от сбора информации и дайте своему мозгу время на обработку. Я рекомендую выделить на этот этап как минимум несколько дней. Постарайтесь намеренно прекратить впитывать новую информацию и потратьте это время на рассмотрение проблемы под разными углами. Делайте заметки, рисуйте диаграммы, играйте в гольф, принимайте душ или делайте что угодно, что помогает вам обдумать проблему и прийти к анализу, который соответствует данным и пригоден для действий.
- Продолжая приведённый выше пример, на этом этапе вы можете попытаться выдвинуть различные гипотезы о том, почему этот человек показывает недостаточную эффективность: как он тратит своё время? Это несоответствие навыков, несоответствие ожиданий или что-то идёт не так в его личной жизни и т. д.?

3. Действие (Act)

- Как только у вас появился тезис, пора действительно претворить план в жизнь. Когда вы предпринимаете действия, важно проверять, что ваш план достигает желаемых результатов. По возможности тестируйте, проверяйте и при необходимости запускайте цикл заново.

Модели принятия решений на основе команды

Существует три модели принятия решений с вашей командой. Как руководитель вы можете принимать решения полностью самостоятельно и преподносить результат команде как *fait accompli* (свершившийся факт). Есть и противоположный подход: вы начинаете с нуля и полностью совместно разрабатываете материал с частью команды или со всей командой. Третий подход — это компромисс между первыми двумя: вы сами разрабатываете черновик и преподносите его команде как «соломенное чучело» (straw man), отправную точку для сбора обратной связи и итерирования к финальной версии. Ключевые различия между этими техниками — это количество времени, которое они занимают, и то, насколько большую вовлечённость (buy-in) вы получаете от команды. И я призываю вас оптимизировать ради вовлечённости: обеспечение того, чтобы каждый член вашей команды понимал решения и мог быть их сторонником, — единственный способ гарантировать, что вы все шагаете в одном направлении. Как называет это Marty Cagan из Silicon Valley Product Group, вы хотите, чтобы ваша команда вела себя как миссионеры, а не как наёмники.

Независимая модель: Разработка решений руководителем самостоятельно занимает меньше всего общего времени, но также порождает наименьшую вовлечённость.

Модель «соломенного чучела»: Совместная разработка решений, начинающаяся с «соломенного чучела», занимает среднее количество времени и, в зависимости от исполнения, может породить здоровую вовлечённость.

Модель совместной разработки: Коллективная разработка с нуля может занять значительное количество времени, хотя она порождает наибольшую вовлечённость со стороны тех, кто внёс вклад в эту разработку.

Какую модель использовать для того или иного решения — решать вам, и я призываю вас подходить к этому выбору вдумчиво и осознанно. Не бойтесь пересмотреть его, если почувствуете, что выбрали неправильную модель.

Два типа решений

В ежегодном письме акционерам в 1997 году Jeff Bezos изложил фреймворк для классификации решений на решения Типа 1 и Типа 2. Решения Типа 1 необратимы, и их следует продумывать методично, тщательно, медленно, с большой обстоятельностью и консультациями, писал Bezos. Если вы проходите через дверь и вам не нравится то, что вы видите по другую сторону, вы не можете вернуться туда, где были раньше. Например, то, какой язык программирования вы используете, — это решение Типа 1.

Решения Типа 2 — противоположность. Они могут и должны приниматься быстро людьми с хорошим суждением или небольшими группами. То, какой именно оттенок серого будет у кнопки, может быть решением Типа 2, поскольку его легко изменить впоследствии.

Совет Bezos, который я повторяю здесь, состоит в том, что применение подхода к принятию решений Типа 1 для решений Типа 2 ведёт к медлительности и неспособности экспериментировать и внедрять инновации.

Большинство ваших повседневных технических решений относятся к Типу 2, и их лучше всего принимать быстро, а затем пересматривать или подтверждать после того, как вы соберёте больше данных через прототип или реализацию MVP. Это потому, что самым дорогим элементом большинства технических решений в стартапе является время инженерной команды, вложенное в решение. Если вы намеренно ограничите время (а значит, и стоимость), вложенное в проверку обратимого решения, вы потеряете лишь малость. Большинство технических решений Типа 2 становятся необратимыми только после того, как вы вложили в них значительное время и новую инженерную работу, поэтому будьте строги в оценке прогресса на раннем этапе. В случае сомнений принимайте решение об откате как можно раньше.

Разрешение ничьих

Как руководитель своей команды, в конечном счёте именно вы несёте ответственность за достижение целей команды. Если команда в целом не достигает контрольных точек, это на вашей совести. Поэтому, когда внутри команды возникает конфликт или разногласие, вам нужно вдумчиво вовлечься, а затем быть готовым принять решение, следуя одному из этих трёх сценариев:

1. Мы пойдём вашим путём, потому что вы привели ясный и убедительный аргумент в пользу того, что он лучше.
2. Мы пойдём моим путём, потому что он лучше, и я объясню почему, используя дополнительный контекст, который у меня есть как у руководителя с более широкой сферой ответственности.
3. Мы пойдём моим путём, потому что мы не можем выявить никакой объективной причины, по которой один путь лучше другого. Иными словами, это ничья, и поскольку в конечном счёте именно я отвечаю за успех, мы пойдём моим подходом. Я возьму на себя успех или неудачу этого решения.

Сортировка задач — матрица «Срочное/Важное»

В книге *The 7 Habits of Highly Effective People* доктор Stephen Covey предлагает матрицу «Срочное/Важное», адаптированную из концепции, представленной президентом Dwight Eisenhower в речи 1955 года. В дихотомии «Срочное/Важное» работа классифицируется как по её срочности (т. е. чувствительности ко времени), так и по важности (т. е. влиянию). В результате получается четырёхквadrантная диаграмма:

Я привожу здесь этот фреймворк как напоминание о необходимости оценивать ценность различных возникающих задач. Технического руководителя регулярно бомбардируют запросами на функционал, долгом, который нужно приоритизировать, дефектами и т. д., и наличие перспективы задаться вопросом, является ли тот или иной пункт важным и/или срочным, — это очень полезный и быстрый механизм сортировки.

Решатель головоломок из людей

Большая часть работы, которую вы будете выполнять как руководитель, — это работа решателя головоломок из людей. Вам придётся выяснять, как помочь двум людям продуктивно работать вместе, направлять кого-то на улучшение навыка или проектировать структуру команды для обеспечения сотрудничества. Поведение людей трудно предсказать, и иногда член команды может говорить одно, а думать или чувствовать другое.

В свете этой постоянно меняющейся сложности я призываю вас мыслить как детектив или учёный: всегда собирайте данные. Сформулируйте гипотезу о том, что может произойти, если вы предпримете то или иное действие, предпримите это действие, а затем соберите данные о результатах. Намеренное прохождение через эти шаги будет побуждать вас слушать, наблюдать, как люди реагируют в различных сценариях. Это, в свою очередь, обеспечит полезные данные на следующий раз, когда вы окажетесь в схожем сценарии с теми же людьми.

Присоединение к команде

Есть два способа присоединиться к команде в качестве технического лидера: либо вы начинаете работу в команде на нелидерской позиции и вырастаете или продвигаетесь в эту роль, либо вас нанимают возглавить команду. Если вас продвигают в роль, предположительно, у вас есть хороший бизнес-контекст и вы продемонстрировали техническую компетентность, но ещё не проявили себя в управлении и лидерстве. И наоборот, если вас нанимают на лидерскую позицию, у вас, вероятно, есть послужной список в управлении, но не хватает контекста, истории и фоновых знаний о бизнесе организации, технологиях и людях. Отсюда следует, что ваш подход при вступлении в роль должен различаться, чтобы компенсировать соответствующие слабые места.

Продвижение в управление/лидерство

Если вас продвигают на управленческую позицию и вы вложили время в развитие управленческих навыков, или если у вас есть опыт и послужной список руководителя, то, скорее всего, этот переход не покажется вам очень страшным, и вашей целью будет продолжать расти как руководитель. Однако если это ваша первая управленческая роль, вам предстоит покорить гору повыше.

Управление людьми — это совершенно новый набор навыков по сравнению с техническими навыками, которые принесли вам повышение. Технические навыки, разумеется, являются предпосылкой для того, чтобы быть хорошим техническим руководителем, но их далеко не достаточно. Понимание этого и развитие дополнительного набора навыков, которого требует ваша новая роль, будут ключом к успеху в качестве руководителя.

Если бы ваша рабочая роль изменилась с backend-инженера, пишущего код на C, на frontend-инженера, пишущего код на TypeScript, что бы вы стали делать? Вы могли бы прочитать книгу по TypeScript, выполнить несколько упражнений по программированию на TypeScript, вступить в группу пользователей TypeScript, почитать несколько блогов о TypeScript и т. д.

Переход от написания кода на C к управлению людьми на самом деле представляет собой бóльшую переменную, чем переход с C на TypeScript. Успешное совершение этого перехода требует такого же или большего уровня проактивного обучения. Для большинства людей становление отличным руководителем людей — это путешествие длиною в жизнь, а не навык, освоенный за выходные. Примите этот вызов и относитесь к работе как к упражнению по обучению длиною в жизнь, которое начинается прямо сейчас.

Применимые на практике советы по управлению

Памятуя об изречении канцлера Германии Otto von Bismarck — **Глупцы учатся на собственном опыте; я предпочитаю учиться на опыте других**, — вот несколько применимых на практике советов для новых руководителей:

- Найдите управленческого ментора или коуча. Обсуждение чужих проблем и получение дополнительной перспективы бесценно. (См. врезку «Найдите управленческого ментора», стр. 13.)

- Научитесь делегировать. В книге *Immunity to Change* Robert Kegan подробно разбирает препятствия к делегированию и то, почему обучение делегированию столь критично для успеха руководителя. Суть в том, что ваша ценность для организации теперь не ваш собственный личный результат, а результат вашей команды. Для многих технических руководителей это самая трудная часть перехода, но она же и самая критичная.
- Заботьтесь о себе.
 - Управление людьми может очень сильно истощать эмоционально. Чтобы стабильно быть лучшей версией себя — уравновешенным, позитивным и продуктивным, — критически важно заботиться о себе.
 - Выясните, что работает именно для вас; возможно, это медитация, гольф, видеоигры или время с семьёй. Делайте то, что приносит вам хорошее самочувствие и оставляет вас отдохнувшим и готовым к следующему вызову.
 - Обращайте внимание на признаки выгорания и делайте перерыв до того, как оно наступит. Управление людьми не обязательно должно быть деятельностью на восемьдесят часов в неделю.

См. список рекомендованной литературы в конце этой книги, чтобы найти больше ресурсов для развития ваших управленческих навыков.

Найм на лидерскую позицию

Если вас наняли возглавить команду (в отличие от продвижения на лидерскую должность или принятия лидерской роли в качестве сооснователя стартапа впервые), предположительно, у вас есть как технический, так и управленческий опыт. Ваш вызов как нанятого руководителя существующей команды — максимально плавно интегрироваться в новую команду и выстроить доверие с новыми коллегами. Неудивительно, что мой совет — сосредоточиться больше на людях, чем на технологиях, при управлении вашей новой командой.

Ниже я обрисовываю некоторые краткосрочные цели для технического руководителя, нанятого извне, плюс несколько вопросов, на которые вам стоит попытаться ответить очень рано.

Цели:

- Выстроить доверие с технической командой. Слушайте и будьте вдумчивы относительно того, когда и насколько быстро вы начинаете приносить пользу или менять что-либо.
- Выстроить доверие с другими командами/руководителями, беря на себя разумные обязательства и доводя их до конца.
- Узнать о людях, с которыми вы работаете, и об их истории отношений с компанией/продуктом/технологией.
- Диагностировать самые влиятельные проблемы, связанные с людьми, внутри команды. Есть ли сотрудники, неправильно оценённые по уровню — недотягивающие или, наоборот, перерастающие свои роли? Нуждаются ли какие-либо культурные вызовы в корректировке курса? Решительные действия на раннем этапе для корректировки культуры — отличный способ выстроить доверие с членами команды, которые, вероятно, сознательно или подсознательно страдали от культурной проблемы.
- Диагностировать самые влиятельные технические проблемные области для команды в целом и составить набор краткосрочных, среднесрочных и долгосрочных целей для команды.

Вопросы:

- Кто руководил технологиями раньше? Этот человек всё ещё в команде?
 - Технические руководители часто обнаруживают, что хотят дорасти до управления людьми. Там, где это произошло, вы будете вступать в вакуум управления людьми. Другой распространённый сценарий — это что предыдущего технического руководителя либо вообще не было, либо он ушёл с

должности из-за недостаточной эффективности, и вы наследуете большой объём технического долга.

- Какую проблему, по словам CEO, вас наняли решить? Какую проблему, по вашему мнению, вас наняли решить?
- Какие болевые точки существуют между технической командой и остальной частью компании сегодня?
- Какие болевые точки имеют наивысший приоритет внутри технической команды?

Технические советы друзьям/незнакомцам

После того как у вас в резюме какое-то время значится та или иная форма технического лидерства, к вам, скорее всего, начнут обращаться за советом друзья или друзья друзей. По большей части я рекомендую принимать эти звонки — не только потому, что нетворкинг ценен, но и потому, что задаваемые вам вопросы могут заставить вас продумать и облечь в слова идеи, над которыми вы подсознательно работаете. Говорят, обучение других — лучший способ по-настоящему чему-то научиться самому.

Короткое замечание о консультировании нетехнических основателей: время от времени к вам, скорее всего, будет обращаться кто-то с самопровозглашённой идеей на миллиард долларов. Принять такие звонки стоит очень немного, и это может быть отличным способом накопить немного социального/отношенческого капитала. Однако помните, что блестящие идеи сами по себе не создают успешных бизнесов. Между каждой великой идеей и успехом лежит гигантская гора исполнения, и большинство восходителей не экипированы для покорения Эвереста. Поэтому будьте очень осторожны, беря на себя обязательства перед тем, у кого есть хорошая идея, но нет альпинистского снаряжения.

Трафик: воронки и зонтики

Вы можете быть либо воронкой для дерьма, либо зонтиком от дерьма. — Todd Jackson, продакт-менеджер Gmail (см. ctohb.com/umbrella и ctohb.com/keytogmail)

Вопросы, опасения и идеи о вашем продукте, при отсутствии какого-либо строгого процесса их перенаправления в другое место, найдут свой путь к руководству. Это касается не только вас, но и всех в руководстве вашей организации. Руководители — это входящий ящик по умолчанию, и суть высказывания Jackson в том, что ваша команда — это исходящий ящик по умолчанию. Вы слышите: «Эй, в X есть баг», — и думаете: «Ладно, эту функцию написал инженер Y, отправлю-ка ему этот баг». Это был бы пример переправления входящего потока напрямую вашей команде через воронку.

Лучшая стратегия — вместо этого выступать зонтиком для команды. Вместо того чтобы направлять весь входящий поток в реальном времени на команду, хороший руководитель организует, приоритизирует и даёт команде структурированную очередь, с которой можно работать. Ваша цель — помочь команде сосредоточиться, ограничить отвлечения и обеспечить место, куда должен поступать входящий поток, чтобы его можно было эффективно обрабатывать.

Лишь в редких случаях вам как руководителю следует выделять баг для отдельного инженера. Если у вас есть очередь багов и процесс работы с этой очередью, вы можете в значительной степени устранить регулярные разовые эскалации.

Руководство должно отслеживать этот процесс очереди багов, чтобы очередь оставалась управляемой длины, и корректировать штат или процесс, если качество продукта не соответствует целевым показателям.

Вы должны приоритизировать свои очереди на основе важности и срочности. Если возникает что-то критически важное с крайним временным давлением, это следует поместить в очередь и поднять на самый верх. Затем применяйте здравый смысл к тому, как вы с этим справляетесь. Если вам нужно позвонить кому-то, чтобы убедиться, что они в курсе, то так тому и быть, — лишь бы это было исключением, а не правилом.

Ваша первая команда

Как у технического руководителя ваша работа состоит не только в том, чтобы управлять технической командой; она также включает выполнение роли технического представителя в C-suite (высшем руководстве). Ваша роль —

представлять инженерию и технологии во всех стратегических дискуссиях компании самого высокого уровня и обеспечивать, чтобы инженерия и технологии находились на правильном курсе для бизнеса. Порой это означает необходимость вести трудные разговоры с другими руководителями, которые требуют уязвимости и смирения, разговоры, которые позволяют вам прорабатывать конфликт и которые основаны на взаимном доверии. Эти разговоры — ключ к работе, и чтобы они стали возможны, вы должны быть глубоко вовлечены в свою команду руководителей, считая их своей первой командой.

Вы, вероятно, очень технический человек. Вам, вероятно, больше всего нравятся ваши технические встречи, или, по крайней мере, решение технических проблем с вашей командой кажется вам привычным и крайне приносящим удовлетворение. Поэтому легко скатиться в паттерн, при котором вы проводите большую часть времени с техническими командами, и принять установку «моя команда — это техническая команда». Это отличный способ выстроить взаимопонимание внутри технической команды, и бывают моменты, когда это и есть самые критичные отношения, в которые стоит вкладываться.

Мой совет прост: не позволяйте техническим блестящим штучкам отвлекать вас или ограничивать ваши вложения в отношения с нетехническими руководителями. Ваши отношения с другими руководителями и ваше доверие с нетехническими коллегами придадут вам авторитет и позволят направлять бизнес к принятию хороших технических решений в целом. Доверие за пределами технической команды строится так же, как строятся любые другие доверительные отношения: отличной коммуникацией, регулярной установкой ожиданий через принятие и выполнение обязательств и признанием ошибок/неудач, если и когда они происходят.

Технический руководитель или СТО, который проводит всё своё время погружённым в код с инженерией и едва участвует в команде руководителей, будет иметь мало авторитета, когда попытается убедить остальных C-level вкладываться в инженерию дальше. Или, что ещё хуже, его даже не спросят о мнении, когда придёт время принимать трудное решение. У других руководителей не будет контекста, чтобы понять ценность того, о чём просит техническая команда, или перспективы того, как инженерия функционирует в данный момент, и у них не будет общего видения того, какой она может быть в будущем. Только регулярно взаимодействуя с остальной командой руководителей, делаясь этим контекстом и будучи частью разговора на всём пути, вы как технический руководитель можете гарантировать, что у команды руководителей есть общее понимание того, как инженерия помогает организации и как ей нужно расти со временем.

Работа с CEO

Каждые отношения СТО и CEO различны, хотя есть несколько элементов, общих для всех таких партнёрств, а также некоторые ключевые предпосылки для здоровых отношений.

Согласование конкретных целей

CEO должен полностью доверять вашей способности вести техническую команду к достижению бизнес-целей. Выстраивание этого доверия означает, что вам нужно уметь хорошо коммуницировать с CEO — как через проактивную коммуникацию, так и обеспечивая, чтобы у CEO всегда было достаточно контекста, чтобы задавать хорошие вопросы.

Хорошо коммуницировать означает научиться говорить на языке бизнеса и избегать скатывания в технический жаргон во время разговоров с руководством. Вы хотите дать CEO возможность коммуницировать с вами, и чем больше вы говорите на их языке (если они не технические), тем больше информации вы сможете извлечь из ваших взаимодействий и тем лучше вы двое будете ладить.

Согласование направления бизнеса

Вам и CEO нужно иметь общее понимание направления бизнеса и быть способными вступать в конструктивные и, возможно, даже спорные разговоры, чтобы обеспечить глубину этого понимания. Доверие применимо как к общему направлению бизнеса, так и к конкретным целям.

Есть много способов выстроить это, но вам нужно установить доверие независимо от выбранного подхода. Занимайтесь совместными нерабочими активностями, находите области, где вы разделяете личные ценности, и используйте конкретные инструменты и упражнения для выстраивания этого доверия (см. Brene Brown's BRAVING Inventory на ctohb.com/braving).

Согласование культуры и ценностей

Как и со всеми C-level, СТО и CEO должны иметь сильное согласование по культуре и ценностям компании. Особенно важно, чтобы вы сосредоточились на построении позитивной культуры внутри инженерии, а также между инженерией и остальной частью компании. Техническая команда зачастую представляет собой очень крупную, если не самую крупную статью расходов в бюджете стартапа. Технический персонал также часто относится к самым конкурентным ролям для найма, что делает рекрутинг и неизбежную непроизвольную текучесть кадров более дорогими в инженерии, чем в других отделах.

Сильное согласование между СТО и другими руководителями уровня C-level по культуре и ценностям — ключевой фактор в обеспечении того, чтобы техническая команда чувствовала себя уважаемой и включённой в компанию, что, в свою очередь, должно помочь с удержанием.

Сообщение плохих новостей

Один общий совет по работе с другими руководителями или топ-менеджерами — не уклоняться от сообщения плохих новостей коллегам-руководителям, особенно CEO. Поскольку вы несёте ответственность за результаты команды, у вас может возникнуть соблазн приукрасить реальность, заявить, что всё в порядке. Проблем у такого подхода множество:

- Если дела не в порядке — то есть дедлайны постоянно срываются или качество падает ниже ожиданий, — ваши коллеги будут это знать и будут недоумевать, почему вы не берёте на себя ответственность за эти неудачи и не объясняете, как вы будете улучшать ситуацию. Этот разрыв между реальностью и тем, как вы её представляете, подрывает доверие к вашему лидерству.
- Время от времени вам нужно будет выделять время команде программной инженерии на инженерную работу, не обращённую к пользователю, как на инвестицию — либо в технический долг, либо в будущую архитектуру. Вам понадобится доверие других руководителей и авторитет, чтобы другие верили вам и понимали ROI на это время.

См. раздел «Принцип» главы 1 книги Jocko Willink и Leif Babin *Extreme Ownership: How U.S. Navy SEALs Lead and Win*, чтобы узнать больше о важности принятия ответственности за неудачу.

Говорите на языке вашей аудитории

Технические темы часто крайне нюансированы; детали имеют значение. Технический жаргон отлично справляется с задачей помочь передать этот нюанс, поэтому неудивительно, что, объясняя технические предметы, инженеры часто используют язык, непонятный сотрудникам из других отделов. Я уверен, вы были свидетелем того, как чрезмерно рьяный инженер с энергией, страстью и воодушевлением пытается объяснить свой проект неинженеру, но в ответ получает лишь пустой взгляд, и никакого нового общего понимания не возникает. Как технический руководитель вы должны справляться лучше.

Если, например, у вас есть крупная область технического долга и вы хотите отстоять внутри команды руководителей идею потратить целый месяц на переархитектуру этой области кода, вам нужно сообщить свои причины понятным образом. Если вы войдёте в этот разговор, рассуждая о задержках (latencies), RPC-вызовах, dependency injection и аббревиатурах от вашего облачного провайдера, велики шансы, что ваши CEO и CFO отключатся почти сразу.

Если же, с другой стороны, вы выстроите этот разговор вокруг продуктивности разработчиков и морального духа команды и объясните погашение долга в контексте скорости работы команды (velocity) на ближайшие шесть месяцев, ваш аргумент будет гораздо убедительнее.

Лучшие практики технической коммуникации

Несколько общих советов для обеспечения более успешных обсуждений и презентаций — особенно с неинженерами — когда речь идёт о технике:

Заранее установите общий язык/словарь. Если вам нужно использовать какие-либо слова, которые средний старшеклассник не понял бы, убедитесь, что ваша аудитория уже знает их, или чётко определите их, прежде чем

пускаться в объяснение.

Используйте понятные концепции. Технические вызовы часто сравнивают с другими техническими вызовами: это не работает в разговоре с нетехническим персоналом. Вместо того чтобы описывать вашу медленную передачу данных в байтах в секунду, сравните её с трафиком на автомагистрали.

Подтверждайте понимание по ходу дела. Задавайте вопросы своей аудитории во время объяснения. Если они могут произнести развязку раньше вас, значит, вы на верном пути.

Не предполагайте, что вы хоть в чём-то превосходите других благодаря своему владению техническим языком, или, что ещё хуже, не заставляйте свою аудиторию чувствовать себя ниже из-за нехватки знаний. «Я не хочу слишком углубляться в технические детали для вас» — отличный способ оттолкнуть аудиторию.

В целом старайтесь делать свои объяснения настолько простыми и сжатыми, насколько можете. Избегайте ухода в «кроличьи норы», которые могут отвлекать или иным образом терять вовлечённость аудитории.

Найм и проведение собеседований

Эффективный найм — одна из ваших активностей с наибольшим влиянием как технического руководителя и одна из самых сложных, чтобы сделать её правильно. Вы часто будете оказываться в ситуации попытки рекрутировать таланты на рынке с ограниченным предложением и конкуренции с другими компаниями, у которых могут быть карманы поглубже ваших. Ваши лучшие кандидаты, скорее всего, будут получать другие конкурирующие предложения о работе, а это значит, что вам нужно не только квалифицировать кандидатов, но и убедить их, что ваша возможность — правильная для них.

В этом смысле найм — это в той же мере деятельность по продажам (где кандидаты квалифицируют вас/вашу компанию), как и процесс фильтрации (где вы/ваша компания квалифицируете кандидатов). Важно держать это в уме на каждом шаге пути по мере того, как вы определяете процессы найма своей команды.

Этот раздел книги охватывает различные части пути найма и проведения собеседований последовательно — от планирования численности штата до онбординга.

Нанимайте как стартап

Как стартап, вы обладаете рядом ключевых преимуществ в найме, и крайне важно использовать их в процессе, чтобы привлечь лучших специалистов. Вот несколько особенностей, которые дают вам фору:

- Вы меньше, а значит, вы должны быть более гибкими, более внимательными к каждому кандидату и нанимать быстрее, чем крупные конкуренты.
- Вы можете «продать» по-настоящему привлекательную траекторию роста компании и личного развития.
- Вы можете «продать» творческую и вдохновляющую культуру рабочего места.
- Вы можете «продать» тот эффект (impact), который успешные кандидаты смогут создать.
- Вы можете предложить значимую долю в капитале компании (equity) и, как следствие, возможность разделить выгоды от её успеха.

Вот несколько практических советов, как использовать эти преимущества:

- Чтобы действовать быстро, обучите и привлечите коллег к проведению собеседований ещё до публикации описания вакансии. Убедитесь, что все понимают процесс планирования встреч, сценарии собеседований и критерии оценки, а также умеют пользоваться системой отслеживания кандидатов (Applicant Tracking Software, ATS), чтобы читать и оставлять обратную связь, ещё до того, как вы начнёте (см. раздел «Поиск кандидатов», стр. 59).
- Назначайте все собеседования с каждым кандидатом сразу. Если ваш процесс включает четыре собеседования, поставьте все четыре в календарь с самого начала, в идеале в течение пяти рабочих дней. Если ваша команда оперативно оставляет обратную связь по собеседованиям (а вы должны на этом

настаивать), то любым кандидатам, которые не прошли отбор, вы сможете заранее сообщить об этом и отменить все ожидающие приглашения в календаре. Альтернатива — назначать каждое следующее собеседование только после того, как кандидат прошёл очередной этап — добавляет по несколько дней между шагами, легко превращая процесс, который мог бы занять неделю, в три недели или более.

- Хорошее эмпирическое правило, особенно в стартапе: никто не настолько занят или важен, чтобы не найти время встретиться с кандидатами, если это имеет смысл, особенно при найме на более старшие позиции. Если сильный кандидат просит поговорить с вашим CEO и COO, то вам следует назначить с ними встречи.
- Убедитесь, что у каждого интервьюера есть уникальный сценарий или гайд, который охватывает иной материал или тот же материал под другим углом, нежели другие собеседования. (Подробнее об этом — в разделе «Задавайте только новые вопросы» главы «Лучшие практики собеседований», стр. 63.)

Как всегда, бесплатного сыра не бывает, и преимущества найма в качестве стартапа сопровождаются компромиссами, прежде всего в виде рисков. Кандидаты почти наверняка зададут вам вопросы о соответствии продукта рынку (product market fit), о наличии денежных средств или о финансовой подушке (runway), о культуре компании и о балансе между работой и личной жизнью. Я призываю вас быть откровенными с кандидатами по этим аспектам, согласовать фактическое положение дел с вашей управленческой командой и иметь хорошие ответы на эти вопросы.

Скорость — ваш друг!

Помните: скорость — ваш друг при привлечении лучших специалистов. Если вы способны провести человека через весь ваш процесс за неделю, вы даёте своему стартапу серьёзное преимущество перед крупными компаниями, чьи процессы зачастую растягиваются на месяцы, прежде чем будет принято решение.

Когда нанимать: планирование штата

Деньги — это кровь молодого и пока неприбыльного стартапа, поэтому решение взять на себя ежегодные регулярные расходы в размере более 100 000 долларов в виде зарплаты нового инженера нельзя принимать легкомысленно. На решения о численности штата влияет несколько ключевых факторов, и главные из них — потребность, приоритизация, тайминг и бюджет.

Потребность в роли / пробел в команде

Первый шаг к решению о найме — выявление пробела в команде. Пробелы бывают разных видов. Часто на ранней стадии развития компании это просто пробел в навыках. Например, ваш бизнес решает, что мобильные приложения станут ключевым элементом стратегии выхода на рынок (go-to-market), а ваша команда-основатель никогда раньше не работала с мобильной разработкой. Безусловно, они могли бы научиться и со временем стать эффективными, но и в краткосрочной, и в долгосрочной перспективе гораздо эффективнее нанять опытного инженера, у которого есть опыт работы с мобильной разработкой и желание этим заниматься, чтобы создавать и поддерживать этот проект.

К другим видам пробелов относятся пробелы в уровне (seniority) — недостаточно старшего опыта для принятия хороших решений или недостаточно младших специалистов для выполнения менее сложных задач, — управленческие пробелы (один руководитель отвечает за слишком много людей) или пробелы в предметной экспертизе (нет никого в команде, кто понимал бы какую-то область отрасли достаточно хорошо, чтобы направлять принятие решений).

Другое важное обоснование найма — увеличение общей пропускной способности (bandwidth) команды. Такие наймы должны быть согласованы с какой-либо бизнес-целью или дорожной картой продукта (roadmap), которая оправдывает добавление нового постоянного члена команды в конкретный момент.

Приоритизация роли и тайминг

После того как вы выявили пробел, следующий вопрос — когда этот пробел нужно закрыть. С учётом времени, необходимого для найма отличного специалиста, когда имеет смысл начинать процесс найма?

Как правило, существует давление нанимать как можно скорее. Постарайтесь сопротивляться этому давлению, поскольку это не всегда правильный ответ. Каждый новый нанятый человек добавляет вашей команде сложности и накладных расходов. Если боль от наличия пробела не слишком сильна и вы можете обойтись меньшей командой ещё полгода и отложить найм, это может быть хорошей идеей: так вы и снижаете затраты, и получаете больше времени, чтобы обосновать необходимость найма.

Запросы на расширение штата или найм от вашей команды часто будут конкурировать с запросами от других команд, поэтому полезно выработать в компании общий язык для обсуждения того, насколько срочен или важен тот или иной найм. Это не обязательно должно быть сложным; например, это может быть система ранжирования от 0 до 5, где 0 означает срочную потребность, а 5 — найм, который было бы неплохо сделать, но он может подождать несколько месяцев или кварталов, прежде чем станет срочным.

Бюджетирование новых наймов

В неприбыльном стартапе у вас должна быть финансовая модель, которая соотносит расходы с доходами и примерно прогнозирует, на сколько хватит ваших текущих денежных средств до того, как понадобится новый раунд привлечения инвестиций. Большинство CEO и CFO глубоко и досконально понимают эту модель; как технический руководитель вы не будете тратить на неё столько же времени.

Тем не менее крайне важно, чтобы вы поддерживали ясную картину вклада вашего отдела в эту модель, который в первую очередь будет выражаться в виде расходов на персонал (текущих и будущих). Эта модель должна задавать определённое ограничение — в виде годового бюджета или темпа расходов (run-rate), — которое будет направлять тайминг ваших наймов.

Цели и задачи найма

Так же как и при проектировании программных систем, садясь за проектирование процесса собеседований, вам следует начать с обдумывания ваших требований и целей. Хотя у каждой компании должны быть и будут свои собственные требования и точка зрения, вот некоторые из вещей, которые я учитываю при проектировании процесса собеседований:

Эффективность: Сколько времени и средств уходит на найм кандидата?

Успешность: Насколько успешны нанятые кандидаты в работе и как долго они остаются в компании?

Опыт кандидата: Уходят ли кандидаты с высоким мнением о вашей компании после прохождения процесса, независимо от того, были ли они наняты?

Равные возможности: Обеспечили ли вы каждому человеку справедливый шанс быть нанятым и максимально ли избежали неосознанной предвзятости?

Масштабируемость: Могут ли люди, кроме вас, проводить процесс и быть столь же эффективными / иметь успешность, близкую к вашей?

Эффективность

Качественный найм — дорогостоящее мероприятие для вашей компании. Эта стоимость измеряется как в реальных деньгах — будь то рекрутеры, размещение на досках вакансий или рекламные объявления о работе, — так и во времени, прежде всего во времени сотрудников, потраченном на проведение собеседований. Проектируя процесс собеседований, подумайте, каково ваше намерение на каждом этапе, что именно вы фильтруете и какой способ выполнить эту фильтрацию наиболее эффективен.

Один из способов сократить или хотя бы распределить вложение времени — включить в процесс найма других членов команды. В зависимости от тематики конкретного собеседования вам не всегда нужны в комнате самые старшие инженеры. Координатор найма при должном обучении может провести телефонный скрининг, собеседование по культуре или проверку рекомендаций столь же эффективно, как старший инженер или руководитель.

Успешность

Не каждый найм окажется для вашей компании попаданием в десятку. Кто-то будет неверно оценён по уровню, кто-то не впишется в культуру, а кого-то уволят или он уйдёт в первый же год. Особенно по мере масштабирования процесса собеседований вам нужно измерять, сколько наймов оказываются успешными. Это одна из немногих возможностей для технического руководителя рассчитать чёткие, согласованные и показательные метрики, так что воспользуйтесь этим и убедитесь, что ваш процесс на высшем уровне. Рассмотрите возможность отслеживать время найма (от публикации описания вакансии до даты выхода нового сотрудника), общую удерживаемость сотрудников, отток новых сотрудников (или понижение в уровне), а также сколько новых сотрудников получают повышение в первые два года.

Как обсуждает Энди Гроув в книге *High Output Management*, даже первоклассный процесс собеседований успешен лишь примерно в 70 процентах случаев. По сути, в найме много рисков: вы пытаетесь предсказать, как человек будет работать сорок часов в неделю, неделя за неделей, на основе всего нескольких разговоров и данных, собранных в процессе собеседований.

Лучшие руководители отслеживают свою успешность, не боятся признавать ошибки в найме и придерживаются принципа «нанимай медленно, увольняй быстро».

От этого никуда не деться: увольнение нового сотрудника, который не справляется, вскоре после его найма — социально неловко и некомфортно для всех. Однако это ответственный поступок по отношению к вашей команде. Среди практик, которые помогают обеспечить прозрачность для новых сотрудников и помочь руководителям принимать хорошие решения, — введение формального девятидневного испытательного или вводного периода и обязательные сверки между новым сотрудником и руководителем каждые пятнадцать или тридцать дней, либо использование схемы найма «контракт с переходом в штат» (contract-to-hire).

Опыт кандидата

Опыт кандидата — это то, что кандидаты чувствуют по отношению к вашей компании во время и после прохождения процесса найма. Многие кандидаты проводят due diligence вашей компании перед подачей заявки или собеседованием. Скорее всего, они заглянут на онлайн-форумы и в социальные сети и посмотрят, что говорят о вас другие кандидаты или сотрудники, прошедшие через ваш процесс.

Вы не всегда можете контролировать то, что говорят о вас, но тем не менее вы хотите обеспечить такой опыт кандидата, который повысит вероятность того, что они прочтут хорошие отзывы онлайн, сами получат отличный опыт и, как следствие, будут более склонны продолжать ваши собеседования и принимать ваши предложения.

Равные возможности

Есть поговорка, что люди склонны нанимать людей, похожих на себя. Часто это результат примитивных методов оценки на собеседовании, которые попросту опираются на «чутьё» интервьюера, а чутьё нередко сильно подвержено неосознанной предвзятости. Эта предвзятость может ставить в невыгодное положение кандидатов другой расы, пола, этнической принадлежности и т. д.

Проектируя процесс собеседований, вам следует сосредоточиться на оценках, основанных на критериях (rubric), которые соответствуют требованиям из описания вакансии, а не просто на чутье интервьюера. Подробнее об избегании предвзятости см. раздел «Избегайте предвзятости» в главе «Лучшие практики собеседований», стр. 63.

Масштабируемость

Всё хорошо, если лично вы способны эффективно нанимать. Но в какой-то момент открытых позиций станет больше, чем вы можете закрыть сами, и вам потребуется масштабировать процесс и привлечь других людей. Чтобы сделать это эффективно, вы должны выстроить воспроизводимую систему, которой смогут пользоваться другие, чтобы выявлять лучших специалистов и нанимать с той же эффективностью и успешностью, что и вы.

Это значит, что кто-то другой должен суметь провести те же собеседования и сделать в конце этого процесса те же выводы, которые, скорее всего, сделали бы вы, провели вы собеседования сами.

Цель оставшейся части этого раздела — помочь вам создать масштабируемую систему собеседований и найма, которая соответствует целям вашей организации и может работать без сбоев без вашего непосредственного участия (после того как вы потратите время на её калибровку). Определив и внедрив такую структуру, создав

продуманную документацию и шаблоны, вы дадите другим возможность проводить собеседования и выставлять кандидатам ранжирующие оценки, которые будут близко отражать ваши собственные.

Описание вакансии

Многие компании недооценивают ценность отличного описания вакансии. По-настоящему хорошее описание вакансии делает для вас две главные вещи: оно помогает создать внутри компании ясность и согласованность относительно того, что эта роль делает и какую ценность она может принести, и оно рекламирует вашу компанию и привлекает тех соискателей, которые могут хорошо подойти.

Изучение рынка

Прежде чем приступить к написанию описания вакансии, я призываю вас провести изучение рынка. Посмотрите на конкурирующие или похожие компании и описания вакансий, которые у них есть для аналогичных ролей.

Часто такие объявления могут дать хорошее вдохновение и калибровку, особенно когда вы нанимаете на менее распространённые роли, которые, возможно, не так хорошо охвачены внедрённой вами масштабируемой системой.

Создание ясности относительно роли

Традиционное описание вакансии содержит краткое описание того, чем будет заниматься роль, за которым следует маркированный список требований к кандидату. Я призываю вас написать больше. Вместо того чтобы фокусироваться на том, что успешный кандидат будет делать в конкретной роли, продумайте, какой цели служит роль. Какие результаты (outcomes) она обеспечивает? Какого эффекта вы ожидаете от этой роли через три, шесть или двенадцать месяцев? Возможно, вы захотите, а возможно, и нет публиковать ответы на эти вопросы как часть описания вакансии, но само упражнение по детальной проработке ожиданий всё равно окажется ценным.

Обсудите ответы на эти вопросы с другими руководителями вашей организации и убедитесь, что они с ними согласны. Не удивляйтесь, если получите значительную обратную связь по первой версии обязанностей и результатов роли. В большинстве стартапов, прежде чем единица штата официально открывается, происходит высокоуровневый, неструктурированный разговор вокруг конкретной должности. «О, нам нужно нанять старшего backend-инженера на JavaScript». Сам акт написания и обсуждения описания вакансии позволяет вашей команде точно определить, что компании действительно нужно, так что вполне естественно, что вам потребуется сделать несколько правок.

Описания вакансий как реклама

Всё, что ваша компания публикует публично, является отражением вашей культуры и бренда. Описание вакансии — не исключение. Описание вакансии нацелено на самого ценного потребителя этой культуры и бренда: на ваших сотрудников, нынешних и будущих. В идеале нужный кандидат — тот, кто не только соответствует требованиям вакансии, но и отлично вписывается в культуру, — читает ваше описание вакансии и приходит в восторг и от роли, и от самой компании.

Несколько способов помочь описанию вакансии отразить вашу культуру:

- Включите ключевые ценности, миссию или видение вашей компании — что бы у вас ни было — на видное место.
- Включите тот эффект, который роль окажет на вашу команду, компанию и клиентов.
- Расскажите о структуре, рабочей среде и размере вашей команды.
- Подчеркните вашу систему вознаграждения и льготы (в некоторых рынках указание диапазона зарплаты обязательно по закону).

Помимо элементов, призванных вызвать интерес кандидата, неплохо включить некоторые часто упускаемые детали, чтобы помочь кандидатам произвести самоотбор:

- Укажите уровни (leveling) и зарплатные «вилки».

- Укажите локацию, требования к присутствию в офисе, а также допускается ли удалённая работа и в какой степени.
- Укажите требования к часовому поясу / рабочим часам.

Поиск кандидатов

Когда дело доходит до закрытия ролей в вашей организации, вы будете находить квалифицированных кандидатов тремя способами: входящий рекрутинг, исходящий рекрутинг и реферальные рекомендации. Эффективный, масштабируемый процесс найма должен быть спроектирован так, чтобы задействовать все три метода.

Входящий рекрутинг

Входящий рекрутинг — это маркетинг вашей открытой вакансии и сбор добровольных заявок от кандидатов. Как и в любом другом маркетинговом мероприятии, одноканального подхода может оказаться недостаточно для получения результатов.

Поэтому публикация описания вакансии на доске вакансий — это самый минимум. В зависимости от состояния рынка, от того, на сколько ролей вы нанимаете, и от качества/ясности вашего описания вакансии одной только публикации может быть достаточно. Часто вам придётся делать больше, чтобы привлечь лучших специалистов, включая активное продвижение ваших ролей в специализированных технических сообществах и/или маркетинг вашего бренда через участие в конференциях / спонсорство, корпоративный блог, активность в социальных сетях и т. д.

Не существует универсально лучшей площадки для размещения объявлений, на которую обращаются отличные сотрудники. Держите руку на пульсе того, какая платформа / сайт вакансий кажется наиболее распространённой, и публикуйте описание вакансии соответственно. В этом вам поможет хорошая система отслеживания кандидатов (ATS): она может отслеживать источник перехода (referral source) для каждого кандидата и предоставлять метрики о том, какие доски вакансий приносят более качественных / большее число кандидатов, которые проходят глубже по вашему процессу, чем другие. При найме дизайнеров, в частности, важно поговорить с практикующими дизайнерами о том, какие сайты для размещения портфолио наиболее популярны, и поддерживать присутствие на этих досках вакансий, чтобы находить лучших кандидатов.

Вам также стоит отслеживать, сколько заявок вы получаете на каждую роль. Как минимум, ваш(и) нанимающий(е) руководитель(и) должны еженедельно смотреть на состояние воронки по своим ролям и соответственно корректировать подход. Если на роль не приходит достаточно соискателей (или приходят не те соискатели), то что-то меняйте! Попробуйте подкорректировать название должности или разместить описание вакансии на новых каналах. Ключевой элемент сильного процесса найма тот же, что и у любого другого процесса, который вы выстраиваете для своей команды: смиренная готовность пересматривать прошлые решения и улучшаться со временем.

Исходящий рекрутинг

Исходящий рекрутинг предполагает проактивное обращение к целевым кандидатам и побуждение их откликнуться на вашу роль. Это можете делать вы, ваша команда, внутренний рекрутер и/или внешний рекрутер. Я призываю команды начинать процесс найма сначала с входящего рекрутинга и собственного исходящего рекрутинга. Занимаясь рекрутингом лично, разговаривая с кандидатами и слушая их реакцию на ваш питч, вы многое узнаете о рынке и о том, что лучшие кандидаты думают о том, что вы предлагаете. Вы также поймёте, насколько конкурентоспособно ваше предложение и насколько легко или трудно находить кандидатов, соответствующих описанию вакансии — возможно, это даже подтолкнёт вас его подкорректировать. Как только вы отточите роль и будете точно знать, кого и что ищете, вы будете готовы дать оптимизированные указания внешнему рекрутеру, что поможет ему эффективнее находить кандидатов от вашего имени.

Не все внешние рекрутеры одинаковы. Вам нужен тот, кто соответствует всем этим критериям:

- Высокоорганизован
- Способен эффективно «продавать» вашу роль (ваша задача — обучить его и спросить с него, чтобы он делал это хорошо)

- Мотивирован неустанно поддерживать контакт без навязчивости и назойливости
- Склонен ценить отношения и с вами, и с кандидатом выше, чем комиссию за одно конкретное трудоустройство

Реферальные рекомендации

Наивысшую отдачу от вложений в найм дают внутренние реферальные рекомендации, то есть рекомендации от ваших существующих сотрудников. Люди гораздо охотнее хотят иметь дело с компанией, о которой хорошо отзывается её нынешний сотрудник, и зачастую легче найти соответствие по культуре, когда кандидат уже проверен кем-то, кто знаком с вашей культурой. Вы можете стимулировать внутренние рекомендации, предлагая денежные вознаграждения (см. врезку) или поддерживая хорошую коммуникацию с рекомендующими о статусе их рекомендаций.

Учитывая, что у рекомендаций столь высокий шанс на успех, вы захотите обеспечить наилучший возможный опыт кандидата. Возможно, вам также стоит рассмотреть сокращённый (но справедливый) процесс найма. Может быть уместно пропустить или сжать какие-либо грубые фильтры в начале воронки, такие как телефонные скрининги или квалификационные анкеты. Также имеет смысл попросить рекомендующего письменно изложить абзац или два, обосновывающие его рекомендацию.

Заметка о математике стимулирования рекомендаций

На основе данных, которыми вы, вероятно, уже располагаете, относительно легко приблизительно оценить стоимость (как во времени, так и в реальных потраченных деньгах) найма нового инженера для вашей компании. Если учесть, что у рекомендаций часто существенно выше коэффициент конверсии в найм, становится очевидно, что рекомендации экономят тысячи и десятки тысяч долларов, что может помочь вам обосновать бонус в несколько тысяч долларов любому сотруднику, который порекомендует кандидата, в итоге нанятого и проработавшего в своей роли более нескольких месяцев.

Лучшие практики собеседований

Процесс собеседований — это момент истины для вашей способности определить, насколько хорошо кандидат подходит на роль, на которую вы нанимаете. Имейте в виду, что идеального собеседования не существует. Объём данных, который интервьюер собирает за скудные несколько часов с кандидатом, разумеется, не может идеально предсказать, насколько хорошо человек будет работать полный рабочий день месяцами и годами вперёд.

В этом разделе я рассматриваю некоторые высокоуровневые лучшие практики собеседований, а затем привожу некоторую предысторию и контекст по различным этапам собеседований, включая анкеты приёма кандидата/резюме, телефонные скрининги, собеседования по культуре, технические собеседования, задания по написанию кода или домашние задания, собеседования с руководством и, наконец, проверки рекомендаций.

Мнения отклонённых кандидатов имеют значение

Проектируя процесс собеседований, опыт кандидата должен быть у вас в фокусе и в высшем приоритете. Даже если вы решите не нанимать кандидата, этот человек уйдёт с впечатлением — хорошим или плохим — о вас и вашей компании. Это впечатление может привести к тому, что они будут петь вам дифирамбы тем в своей профессиональной сети, кто однажды может откликнуться на ваши роли. Или это впечатление может привести к тому, что они будут гневно отзываться о вас при каждом удобном случае.

Доски вакансий и отзывы в Google усыпаны свидетельствами вышедших из-под контроля собеседований, и крайне трудно отменить нанесённый репутации ущерб после того, как он уже причинён. Хотя верно, что для некоторых кандидатов никакое уважение и внимание с вашей стороны не помешают горечи отказа отравить их итоговое мнение о вас, такие люди в меньшинстве. Для большинства кандидатов, добравшихся до этапа собеседований, уважительный и продуманный процесс собеседований оставит у них нейтральное или положительное чувство о вашей компании и поможет вам избежать негативной огласки онлайн.

Будьте оперативны и упростите назначение встреч

В идеале вы / ваша команда заранее сообщаете кандидатам этапы и объём вашего процесса найма и используете простое, надёжное решение для назначения этих этапов в реальном времени. Например, вы можете выбрать один из вариантов: (А) назначить нанимающего руководителя, который занимается всем планированием в рабочие часы, (В) назначить все собеседования заранее или (С) предоставить онлайн-инструмент, которым кандидаты могут пользоваться, чтобы асинхронно назначать свои собеседования в удобное им время.

Поистине, что угодно лучше, чем требовать, чтобы каждый интервьюер писал каждому кандидату перед каждым собеседованием для последовательного согласования расписания, что может растягивать процесс собеседований на недели или месяцы.

Задавайте только новые вопросы

Каждая точка контакта в собеседовании должна ощущаться кандидатом как продолжение разговора, а не как пересказ деталей, обсуждавшихся на предыдущих сессиях. Чтобы избежать последнего, требуется продуманная структуризация и тщательное планирование заранее, до собеседований.

В идеале последующие собеседования следует использовать для более глубокого погружения и изучения областей, специфичных для кандидата или роли, где обе стороны стремятся полностью понять ключевые сильные и слабые стороны. Делиться с последующими интервьюерами предлагаемыми областями для фокуса или новыми вопросами через систему отслеживания кандидатов (ATS) — отличный способ обеспечить непрерывность, эффективность и прекрасный опыт кандидата, который может показать, действительно ли ваши потенциальные сотрудники подходят вашей команде.

Избегайте неосознанной предвзятости

Если вам незнакомо словосочетание «неосознанная предвзятость», я призываю вас прочитать *«Думай медленно... решай быстро»* (Thinking Fast and Slow) Дэниела Канемана. Это моя настольная книга для понимания многих типов систематических ошибок, которые совершает наш мозг.

На самом деле очень легко непреднамеренно дать кандидату преимущество или поставить его в невыгодное положение неоправданными способами. Это неизбежно приведёт к худшим результатам найма или потенциально к дорогостоящим судебным разбирательствам.

Предвзятость принимает множество форм. Большинство предубеждений неосознанны и могут касаться пола, расы, статуса выпускника определённого вуза или социально-экономического происхождения. Но предвзятость может также означать, что выводы, сделанные интервьюером о кандидате до собеседования, основаны исключительно на оценках предыдущего интервьюера. Не существует системы, которая гарантирует устранение всех вредных предубеждений, но есть определённые шаги, которые вы можете предпринять, чтобы минимизировать неосознанную предвзятость, например скрывать имена или фотографии кандидатов (которые часто намекают на пол и этническую принадлежность) во время отбора резюме.

Чтобы избежать «эффекта якоря» (anchoring) или предвзятости у последующих интервьюеров, я призываю интервьюеров оставлять два разных вида обратной связи по беседам с кандидатами:

1. Подробные заметки и оценки
2. Предлагаемые вопросы для последующих бесед.

Большая часть обратной связи по собеседованию должна состоять из подробных заметок и оценок по специфичному для роли оценочному гайду, в котором ваши вопросы для собеседования спланированы заранее (подробнее см. раздел «Технические собеседования», стр. 76). В идеале эту обратную связь не должны читать последующие члены команды до их собеседования, чтобы избежать предвзятости. Например, если вы знаете, что предыдущий интервьюер оценил кандидата плохо, вы можете подвергнуться предвзятости подтверждения (confirmation bias) и переоценить любые области, в которых кандидат плохо проявляет себя на вашем собеседовании.

Второй тип обратной связи — предложения для последующих собеседований — должен фокусироваться на областях, которым стоит уделить особое внимание или которые стоит изучить более подробно на последующих собеседованиях, и не должен раскрывать данные, которые могли бы явно создать предвзятость для дальнейших собеседований.

Использование системы отслеживания кандидатов (ATS)

Когда вы собеседуете более двух-трёх кандидатов одновременно, может потребоваться существенное усилие, чтобы управлять логистикой того, на каком этапе процесса находятся кандидаты, координировать заметки интервьюеров и поддерживать последовательную и оперативную коммуникацию с кандидатами по мере их продвижения по воронке. Без тонко отлаженной системы управления всей этой логистикой легко допустить, чтобы опыт кандидата пострадал, а затраты на найм выросли. Это всеобщая проблема, и было разработано несколько высококачественных готовых решений системы отслеживания кандидатов (ATS) в различных ценовых категориях и уровнях сложности, чтобы решить эту задачу.

Совет здесь прост: выберите и внедрите ATS заранее. Не ждите, пока ваш процесс уже не уйдёт под воду, чтобы принять меры. Обучите свою команду, потребуйте повсеместного использования системы и задайте ожидания по её применению для HR, нанимающих руководителей и интервьюеров.

«Продажа» кандидатам

Как упоминалось ранее, я настоятельно призываю нанимающих руководителей думать о процессе собеседований как о процессе продаж. Это естественно приводит к нескольким хорошим привычкам, которые без проблем переносятся из продаж в собеседования:

- Во время процесса продаж клиенту вы всегда сосредоточены на «продаже» продукта потенциальному покупателю, даже когда квалифицируете клиента. Хороший процесс продаж рассматривает квалификацию кандидатов как воронку, с лёгкой квалификацией наверху и постепенно более тонкой / трудозатратной по времени квалификацией вниз по воронке, наряду с постепенно более кастомизированным и индивидуализированным питчем.
- Вы должны всегда «продавать» кандидатам преимущества и положительные выгоды от присоединения к вашей компании, а также ту роль/возможность, которую вы предлагаете. К тому моменту, как они пройдут через все ваши «обручи» собеседований, они должны страстно хотеть работать в вашей компании и быть в восторге от того, чтобы принять ваше предложение о работе вместо других, которые они получили (или ещё могут получить).
- Убедитесь, что вы рано задаёте хотя бы пару открытых вопросов о том, что кандидат ищет в своей следующей роли. Это поможет вашим интервьюерам составить представление о том, насколько хорошо ожидания кандидата соответствуют роли, на которую вы нанимаете. Эту информацию следует зафиксировать в профиле кандидата и использовать, чтобы по ходу дела адаптировать и кастомизировать питч под кандидата.
 - Например, младший кандидат, пришедший из преимущественно младшей и среднего уровня команды, может искать возможность работать с более старшими JavaScript-инженерами в среде, которая способствует его росту. Если ваша команда предлагает поддержку старших специалистов, а ваша культура делает упор на наставничество, обязательно подчеркните это преимущество, особенно на этапе оффера.
- Всегда оставляйте кандидатам немного времени (пять или десять минут) на вопросы в конце собеседования. Большинство квалифицированных кандидатов приходят на собеседования вооружёнными вопросами, и вы можете многое узнать о том, что для человека важно, по тому, что он решает спросить. Это хорошая возможность для вашего интервьюера «продать» преимущества вашей компании в своих ответах.
- По ходу дела следите за тем, чтобы кандидаты чувствовали себя уважаемыми и постепенно знакомились со всё большей частью вашей компании. Ваши лучшие кандидаты должны чувствовать, что их вдумчиво проверили и что они узнали достаточно о компании, чтобы загореться. В идеале вы хотите, чтобы даже отклонённые кандидаты могли оставлять положительные отзывы в Google и на Glassdoor. Этого можно добиться, «продавая» преимущества вашей компании на протяжении всего собеседования, уважая время людей как своё собственное, поддерживая последовательную и оперативную коммуникацию и обеспечивая, чтобы каждый чувствовал, что процесс был максимально справедливым и прозрачным.

Анкеты для приёма заявок

Начало воронки собеседований — это форма, которая решает две задачи: она предоставляет кандидату некоторую информацию о вашей компании и процессе найма (а значит, и пример вашей культуры), а также собирает от кандидата набор данных, выступая в роли недорогого, грубого фильтра.

Преамбула анкеты

В начале анкеты вам следует изложить несколько ключевых сведений для кандидатов:

- Повторите, на какую роль они претендуют, а также её ключевые требования и влияние.
- Повторите основные ценности вашей компании и дайте пример вашей культуры.
- Задайте ожидания относительно процесса найма: сколько он займёт времени, сколько в нём этапов и как в целом выглядит процесс.

Опросник анкеты

Опросник должен включать запрос резюме кандидата (или URL профиля в LinkedIn), несколько вопросов, требуемых юридическим отделом и HR в отношении права на трудоустройство, а затем, в идеале, несколько квалификационных вопросов к кандидату. Квалификационные вопросы должны быть необременительными, как правило в свободной форме, и, возможно, даже техническими, чтобы убедиться, что кандидат в целом подходит под роль. Например, для роли, требующей опыта работы с JavaScript, вполне разумно подтвердить этот опыт в опроснике вопросом вроде: «Оцените свой уровень комфорта при работе с JavaScript по шкале от "некомфортно" до "крайне комфортно"».

Это может показаться излишним по сравнению с требованиями, перечисленными в описании вакансии, и это действительно так, хотя вы удивитесь, как много резюме приходит без базовой квалификации. На эти вопросы кандидату быстро/легко ответить, и точно так же быстро нанимающему менеджеру использовать их, чтобы отсеять часть откликов.

Если вы завалены кандидатами и хотите немного больше фильтровать на этом этапе, опросник также может включать один-два более интересных или сложных вопроса. Если вы их добавляете, обязательно делайте их короткими: вы не хотите терять кандидатов на этой форме из-за того, что вопросы оказались слишком утомительными. Если вы перегружены откликами, склоняйтесь к тому, чтобы собрать здесь больше данных для фильтрации, в противном случае, возможно, лучше отложить более тонкую квалификацию на следующие этапы воронки.

Несколько примеров вопросов для анкеты, охватывающих общую совместимость и заявленное самим кандидатом техническое знакомство (образец я привёл на cto.hb.com/templates):

- Как хороший кандидат, вы получите массу предложений. При равной компенсации и льготах, что заставит вас выбрать одну компанию вместо другой?
- Что является решающими плюсами и решающими минусами при вашем следующем переходе?
- Что даёт вам энергию в работе? Что отнимает у вас энергию?
- Каковы ваши географические ожидания (локация, удалённо, в офисе)?
- Насколько вы знакомы с базовыми техническими квалификаторами: оцените знакомство с [соответствующий язык программирования или инструмент] по шкале от 1 до 10?

Телефонный скрининг

Первичный телефонный скрининг, как и всё в процессе собеседований, выполняет двойную функцию: это возможность узнать больше о кандидате, и это первое взаимодействие кандидата с человеком из вашей компании (и его оценка этого человека).

Учитывая, что это первый человек, с которым кандидат будет взаимодействовать в вашей компании, стоит тщательно подумать, кто проводит это собеседование. Вопросы на этом этапе не должны носить сильно технический характер, поэтому необязательно, чтобы собеседование проводил член технической команды. Часто это делает HR или выделенная команда рекрутеров.

Независимо от того, кто проводит телефонный скрининг, убедитесь, что этот человек является хорошим представителем культуры вашей команды/компании и вооружён информацией, которую технические кандидаты, вероятно, запросят на этом этапе, включая:

1. Как выглядит программный стек: включая ключевые языки, инструменты и целевых клиентов (например, мобильные устройства, десктоп и т.д.). Интервьюер должен иметь хотя бы зачаточное понимание слов, которые он здесь использует, а не просто зачитывать список.
2. Размер технической команды — как в целом по компании, так и той, с которой будет работать кандидат. Сюда же стоит включить общие прогнозы по найму и примерно сколько людей добавляется со временем.
3. Кому будет подчиняться кандидат. Дайте некоторую базовую информацию об этом руководителе, включая его стаж в компании и, возможно, чем он занимался до прихода в компанию.
4. Хорошее представление об основных ценностях/культуре компании и о том, как в ней принято работать.

Цель интервьюера — познакомить кандидата с компанией, её культурой, ролью и процессом найма. Также он задаст несколько высокоуровневых вопросов кандидату, чтобы подтвердить его структурное соответствие роли. Вы хотите, чтобы кандидат ушёл с этого собеседования мотивированным хорошо проявить себя на остальных этапах процесса и воодушевлённым идеей работать в вашей компании.

Точные вопросы, задаваемые на телефонном скрининге, поэтому не так уж важны. Вот набросок некоторых областей, которые вы, возможно, захотите охватить:

- Есть ли у них что-то, ограничивающее их сроки найма (например, другие предложения о работе)?
- Где находится кандидат и готов ли он переехать при необходимости?
- Примерно когда они могут приступить или хотят приступить к работе?
- Подтвердите, что ожидания по компенсации совпадают, и объясните льготы/бонусы.

Помимо хороших ответов на вопросы, интервьюер должен оценить общее соответствие кандидата роли. Ясно ли кандидат излагает мысли, кажется ли он подходящим по культуре, совпадает ли заявленный им опыт с тем, что указано в резюме, и заинтересован ли он в компании и возможности?

Культурное собеседование

Один из главных критериев, который вы ищете в процессе собеседований, — это культурное соответствие (culture fit). Культурное соответствие — это все элементы личности кандидата, помимо его опыта и навыков, которые позволят ему быть успешным в вашей организации. Чтобы эффективно отбирать кандидатов на культурное соответствие, ваша компания должна иметь достаточно чёткое представление о том, в чём состоит её культура. Это может выглядеть по-разному, например: список основных ценностей, формулировка миссии, формулировка видения, руководящие принципы. Чем бы они ни были, они должны быть искренними и подлинными для компании. Если у вас с этим трудности, я порекомендую вам *Team of Teams* Стэнли Маккриснала, *Work Rules!* Ласло Бока и *Good Authority* Джонатана Рэймонда.

В настоящее время существует мало формально структурированных программ собеседований, которые широко используются. Та, что встречается довольно регулярно, называется *topgrading* и относится как минимум к двум разным вещам: методу *topgrading* и собеседованию по *topgrading*. Метод *topgrading* (cto.hb.com/topgrading) — это методология найма, которая, как утверждается, была разработана General Electric в 1980-х/90-х годах и описана в книге Верна Харниша *Scaling Up*. Собеседование по *topgrading* (cto.hb.com/interview), которое я называю культурным собеседованием, — это конкретная повестка, стиль и структура собеседования, призванные узнать о бэкграунде кандидата и его культурном соответствии.

Как формально задумано, собеседование по *topgrading* проводит кандидата по его трудовой истории и задаёт один и тот же набор вопросов о каждой из нескольких предыдущих ролей кандидата. В зависимости от истории

кандидата и того, как долго он провёл на последних местах работы, вам следует охватить от двух до пяти прошлых позиций. Вы хотите захватить достаточно длительный период времени, чтобы попытаться выявить тенденции и увидеть рост, но при этом не держать кандидата на собеседовании три часа, обсуждая стажировки, которые у него были в колледже двадцать лет назад.

Для каждой роли topgrading предписывает интервьюеру задавать следующие вопросы:

- Какие были заметные успехи или достижения на этой позиции?
- Какие были ошибки или неудачи на этой позиции?
- Как звали вашего руководителя и какова была его должность?
- Как, по-вашему, ваш руководитель честно оценил бы ваши сильные и слабые стороны?
- Каковы, на ваш взгляд, были сильные и слабые стороны вашего руководителя?

В дополнение к этой формуле собеседования topgrading предлагает подход с двумя интервьюерами: ведущий, который активно вовлечён в разговор и проявляет любопытство к кандидату, и выделенный человек, ведущий заметки.

Используете ли вы одного или двух интервьюеров, ведение заметок критически важно. Чтобы оценивать кандидатов справедливо, вам стоит создать оценочную карту (scorecard), которая оценивает ответы кандидата, отслеживая их соответствие культуре вашей компании. Например, если уважительный вызов (respectful challenge) — это основная ценность компании, спросите кандидата, может ли он привести примеры того, как он уважительно бросал вызов. Или же он отзывался неуважительно о ком-то из прошлых коллег? Использование заметок после собеседования для заполнения и обоснования баллов в оценочной карте крайне важно.

Кодинг-челлендж

Требование выполнить тестовое задание на дом, также называемое кодинг-челленджем или домашним заданием на собеседовании, — спорная тема. Тестовые задания на дом часто представляют для кандидатов значительные затраты времени и поэтому являются существенным источником отсева кандидатов в воронке найма. Нетрудно представить себе востребованных кандидатов, которых просят выполнить несколько тестовых заданий, каждое из которых требует многих часов или дней работы, что в сумме складывается в недели работы. Сталкиваясь с такими запросами, кандидаты вполне понятным образом будут отдавать приоритет заданиям тех компаний, которыми они больше всего воодушевлены и/или у которых самые посильные задания.

Несмотря на эти структурные сложности, с точки зрения нанимающего менеджера, иметь такое задание критически важно. Как можно нанять инженера-программиста, не дав ему написать для вас код?

Подводя итог, есть три конкурирующих фактора:

Установить прогностическую способность: работодатели хотят, чтобы кандидаты действительно производили код в процессе собеседования инженера-программиста, чтобы попытаться спрогнозировать их производительность на работе.

Минимизировать отсев: работодатели хотят, чтобы кандидаты действительно выполняли задания по программированию и не выпадали из воронки.

Улучшить опыт кандидата: кандидаты хотят чувствовать, что их время уважают и что назначенные задачи разумны. В идеале кандидат должен узнать больше о вашей компании через это задание и стать ещё более воодушевлённым возможностью у вас работать.

Прогностическая способность

Существует несколько стилей собеседования или задания по программированию. Задания варьируются от проектов на дом с заданием-описанием до использования онлайн-платформы для упражнений по программированию (также иногда известных как code katas) и до живого парного программирования. В отсутствие каких-либо эмпирических данных о прогностической способности этих стилей я призываю вас спроектировать упражнение, которое максимально похоже на обычную повседневную работу в вашей компании. Если в вашей

компании вы не занимаетесь парным программированием, то оценка того, как кандидат проявляет себя в условиях собеседования при парном программировании, интуитивно не ощущается сильно коррелирующей/прогностической. По меньшей мере это сбор косвенных сигналов.

Как руководитель, ваша цель — получить лучшее от людей, с которыми вы работаете. Имея это в виду, постарайтесь вспомнить, когда вы в последний раз проходили собеседование, и задействуйте свою мышцу эмпатии при разработке задания по программированию. Проходить собеседование для большинства — очень стрессовый процесс, и просьба проявить креативность или решить задачу в таком сценарии не всегда выявляет лучшую производительность. Вот некоторые способы помочь кандидатам выполнить задание по программированию наилучшим образом:

- По возможности предоставляйте гибкость в выборе языка/инструментов.
- Позволяйте выполнять работу асинхронно (т.е. задание на дом вместо живого кодинга).
 - Если вы сильно ориентируетесь на сигналы из самого процесса написания кода, так что задания на дом не дают вам достаточно информации, рассмотрите возможность попросить кандидата записать себя (через Loom или другие подобные инструменты) за выполнением части упражнения.
- Чётко обозначайте, что вы ищете в результате работы кандидата. Например, если ваша оценочная карта измеряет, насколько хорошо он задокументировал свой код, то убедитесь, что в задании, которое получает кандидат, ему сказано включить документацию. Или, если вы планируете запускать код, дайте кандидату знать, будете ли вы оценивать только корректность, или важны и другие элементы, такие как производительность, обработка негативных случаев и т.д.

Опыт кандидата и отсев

Кандидаты с большей вероятностью выполнят ваше задание на дом по программированию, если найдут его интересным и легко начинаемым. Лучшие задания тематически связаны с вашим бизнесом и в идеале знакомят кандидата с теми проблемами, с которыми ваша компания действительно сталкивается ежедневно.

Плохой пример: вы — веб-SaaS-платформа, и вы даёте кандидату челлендж, связанный с разработкой под мобильные телефоны.

Хороший пример: ваша компания интегрируется со множеством устаревших сторонних API, и ваш челлендж — построить ограниченную интеграцию с Sandbox API, у которого схожие с реальным бизнесом доменные существительные/глаголы.

Предоставление кандидатам готовых репозиторий кода с работающими системами сборки/тестами для старта может сэкономить кандидату время на самостоятельную начальную настройку сборки.

Чтобы уважать время кандидата, я предлагаю задавать жёсткий лимит времени для задания на дом. Цель лимита времени не в том, чтобы создать временное давление и форсировать быструю сдачу, а в том, чтобы кандидаты не вкладывались в челлендж чрезмерно и чувствовали, что челлендж — разумный запрос. Чтобы кандидаты понимали лимит времени, вам следует:

- Дать достаточное объяснение лимита времени
- Убедиться, что задача легко достижима в указанный лимит времени
- Дать кандидатам знать, как будет оцениваться их работа. Если ваша оценочная карта оценивает файл README кандидата, то дайте кандидату знать, что ему стоит потратить время на написание README. Если код будет запускаться либо вживую с кандидатом, либо интервьюером асинхронно, дайте им знать, что время выполнения будет оцениваться. Если вас в основном волнуют архитектурные решения и вы меньше озабочены производительностью во время выполнения, дайте им знать и это тоже, чтобы они могли потратить время правильно.

Технические собеседования

Насколько спорны и разнообразны методологии собеседований с заданиями на дом по программированию, настолько же сами технические собеседования ещё более разнообразны. В целом я призываю вас следовать тем

же фундаментальным принципам: убедитесь, что вы собираете сигналы, релевантные реальной работе, и будьте уважительны и внимательны к самим кандидатам.

Классическое техническое собеседование, практикуемое многими из крупнейших технологических компаний, включает некую форму совместной работы у доски, где кандидата просят решить техническую задачу в реальном времени. Задачи варьируются от академических — отсортировать массив с некоторыми особыми условиями — до высокоуровневых/расплывчатых архитектурных — спроектировать систему для обработки 100 миллионов пользователей, публикующих обновления в новостной ленте. Классический подход к собеседованию, должно быть, работает для крупных компаний, поскольку они продолжают использовать его год за годом, но я не вижу, как он работает в стартапе. Эти задачи зачастую либо чрезмерно широки, либо чрезмерно узки, и потому их трудно справедливо оценить. Академические вопросы редко коррелируют с типами задач, которые решаешь ежедневно на работе.

Самое разоблачительное — они не настраивают кандидатов на успех в условиях собеседования. В конце концов, я уверен, что в крупных компаниях очень мало инженеров пишут алгоритмы сортировки массивов в рамках своей повседневной работы.

Методология, которую я описываю в этой главе, представляет собой альтернативный подход, который я видел и сам успешно использовал в стартап-среде.

Методология

Далее следует методология, которую я использовал, выведенная из уроков topgrading, для отбора и найма кандидатов на роли старших инженеров-программистов. В духе topgrading я называю её собеседованием с техническим фокусом (technical focus interview).

Руководство по собеседованию с техническим фокусом

Чтобы выяснить, где находятся сильные и слабые стороны кандидата и насколько это важно для роли, на которую вы нанимаете, сначала вам нужно решить, какие тематические области важны для вашей роли. Это делается путём создания руководства по собеседованию с техническим фокусом, которое должно включать список из примерно четырёх-восьми технических областей, а внутри каждой области — набор примерных вопросов, ответов по лучшим практикам и руководство по оценке.

Примерные ответы и руководство по оценке включаются для обеспечения справедливости и единообразия оценивания у разных интервьюеров и у разных кандидатов. Вы пытаетесь различить, где у конкретного кандидата есть пробелы, а где — подлинная экспертиза, поэтому ваши вопросы должны быть спроектированы так, чтобы вызывать один из трёх видов ответов: плохой, хороший и потрясающий. Таким образом, они должны поддаваться соответствующей оценке. Когда дело доходит до оценки вопроса, чтобы сделать разницу между пробелом в знаниях и подлинной экспертизой очевидной, я рекомендую, чтобы плохой ответ получал балл 0–2, хороший ответ получал балл 3–6, и только потрясающий ответ получал от 7 до 10.

Под плохим ответом я подразумеваю ответ на вопрос, демонстрирующий либо отсутствие, либо минимальный опыт или экспертизу по рассматриваемой теме. Хороший ответ демонстрирует компетентность, возможно даже очень высокий уровень компетентности по теме. Потрясающий ответ демонстрирует не только компетентность, но и подлинное понимание и интеллектуальную глубину по теме. Например, если вопрос касается того, как кандидат думает о проектировании набора модульных тестов, и его ответ — что он никогда об этом не задумывался, это 0, и вы нашли пробел. Если его ответ включает описание нескольких наборов тестов, которые он проектировал, и некоторое обоснование, это хорошо, возможно 5 или 6. Если его ответ включает полное изложение философий проектирования наборов тестов, плюсов и минусов каждой и того, как применять их в разных сценариях, то теперь вы смотрите на настоящую экспертизу и балл 7–10.

В духе предоставления кандидатам наилучшего шанса на успех я не рекомендую оценивать каждый вопрос. Вместо этого ставьте балл за тематическую область. Так вы можете попробовать несколько вопросов внутри темы, ища у кандидата области экспертизы и оценивая итоговый результат по этой теме.

Не заблуждайтесь, написание этих вопросов, примерных ответов и руководств по оценке — большой труд. Хорошая новость в том, что любой конкретный вопрос полезен для нескольких ролей и может переиспользоваться на протяжении длительного периода. Более того, я призываю вас вести центральный репозиторий вопросов (и

связанных с ними примерных ответов/руководств по оценке). Когда придёт время писать следующее руководство по собеседованию с техническим фокусом, вы обнаружите, что ваша задача значительно проще, поскольку вы сможете переиспользовать вопросы из репозитория по мере необходимости.

См. cto.hb.com/templates для примера руководства по фокусу из моего собственного репозитория вопросов.

Найм джуниоров против сениоров

Качества, которые вы ищете в джуниоре, скажем с одним-двумя годами опыта программирования, должны сильно отличаться от качеств сениора с десятью и более годами. Идеальный джуниор должен быть любознательным, жаждущим учиться, иметь прочные основы программирования и быть готовым работать над инкрементальной разработкой фич. Сениор, напротив, должен приходить не только с основами программирования, но и с глубоким мышлением об архитектуре, мнениями и лучшими практиками по широкому кругу инструментов и проблем. Они должны уметь вызывать доверие в том, что могут не только создавать инкрементальные фичи, но и владеть архитектурой и принимать хорошие решения по ней для новых проектов с нуля (greenfield). Поскольку ключевая ценность, которую предлагают эти два типа ролей, столь различна, из этого следует, что и собеседования для них должны быть разными.

Для сениора собеседование с фокусом, где вы глубоко исследуете навыки принятия решений кандидатом, понимание концепций и архитектурное ноу-хау, критически важно и должно весить много. Для джуниорской роли это глубокое погружение в знания должно быть короче и весить меньше, чем практическое упражнение по программированию.

Само собеседование

Собеседование с техническим фокусом для старшего инженера-программиста обычно представляет собой разговор продолжительностью от шестидесяти до девяноста минут между кандидатом и ведущим интервьюером, в идеале с преимущественно молчаливым вторым интервьюером, который ведёт заметки. В зависимости от длины вашего руководства по фокусу и того, сколько тем вы хотите охватить, вы можете рассмотреть разбиение тем на несколько собеседований с фокусом.

Я подчёркиваю, что это собеседование должно быть разговорным; вы стремитесь выяснить, в каких областях программной инженерии кандидат наиболее знающ и увлечён, а в каких он либо никогда не нёс ответственности, либо исторически делегировал. Для этого не требуются головоломки, парное программирование или какое-либо решение задач. Просто спрашивайте!

Начните собеседование неформально с лёгкой беседы. Через минуту-две начните описывать повестку/план встречи. Дайте кандидату знать, что у вас есть документ с руководством по собеседованию, и ваша цель — чтобы кандидат обсудил темы из этого руководства в течение следующих шестидесяти-семидесяти пяти минут, оставив пятнадцать минут в конце на то, чтобы он задал вам вопросы.

После преамбулы вы перейдёте к первому разделу руководства по собеседованию. Ваша цель в каждом разделе руководства не в том, чтобы задать все до единого вопросы. Вы стремитесь сначала определить, к какой из трёх категорий относится кандидат в этой тематической области — плохой, хороший или потрясающий — а затем сузить балл из этого. У вас должно быть довольно хорошее представление о том, куда отнести кандидата, после первого вопроса или двух, после чего используйте уточняющие вопросы, чтобы прозондировать глубже и сузить балл.

Если кандидат полностью промахивается или признаёт, что не знаком с темой, нет нужды продолжать переходить к каждому вопросу; у вас есть балл, и вы можете двигаться дальше.

С другой стороны, если кандидат блестяще отвечает на первый вопрос, он вполне может быть настоящим экспертом в этой области, но вы, скорее всего, не будете уверены в его мастерстве, пока он не даст содержательные ответы на несколько вопросов по теме. Как правило, на выявление мастерства уходит больше времени и вопросов, чем на выявление отсутствия квалификации.

Не стесняйтесь вежливо прервать ответ кандидата и перейти к следующей категории, когда вы знаете, что услышали достаточно. Ваша цель — помочь кандидату продемонстрировать свои навыки и знания по всем темам, которые вы решили считать важными для этой роли и оценивать на этом собеседовании. Позволяя кандидату уйти в кроличью нору и потреблять время на одну тему, когда у вас уже есть вся нужная для балла информация, вы

лишаете его возможности продемонстрировать свои способности в других темах, если у вас на собеседовании закончится время. Управлять темпом собеседования — это ваша работа, а не кандидата.

Собеседования с руководителями

К тому времени, когда кандидат доходит до раунда собеседования с руководителями, вы уже должны были подтвердить, что он обладает навыками, требуемыми в описании вакансии, и будет подходящим по культуре для вашей компании. Собеседование с руководителем в большинстве сценариев — это в меньшей степени отбор кандидата руководителем и в большей степени шанс для кандидата познакомиться с руководителем и задать ему вопросы.

Однако если кандидат претендует на очень высокую роль или будет подчиняться непосредственно руководителю, то может быть уместно, чтобы это последнее собеседование было длиннее или более тщательным, чем простая сессия вопросов и ответов кандидата.

Проверка рекомендаций

С проверкой рекомендаций вам нужно найти баланс между тем, чтобы планировать их достаточно рано в процессе собеседований, дабы они не создавали узкое место, и тем, чтобы не тратить время на проверку рекомендаций для кандидатов, которые не получают оффер. Имейте в виду, что кандидаты, и это справедливо, могут не желать предоставлять рекомендации, пока не дойдут до конца процесса, чтобы защитить собственные отношения с теми, кто их рекомендует.

Тайминг

Из этого следует, что проверка рекомендаций почти всегда происходит последней в процессе собеседований. Чтобы избежать необходимости откладывать оффер из-за завершения проверки рекомендаций, вот несколько советов:

- Начинайте планировать встречи параллельно со всеми рекомендателями, как только они предоставлены. Учитывая их краткий характер, самой эффективной стратегией может быть возможность позвонить рекомендателям без планирования.
- Рассмотрите возможность сделать оффер до завершения проверки рекомендаций, но ясно дайте понять кандидатам, что финализация оффера зависит от того, что рекомендации поступят и оправдают ожидания. Проверка рекомендаций, при условии что вы хорошо поработали со своим процессом фильтрации до этого момента, имеет высокий процент успеха, поэтому редко условные офферы придётся отзываться из-за провальных рекомендательных интервью.
- Будьте гибки в том, кто проводит рекомендательные интервью, поскольку это необязательно должен быть член технического персонала. Это должен быть кто-то, кто очень отзывчив на email и имеет значительную доступность в своём календаре, чтобы подстроиться под рекомендателей.

Содержание

В целом рекомендательные интервью должны быть краткими и уважающими время рекомендателя и его готовность помочь. Большинство рекомендательных интервью дают обратную связь в диапазоне от нейтральной до восторженно положительной. Очень редко вы получите откровенно негативную обратную связь, поэтому ваша цель — быстро отличить нейтральную от восторженно положительной, подтвердить любые сильные/слабые стороны, выявленные в процессе собеседований, и двигаться дальше. Если вы всё же получаете откровенно негативную обратную связь в рекомендательном интервью, обратите на это очень пристальное внимание и постарайтесь получить конкретные детали критики, чтобы передать их нанимающему менеджеру.

Несколько примеров вопросов для рекомендательного интервью:

- В каком контексте вы работали с [имя кандидата]?
- Оцените, насколько достоверен рекомендатель: руководили ли вы многими другими инженерами за свою карьеру?

- Каковы были самые сильные стороны [имя кандидата]?
- Каковы были самые большие зоны роста [имя кандидата] в то время?
- Как бы вы оценили его производительность на той работе по шкале от 1 до 10?
- Что в его производительности заставляет вас дать такую оценку?
- [Имя кандидата] упомянул, что испытывал трудности с _____ на той работе. Можете рассказать мне об этом подробнее?
- В какой среде и при каком стиле управления [имя кандидата] был бы наиболее успешен?
- Как [имя кандидата] управляет конфликтами?
- Наняли бы вы [имя кандидата] снова, будь у вас такая возможность?

Как сделать оффер

К тому времени, когда вы готовы сделать кому-то оффер, у вас должно быть твёрдое мнение, основанное на описании вакансии и обратной связи с ваших собеседований с фокусом, об уровне, на который вы бы взяли кандидата. Отсюда должно быть относительно несложно определить сумму зарплаты/бонуса/долевого участия, используя ваши заранее определённые уровневые диапазоны (leveling bands). (Подробнее об этом см. 1.4.2 «Компенсация и грейдинг».)

После того как вы откалибровали суммы оффера, вам следует решить, как преподнести оффер. Особенно если ваш оффер включает компенсацию долевым участием, вам стоит серьёзно рассмотреть возможность предоставить таблицу, дающую контекст к суммам оффера. Ценность некоторого числа акций сама по себе невозможна для оценки кандидатом. Им нужны дополнительные точки данных, чтобы оценить то, что вы предлагаете, включая такие цифры, как общее число акций в обращении, цена исполнения (strike price) акции, последняя оценка стоимости компании и т.д.

Я подготовил образец таблицы оффера кандидату на cto.hb.com/templates.

Преподнесение оффера

Момент, когда вы преподносите оффер, — это момент, когда вам нужно быть в режиме суперпродаж. В идеале вы продавали кандидатам на всём пути, так что они уже очень воодушевлены компанией и возможностью для себя. В любом случае это большое событие для кандидата, поэтому обязательно отнеситесь к случаю с уважением, которого он заслуживает. На протяжении всего процесса объяснения оффера не забывайте быть особенно жизнерадостным, поздравить кандидата и подчеркнуть, как весело вам будет и какие замечательные вещи вы построите вместе. Также критически важно, чтобы вы были прозрачны и изложили все ключевые пункты оффера заранее, особенно всё, чего они могут не ожидать или к чему не привыкли, например компенсацию долевым участием или испытательный/пробный период.

Я рекомендую делать оффер в трёх частях: телефонный звонок, email и ужин. Что касается телефонного звонка, я предлагаю позвонить кандидату без предварительного планирования. К этому моменту вы уже провели с кандидатом массу планирования, так что нет нужды нагнетать его тревогу ещё больше, назначая очередную встречу. В качестве альтернативы вы можете сообщить им в письменном виде, что намереваетесь сделать оффер, и спланировать встречу исходя из этого, но вы теряете эффект от того, чтобы быть на линии с ними, когда они узнают новость. Я нахожу, что просто проще позвонить человеку и поделиться новостью всю сразу.

На звонке вам следует выразить воодушевление, передать ключевые пункты оффера и ответить на любые первоначальные вопросы. Объясните, что после звонка вы пришлёте им по email письменные материалы, чтобы помочь обеспечить контекст по долевому участию, и, конечно, от компании придёт формальное письмо с оффером. И наконец, если это логистически возможно, запланируйте совместную трапезу с кандидатом для более личного, углублённого разговора.

Онбординг

Онбординг новых инженеров в команду в большинстве случаев не требует от команды строго больших вложений: хороший инженер рано или поздно во всём разберётся сам. Тем не менее, если не делать ничего, ваш новый сотрудник получит неудачный опыт. Это замедлит выход на продуктивность, а также может затруднить оценку того, насколько удачным был ваш найм. Иначе говоря, хороший онбординг оптимизирует три цели:

1. **Он уважает сотрудника:** хороший опыт онбординга помогает новому сотруднику почувствовать себя интегрированным в вашу компанию и культуру и как можно быстрее выйти на продуктивность.
2. **Он помогает оценить качество найма:** хороший онбординг задаёт структуру как для нового сотрудника, так и для его руководителя, включая чёткие цели, достижение которых показывает, что вы удачно наняли человека на эту роль.
3. **Он формирует вашу культуру:** хороший онбординг подчёркивает культуру непрерывного улучшения, помогая оптимизировать процесс для будущих сотрудников и повысить масштабируемость ваших процессов в целом.

Есть много правильных способов сделать это. Далее приведены относительно простые и недорогие приёмы и практики, которые я использовал сам. Не стесняйтесь расширять их или отступать от этих идей.

Правило бойскаута: онбординг

Я призываю вас подчёркивать своим руководителям, новым сотрудникам и в документации по онбордингу, что успешный онбординг — это общая ответственность всех членов вашей команды (команд), включая недавно нанятых. В зависимости от того, как часто вы нанимаете, документация по онбордингу склонна устаревать. Если новый сотрудник сталкивается с чем-то, что неясно, неверно или вовсе отсутствует в его материалах, дайте ему понять, что вы ожидаете от него усилий по уточнению и улучшению документации для следующего человека.

Онбординг в команду против онбординга в компанию

Есть элементы онбординга любого инженера, нового в вашей компании, которые должны быть единообразными для всех нанятых. Сюда входят высокоуровневый процесс, акцент на организационной культуре, типы документации, которую получают новые сотрудники, а также структура передачи документации и постановки контрольных точек онбординга. Вы вряд ли захотите, чтобы у ваших фронтенд-команд онбординг проходил по-звёздному гладко, а ваши бэкэнд-команды оставались в неведении. Первое впечатление имеет значение, и онбординг — это ваша возможность убедиться, что все члены команды получают отличный первый опыт знакомства с вашей организацией и единообразно знакомятся с ценностями вашей компании и лучшими практиками вашей команды.

Это не значит, что детали онбординга будут идентичны во всех командах. Вы можете и должны иметь разные материалы для разных команд, когда это имеет смысл, и у каждой команды и каждого нанятого сотрудника должен быть индивидуальный план онбординга и контрольные точки.

Документация по онбордингу

Есть два ключевых элемента онбординга нового инженера: рассказать ему о вашей культуре и лучших практиках, а также дать ему чем заняться, обеспечив структуру и инструкции. Я предпочитаю разбивать это на два письменных артефакта: «Инженерное руководство» (Engineering Guidebook) и «Добро пожаловать в инженерную команду [Название вашей компании], руководство первого дня» (Day 1 Guide).

Инженерное руководство (The Engineering Guidebook)

Инженерное руководство собирает в одном документе все мнения, лучшие практики, структурные элементы и бизнес-процессы вашей инженерной команды. Это должен быть единственный источник, на который любой инженер может опереться, чтобы узнать о выборе и решениях, которые должны быть единообразны во всей инженерной организации. Будьте осознанны и вдумчивы относительно того, какие именно практики должны оставаться единообразными во всей организации.

Чем больше становится ваша команда, тем больше смысла в том, чтобы отдельные части команды вырабатывали собственный специализированный способ выполнения работы. Тем не менее, для большинства небольших/средних стартапов, скажем, менее чем с семьюдесятью пятью — ста разработчиками, огромную ценность и эффективность можно раскрыть, придерживаясь здорового и единообразного набора лучших практик.

Руководство может принимать множество форм, хотя я предпочитаю формат слайд-колоды/презентации. Вот несколько примеров практик, которые ваше руководство должно описывать:

Разработка ПО

- Выбор языков программирования
- Мнения/требования к CI/CD
- Стандарты именования (регистр в коде, регистр в контрактах)
- Стандарты обработки, защиты, резервного копирования и безопасности данных
- Мнения о том, как использовать систему контроля версий (Git Flow, GitHub Flow)
- Мнения о тестировании (виды, инструменты, сколько делать)
- Стандартные паттерны фронтенд- и бэкенд-аутентификации и авторизации
- Стандарты сетевых протоколов (REST, gRPC, GraphQL и т.п.)
- Универсальные требования (Поддерживаем ли мы мобильные устройства, адаптивность, перевод?)
- Сертификационные фреймворки и связанное обучение (например, PCI, SOC2, GDPR)
- Прочая логистика кода: доступ к приватным репозиториям, линтинг, статический анализ кода, формат/стиль сообщений коммитов.

Инженерный процесс

- Мнения о ритме/церемониях (Agile, Kanban, ретроспективы)
- Мнения о требованиях к технической документации/спецификациям
- Мнения о том, как использовать систему тикетов (Что такое эпик? Используем ли мы стори-поинты?)
- Любые метрики, которые важны команде в целом (Измеряете ли вы время цикла?)
- Как обрабатываются инциденты в продакшене (PagerDuty? Документы RCA?)
- Как новые технологии включаются в стек
- Процесс работы с багами и техническим долгом.

Управление людьми

- Ожидания относительно того, как проводятся оценки эффективности, как сотрудников оценивают/повышают
- Ожидания относительно вклада в процессы онбординга/найма.

Руководство должно быть чётко обозначено как живой документ, с хорошо определённым процессом предложения, сбора обратной связи и внесения изменений в руководство. Например, я использовал процесс Request for Comments (RFC) для обновлений.

Добро пожаловать в инженерную команду [Название вашей компании], руководство первого дня (Day 1 Guide)

Распространение «Руководства первого дня» — это ваша возможность задать некоторую структуру для новых сотрудников, дав им конкретный список того, что нужно сделать в первый день в вашей организации, что познакомит их с культурой компании, коллегами, вашими процессами и вашим программным стеком. «Руководство первого дня» должно, разумеется, ссылаться на «Инженерное руководство» как на обязательное чтение первого дня. Кроме того, ваше «Руководство первого дня» должно охватывать следующее:

Инструкции о том, как получить логины/доступы к необходимым системам, включая:

- Систему контроля версий
- Управление тикетами
- Любое логирование dev/stage/prod
- Отслеживание ошибок
- Любые инструменты дизайна (Figma, Sketch)
- Документацию/вики (Confluence, Notion и т.п.)
- Внутренние коммуникации (Slack, email)
- Информацию о корпоративном оборудовании (включая то, могут ли новые сотрудники выбрать ноутбук/телефон) и ожидания по использованию этого оборудования
- Инструкции о том, как настроить локальное окружение для разработки

- Знакомство с командой и организационной структурой компании: кто их руководитель, релевантные кросс-функциональные лидеры, прямые подчинённые, а также релевантные вице-президенты или руководители
- Ожидания относительно прозрачности и обращения через организационную структуру за помощью или эскалацией проблем
- Знакомство с технической архитектурой
- Релевантные книги, блоги и другие письменные ресурсы, которые вы рекомендуете читать всем членам команды

Контрольные точки онбординга, или Девяностодневная оценочная карта

Как обсуждалось в главе о найме, найм очень сложен. Даже самые продуманные процессы найма не достигают 100-процентной успешности. Иначе говоря, неудачные наймы неизбежны.

Лучший способ справиться с вероятностью неудачных наймов — сначала проявить смирение и признать, что ваш процесс найма несовершенен, а затем вдумчиво подойти к тому, как измерять успешность новых сотрудников, и быстро принимать меры для исправления любых ошибок. Процесс должен быть прозрачным для новых сотрудников с самого начала, чётко объясняя ожидания. Руководители должны работать с новыми сотрудниками, чтобы убедиться, что роль подходит обеим сторонам, что новый сотрудник начинает чувствовать себя в роли как дома и что он работает на уровне, соответствующем тому, ради чего его наняли. К шестидесятому или девяностому дню должно быть ясно как новому сотруднику, так и руководителю, выполняются ли эти ожидания.

Если есть разногласия в том, успешен ли сотрудник, это хороший признак, что что-то не работает, и вам следует относительно быстро рассмотреть, есть ли в команде другое место, где новый сотрудник подошёл бы лучше, или же обеим сторонам стоит расстаться.

Оценочная карта

Это ответственность руководителя нового сотрудника — определить и задокументировать измеримые контрольные точки для любой новой роли до того, как новый сотрудник приступит к работе. В первый день руководитель должен пройтись по контрольным точкам с новым сотрудником, собрать обратную связь и совместно поработать над этими контрольными точками, чтобы убедиться, что они справедливы и чётко измеримы. Для некоторых ролей эти контрольные точки могут быть простыми и ясными, например, в роли поддержки — измерение эскалационных тикетов через пропускную способность по тикетам. Для других ролей вам может понадобиться больше креативности, например, реализованные фиши или закрытые сториз-поинты. Независимо от выбранных контрольных точек, оценочная карта должна делать следующее:

- Устанавливать чёткие и прозрачные ожидания между руководителем и новым сотрудником.
- Давать новому сотруднику ориентир относительно того, что он будет делать и как его будут оценивать в первые девяносто дней.
- Давать очевидные критерии соответствия или несоответствия ожиданиям его роли.

Оценочная карта не обязана быть длинной или высококонцентрированной. Ключевое в том, что, какую бы форму она ни приняла, после девяноста дней сотрудник и руководитель могут посмотреть на оценочную карту и согласиться, как сотрудник себя проявил, и иметь общее ощущение уверенности в том, будет ли это хорошим долгосрочным сотрудничеством.

Пара слов о продолжительности в девяносто дней: девяносто дней — это часто используемый срок для онбординга новых сотрудников, но это не жёсткое правило. Интервал в тридцать дней обычно слишком короток в инженерии, где есть значительная кривая обучения для освоения вашей технологии, инструментов и продукта. С другой стороны, ожидание полного цикла оценки эффективности, например, шесть или двенадцать месяцев, слишком надолго оставляет потенциально неподходящего человека в роли, не давая ему получить необходимую коррекцию для выхода на продуктивность и обходясь компании в потерянное время и продуктивность. Правильный ответ, вероятно, где-то посередине, и точное количество времени зависит от вас и ваших руководителей.

Что делать при провале оценочной карты

Если после девяноста дней руководитель и сотрудник согласны, что дела не соответствуют ожиданиям, или нет согласия в том, выполняются ли ожидания, что-то должно измениться. Это не значит, что вы обязаны уволить нового сотрудника, но это значит, что вы обязаны что-то сделать. Рассмотрите следующие варианты в этом сценарии:

- Проблема в руководителе? Был бы этот человек более успешным в другой команде или с другим руководителем?
 - Если вы подозреваете, что это так, рассмотрите горизонтальный перевод, прежде чем переходить к увольнению.
- Есть ли культурное рассогласование?
 - Перенастроить сотрудника на вашу культуру после выявленного рассогласования сложно и редко удаётся. Если вы опасаетесь, что находитесь в этом сценарии на девяностый день, почти наверняка правильный вариант — расстаться, и более чем вероятно, что кандидат будет так же облегчён, как и руководитель.
- Это нехватка опыта или навыков?
 - Если вы наняли кого-то на сеньорский уровень, но он работает на уровне мидл, у вас есть вариант попытаться понизить его уровень. В конце концов, нечестно по отношению к другим сотрудникам держать этого человека и платить ему как сотруднику сеньорского уровня, если он не работает на этом уровне. Однако будьте осторожны: понижение уровня очень непросто. Если ожидания не управляются очень тщательно, понижение уровня часто приводит к уязвлённому эго и в итоге оказывается непродуктивным или даже токсичным для вашей команды.

Увольнение нового сотрудника

В целом, если после девяноста дней неясно, сложится ли с новым сотрудником, вряд ли это волшебным образом станет лучше через 120 или 150 дней, и лучше его отпустить. Вам следует увольнять этого сотрудника так же, как любого другого, с полным выходным пакетом и с максимальной добротой, насколько это возможно.

Я призываю вас взять полную ответственность за неудачный найм. Если вы наняли их, возьмите ответственность; это значит, что ваш процесс найма несовершенен. Не наказывайте за это сотрудника. Стандартный для отрасли выходной пакет в стартапе — это четыре недели зарплаты, льготы, если вы можете их продлить, и помощь в поиске другой работы любым способом, который вам комфортно предложить.

Таймлайн онбординга

Онбординг начинается в ту секунду, когда кто-то соглашается работать в вашей компании и подписывает оффер. Вам следует думать о том, как сделать вашего нового сотрудника успешным, ещё до его первого дня. Не каждый новый сотрудник будет рад тратить собственное время на изучение компании или своей роли до даты выхода, но в зависимости от задачи или того, что предлагается, многие добровольно согласятся это сделать.

Я призываю вас отправлять кандидатам ваше «Руководство первого дня», а также ваше руководство в тот день, когда они подписывают оффер. Если у вас есть корпоративный список для чтения, сейчас хорошее время заказать эти книги и либо отправить их кандидату, либо предложить их в формате электронной книги/аудиокниги. Большинство кандидатов совершенно не заинтересованы в чтении/написании кода до первого дня, но изучение вашей культуры или чтение высококлассных книг о бизнесе/культуре/инженерии редко воспринимается как обуза. Вам не следует требовать этой активности, но, сделав её доступной, вы, вероятно, получите вполне здоровое добровольное участие.

В фактическую дату выхода кандидат должен встретиться со своим новым руководителем первым делом утром и созвониться/сверить статус. Если он не прочитал материалы, которые вы отправили ему до прихода, задайте ожидание, что он должен сделать это в первый день. Им следует запланировать последующее время для разбора девяностодневной оценочной карты после того, как кандидат успел изучить вводные материалы и настроить своё окружение/логины. Это также хорошее время, чтобы укрепить идею непрерывного улучшения и поощрить кандидата взять на себя ответственность за любые шероховатости в его онбординге и внести вклад в улучшение документации и процесса для того, кто придёт в команду следующим.

Управление эффективностью

Один из ключей к улучшению морального духа сотрудников и продвижению позитивной рабочей культуры — обеспечить, чтобы каждый чётко понимал, как его воспринимают на рабочем месте, и имел надёжный ориентир

относительно того, как расти внутри организации. Цель любой системы управления эффективностью — настолько объективно и справедливо, насколько возможно, обеспечить сотрудникам эту прозрачность и структуру. Плохая система управления эффективностью приведёт к нежелательным сюрпризам или неловким и демотивирующим ситуациям, тогда как сильная система управления эффективностью мотивирует вашу команду и поощряет всех расти вместе.

Плохое управление эффективностью часто приводит к негативным результатам. Вот два примера:

Сотрудника А повышают, и в результате сотрудник В удивлён и чувствует, что его обошли. Ситуация усугубляется, если руководитель не может предоставить конкретное обоснование решения, когда сотрудник В впоследствии его оспаривает, что приводит к чувству недоверия, недосмотра и обрушившемуся моральному духу.

Сотрудник X, слишком долго пробыв на 4-м уровне, чувствует себя измотанным и деморализованным, не зная, как добраться до 5-го уровня и получить связанное с этим повышение зарплаты.

Ваша система управления эффективностью должна давать каждому ясность относительно того, где именно он находится, что ему нужно улучшить (и как), когда его будут оценивать и как эти оценки учитываются при повышениях и корректировках компенсации.

Матрица компетенций и грейдирование

Управление эффективностью и проектирование компенсации не должны выполняться целиком вами, техническим руководителем. Здесь есть масса способов совершить ошибки, которые могут подвергнуть вашу компанию юридической ответственности. Их легко избежать, обеспечив активное вовлечение вашего руководителя HR в процесс. На самом деле, в идеале ваш руководитель HR выполнял бы большую часть проектирования и опирался бы на вас только в помощи с определением технических компетенций. Независимо от того, кто берёт на себя ведущую роль, HR — ваш партнёр в этом.

Обзор

Ядро системы управления эффективностью — это документ, таблица или другой рабочий артефакт, который здесь я назову матрицей компетенций (иногда её также называют матрицей влияния или планом продвижения), перечисляющий навыки и области влияния для каждой роли. Матрица компетенций обеспечивает детализацию, конкретность и ожидания относительно того, как каждый навык/область влияния выглядит на различных уровнях.

Например, матрица компетенций индивидуального контрибьютора (IC) — инженера-программиста — может включать строку для скорости вывода кода/фич. В системе от 1-го до 5-го уровня матрица указывала бы, что для инженера 1-го уровня ожидания по скорости кода составляют X pull request'ов в неделю или способность закрывать Y стори-поинтов за спринт — что имеет наибольший смысл для вашей команды. В идеале каждое описание либо напрямую измеримо и задано количественно, либо качественно осязаемо и единообразно интерпретируется командой. Ожидания для остальных уровней возрастали бы постепенно и кульминировали бы в очень высокой планке скорости написания кода на 5-м уровне. Таким образом, при полной матрице компетенций любой член команды должен быть способен оценить себя в каждой категории и выдать набор оценок, который тесно совпадал бы с оценками, данными его руководителем или коллегами.

Как только у вас есть выписанные описания для каждого уровня, остаётся лишь опубликовать формулу для сведения оценок отдельных навыков в единый рабочий грейд. С этим у вас есть прозрачная, объективная, измеримая система, которую любой сотрудник может использовать, чтобы понять свою эффективность на работе и точно понять, где он может улучшиться, чтобы вырасти в уровне. Я привожу пример формулы для этого процесса суммирования в разделе «Оценки эффективности», страница 101.

Имейте в виду, что разные роли должны оцениваться за разный вклад в команду и должны иметь разные (хотя, возможно, и пересекающиеся) матрицы компетенций. Особенно важно создать отдельную матрицу для менеджмента, отличную от индивидуальных контрибьюторов-инженеров, чтобы поощрять руководителей развивать свои навыки за пределами написания кода.

Создание матрицы компетенций

Детали матрицы компетенций влияют на каждого члена команды, поэтому логично, что команду следует включить в определение этих деталей. Ссылаясь на раздел «Модели командного принятия решений» в «Мини-фреймворках

управления», страница 33, это определённно задача либо для модели «соломенного человека» (straw man), либо для модели совместной разработки (codevelopment).

Я рекомендую модель «соломенного человека»: набросайте ключевые навыки и области влияния, которые вы хотели бы видеть для любой роли, и попробуйте заполнить большую часть матрицы компетенций. Затем представьте идею своей технической команде и дайте им знать, что вы хотели бы их вклада в то, как доработать этот первый черновик. Выделите фиксированное время как команда и сделайте так, чтобы было безопасно и поощряемо прорабатывать матрицу вместе, возможно, используя группы-брейкауты для проработки отдельных категорий.

Какую бы структуру вы ни выбрали, сделайте её явной, выделите хотя бы несколько часов защищённого времени для работы над ней и установите дедлайн, к которому нужно получить финальную обратную связь для включения и превращения в кандидатский финальный черновик для команды.

Стремитесь максимум к пяти общим категориям и не более чем к трём областям внутри этих категорий. Более примерно пятнадцати навыков/областей влияния сделают матрицу слишком громоздкой для использования как эффективного инструмента оценки эффективности (и наверняка окажутся слишком обременительными для сбора своевременной обратной связи от команды).

Каждый уровень должен иметь свою систему оценки и ожидания по эффективности, чётко описанные, и/или может разделять описание с соседним уровнем, где это уместно.

Я привёл пример плана продвижения на моём сайте к этой книге по адресу (ctohb.com/templates) [\[https://ctohb.com/templates/templates\]](https://ctohb.com/templates/templates). У Codecademy.com также есть отличный шаблон по адресу ctohb.com/competencies.

Компенсация и грейдирование

Я рекомендую привязывать компенсацию к системе грейдирования матрицы компетенций, поскольку это расставляет приоритеты на двух целях: справедливости и стимулировании высокой эффективности.

Что касается справедливости, если любые два сотрудника на одной роли и одном уровне получают равную компенсацию, то логично, что справедливость этой компенсации будет зависеть от того, насколько справедливо грейдирование. Если команда в целом внесла вклад в матрицу компетенций и верит в её справедливость, то по большей части они также будут верить, что компенсация, привязанная к этой матрице, справедлива.

Что касается стимулирования высокой эффективности, привязка компенсации к грейдированию финансово стимулирует каждого расти в уровне. Если матрица компетенций спроектирована хорошо и демократично, ваша команда сосредоточится на навыках/областях влияния, которые действительно помогают бизнесу, что принесёт им более высокие уровни (и более высокую компенсацию), одновременно ускоряя вашу команду.

Перевод уровней в справедливую компенсацию чуть более нюансирован, чем большинство могло бы предположить. Проще всего создать прозрачную таблицу, которая говорит, что каждый на уровне X получает \$Y в год, но из такой строгой системы возникает несколько проблем: корректировки на стоимость жизни (также известные как локальные ставки) и бонусы компенсации, не основанные на эффективности.

GitLab опубликовал отличный пост в блоге, объясняющий, почему они платят локальные ставки (см. ctohb.com/local). Их калькулятор компенсации также публичен на ctohb.com/gitlabcompcalc). При этом нет единственно правильного способа обращаться с локальными ставками, и вам стоит подумать, имеет ли смысл их платить именно для вашего бизнеса. Если да, рассчитывайте эти ставки способом, который одновременно прозрачен и основан на данных.

Когда уровень эффективности переводится в конкретный диапазон оплаты, а не в точную сумму компенсации, это решает многие проблемы компенсации. Любая роль должна быть откалибрована под рыночную ставку, но как определяются рыночные ставки? Вообще говоря, инструменты и данные, доступные для определения рыночной ставки, будут несколько неточными и в лучшем случае дадут диапазон в пределах 10–20 процентов. Причина, почему это не точнее, проста: роль инженера-программиста в вашей компании вряд ли будет на 100 процентов идентична по требованиям той же роли в другой компании. В конце концов, ваша кодовая база и инструментарий не на 100 процентов одинаковы.

Наличие диапазона оплаты также оставляет пространство для повышения компенсации вне системы управления эффективностью. Изменения, не связанные с эффективностью, включают повышения, основанные на стаже, и корректировки на инфляцию. Вы также можете использовать диапазоны оплаты как грубую замену и оставить место для корректировок на стоимость жизни, прежде чем ваша организация формализует более сложную систему локальных ставок.

Конкурентные ставки и данные о рыночных ставках

Итак, вы спроектировали свою матрицу компетенций и решили переводить уровни в диапазоны оплаты. Теперь вам остаётся лишь рассчитать ваши диапазоны оплаты. Это область, в которой вам определённо стоит тесно сотрудничать с вашим руководителем HR, чтобы убедиться, что вы соблюдаете любые существующие регуляторные требования. Определение диапазонов оплаты должно быть максимально основано на данных, и благодаря горстке существующих платформ (как платных, так и тех, что требуют лишь обмена данными) эти данные относительно доступны. Платформы вроде Pave, Option Impact by Advanced-HR, Levels.fyi и Glassdoor могут предоставить богатые наборы данных, которые можно отфильтровать под размер/форму/стадию вашей компании, чтобы определить релевантный диапазон оплаты для конкретной роли.

Должностные титулы

Многие основатели стартапов скажут вам, что их организация очень плоская и что титулы ничего не значат. Это действительно может быть правдой время от времени в отдельных случаях, но это исключение, а не норма. В подавляющем большинстве компаний, включая стартапы, есть устойчивые тенденции в том, как используются титулы. Присвоение титулов создаёт ожидание относительно уровня и масштаба ответственности. Титулы также легко дать и трудно забрать, поэтому стоит быть вдумчивым и внимательным относительно того, какой именно титул вы поставите в описании вакансии или при повышении.

Для неисполнительных ролей, прежде чем определяться с титулами, я сначала призываю вас определить ваши уровни, используя только числа, например, уровень 1, уровень 2 и т.д., через матрицу компетенций (см. «Матрица компетенций и грейдирование», страница 95). Как только вы поймёте, чего ожидать от каждого из этих уровней, вы можете сопоставить уровни с титулами. Не бойтесь добавлять числовой суффикс к титулам; проще и яснее использовать титулы вроде Junior Engineer 1 и Junior Engineer 2, чем изобретать новое прилагательное, означающее чуть более опытного, чем джуниор, но ещё не мидл-уровня.

Титулы инженеров — индивидуальных контрибьюторов

Титулы для инженеров — индивидуальных контрибьюторов довольно просты и используют описательные прилагательные, передающие seniorность и размер ответственности. Большинство стартапов используют три основных титула: junior, mid-level и senior engineer. За пределами senior часто используются такие выражения, как principal, fellow и architect, хотя у них менее единообразное определение и иерархия.

Сеньорные индивидуальные контрибьюторы часто имеют неформальный титул тимлида (tech lead). Тимлид подразумевает, что часть времени индивидуального контрибьютора тратится на обязанности управленческого характера, но его основная ответственность по-прежнему — заниматься инженерией. Редко понятие тимлида отмечается титулом в резюме или организационной структуре; это просто дополнительная ответственность для более сеньорных сотрудников и часть ожиданий на этом уровне сеньорности. Если основной результат работы тимлида — это менеджмент, а не код, то ему следует находиться на управленческом треке с ожиданиями, титулом, обучением, коучингом и т.д. руководителя.

Титулы руководителей

Титулование менеджмента имеет более нюансированные последствия, чем у индивидуальных контрибьюторов. Самые распространённые титулы — software development manager (SDM) или software engineering manager (SEM), с соответствующими указаниями seniorности, например, mid-level software engineering manager или senior software engineering manager. SDM или SEM обычно отвечает за одну команду инженеров, которые, в свою очередь, работают над одной фичей или продуктом.

Следующий уровень — это обычно director of engineering (директор по инженерии). Директора отвечают за эффективность, согласованность и результат нескольких команд в рамках одного или сильно смежных продуктов. В большинстве организаций от директора не ожидается стратегическая роль. Иначе говоря, директор не задаёт фундаментальное техническое направление или продуктовую стратегию.

За пределами директора находится роль vice president of engineering (VPE, вице-президент по инженерии). Единой реализации VPE не существует. Она варьируется от организационного лидера всех инженеров компании (вместо СТО) до стратегического технического лидера по нескольким продуктовым областям. Иногда VPE подчиняется СТО, а иногда CEO. Однако общее у VPE — это ожидание быть технически очень сеньорным, опытным и умелым в управлении людьми — отличным коммуникатором и стратегическим мыслителем.

Оценки эффективности, опросы и повышения

Не все области навыков в матрице компетенций можно оценить количественно руководителем или таблицей, поэтому каждой команде нужен процесс оценки эффективности, который также может собирать качественную обратную связь от руководителей и коллег.

Задача в том, чтобы собрать качественную обратную связь таким образом, чтобы её можно было использовать для согласования с вашим грейдингом. В этой главе я представляю методологию, которую использовал для сбора обратной связи, в итоге приводящую к относительно простой для понимания системе подсчёта баллов, которая может выдавать индивидуальные уровни эффективности.

Ритм оценок

Оценки эффективности требуют значительного количества времени, могут быть эмоционально изматывающими и очень затратны для вашей команды — всё это подталкивает менеджмент проводить их реже. Конкурирующий стимул — тот факт, что сотрудники хотят более плотных циклов обратной связи и больше возможностей для повышения. Баланс достигается планированием оценок раз в шесть или двенадцать месяцев (более оперативную обратную связь можно и нужно давать через регулярные встречи 1:1 сотрудника с руководителем). Помните, что достаточное количество времени между оценками необходимо, чтобы люди могли расти и демонстрировать этот рост. Как правило, шесть месяцев — это безопасная нижняя граница, балансирующая эти соображения.

Выбор рецензентов

Вопрос «кто кого оценивает» не имеет лёгкого ответа. Многие компании просто поручают оценку руководителю, и всё. Хотя обратная связь руководителя ценна, она также может оставлять слишком много места для предвзятости и игнорировать не менее важную перспективу коллег и прямых подчинённых. Самый простой (хотя и немного более затратный) способ провести справедливый и всесторонний процесс — обеспечить, чтобы каждый сотрудник получал несколько оценок, включающих эти другие перспективы, что часто называют оценкой 360.

Здесь я рекомендую использовать технику «соломенного человека» (см. раздел «Модели командного принятия решений» в «Мини-фреймворках управления», страница 33): каждый руководитель должен составить список прямых подчинённых и коллег, которые достаточно соприкасаются с этим членом команды, чтобы написать ценную оценку, затем провести разговор с сотрудником и получить обратную связь, прежде чем финализировать список. Руководителям также следует отслеживать, сколько оценок просят выполнить каждого сотрудника, чтобы поддерживать управляемое количество запросов.

Вопросы оценки

Ваш опросник должен отражать вашу матрицу компетенций, и рецензентов следует поощрять делать явные ссылки на матрицу.

Один и тот же набор вопросов может применяться к каждой категории матрицы:

- Какие есть примеры того, как этот человек преуспевает в этой области?
- В чём этот человек демонстрирует пространство для улучшения в этой области? На каком уровне, по вашему мнению, этот человек работает в этой области?

Обратите внимание, что с точки зрения бессознательной предвзятости лучше попросить рецензента перечислить примеры, прежде чем спрашивать об уровне. Альтернатива может побудить рецензентов выбрать уровень, а затем выборочно подобрать примеры, чтобы оправдать уже выбранный уровень.

Может иметь смысл включить несколько более высокоуровневых/мягких вопросов в конце:

- Насколько вы стремитесь и рады работать с этим человеком? (Шкала: совсем не рад — очень рад. Этот вопрос из Keeper Test от Netflix [cto.hb.com/keeper])

- Этот человек сейчас на уровне X. Чувствуете ли вы, что он готов к повышению до уровня X+1?
- Есть ли какие-то другие сильные стороны, которые привносит этот человек и которые вы хотите выделить?
- Есть ли какие-то другие области для улучшения, которые вы хотите выделить?

Формат оценки

Вы можете проводить оценки с помощью формального инструмента оценки или без него (также известного как инструменты управления эффективностью или культурой, такие как Culture Amp и 15Five). Разумеется, специализированный инструмент экономит время и быстро масштабирует этот процесс для более крупных команд. Критически важно сохранять всю индивидуальную обратную связь анонимной, за исключением пометки о том, какие оценки пришли от менеджмента (мы будем использовать эти оценки отдельно как проверку на адекватность относительно оценок коллег позже в процессе).

Расчёт результата

После того как оценки поданы, для каждого человека в каждой категории матрицы компетенций должен отразиться набор баллов, в идеале разбитый между оценками коллег, прямых подчинённых и руководителей. Вот пример матрицы баллов:

<TODO: Insert Table Here>

Задача теперь в том, как агрегировать эти баллы в итоговый расчёт рабочего грейда. Несколько ключевых соображений в этом расчёте:

- Защитить целостность процесса (например, чтобы сотрудник не сговорился со своими коллегами искусственно завысить или занижить чьи-либо баллы)
- Обеспечить, чтобы формула была справедливой и могла рассчитываться единообразно
- Подтвердить, что перспектива руководителя относительно влияния сотрудника согласуется с обратной связью коллег/подчинённых
- Решить, все ли категории в матрице взвешены одинаково или неодинаково.

Вот метод, который я рекомендую для определения уровня: присвойте уровень, на котором кумулятивный балл составляет 66 процентов или выше. Кумулятивный балл для данного уровня — это процент всех оценок, которые находятся на этом уровне или выше. У самого низкого уровня кумулятивный балл всегда будет 100 процентов, у уровня 2 — это 100 процентов минус процент голосов с уровня 1. Уровень 3 — это 100 процентов минус процент с уровня 2 и уровня 1, и так далее.

В примере выше человек был бы уровнем 2. На уровне 2 его кумулятивный балл — 90 процентов. По правилу 66 процентов он очень близок к уровню 3 (который на 60 процентах) — всего в двух голосах. То, что он так близок, можно использовать в коучинге, чтобы поощрить дальнейшее улучшение перед повышением, или использовать для обоснования корректировки в пределах диапазона оплаты.

Когда вы завершили общий расчёт, если вам удалось отследить оценки руководителя отдельно, вы можете применить ту же формулу к оценкам руководителя в изоляции и рассчитать разницу между уровнем, полученным из кумулятивного балла оценок руководителя, и уровнем из кумулятивного балла всех оценок коллег. Большая дельта здесь — что-либо больше одного уровня — заслуживает пристального внимания и дополнительной проверки, так как это означает, что у руководителя и коллег значительно разные перспективы относительно эффективности человека. Или это может указывать на некоторую нерегулярность в процессе голосования/оценки.

Обсуждение результата

Я призываю руководителей предоставлять данные оценки эффективности заранее, до фактической встречи 1:1 по оценке эффективности, чтобы максимизировать ценность самой встречи. Лучше дать человеку данные и некоторое время на их осмысление, чтобы он мог быть полностью вовлечён, когда встреча состоится.

Повестка встречи по оценке эффективности должна быть простой:

1. Обсудить любые выявленные сильные или слабые стороны, которые неожиданны или иначе регулярно не освещаются на встречах 1:1.
2. Синтезировать небольшой список областей фокуса для работы до следующего периода оценки. Многие лидеры выступают за единственную область фокуса, но я видел, как несколько людей росли более чем в одном направлении за данный период, поэтому две или три области фокуса — разумный верхний предел, по обстоятельствам.
3. Установить расписание для руководителя и сотрудника, чтобы регулярно сверяться по этим областям фокуса и обеспечить продвижение до следующей оценки.

Развёртывание корректировки компенсации

Во многих компаниях обратная связь по оценке и изменения компенсации обрабатываются в разное время. Я не считаю критичным или даже выгодным обсуждать изменения компенсации на встрече по оценке эффективности, поскольку это может отвлечь от того, что в остальном может быть сложным, но важным обсуждением. Ключевое — задать ожидания относительно того, когда изменения компенсации будут решены, сообщены и реализованы, до процесса оценки эффективности, чтобы сотрудники знали, чего ожидать, прежде чем войти на встречу.

Планы повышения эффективности (PIP)

Планы повышения эффективности, обычно называемые PIP (performance improvement plans), — это инструмент, который придаёт структуру процессу либо улучшения работы сотрудника, либо его увольнения. Несколько ключевых аспектов PIP:

- Каждый член вашей команды должен понимать, как устроен процесс PIP в вашей компании.
- Многие люди приходят на работу с одними и теми же жгучими вопросами в глубине сознания: «Уволят ли меня сегодня?» или «Достаточно ли хорошо я работаю?»
- Знание о том, что в компании есть формальный процесс увольнения сотрудников, включающий возможность исправиться, помогает снизить эту тревогу.
- PIP следует использовать только при искренней попытке устранить и исправить недостаточную эффективность, поскольку они требуют значительных усилий как от сотрудника, так и от руководителя.
- Руководителю стоит потратить время на вдумчивое заполнение документа PIP, убедившись, что он чётко формулирует проблему недостаточной эффективности, по возможности задаёт количественные, ясные критерии оценки и структуру, а также предлагает поддержку и наставничество, чтобы помочь человеку улучшиться.
- PIP должны предусматривать разумный период времени для демонстрации улучшений — скажем, тридцать дней для индивидуальных контрибьюторов и шестьдесят дней для старших сотрудников или руководителей.

PIP всегда должны включать полную письменную версию — не только для того, чтобы обеспечить ясность между сотрудником и руководителем, но и чтобы предоставить документацию для HR/юристов, которая будет храниться на случай любых последующих запросов.

Есть несколько ситуаций, в которых вам следует обойти процесс PIP и сразу перейти к увольнению:

- Нарушения политики компании, нарушения HR-правил, неприемлемое поведение на рабочем месте и т. п. не исправляются с помощью PIP и должны встречать нулевую терпимость.
- Некоторые пробелы в навыках нельзя исправить с помощью PIP, например общее отсутствие здравого суждения в определённой теме, плохое соответствие культуре или нехватка опыта в критически важных областях навыков. Чаще всего это соображение актуально для управленческих или очень старших позиций, где грамотное профессиональное суждение критично для роли.

Смена места

Прежде чем разрабатывать PIP или увольнять сотрудника с недостаточной эффективностью, стоит задать вопрос: возможно, этот человек лучше подойдёт для другой команды или роли в компании? Если природа недостаточной эффективности связана с навыками, а у сотрудника есть навыки, которые можно было бы лучше применить в другой роли, то смена команды может оказаться очень продуктивной. Если же недостаточная эффективность

связана с культурой, то, скорее всего, ни у одной из сторон не будет мотивации пытаться организовать внутренний перевод.

Гениальные засранцы

Гениальный засранец (brilliant jerk) — это устоявшийся в индустрии термин, описывающий человека, который сам по себе крайне продуктивен, но чьё присутствие вредит моральному духу или продуктивности окружающих. Таких людей часто называют токсичными личностями.

Из-за их порой выдающейся индивидуальной продуктивности решение уволить гениального засранца зачастую может казаться трудным или неправильным. Почти универсальная рекомендация — всё равно увольнять. Каждый день, в который вы как руководитель позволяете токсичному поведению сохраняться, может усиливать обиду вашей команды на вас за то, что вы это допускаете. Нет такого объёма индивидуальной продуктивности, который компенсировал бы удар по культуре компании и по вашему авторитету как руководителя, который влечёт за собой сохранение токсичного человека в команде.

Увольнение

В вашей компании должна быть чёткая процедура того, как именно увольнять сотрудника. Мой лучший совет: следуйте ей. Это та область, которая, если её обработать неправильно, может стать существенным юридическим риском для компании. Ключевые соображения включают:

Документация: убедитесь, что у вас достаточно документации, чтобы обосновать решение об увольнении этого человека и доказать, что увольнение основано на эффективности или другой обоснованной причине (for-cause).

Сроки: как только вы решили кого-то уволить, делайте это как можно скорее. Распространённая мудрость гласит, что после увольнения сотрудника руководители обычно беспокоятся не столько о том, стоило ли вообще увольнять этого человека, сколько о том, не слишком ли долго они тянули с этим. Механически не так уж важно, какой день недели вы считаете лучшим для увольнения, но если вы можете перенести это на первый день месяца вместо последнего, вы даёте сотруднику бонус в виде лишнего месяца на корпоративной программе медицинского страхования.

Свидетели: убедитесь, что руководитель и HR присутствуют как свидетели на самой встрече по увольнению. Встреча должна быть очень короткой и по существу, а HR должен отвечать на большинство последующих вопросов, касающихся логистики увольнения.

Офбординг: заранее, до увольнения, разработайте план того, как и когда отключить доступ сотрудника к системам компании и вернуть всё корпоративное оборудование.

Выходное пособие: увольнение сотрудника чаще всего является в той же мере провалом компании/руководства, что и самого сотрудника. Увольнение — не место для злобы или мелочности; этот человек вложил время и энергию в вашу компанию, и вам следует сделать всё возможное, чтобы подготовить его к успеху на следующей роли, включая стандартный для индустрии пакет выходного пособия (диапазон довольно широк — от нескольких недель для индивидуального контрибьютора до двух-трёх месяцев для старшего руководителя, причём пакеты часто также учитывают стаж).

Состав команды

Ключевое различие во влиянии между джуниорскими и сеньорскими специалистами — это стабильность, с которой они могут надёжно решать разные типы задач. По мере того как инженеры набираются опыта, их суждения и принятие решений улучшаются на всё более широких областях. Аналогично, вам стоит ожидать, что более старшие специалисты будут разрабатывать решения, в которых меньше дефектов, которые служат дольше и более устойчивы к изменению требований по ходу дела. Это не значит, что все обязаны быть сеньорами; на самом деле редко когда большинство проектов связаны с проектированием совершенно новых решений «с нуля» (greenfield).

Правильный баланс для каждой конкретной команды учитывает типы предстоящей работы и в результате вдумчиво формирует штат команды.

Состав по уровню сеньорности

Ваша команда должна быть в большей степени смещена в сторону старших инженерных специалистов, если ваша кодовая база:

- Очень новая, требующая много архитектуры и создания базовых контрактов
- Очень старая, плохо поддерживаемая или плохо продуманная и считающаяся трудной для работы — короче говоря, brownfield-кодовая база
- Существенно меняется в требованиях, особенно если новые требования не очень похожи на старые
- Использует новые инструменты, техники или паттерны, требующие проверки применимости для вашей задачи
- Требуется установление новых паттернов/способов выполнения работы, особенно в экосистемах, которые не предоставляют жёстких ограждений, поощряющих здоровые паттерны.

Техническая специализация

В первый день существования большинства стартапов команда состоит из небольшой горстки инженеров, обычно двух или трёх. Команда из трёх человек оставляет ограниченные возможности для специализации. Есть двадцать категорий технической работы и всего три человека, так что по принципу Дирихле (pigeonhole principle) как минимум один человек, а вероятнее все три, будут заниматься многими типами технической работы. Иначе говоря, на раннем этапе в вашей компании от каждого ожидается, что он будет носить много технических шляп. По мере роста компании и добавления новых людей в команду вы и ваши сотрудники найдёте больше возможностей для специализации.

Итак, вы привлекли раунд финансирования и впервые собираетесь расширять команду. Как решить, нужны ли вам frontend-инженеры, backend-инженеры, DevOps-инженеры и т. д.? Вот несколько общих рекомендаций:

Слушайте свою команду: люди, которые сейчас занимаются инженерией, с большой вероятностью будут открыты говорить о том, где находятся самые крупные источники неэффективности и где больше всего нужна помощь. Ваша задача как руководителя — воспринять эту точку зрения и экстраполировать на будущее. Команда указывает на проблему, которая исчезнет через два месяца, — в таком случае нанимать кого-то было бы неуместно? Или проблема носит системный характер и, скорее всего, сохранится надолго?

Ищите факторы, которые вредят продуктивности: если ваша команда в основном состоит из frontend-инженеров, а вы испытываете трудности с надёжностью backend, то это должно быть признаком того, что вам нужна помощь backend- или DevOps-инженерии. Тот же принцип применим к тестированию, опыту разработчика (developer experience) и т. д.

Специализируйтесь по мере масштабирования: пока ваша команда не превысит дюжину человек, велики шансы, что вам лучше иметь команду преимущественно из универсалов (generalists).

Вот несколько приблизительных цифр для состава команды на основе опыта стартапов:

- Размер команды 1–5
 - Ваша команда — это все универсалы, специализированные максимум по разделению на frontend, backend и мобильную разработку.
- Размер команды 5–15
 - Ваша команда специализирована по продукту или общей области навыков, такой как backend, frontend-архитектура, frontend-дизайн, DevOps и тестирование.
 - Скорее всего, вам захочется начать думать о выделенных ресурсах в тестировании и DevOps, когда вы вырастаете до пятнадцати инженеров (или превысите это число).
- Размер команды 15–30
 - К этому моменту у вас должна быть реальная специализация, и вы должны нанимать только людей с экспертизой в какой-то подобласти разработки ПО.
 - На этом этапе любая неэффективность в том, как выполняется работа, скорее всего, начнёт обходиться очень дорого в масштабах команды, поэтому убедитесь, что вы вкладываете либо

штатные единицы, либо время в обеспечение того, чтобы разработчики могли выполнять работу, чтобы их инструменты работали и чтобы операционная логистика была отлажена.

- Размер команды 30 и более
 - На этой стадии существует множество методологий разбиения команд на более мелкие единицы, чтобы работа оставалась эффективной. Если у вас несколько продуктовых линий, выравнивание членов команды с конкретными продуктовыми линиями — довольно естественная отправная точка для организации.
 - Многие компании на этой стадии используют концепцию подов (pods), где под имеет фокусную область и состоит из разнородной/кросс-функциональной команды, способной самостоятельно выполнять задачи в этой области.

Сопровождение проектов: философия двух команд

Если вы выпускаете ПО для конечных пользователей, вашей инженерной команде приходится соблюдать баланс между новой работой и обработкой тикетов поддержки, поступающих от активных клиентов. Если оставить это без контроля, необходимость обрабатывать тикеты поддержки может стать серьёзным отвлекающим фактором для команды, вредящим эффективности, истощающим моральный дух и выжигаящим ваших лучших людей. У этой проблемы есть много правильных решений; важно то, что вы признаёте её влияние на вашу команду и проектируете решение, помогающее им быть продуктивными и обеспечивать отличные результаты для клиентов.

Microsoft опубликовала отличную статью на эту тему под названием [Building productive teams \(cto.hub.com/teams\)](https://cto.hub.com/teams), описывающую то, что они называют моделью двух команд (two crews model). Модель двух команд выделяет команду фич (feature crew) и клиентскую команду (customer crew).

Команда фич сосредоточена на будущем, создавая новые фичи. Клиентская команда сосредоточена на настоящем, работая над текущими проблемами клиентов, диагностируя баги и приоритезируя здоровье сайта.

Другими названиями клиентской команды могут быть команда сопровождения или команда поддержки уровня Tier 2 (где Tier 1 — это ваш нетехнический персонал клиентской поддержки).

Выделение работы по сопровождению в отдельную команду имеет много преимуществ:

- Это позволяет выделенной команде постоянно мониторить очередь обращений клиентов, проводя триаж и решая всё важное и срочное.
- Это позволяет вашей команде фич оставаться на 100 процентов сосредоточенной на будущем, не отвлекаясь на работу по клиентской поддержке.
- Это позволяет развивать специализацию внутри клиентской команды, создавая инструментарий и экспертизу в обработке проблем, делая решение проблем со временем менее затратным.
- Это даёт ещё один карьерный путь для отдельных инженеров, особенно джуниорских, чтобы учиться и расти в вашей команде.

Первый вопрос, который мне задают по поводу подхода с двумя командами: как долго человек остаётся в клиентской команде? Есть четыре подхода к определению срока пребывания в клиентской команде:

- Сделать клиентскую команду постоянной, отдельной командой или департаментом.
 - У вас есть опубликованные описания вакансий для инженеров, ориентированных на поддержку и отладку. Учтите, что для многих инженеров работа, сосредоточенная только на отладке, может звучать непривлекательно. Мне описание вакансии, которое подчёркивает ввод данных или бухгалтерию, кажется очень неприятным, и тем не менее есть много людей, которым такая работа нравится и которые даже стремятся к ней.
 - Не предполагайте, что раз вы сами не стали бы делать эту работу, то нет других, кого такая перспектива могла бы воодушевить. В частности, работа в клиентской команде знакомит инженера

с огромным объёмом кода, часто даёт возможности общаться с клиентами и связана с меньшим продуктовым давлением дедлайнов — всё это может привлечь подходящего кандидата.

- Создать явно обсуждаемую карьерную траекторию для инженеров, чтобы они начинали в клиентской команде и через некоторый период времени (обычно двенадцать с лишним месяцев) переходили в команду фич.
- Инженеры регулярно ротируются между командами. Упомянутый выше пост в блоге Microsoft рекомендует менять местами некоторых членов команды между двумя командами каждую неделю.
- Определить клиентскую команду как временную команду. Это может означать либо что сама клиентская команда не существует на постоянной основе (возможно, только одну неделю в месяц), либо что члены команды постоянно ротируются между клиентской командой и командой фич.

Я рекомендую создавать выделенную, постоянную клиентскую команду только для тех команд или продуктов, где проблемы клиентов достаточно затратны, чтобы это оправдало. Если ваш бизнес связан с большим объёмом поддержки, выделенная клиентская команда может стать множителем силы как для инженерной продуктивности, так и для удовлетворённости клиентов.

Организация команды

В целом вы обнаружите, что технические организационные структуры построены одним из двух способов: функциональная организация и продуктовая организация. Функциональная группа организована вокруг типа работы, которую выполняют члены команды, например frontend-инженерия, тестирование или конкретный внутренний сервис. Продуктовые группировки, иногда называемые бизнес-юнитами (business units), организованы вокруг конкретного бизнес-/продуктового фокуса, например команда корпоративного основного приложения или команда потребительского мобильного приложения. То, как вы решите организовать свою команду, может оказать существенное влияние на коллаборацию в команде, продуктивность и моральный дух. Команды, организованные по продукту, часто называемые подами, в большинстве случаев являются правильным ответом.

Когда вы проектируете организационную структуру, вам следует подумать о том, что именно вы оптимизируете. Для стартапа основная цель организационной структуры — обеспечить, чтобы разные люди, которым нужно тесно сотрудничать друг с другом, получали возможность и поощрение делать это благодаря организационной структуре. Лучший способ этого добиться — организовать вместе всех людей, которые напрямую влияют на осуществимость и успех продукта, держать их подотчётными общему набору целей и развивать у них общее чувство ответственности за этот продукт.

Когда ваша команда мала и у вас всего один продукт, вопрос о том, как организовываться, не имеет смысла — это одна кросс-функциональная команда, работающая над одним и тем же продуктом. К моменту, когда ваша команда вырастет до 12 человек или более, вам нужно будет начать более осознанно определять, что такое под, и найти метод, который легко понять и который укоренён в реальности вашего продукта, прежде чем разбивать команду на поды.

Преимущества подов — большая сплочённость команды, автономность, ответственность за владение, подотчётность и общая скорость исполнения. Однако этот подход не лишён компромиссов. Когда долгоживущая группа инженеров работает над одним продуктом, это может создавать силосы знаний (knowledge silos). По мере масштабирования организаций также более вероятно, что продуктовая команда примет архитектурные решения, которые могут оптимизировать локально и привести к дублированию работы или неэффективному использованию вычислительных ресурсов. Если вы предвидите и планируете возникновение этих проблем, боль, которую они причиняют при хорошем управлении, будет намного перевешена преимуществами автономных, высокоэффективных подов.

Управление удалёнными командами

Существует достаточное число успешных на 100 процентов удалённых команд разработки ПО, чтобы быть неоспоримым тот факт, что удалённые организации могут работать. Это не значит, что все удалённые команды успешны или что обязательно легко построить полностью удалённую культуру. Я провёл почти десятилетие,

управляя удалёнными командами. Ниже приведены несколько рекомендаций, которые должны быть применимы к большинству сценариев удалённого управления.

Документация

Удалённость означает, что рядом с вами нет никого, кто ответил бы на вопросы, но это не значит, что вопросы исчезают. Эти вопросы всё равно задаются, только теперь социальный контекст утрачен, и потому, возможно, вопрос какое-то время остаётся без ответа. Или же, вместо того чтобы придержать вопросы до момента, когда другой человек сделает перерыв, теперь они задаются немедленно, вызывая разного рода уведомления и отвлекая от сосредоточенной работы. Наличие надёжного набора внутренней документации с эффективной функцией поиска может ускорить время до получения ответа и сократить количество переключений контекста один на один, которые становятся барьерами для выполнения работы.

Асинхронная работа

Отличный способ превратить удалённую работу в актив, а не в обузу, — это сделать ставку на асинхронные рабочие практики. Сильная асинхронная культура снижает бремя несовпадения часовых поясов и сокращает время, проводимое на удалённых встречах / в работе с удалёнными переключениями контекста. (Подробнее об асинхронной коммуникации и ценности асинхронной работы см. «Преимущества избыточной коммуникации», стр. 20.)

Очные встречи

Какими бы полезными они ни были, видеозвонки не заменяют возможность сесть и поесть всей командой. Связи, формируемые при очных встречах, как правило, сохраняются в течение долгого периода времени, поэтому вложение даже в нечастые очные встречи может улучшать качество социальных отношений между членами команды на месяцы вперёд. Как общее правило, для команды полезно встречаться лично раз в квартал, чтобы поддерживать эти отношения и минимизировать удалённую социальную фрустрацию.

Перекрытие часовых поясов

Моя общая рекомендация для команд, работающих над одним проектом, — иметь минимум четыре рабочих часа перекрытия. Это оставляет достаточное окно для любых регулярно запланированных встреч и возможность для спонтанных разговоров и вопросов.

Создавайте возможности для общения

В целом режим людей по умолчанию при видеозвонках — заниматься несколькими делами одновременно во время звонка, а затем как можно быстрее завершить звонок. Как только рабочий разговор окончен, все кладут трубку. Не так люди взаимодействуют лично; например, встреча заканчивается, а затем вы говорите о спорте в коридоре, возвращаясь к своим рабочим местам. Эта толика социального времени до/после встреч — ценный способ для людей выстраивать отношения и доверие, и оно не возникнет, если только руководство и культура активно его не поддерживают. Дешёвый и простой способ делать это на регулярной основе — регулярно задавать лёгкий, по кругу, в стиле «ледокола» вопрос, например: «Какой одной хорошей новостью — личной и профессиональной — вы можете поделиться?»

Вы можете поддерживать удалённое выстраивание отношений с помощью удалённых happy hours или виртуальных командных ужинов и социальных активностей. После COVID-19 появилось много онлайн-фасилитаторов социальных мероприятий, которые проводят удалённые события — от цифровых «вечеров казино» до виртуальных квестов (escape rooms).

Камера включена

В зависимости от того, какую статью вы читаете, 70–90 процентов коммуникации невербально. Использование веб-камеры во время видеозвонка не заменяет все эти 70 процентов, но это, бесспорно, улучшение по сравнению с полным отсутствием видео. Поскольку веб-камеры сейчас так дешёвы, на самом деле нет оправдания тому, чтобы не иметь этой дополнительной полосы пропускания социальной коммуникации внутри вашей компании. Распространённое возражение состоит в том, что сотрудники беспокоятся, что другие могут судить об их внешности на камере. Если вы сталкиваетесь с такой обеспокоенностью, я призываю вас пройти по бизнес-обоснованию желания иметь максимально качественную коммуникацию в удалённой среде и проводить политику

нулевой терпимости к любому сотруднику, который делает неуместные замечания о чьей-либо внешности. Также бывает полезно создать пространство, где люди могут делать всё, что считают уместным, чтобы привести себя в готовность, например разрешая/поощряя размытие фона и периодическое отключение видео, чтобы разобраться с чем-то в реальности или привести себя в порядок.

Запись встреч

Записанные встречи дают отличный способ позволить сотрудникам войти в курс темы, не требуя остановки работы ради встречи по календарю. В целом вы обнаружите, что очень немногие сотрудники или кандидаты возражают против идеи записи видеозвонков. Записи также отличный способ познакомить с каким-то контентом больше людей, чем это, возможно, комфортно в моменте. Например, вы можете захотеть, чтобы команда по найму из четырёх-пяти человек посмотрела собеседование с кандидатом, но присутствие пяти человек в комнате в тот момент может оказаться пугающим. Запись собеседования один на один (1:1) — отличный способ обеспечить, чтобы у каждого была возможность оценить кандидата, не создавая некомфортной или несправедливой ситуации перегрузкой самой встречи-собеседования.

Обязанности лидерства

Как руководитель, ваши обязанности лидера выходят за пределы технических компонентов создания и развития инженерной команды. Вам следует стремиться помогать компании быть успешной в целом, с особым акцентом на то, как технологии вносят вклад в этот успех. Это означает следить за тем, как технологии работают с остальной частью компании, способствуя здоровой коллаборации с другими командами, как внутренними, так и внешними, а также внедряя хорошие процессы и управленческие практики вокруг технологий и разработки продукта.

Продуктовые и дизайн-команды

Это ваша ответственность как технического лидера — обеспечить, чтобы ваша инженерная команда эффективно работала с продуктовыми и дизайн-командами, даже если эти команды подчиняются кому-то другому. При проектировании этого процесса, как общее правило, перебрасывать задачи через стены между группами неэффективно. Хорошее взаимодействие между продуктовой, инженерной и дизайн-командами требует эмпатии и понимания. Чем занимается другая команда, с какими вызовами она сталкивается и как вы и ваша команда можете облегчить им жизнь? Ниже я дам немного контекста об этих других функциях и несколько конкретных предложений о том, как эффективно работать вместе.

Дизайн-системы

Дизайн-команды в идеале хотели бы, чтобы их работа была реализована точно, пиксель в пиксель, командой разработки ПО. В отсутствие какой-либо структуры или набора ограничений добиться пиксельной точности на деле дорого или даже невозможно; однако немного общего понимания и дизайн-система могут радикально удешевить эту задачу.

Дизайн-система — это набор стандартов для управления дизайном в масштабе с использованием переиспользуемых компонентов и паттернов. Крупные компании, как правило, создают собственные дизайн-системы. Atlassian, например, делает свою публично доступной (cto.hb.com/design). Для маленького стартапа инновации в дизайн-системах вряд ли являются ключом к успеху, поэтому из этого следует, что вам стоит использовать готовую дизайн-систему. В наши дни вы можете выбирать из множества готовых систем с богатыми наборами функций и разнообразием эстетических стилей, которые, скорее всего, можно настроить под ваш бренд.

Дизайн-системы не только дают экономящий время набор ограждений и компонентов, используемых дизайнерами, — они часто поставляются с готовой поддержкой различных frontend-языков и фреймворков. Material Design, например, имеет опубликованную дизайн-систему в Figma (cto.hb.com/figma), а также набор react-компонентов на JavaScript (mui.com) и angular-компонентов (material.angular.io).

Приняв систему вроде Material (или AntD, Chakra UI, Blueprint, Bootstrap, Semantic UI и т. д.), вы получаете не только набор готовых технических компонентов, но и систему, которая аккуратно интегрируется с инструментами дизайнеров. Интегрируя одну и ту же систему с инструментами как инженеров, так и дизайнеров, вы обеспечите, что то, что создают ваши дизайнеры, будет чисто отображаться на компоненты, доступные инженерам. Это

сокращает или даже устраняет необходимость для инженерии заниматься кастомной стилизацией или frontend UI и упрощает соответствие дизайну вплоть до пикселя.

Помимо эффективности, получаемой от согласованности между дизайном и инженерией, большинство реализаций компонентов дизайн-систем также учитывают или автоматически решают другие дизайнерские приоритеты, такие как доступность (accessibility) (как для программ чтения с экрана, так и управление цветовым контрастом для людей с дальтонизмом), соответствие стандартам UI и даже готовая поддержка тёмной темы.

PRD и спецификации

Документ требований к продукту (Product Requirements Document, PRD), иногда также называемый продуктовой спецификацией или просто спецификацией (spec), — это неотъемлемая часть процесса разработки продукта. Учтите, что продуктовая спецификация имеет иную цель и является отдельным документом от технической спецификации (см. «Техническое планирование и спецификации», стр. 164). Существует множество методологий и шаблонов PRD. Источники вроде lennysnewsletter.com регулярно каталогизируют некоторые из наиболее распространённых и подробных. У PRD есть общая цель — описать предысторию проблемы, а также «почему», которое обосновывает проект или фичу. PRD часто включают списки требований, которые нужно выполнить для достижения бизнес-цели. Большинство PRD оставляют «как» реализации фичи технической спецификации.

PRD, как и техническая спецификация, — это живой документ. По мере того как ваша команда узнаёт больше о проблеме или по мере изменения факторов во внешнем мире, эти документы могут и должны меняться и обновляться в соответствии с этим. Я призываю вас и вашу команду использовать эти документы как непрерывно обновляемый источник истины для документирования ваших соображений и итоговых решений, принятых в процессе разработки продукта.

EPD

EPD — это отраслевая аббревиатура для Engineering Product and Design (инженерия, продукт и дизайн).

Подразумевается, что все три департамента существенно важны для жизненного цикла разработки продукта и нуждаются в здоровом способе совместной работы для получения отличных результатов. С точки зрения бизнеса, все три департамента вместе отвечают за создание продукта, который любят клиенты, поэтому полезно иметь одного человека с единым набором бизнес-целей, задающего направление для всех трёх единиц.

Продуктовый менеджмент против проектного менеджмента

Продуктовый менеджмент и проектный менеджмент — это отраслевые термины с различающимися значениями. Продуктовый менеджер отвечает за проектирование и создание продукта, а также за ключевые показатели эффективности (KPI), которым продукт должен соответствовать для бизнеса. Книга Мелиссы Перри *Escaping the Build Trap* — феноменальный ресурс, который глубоко погружается в роль и влияние, которое отличный продуктовый менеджер может оказать на вашу организацию. Проектный менеджер отвечает за руководство внутренней организацией команды, управление внутренней коммуникацией и соблюдение дорожной карты (roadmap) и сроков.

На раннем этапе вашего стартапа вы как СТО можете выполнять обе эти роли. Очень рано в своём плане найма вам следует запланировать наличие отличного продуктового менеджера, который сможет снять с вас часть этих обязанностей. Некоторые продуктовые менеджеры превосходно справляются с проектным менеджментом, тогда как другие не находят в нём радости и потому не уделяют ему время и внимание. Можно поспорить, что на раннем этапе вашего стартапа это нормально в любом случае; при маленькой команде последствия слабого проектного менеджмента минимальны. Однако по мере роста формализация проектного менеджмента становится важнее, и если ваш продуктовый менеджер не выполняет эту роль, вам стоит стремиться дополнить его проектным менеджером.

Моя общая рекомендация — формально делегировать проектный менеджмент, когда EPD выросла примерно до двадцати человек. Это будет означать формальное назначение этой роли себе, передачу её продуктовому менеджеру или наём выделенного проектного менеджера. Вам захочется участвовать на раннем этапе в выстраивании процессов проектного менеджмента, обеспечить, чтобы внедряемые механизмы проектного менеджмента поддерживали ту культуру, которую вы хотите построить, и проявлять эмпатию ко всем вовлечённым сторонам.

Управление руководителями и обучение руководителей

Есть отраслевое выражение, что вашей компанией в конечном счёте управляют ваши руководители среднего звена. Подразумевается, что, несмотря на любые стратегии и процессы, которые внедряют топ-менеджеры, именно руководители среднего звена в конечном счёте оказывают наибольшее влияние на количество и качество результата. Руководители среднего звена нанимают индивидуальных контрибьюторов, задают им повседневные цели и держат их подотчётными стандартам качества. Лучшие руководители среднего звена — это мини-руководители, сосредоточенные на культуре, построении команд, склонных к коллаборации, и работающие над тем, чтобы дать им возможность выполнять свою лучшую работу. Из этого следует, что вам как руководителю стоит вкладывать много усилий в наём, управление и обучение этих руководителей среднего звена.

Сложная тема обучения руководителей заслуживает целой отдельной книги, и это навык, на освоение которого нужно время. При этом два главных урока, которые я могу вам предложить и которые дадут рычаг всем остальным навыкам здесь, — это подавать пример самому и строить культуру непрерывного обучения управлению. В ту же минуту, как вы нанимаете или повышаете кого-то в менеджмент, вам следует ясно дать понять, что вы ожидаете от него готовности вкладывать время и усилия в оттачивание своего ремесла управления. Вы будете идти на всё, чтобы облегчить им это, включая обучение управлению в их личные цели и план развития, предоставляя им ресурсы для прокачки управленческих навыков, помогая им прорабатывать управленческие проблемы и обучая их всему, что знаете сами.

Обучение управлению не обязано быть чрезмерно обременительным и дорогим. Если позволяет бюджет, я настоятельно рекомендую нанимать внешних коучей по управлению для ваших руководителей. Внутреннее ежемесячное чтение/обсуждение управленческой книги среди руководителей также может быть эффективным для запуска маховика непрерывного развития.

Финансы и бюджетирование

Часть вашей роли, вероятно, состоит в том, чтобы нести ответственность за текущие и будущие затраты департамента разработки ПО (а иногда и дизайн- и продуктовых департаментов). Как ответственный распорядитель этого бюджета, вы должны знать, сколько было потрачено на различные статьи в прошлом и сколько ваша компания в целом заложила на будущее. Самое важное — вам понадобится план, чтобы обосновать разумное расходование этого выделенного бюджета.

В целом вы обнаружите, что две крупнейшие статьи расходов в техническом департаменте — это люди (фонд оплаты труда) и инфраструктура/SaaS. Имейте в виду, что фактическая денежная стоимость сотрудника выше, чем просто его зарплата, поскольку она включает льготы и налоги на фонд оплаты труда. Хорошее эмпирическое правило: денежная стоимость сотрудника на 20 процентов выше его зарплаты; этот процент называется ставкой накладных расходов (burden rate).

Бюджет

Большинство финансовых команд используют сложное бухгалтерское и бюджетное ПО для ведения бухгалтерии компании. Если нет, у них будет общекорпоративная электронная таблица, которая намного больше и сложнее, чем вам нужно как СТО. По моему опыту, финансовые департаменты обычно не готовы позволять людям вне финансов вносить изменения в основную систему бюджетирования, поэтому, если только они не дадут вам что-то для старта, на вас лежит задача создать финансовую модель для вашего департамента.

Учитывая, что затраты вашего департамента довольно предсказуемы и сосредоточены в нескольких статьях расходов, создаваемая вами модель не обязана быть очень сложной. Моя рекомендация — вести электронную таблицу, которая включает следующее:

- Вкладка фонда оплаты труда (Payroll)
- Вкладка SaaS/Себестоимость проданных товаров (Costs of Goods Sold, CoGS)
- Вкладка инфраструктуры
- Вкладка «Прочее» (включая поездки, оборудование) Сводная вкладка

Вы можете найти образец бюджетной таблицы технического департамента на cto.hb.com/templates, который проведёт вас большую часть пути в собственно моделировании.

Работа с вашим CFO

Если ваша компания похожа на большинство стартапов, вы будете самым дорогим департаментом, и ваш CFO (финансовый директор) должен хорошо это осознавать. Несколько вещей, которые стоит учесть, которые полезны для вас и сделают вашего CFO вашим лучшим другом:

- Помогайте вашему CFO понимать, как деньги тратятся и будут тратиться. По возможности избегайте сюрпризов. Заранее давайте ориентиры по таким вещам, как стоимость оборудования, стоимость поездок/конференций и стоимость облака.
- Ведите бюджет вашего департамента и держите его в актуальном состоянии с учётом изменений в вашем прогнозе.
- Регулярно обновляйте свой бюджет фактическими данными (actuals) из финансового департамента и обеспечивайте, чтобы разница между прогнозом и фактом была понятна и управляема.
- Установите план найма и включите оценочные зарплаты.
- Счета за SaaS часто бывает обременительно отслеживать. Рассмотрите либо использование инструмента анализа выписки по кредитной карте (то есть платформы управления SaaS, она же SMP), либо наём ассистента, который будет регулярно категоризировать и сверять эти расходы с вашим бюджетом.
- Финансовые департаменты часто очень заботятся об атрибуции затрат, чтобы различать, например, затраты, которые являются частью себестоимости проданных товаров (COGS). Указание в вашем бюджете очень грубого «почему» для каждой статьи расходов может завоевать вам друзей в финансах.

Измерение скорости/здоровья инженерной разработки

Мне ещё не доводилось встречать техническую команду или руководителя, которым удалось бы последовательно и с пользой количественно оценить реальную производительность инженерной разработки. Сюда же относится и измерение скорости как суммы оценок выполненных задач. (См. раздел «Оценки» в главе «Рабочий процесс» (Workflow) на странице 160, где обсуждается ненадёжность технических оценок.)

При этом я видел немало компаний, которые эффективно измеряют здоровье инженерной разработки и факторы, влияющие на её скорость, — а именно время цикла (cycle time) и распределение рабочего времени.

Время цикла

Время цикла — это измерение того, сколько времени проходит от идеи до выпущенной функции, и его часто разбивают на следующие промежуточные этапы:

- Время, потраченное на написание кода
- Время до начала ревью кода; время до утверждения кода; время до развёртывания кода

В целом команды с низким временем цикла более эффективны и способны быстрее итерировать, внедрять инновации и доставлять ценность клиентам. Существует множество инструментов, облегчающих измерение времени цикла, включая LinearB (linearb.io) и Code Climate (codeclimate.com). LinearB опубликовала набор бенчмарков по метрикам, связанным со временем цикла, на основе данных тысяч инженерных команд.

Распределение рабочего времени

Идея измерения распределения рабочего времени состоит в том, что, хотя измерить объём выполненной работы сложно, относительно легко измерить, на какие типы работы участники команды тратят своё время. Полученная информация полезна как ориентир. Например, если команда тратит большую часть времени на устранение багов, то разумно предположить, что повышение качества программного обеспечения и снижение этого процента времени приведёт к тому, что больше времени будет уходить на разработку новых функций, а значит, к общему улучшению здоровья и скорости разработки.

Фактическое измерение распределения рабочего времени можно выполнять полуавтоматически, выгружая отчёты из систем учёта задач, либо измерять с помощью регулярных лёгких пульс-опросов команды.

Привлечение инвестиций и due diligence

В целом, как у СТО, ваша роль в привлечении инвестиций и due diligence довольно невелика. В большинстве стартапов основную часть этой работы выполняют CEO и, возможно, CFO. Ваше участие, скорее всего, наступает

ближе к концу процесса, когда инвесторы проводят due diligence по компании. Многие (хотя, к сожалению, не все) запросы в рамках due diligence касаются информации, которую хорошо организованная инженерная команда уже поддерживает в рамках повседневной работы. Следите за следующими пунктами и будьте готовы предоставить их для due diligence:

- Организационную структуру
- Бюджет департамента
- Полное описание всех продуктов, которые инженерная команда создала и поддерживает
- Дорожные карты разработки (roadmaps) (обычно ищут краткосрочные и среднесрочные дорожные карты)
- Список основных областей технического долга — то, что я называю балансовым отчётом технического долга (см. «Технический долг» (Tech Debt) на странице 145)
- Высокоуровневые диаграммы системной архитектуры
- Полное описание того, как программное обеспечение распространяется и обновляется — либо как SaaS, либо как версионированные настольные/мобильные пакеты ПО
- Высокоуровневое описание систем, того, как они размещены, и ваших практик безопасности
- Информацию о лицензировании программного обеспечения, включая сканирование кода компании, подтверждающее отсутствие нарушений лицензий и нелегализованного проприетарного ПО

Нередко инвестор нанимает стороннюю фирму для проведения аудита технического due diligence. Такие аудиты могут включать интервью или встречи с вами и, возможно, с несколькими другими старшими членами команды. Будьте готовы обсудить ваш процесс инженерной разработки, оценить общую продуктивность команды и провести разбор части вашего кода (code walkthrough).

Мой совет — быть откровенным с этими аудиторами. Они изучили множество технологических компаний и прекрасно понимают, что у всех инженерных команд есть долг и части системы, которыми команды гордятся больше, чем другими. Чем больше ваши инвесторы знают о ваших сильных и слабых сторонах, тем лучше они смогут поддерживать вашу команду и требовать от неё ответственности за устранение этих слабостей в дальнейшем.

Управление поставщиками

В целом ответственность за поиск, переговоры и управление сторонними техническими поставщиками ложится на вас как на СТО. Для большинства это неприятная, но необходимая часть работы, и, как следствие, ей не уделяют особого внимания. Однако хорошее выполнение этой обязанности может обеспечить бизнесу значительную экономию средств. Здесь я шаг за шагом проведу вас через типичный процесс переговоров и подписания SaaS-контракта и дам несколько советов о том, как повысить эффективность и сэкономить деньги.

Инструменты самостоятельной регистрации

Обычно либо вы, либо кто-то из вашей инженерной команды обозначает потребность в определённом типе инструмента, и если это один из ваших инженеров или менеджеров, то он либо попросит вас одобрить расход, либо попросит вас зарегистрироваться в инструменте. Многие инструменты стоят незначительно и имеют тривиальные процессы самостоятельной регистрации. Я рекомендую вам поработать с командой по бюджету и финансам и установить порог, ниже которого ваши менеджеры уполномочены просто самостоятельно регистрироваться в инструменте. Для инструментов, стоимость которых превышает этот порог или которые не имеют самостоятельной регистрации, вам потребуется вступить в процесс изучения и переговоров с корпоративным поставщиком, который обычно состоит из четырёх шагов.

Корпоративные инструменты

Теперь вы сталкиваетесь с процессом корпоративных продаж. Прежде чем обращаться к отделу продаж поставщика, убедитесь, что именно эта компания лучше всего подходит для решения вашей проблемы. В большинстве случаев я рекомендую завести таблицу, провести небольшое исследование всех поставщиков в этой области и отфильтровать список до двух-трёх лучших. Даже если один из них является вашим явно предпочтительным выбором, не помешает иметь более глубокое понимание сферы и знаний/рычагов/BATNA (best alternative to a negotiated agreement — лучшая альтернатива переговорному соглашению) для переговоров.

Квалификация продажи

Когда вы впервые обращаетесь к корпоративному поставщику, вы почти наверняка вступите в так называемый процесс квалификации продажи. На этом этапе, особенно как технический руководитель, ваши потребности ещё не совпадают с потребностями поставщика. Вы, вероятно, ищете цену, контракт и кратчайший путь к началу работы. Поставщик же стремится убедиться, что вы — его целевой клиент и, вероятно, зарегистрируетесь и не уйдёте (не «отвалитесь») хотя бы несколько лет.

В большинстве SaaS-компаний первичный телефонный звонок принимают торговые представители первой линии, и их основная задача — узнать о вашей компании и понять, соответствуете ли вы их профилю. У них может не быть особых технических знаний, и часто они не смогут обсуждать с вами цены. В результате вы вряд ли получите большую пользу от этой первой встречи. Мой совет — либо делегировать эти вводные встречи, либо попытаться получить вопросы для квалификации продажи от представителя по электронной почте и ответить на них достаточно полно, чтобы вообще пропустить вводную встречу.

Переговоры

После того как поставщик подтвердил, что вы соответствуете шаблону его целевого клиента, он назначит вторую встречу с более старшими специалистами со своей стороны. Обычно это менеджер по продажам или, возможно, технический торговый представитель. Это этап, на котором вы начнёте получать ответы на большее количество технических вопросов о решении, а также получите некоторую прозрачность в отношении ценовых моделей, которые поставщик уполномочен использовать.

Вот несколько общих советов по переговорам:

- Помните, что в конечном счёте работа продавца — продать вам продукт, поэтому вы оказываетесь в одной команде, когда речь идёт о выработке взаимно приемлемых условий. Чем прозрачнее вы будете в отношении того, что для вас важно, тем лучше они смогут составить торговое соглашение, отвечающее этим потребностям.
- Не недооценивайте факторы помимо общей стоимости, такие как общая длительность контракта (более длинные контракты должны сопровождаться существенными скидками), частота платежей, условия оплаты (например, net 30, net 90) или то, что происходит с контрактами в случае смены контроля над одной из компаний.
- В целом ищите контракты, которые растут по мере вашего роста. В идеале начальная стоимость низкая и растёт со временем, по мере того как инструмент используется более активно и приносит больше ценности. Поощрение установки «мы растём вместе» может помочь снизить эти начальные затраты.
- Держите свои финансовую и комплаенс-команды в курсе. Если ваш CFO отлично ведёт переговоры, поручите эту часть процесса ему.
- Если ваши общие бюджеты на SaaS велики или вы ведёте много таких сделок, подумайте об использовании стороннего переговорщика, например Vendr. Часто такие переговорщики берут плату в зависимости от того, сколько они вам сэкономят, и у них есть данные по гораздо большему числу контрактов, чем у вас, что помогает понимать ценообразование.
- Учитывайте, что квоты на конец месяца/квартала реальны и скидки в это время очень распространены.

Подписание

После того как вы согласовали условия и обменялись документами, следующий шаг — выяснить, кто является уполномоченными подписантами от вашей компании (предполагая, что вы один из них), подписать документацию и держать её в порядке.

Не теряйте и не помещайте не на своё место эти контракты, так как они будут полезны для будущих переговоров или due diligence. Убедитесь, что вы отслеживаете расходы в своих бюджетах или с помощью платформы управления SaaS (SaaS management platform, SMP).

Постпродажное обслуживание

После подписания сделки вас, скорее всего, передадут другому представителю поставщика — человеку с должностью, похожей на «специалист постпродажной поддержки» или «менеджер по обслуживанию клиентов» (customer service manager, CSM). Эти люди мотивированы, оцениваются и ориентированы на удержание клиентов

или допродажи продукта (upsell). Они хорошо разбираются в продукте и по крайней мере отчасти технически подкованы. В дальнейшем они будут служить вашим адвокатом в продвижении новых функций и устранении дефектов.

Какой тип стартап-СТО вы?

Являетесь ли вы СТО, CEO, играющим роль СТО, или основателем, надеющимся нанять СТО, полезно иметь чёткое понимание того, чем именно занимается СТО в стартапе и чем эта роль отличается от руководящих и лидерских ролей в более устоявшихся компаниях. Как и в большинстве случаев, ответ зависит от контекста и будет меняться со временем. У Calvin French-Owen есть

отличная статья (cto4b.com/founder2cto), в которой роль СТО разбивается на четыре архетипа: People Leader (лидер людей), Architect (архитектор), R&D (исследования и разработки) и Marketing/Consumer-facing (маркетинг/работа с потребителями). Мне нравится эта разбивка, и я немного уточню её здесь до трёх типов:

- Сфокусированный на технологиях
- Сфокусированный на людях
- Сфокусированный вовне

СТО, сфокусированный на технологиях *Он же главный архитектор*

СТО, сфокусированный на технологиях, может также возглавлять «офис СТО» — внутреннюю техническую группу-«скунсворкс» (skunkworks), основным результатом работы которой является дальновидная стратегия, архитектура, а иногда и реализация proof-of-concept того, как помочь бизнесу в перспективе. У такого СТО будет меньше подчинённых, а основная часть инженерной организации будет подчиняться отдельному вице-президенту по инженерной разработке. В этом случае нередко вице-президент по инженерной разработке (VP of Engineering, VPE) подчиняется кому-то помимо СТО — чаще всего напрямую CEO или директору по продукту (chief product officer, CPO).

Внутренний СТО, сфокусированный на технологиях, может также выступать главным архитектором технических процессов, настраивая инструменты, системы и процессы того, как выполняется техническая работа.

Как внутренний СТО, сфокусированный на технологиях, вы можете также выполнять функции продакт-менеджера, если продукт вашей компании имеет ярко выраженный технический характер (то есть инструменты для разработчиков, API-as-a-service и т. п.).

СТО, сфокусированный на людях *Он же VP of Engineering (VPE)*

В типичном стартапе не будет одновременно и VPE, и СТО, поэтому часто именно на СТО ложится задача выполнять роль VPE. Это также зачастую самая трудная роль для СТО-кофаундера, поскольку обязанности технического кофаундера на «день первый» мало пересекаются с обязанностями внутренне сфокусированного на людях технического руководителя.

СТО, сфокусированный на людях, отвечает за формирование внутренней технической культуры, процесса найма и за контроль внутренних процессов. Такой СТО тратит значительную часть своего времени на фактическое управление либо индивидуальными контрибьюторами, либо другими техническими менеджерами. Из трёх областей фокуса эту важнее всего наладить правильно. Если компания плохо нанимает или её технический персонал плохо управляется, демотивирован, расфокусирован или не выровнен по целям, это может сказаться на продуктивности и даже привести к плохим решениям, которые нанесут вред организации в долгосрочной перспективе.

СТО, сфокусированный вовне *Он же руководитель технических продаж/маркетинга*

Это, вероятно, наименее распространённый фокус для стартап-СТО, но не менее критичный в нужное время и в нужном месте. Чаще всего такого СТО можно встретить в компаниях, создающих продукты для технической аудитории — например, инструменты для разработчиков. Такие СТО тратят много времени на написание статей в блог или выступления на конференциях. Возможно, их привлекают на встречи по продажам, чтобы они выступали в

роли исполнительного технического представителя для закрытия крупных сделок. Обратите внимание, что выстраивание бренда вокруг технической команды не только положительно влияет на продажи продукта, но и может быть отличным инструментом рекрутинга и снижать время и стоимость найма.

Это, пожалуй, самая лёгкая роль для СТО-основателя, поскольку у него есть исторический контекст компании и он может наиболее искренне и страстно рассказать историю компании и продвигать её продукт и ценность.

Переход между типами

В идеале стартап-СТО окажется искусным во всех трёх областях, хотя большинство людей специализируется лишь в одной-двух. Если вашему бизнесу требуется фокус, не являющийся вашей сильной стороной, возможно, стоит задаться вопросом, не сможете ли вы выполнить эту задачу эффективнее, делегировав её коллеге. Особенно на ранней стадии стартапа большинство технических кофаундеров/СТО-кофаундеров будут внутренне сфокусированы на технологиях. Обычно на этом этапе и нет особой другой работы!

Очень распространён паттерн, когда такой человек обнаруживает, что вынужден растягиваться в сторону внутреннего фокуса на людях или внешнего фокуса по мере роста компании, и это не всегда желательный переход для него. Признать, что ваша специализация или ваша мотивация лежит в технологиях, а управление людьми вам не очень подходит, — это не личная неудача, а как раз наоборот. Выявление своих суперсил и проектирование роли в компании так, чтобы использовать эту суперсилу, — это и есть способ принести максимальную ценность, а компании следует нанять кого-то другого, чья суперсила — например, управление людьми — чтобы заполнить эту роль.

Если вы обнаружили, что застряли в роли, которой недовольны, жизненно важно признать это несоответствие как перед самим собой, так и перед своим CEO. Это не означает отказа от вашего положения основателя или руководителя, либо от статуса уважаемого и высоковлиятельного человека внутри компании.

Несколько мыслей о различных переходах:

- Ваша суперсила: внутренние технологии
 - Потребности компании
 - Внутренние люди: наймите VP of Engineering, сфокусированного на людях, проактивно характеризую роль СТО как техническую.
 - Внешний фокус: если у вас ещё нет внутри компании человека, который делает это лучше вас, наймите developer evangelist (евангелиста для разработчиков) и наделите его полномочиями выполнять эту роль. В противном случае продвиньте кого-то изнутри и убедитесь, что чётко понятно, что эта роль подразумевает.
- Ваша суперсила: внутренние люди
 - Потребности компании
 - Внутренние технологии: если в команде есть старший инженер или архитектор, которого вы можете наделить полномочиями/повысить до позиции технического лидера, это стоит рассмотреть. В противном случае технический архитектор должен быть в числе ваших главных приоритетов найма.
 - Внешний фокус: если у вас ещё нет внутри компании человека, который делает это лучше вас, наймите developer evangelist (евангелиста для разработчиков) и наделите его полномочиями выполнять эту роль. В противном случае продвиньте кого-то изнутри и убедитесь, что чётко понятно, что эта роль подразумевает.
- Ваша суперсила: внешний фокус
 - Потребности компании
 - Внутренние технологии: если в команде есть старший инженер или архитектор, которого вы можете наделить полномочиями/повысить до позиции технического лидера, это стоит

рассмотреть. В противном случае технический архитектор должен быть в числе ваших главных приоритетов найма.

- Внутренние люди: наймите VP of Engineering, сфокусированного на людях.

Во многих случаях итоговым результатом может стать наём СТО, чья суперсила лучше соответствует тому, что нужно компании в данный момент. В других случаях это может означать наём очень способного VP of Engineering, сфокусированного на людях, в дополнение к высокотехническому СТО.

Управление технической командой

Ваш результат как технического руководителя измеряется результатом вашей команды, поэтому

именно на вас лежит обязанность обеспечить, чтобы команда в целом работала хорошо. Что значит «работать хорошо», будет различаться в зависимости от команды и обстоятельств, но существуют общие паттерны и тенденции, коррелирующие с высокой производительностью. В этом разделе моя цель — дать рекомендации по ситуациям, общим для всех технических руководителей.

Техническая культура и общая философия

Я призываю всех руководителей перенять общий стиль лидерства, известный как *служашее лидерство* (servant leadership). Как служащий лидер, вы прежде всего сосредоточены на служении потребностям своей команды. Это означает фокус на расширении возможностей других, на построении

культуры прозрачности, коммуникации, сотрудничества и роста. Размышляя о своей команде и культуре, а также принимая повседневные решения, спрашивайте себя, какой вариант позволяет команде делать свою работу наилучшим образом и тем самым приносить максимум для бизнеса.

Десять столпов технической культуры

1. Тратьте время команды на то, что важно для бизнеса

В общем случае вы хотите, чтобы ваша команда тратила время на то, что двигает бизнес вперёд, и именно ваша обязанность как технического руководителя — создать среду, в которой ваши инженеры могут сосредоточить свои усилия таким образом последовательно и с минимумом отвлекающих факторов. Эта установка может показаться очевидной, но при правильном применении она является мощным инструментом для принятия решений.

Например, предположим, что ваша команда обсуждает, использовать ли готовый фреймворк для бэкенда или написать что-то с нуля. Можно составить нюансированный список плюсов и минусов, где обсуждались бы такие вещи, как дополнительная гибкость от написания с нуля против более короткого времени до первого развёртывания с фреймворком. В этой гипотетической реальности вашему бизнесу на самом деле нужно итерировать фронтенд и оптимизировать путь клиента, поэтому каждый момент, потраченный на бэкенд, — это момент, не потраченный на продвижение фронтенда вперёд. Если готовое решение достаточно хорошо, чтобы обеспечить эту итерацию фронтенда, то даже если вам придётся выбросить фреймворк и переписать бэкенд через восемнадцать месяцев с помощью кастомного решения, быстрая итерация фронтенда сейчас и нахождение соответствия продукта рынку (product-market fit) дают вашему бизнесу право выполнить это переписывание в дальнейшем.

2. Используйте надёжные инструменты там, где можете, и внедряйте инновации там, где это важно

Эту мысль также называют «не изобретай велосипед» и «стой на плечах гигантов»; идея здесь в том, чтобы использовать готовые компоненты (библиотеки, облачные сервисы, приложения, пакеты), когда это возможно. Использование готовых компонентов сопряжено с внутренним компромиссом между лёгкостью старта и подгонкой решения под вашу конкретную задачу. Я утверждаю, что в большинстве случаев, когда уже существует готовая реализация, компромисс сильно склоняется в сторону использования готового сервиса, библиотеки или приложения.

В редких случаях, когда вам в итоге всё же потребуется переписать или сильно кастомизировать зависимость, опыт работы с готовым инструментом окажется очень ценным для влияния на дизайн кастомной реализации и ускорения его проектирования.

3. Автоматизация раскрывает скорость

Ваша цель как архитектора вашей команды — обеспечить, чтобы команда тратила как можно большую часть своего времени на код, приносящий ценность бизнесу. Существует ряд трудоёмких задач, которые разработчики регулярно выполняют и которые необходимы для написания кода, но сами по себе не добавляют ценности бизнесу (например, воспроизведение продакшн-окружений, запуск кода локально, выяснение того, как запустить наборы тестов, развёртывание окружений для фича-веток в облаке, отлов багов, не покрытых тестами, и т. д.).

Такие типы задач налагают постоянный «налог» на общую продуктивность. Вы можете избежать уплаты этого налога, вложившись в автоматизацию этих задач всякий раз, когда они выявлены. Поощряйте свою команду документировать каждый случай, когда они тратят более тридцати минут на непродуктивную техническую работу, и предусмотрите в своём процессе место для автоматизации таких задач, чтобы никто больше никогда не терял этот час.

4. Частота снижает сложность

Под заголовком «Частота снижает сложность» Martin Fowler развивает фразу «Если что-то болит, делай это чаще» (c4hnb.com/fowler). Любой процесс или задача, которые болезненны, подвержены ошибкам или иным образом дорогостоящи для вашей команды, по утверждению Fowler, являются симптомом недоразвитости этой задачи. Без давления с вашей стороны болезненные технические задачи, как правило, оказываются последними, за которые добровольно берутся. В результате ими пренебрегают, и боль со временем усиливается. И наоборот, если культура вашей команды делает упор на приоритизацию этих болезненных задач, то больше усилий будет уходить на автоматизацию, документирование и улучшение этих задач, делая их в конечном счёте менее болезненными или даже полностью автоматизированными. Как отмечает Fowler, выполнение задач чаще также даёт больше обратной связи по ним и развивает навык за счёт практики, и всё это дополнительно снижает сложность и болезненность задачи.

Для многих команд выпуск кода в продакшн является примером задачи, которая часто болезненна. Выпуск кода может требовать часов на фактическое развёртывание, и поэтому он делается редко. Однако если ваша команда придерживается философии «Если что-то болит, делай это чаще», то вы как технический руководитель будете настаивать на регулярных релизах с более частыми интервалами. Если вы начнёте с еженедельного релиза, то на первой неделе команда почувствует боль, и вторая неделя будет такой же болезненной, как и первая, но, возможно, инженеры заметят возможность автоматизировать часть развёртывания. К третьему, четвёртому или пятому релизу у вас, скорее всего, будет совершенно новый набор скриптов и инфраструктуры для выпуска кода, что позволит вам ускорить частоту релизов до двух раз в неделю. Спустя какое-то время вы сможете выпускать код нажатием одной кнопки. Выпуск чаще одного раза в день называется непрерывным развёртыванием (Continuous Deployment) (см. «Непрерывное развёртывание» (Continuous Deployment) на странице 216).

5. Стандартизируйте процесс RFC

RFC, или Request for Comment (запрос на комментарии), — это документ, который описывает техническую идею, процесс или спецификацию и пишется с целью рецензирования коллегами (peer review) и последующего принятия. У большинства протоколов, с которыми вы знакомы, есть соответствующий RFC, например RFC 2616 для HTTP или RFC 1035 для DNS. Идея рецензирования коллегами с последующим принятием и стандартизацией — мощный инструмент для псевдодемократического принятия технических решений и получения поддержки результатов, и это может быть отличным инструментом для использования с вашей командой.

Я рекомендую формализовать процесс RFC и предоставить некоторые рекомендации относительно того, какие виды решений следует пропускать через процесс RFC.

Формальный процесс может выглядеть как простой чек-лист и шаблонный документ, который включает, куда поместить вашу копию этого документа, как собираются отзывы/комментарии и каков процесс голосования, финализации и стандартизации того, как выглядит документ.

Моё предложение — опираться на имеющиеся у вас инструменты и, например, создать markdown-документ в системе контроля версий, который выступает шаблоном RFC. Новый RFC тогда будет pull request в этом репозитории, вводящий новый markdown-файл с предложением. После этого довольно естественно собирать голоса в виде одобрений (approvals) этого pull request, а финализация RFC происходит, когда pull request в конечном счёте сливается (merge). В качестве альтернативы, если вы настроили внутреннюю вики, вы можете создать RFC там и использовать систему комментариев вики для сбора обратной связи.

Вам также следует задать чёткие ожидания относительно того, какие виды решений выносятся на RFC. Я рекомендую ограничить это высокоуровневыми процессами, техническими мнениями и культурой, не используя RFC для выбора инструментов или системной архитектуры. Несколько хороших примеров тем для RFC:

- Стандартизация соглашений об именовании (master vs. main, black/whitelist vs. allow/denylist, camelCase vs snake_case в различных контекстах)
- Использование форматировщиков кода/статического анализа кода; ритмы/повестки/артефакты встреч
- Monorepo vs. manurepo
- Мнения/философии в области дизайна API

Я призываю вас включить в шаблон RFC раздел о том, как результаты RFC институционализируются. Например, если RFC предлагает стандартизировать техническое мнение для вашей команды, то после ратификации RFC это мнение должно быть включено в ваш инженерный справочник и документацию по онбордингу, чтобы оно стало каноном и для нынешних, и для будущих сотрудников.

6. Сделайте скорость целью, а не стратегией

В книге *Good Strategy/Bad Strategy* автор Richard Rumelt определяет хорошую стратегию как ту, что обеспечивает три элемента: диагноз того, как преодолеть вызов, направляющую политику или общий подход и набор действий по тому, как эта политика будет реализована. Стратегия описывает путь.

В отличие от этого, цель — это описание пункта назначения. Я слышал, как бесчисленные команды говорили мне, что их стратегия — «двигаться быстро», без какого-либо описания того, как они туда доберутся. Я полностью согласен, что скорость инженерной разработки — отличная цель, но это не стратегия.

Вот пример стратегии для обеспечения высокоскоростной инженерной разработки:

Вызов: Мы — нерегулируемый стартап на ранней стадии, которому нужно итерировать как можно быстрее, чтобы найти соответствие продукта рынку (product-market fit) прежде, чем у нас закончится капитал.

Диагноз: У нас есть возможность минимизировать сложность и двигаться быстро. Поскольку у нас пока нет product-market fit, ценность того, что мы построили на данный момент, минимальна, поэтому нам нужно игнорировать невозвратные затраты (sunk cost) и выбирать быстрые переписывания, пока мы ещё не уверены, как наш продукт будет выглядеть в долгосрочной перспективе.

План действий: Мы сосредоточимся на найме инженеров-универсалов (generalists), будем укреплять культуру любознательности и находчивости, оставлять эго за дверью и вкладывать лишь умеренное количество усилий в долгосрочное техническое планирование. Вместо этого пока что мы сосредоточимся на нашей способности выпускать на рынок MVP-эксперименты с продуктом.

7. Участвуйте в непрерывном обучении и технологических конференциях

Технологические конференции сегодня повсеместны, и существует собрание единомышленников практически для каждой подспециальности программной инженерии. Ваша команда, если она ещё этого не сделала, в какой-то момент, скорее всего, попросит бюджет на посещение этих конференций. Типичный бюджетный запрос составит около \$1000 на авиабилеты и отель и \$500–\$1000 на входные взносы и суточные расходы. В сумме это обойдётся примерно в \$2000 на то, чтобы инженер посетил конференцию, плюс его время вне офиса. Как в духе обучения и непрерывного совершенствования, так и ради множества сопутствующих преимуществ, я рекомендую вам закладывать бюджет и регулярно или систематически одобрять запросы на конференции.

Если бы единственной пользой от посещения конференции было то, что сотрудник узнал немного больше о релевантной технической теме, которой он увлечён, то затраты были бы оправданы. Однако пользы гораздо больше.

Я призываю вас требовать от участников конференций по возвращении подготовить письменный документ, обобщающий ключевые вещи, которые они узнали. Вам также стоит рассмотреть спонсирование конференций, наиболее релевантных вашему бизнесу, и проведение вами или кем-то из вашей команды семинара по конкретной теме. Эти семинары и спонсорства предоставляют отличные возможности для брендинга вашей компании, доводя ваше имя и ваше сообщение до очень целевой аудитории — аудитории, которая, вероятно, содержит кандидатов, которых вы хотели бы нанять в будущем.

Подводя итог, выделение бюджета на посещение инженерами конференций полезно для индивидуального профессионального развития, полезно для культуры и морального духа, полезно для брендинга, может помочь с будущим наймом и, при небольшой доле документирования, является возможностью для непрерывного обучения всей команды.

8. Применяйте отладку «резиновой уточкой»

Случалось ли с вами такое, что вы пытаетесь разобраться с проблемой и, объясняя её коллеге, находите решение? Отладка «резиновой уточкой» (Rubber Duck Debugging) (cto.hb.com/rdd, cto.hb.com/tpp) — это простая практика, которая пытается воспроизвести этот феномен, не вовлекая коллегу и не заставляя его переключать контекст.

Отладка «резиновой уточкой» — это процесс проработки проблемы путём проговаривания её сначала вслух, возможно, обращаясь к настоящей физической резиновой уточке, сидящей у вас на столе. Идея в том, что, проговаривая проблему вслух, человек часто слышит изъян или видит решение проблемы, которое он не рассматривал, пока проблема существовала только у него в голове. Подход с резиновой уточкой потенциально избавляет коллегу от прерывания и быстрее даёт вам ответ.

9. Создайте библиотеку обучающих видео

Если картинка стоит тысячи слов, то одноминутное видео вашего рабочего стола/IDE/приложения с вашим голосом, обсуждающим техническую тему, стоит \$1000 экономии времени разработчиков. Существует множество инструментов, которые делают запись и обмен этими мини-видео сообщениями тривиально простыми, включая инструменты, встроенные в Slack и Loom. Вы не только сможете яснее донести техническую идею с помощью записи экрана и закадрового голоса, чем текстом, но и сделать это в удобное вам время. Получившееся видео можно ускорить по усмотрению зрителя, а само видео можно заархивировать и пересмотреть позже как часть организационной базы знаний.

Как технического руководителя вашей команды, я призываю вас регулярно создавать эти мини-видеоразъяснения, особенно в моменты, когда вы вносите изменения в архитектуру платформы. Организуйте все видео в библиотеку в вашей внутренней вики. Убедитесь, что они полные и самодостаточные, но при этом короткие и по делу; если вы не доберётесь до критически важного контента до конца чрезмерно длинного видео, ваши члены команды могут так никогда его и не увидеть. Также сделайте подход как можно более последовательным, чтобы зрители знали, чего ожидать.

Я гарантирую, что поначалу, по мере того как вы будете создавать и распространять видео, вы получите относительно низкие показатели просмотров, но со временем эта библиотека принесёт огромную ценность как источник справочных знаний и наберёт просмотры, которые сэкономят вам и вашей команде существенное количество времени.

10. Онбординг — это ответственность каждого

Ваша команда непрерывно создаёт «племенные знания» (tribal knowledge) — будь то то, как запустить сервис, или то, как паттерны кода используются по всей вашей кодовой базе. Есть два способа справиться с этим: вы можете ничего не делать, и тогда ежедневно вы будете увеличивать размер разрыва в знаниях для новых сотрудников, либо вы можете активно работать над преобразованием изолированных «племенных знаний» в масштабируемую документацию для нынешних и будущих сотрудников. По очевидным причинам я рекомендую второе. Это означает, что каждый всегда должен спрашивать себя: «Как я могу задокументировать то, что я только что создал/узнал/обнаружил?» А когда у вас появляются новые сотрудники, которые сталкиваются с чем-то, чего они не могут найти в вашей базе знаний, именно им предстоит найти ответ и задокументировать его.

Технический долг

Мост «Золотые ворота» (Golden Gate Bridge) в Сан-Франциско сделан из стали, которая на самом деле вовсе не золотого цвета. Мост покрашен, и поддержание его знаменитого цвета настолько важно для жителей Сан-Франциско, что они красят его непрерывно (cto.hb.com/painting). Как только перекраска завершается, процесс сразу же начинается заново. Именно такая форма непрерывных вложений, или вечного обслуживания, требуется для того, чтобы самые важные и сложные системы продолжали работать на уровне ожиданий — от моста «Золотые ворота» до программного проекта вашей команды. Только в вашем проекте обслуживание не требует ведёр с краской; оно приходит в форме технического долга.

Каждая функция, которую предоставляет команда разработки, несёт с собой определённый объём будущей работы, или долга. Этот долг может принимать форму багов, которые нужно исправить, быстрых доработок (fast-follow) функции для поставки дополнительной ценности клиенту или небрежности в коде, которую следует устранить ради улучшения сопровождаемости, производительности или безопасности. Определённый объём долга естественным образом накапливается, даже если ваша команда в отпуске: находятся уязвимости безопасности в зависимом ПО, пакеты устаревают, выходят новые версии инструментов, сторонние API объявляются устаревшими или меняются и т. д. Долг неизбежен, и вам нужно его учитывать.

Технический долг и жизненный цикл продукта

Технический долг можно мыслить ещё и как финансовый долг — например, ипотеку на дом. Беря ипотеку для покупки дома, вы осознанно принимаете решение взять на себя долг, понимая последствия (проценты), чтобы получить возможность сделать то, что хотите сейчас (приобрести дом). Затем вы регулярно гасите этот долг на протяжении длительного периода времени (ежемесячные платежи).

То же происходит и с техническим долгом. Ваш стартап может намеренно накапливать его в рамках осознанного компромисса, и часть этого компромисса — составление реалистичного плана его погашения. К погашению технического долга следует применять ту же логику, что и к погашению финансового долга: либо выплатить его сразу, потому что у вас есть лишние ресурсы (и нет лучшего применения этим ресурсам), либо погашать его постепенно со временем, либо выплатить всё в будущем, но, возможно, по более высокой общей цене, включающей проценты.

Каким бы образом вы ни решили гасить технический долг, ключ к успеху — признать, что долг является неотъемлемой частью процесса разработки ПО, и что проактивное погашение долга является необходимой инвестицией в общее здоровье инженерии.

Определение технического долга

Технический долг можно определить ещё и как техническое решение, реализацию или нюанс, которые активно снижают эффективность или результативность бизнеса сегодня или в будущем. Суть в том, что у технического долга есть значимое последствие. Часть вашего кода может быть объективно уродливой или неэффективной, но если эта неэффективность не влияет на бизнес и нет необходимости изменять этот код в будущем ради поддержания качества, производительности или итераций, то такой код не несёт издержек с точки зрения технического долга. В конце концов, цель разработки ПО, особенно в стартапе, — не написать идеальную кодовую базу, а создать программное обеспечение, которое поддерживает бизнес.

Не бойтесь долга. Он может служить определённой цели. Например, при создании первой версии продукта, который ещё не проверен на рынке, техническая команда может остановиться на архитектуре, которая не будет масштабироваться выше сотни пользователей. Если это решение позволяет команде быстро проверить продукт и определить, наберёт ли он вообще хотя бы сотню пользователей, такой путь может оказаться оправданным — особенно с учётом того, что для нахождения той версии, которую полюбят пользователи, может потребоваться несколько таких прототипов.

Существует как минимум семь видов технического долга:

1. **Архитектурный долг** (или **долг проектирования**) возникает, когда дизайн ПО не способен удовлетворить ближайшие или будущие потребности бизнеса. Например, дизайн делает слишком сложным создание нужных бизнесу функций или не масштабируется до требуемого числа пользователей либо требований к производительности.

2. **Долг кода** накапливается, когда сама реализация была сделана без внимания к лучшим практикам, что приводит к коду, который трудно понимать и сопровождать.
3. **Долг тестирования** накапливается, когда вы запускаете недостаточно автоматических тестов, чтобы команда была уверена в корректности кодовой базы.
4. **Инфраструктурный долг** возникает, когда инфраструктура, наблюдаемость и поддерживающие системы недостаточно надёжны или плохо обслуживаются, что приводит к сложностям с масштабированием или развёртыванием обновлений либо к низкому времени безотказной работы и надёжности.
5. **Долг документации** возникает, когда документации недостаточно или она устарела/неточна, что может затруднять онбординг участников команды в проект.
6. **Долг навыков** растёт, когда участникам команды не хватает нужных навыков для сопровождения или обновления кода либо окружающей инфраструктуры.
7. **Процессный долг** накапливается, когда команда непоследовательна в подходах к решению задач, что ведёт к ошибкам, задержкам или росту издержек.

Банкротство по техническому долгу

Если ваш стартап достаточно долго пренебрегает техническим долгом или игнорирует его, он может стать серьёзным препятствием для будущего прогресса. Команды могут непреднамеренно обнаружить, что тратят 80 или даже 100 процентов своего времени на разбор системных проблем или неэффективностей, возникших из-за технического долга, — состояние, известное как банкротство по техническому долгу.

Некоторые признаки того, что ваша команда может быть банкротом по техническому долгу:

- Вы регулярно сталкиваетесь со сбоями в продакшене вплоть до ощутимого влияния на бизнес.
- Вы постоянно получаете возражения или завышенные сроки по новым функциям из-за необходимости разбираться с долгом.
- Команда жалуется, что кодовая база слишком сложна, чтобы выполнять работу.
- Новые функции невозможно выпустить, не сломав случайно старые функции или не внося неприемлемо высокий уровень дефектов.

Если вы оказались в состоянии банкротства по техническому долгу, пора бить тревогу, переустанавливать ожидания со стейкхолдерами, разрабатывать план консолидации долга и немедленно начинать его погашение.

Если вы были честны со своими коллегами из руководства (см. «Сообщение плохих новостей», стр. 46), у вас должно хватить авторитета, чтобы объяснить проблему технического долга и сформировать общее понимание окупаемости (ROI) инвестиций в устранение технического долга.

Измерение долга. Инвентаризация долга

В отличие от ипотеки или автокредита, нет сайта, который вы могли бы посетить, чтобы получить выписку о точной величине вашего технического долга и оставшихся платежах. Некоторые формы долга можно измерить количественно, но большая часть анализа носит качественный характер. Для здорового и ответственного управления долгом в масштабе я рекомендую опрос-инвентаризацию долга.

Опрос следует проводить через регулярные интервалы. Где-то от одного до четырёх раз в год проводите трезвый анализ по разным видам долга, давая честную оценку того, в каком состоянии работает команда. Не проводите опрос в одиночку; делайте это в сотрудничестве с другими инженерами команды, которые ежедневно работают в коде и регулярно сталкиваются с долгом.

Опрос может быть очень простым: для каждого из перечисленных ниже видов долга оцените, сколько его у нас, по шкале от 1 до 10, а затем приведите несколько предложений в обоснование оценки.

Используйте результаты опроса, чтобы понять, как вашей команде распределять усилия на погашение долга, и сравнивайте результаты разных опросов с течением времени, чтобы долг оставался на разумном уровне, а команда регулярно решала свои самые болезненные проблемы с долгом.

Стратегии погашения долга

Чтобы решить, когда гасить технический долг, сначала стоит подумать о том, сколько времени команды имеет смысл на него тратить. Ключевые соображения:

- Сколько долга существует, согласно вашему последнему опросу-инвентаризации долга
- Насколько он мешает способности компании работать изо дня в день, то есть через сбои, отток клиентов или уровень дефектов
- Насколько он вредит способности команды поставлять новые проекты. Насколько сложно будет погасить долг

Если вы не находитесь в состоянии банкротства по техническому долгу и ваша цель — поддерживать здоровый уровень долга, я рекомендую выделять где-то от 10 до 20 процентов времени команды на вложения в нефункциональную инженерию (например, погашение долга, исследование новых паттернов / proof of concept, улучшение опыта разработчика и т. д.). Чем серьезнее текущее влияние долга и чем выше трудозатраты на его погашение, тем больший процент времени команды вам следует выделять.

Погашение «точно в срок» (Just-In-Time)

Самый распространённый способ работы с техническим долгом — гасить его по принципу «точно в срок» (Just-in-Time), то есть долг погашается в рамках проекта, продиктованного бизнесом. Часто это выглядит так: команда добавляет задачи (тикеты) по техническому долгу, связанные с историями, выбранными для спринта на встрече по планированию. Это подход с низкими накладными расходами и малыми усилиями на планирование, и он может хорошо себя показать. Но имейте в виду некоторые потенциальные подводные камни:

- Погашение «точно в срок», будучи менее заметным для более широкой команды, может привести к системному недоинвестированию в технический долг. Убедитесь, что вы честны и прозрачны с командой, выполняя погашение «точно в срок», относительно того, какой суммарный процент времени команды вы ожидаете тратить на долг.
- Добавление технического долга в рамках спринта может подразумевать, что вложения в технический долг — вторичная цель по отношению к целям спринта, и потому, скорее всего, будут вырезаны из объёма работ, если у команды кончается время.
- Работа над техническим долгом в рамках спринта может восприниматься как замедление спринта или причина задержек, а не как инвестиция в скорость и общее здоровье системы.

Периодическое погашение

Периодическое погашение похоже на то, как можно гасить автокредит или ипотеку. Команда выделяет место для погашения долга с фиксированным интервалом (например, день в спринт, пара дней в месяц или пара недель в квартал). Google знаменита тем, что позволяла своим инженерам тратить 20 процентов времени на всё, что они хотят, включая погашение долга или инновации над новыми проектами и инструментами. Идея здесь та же: как руководитель, вы явно выделяете время и поощряете команду вкладываться в инструменты и процессы, используемые для инженерии.

Например, метод Shape Up (см. «Технический процесс», стр. 157) описывает двухнедельный период «остывания» (cooldown) после шестинедельного цикла, или 25 процентов восьминедельного периода, для технических вложений. Имейте в виду, что 25 процентов — не волшебное число; правильный процент будет зависеть от инвентаризации долга вашей команды.

Непрерывное погашение

В зависимости от того, насколько дорог долг вашей команде, вы можете захотеть выделить на общее качество системы больше ресурсов, чем позволяет периодическая стратегия. Это выглядит как наличие выделенной команды — то, что я называю клиентским экипажем (customer crew) в сценарии с двумя экипажами (см. «Сопровождение проекта: философия двух экипажей», стр. 113) — которая гасит технический долг как часть своей повседневной работы и целей.

Важно убедиться, что у любой команды, чья основная цель — внутренняя эффективность, например команды по техническому долгу или клиентского экипажа, есть ясные и измеримые цели для своей работы. Например, если ваша инвентаризация долга ставит долг тестирования на первое место, то измеряйте уровень дефектов и покрытие кода и спрашивайте с клиентского экипажа за улучшение этих метрик. Если ваш инфраструктурный долг наибольший, то сосредоточьтесь на метриках времени безотказной работы и среднего времени восстановления (Mean Time to Recovery).

Коммуникация о техническом долге

Нетехнические руководители не ожидают совершенства от своих технических команд. Но они ожидают высокой производительности и стабильно выполняемых ожиданий.

Когда речь идёт о долге, это означает чёткое информирование о вашей стратегии поддержания долга на управляемом уровне, а также заблаговременную и честную коммуникацию о том, когда долг может встать на пути бизнес-целей, а также о вашей стратегии его погашения, чтобы он перестал быть блокером.

Технологическая дорожная карта

Временные горизонты

Я считаю полезным мыслить технологическую дорожную карту (roadmap) в трёх временных горизонтах, иногда обозначаемых как краткосрочный, среднесрочный и долгосрочный, или горизонт один, два и три. Каждый горизонт должен управляться отдельным процессом и часто принадлежит разным стейкхолдерам.

Краткосрочный / Горизонт один

Ваша краткосрочная дорожная карта — это то, над чем команда работает сейчас. Сюда входят любые функции в работе, активно исправляемые дефекты, технический долг или срочные рабочие задачи. Подробнее об управлении краткосрочной дорожной картой см. «Рабочий процесс» в разделе «Технический процесс», стр. 157.

Среднесрочный / Горизонт два

Если вы единственный технический руководитель в команде, то вы отвечаете и за среднесрочную, и за долгосрочную дорожные карты. Если у вас в подчинении есть директора или старшие менеджеры, вы, скорее всего, будете работать над среднесрочной дорожной картой совместно. Среднесрочная дорожная карта — очень полезный артефакт не только для вашего собственного планирования и организации, но и как инструмент коммуникации с другими отделами/стейкхолдерами о том, чем занимается инженерная команда.

Как правило, среднесрочная дорожная карта реализуется в виде таблицы, где команды или отдельные люди — это строки, столбцы — периоды времени (часто недели или спринты), а содержимое таблицы — высокоуровневые рабочие задачи или области работ. Цель дорожной карты — не точно предсказать, когда будет завершена та или иная задача; для этого потребовались бы точные и аккуратные оценки работы, которые в лучшем случае шатки (даже с гранулярностью в недели) и никогда не являются гарантией. Вместо этого идея в том, чтобы очертить порядок выполнения операций и задать направление для команды.

Вы можете и должны ожидать, что фактическая длительность любого действия будет обновляться по мере того, как инженерия продвигается вперёд. Обновление количества недель по конкретной задаче — отличный момент времени, чтобы оценить, имеет ли смысл продолжать вложения в проект, а также чтобы обновить для внешних стейкхолдеров текущие оценки завершения. Наконец, дорожная карта полезна как ретроспективный инструмент для отслеживания того, сколько заняли крупные инициативы, а также для оценки того, куда команда вкладывает время на очень высоком уровне.

Долгосрочный / Горизонт три

Как руководителю своей команды, именно вам предстоит сосредоточиться на долгосрочном здоровье и продуктивности команды. Вам следует уделить время проектированию этих целей и созданию хорошо продуманного, ясного документа (или слайд-презентации, видео, вики-статьи и т. д.), который объясняет цели команде. Установив первоначальные цели, пересматривайте их нечасто, поскольку изменение стратегических целей вызывает турбулентность в организации. Не менее проблематично и то, что частые смены направления могут сбивать с толку и демотивировать команду. Я призываю вас предоставлять обновление о прогрессе по долгосрочным инициативам на ежеквартальной основе — как всей инженерной команде, так и другим руководителям-исполнителям.

Некоторые примеры долгосрочных инициатив:

Архитектурный технический долг

- Переход с устаревшего фреймворка на что-то активно сопровождаемое
- Миграция из одной среды хостинга в другую (например, онбординг в Kubernetes)

Языковой долг

- Консолидация использования языков программирования
- Переход с более старых версий языков на новые (Python 2 на 3 или .NET 4 на .NET 5+)

Внедрение платформы/архитектуры

- Внедрение или миграция нескольких команд на новые версии внутренних сервисов
- Переход к/от serverless-сред
- Внедрение новых парадигм (например, серверный рендеринг, периферийные вычисления)

Планы найма

- Рост или реорганизация команд
- Найм специалистов или создание новых технических отделов

Коммуникация о сроках

Каждый руководитель в продажах, маркетинге, продукте или поддержке, с которым я работал, ценил прозрачность технического процесса и технической дорожной карты. Напротив, я общался с руководителями в некоторых компаниях, которые описывают свои технические команды как чёрную дыру. Само собой разумеется, что вы не хотите, чтобы вас называли чёрной дырой. Не быть чёрной дырой просто: это выглядит так, что где-то в вашей организации существует регулярный процесс обеспечения прозрачности для других руководителей. В идеале вы также помогаете другим отделам чувствовать, что их слышат, имея форум или механизм для приёма входных данных и включения их в процесс формирования дорожной карты. Вы можете и должны замыкать цикл с другими отделами в своём процессе, сообщая стейкхолдерам обратно, на какой стадии разработки находится их запрос, и управляя ожиданиями относительно того, когда он будет завершён.

Технический процесс

Закон Конвея гласит, что организации, проектирующие системы, вынуждены создавать дизайны, которые являются копиями коммуникационных структур этих организаций. Иными словами, то, как вы структурируете свои команды, и, что важно, процесс работы внутри и между этими командами, окажет существенное влияние на продукт, который вы создаёте. Команды, работающие в информационных силосах, вряд ли создадут продукты, которые красиво интегрируются с дизайнами другой команды, поэтому именно вам, как зрителю и конечному архитектору этих коммуникационных структур, предстоит обеспечить, чтобы эти структуры отвечали потребностям разрабатываемого вами продукта.

Рабочий процесс

Техническая работа — крайне многогранное дело с тысячами мельчайших решений, которые повлияют на то, как в итоге всё будет взаимодействовать и вести себя. Чтобы иметь хоть какую-то надежду на поддержание продуктивности в вашей организации, вам нужен набор стандартов и направляющих принципов, гарантирующих, что повседневные технические решения в целом согласованы и потому управляемы для команды. Это значит, что вам нужно реально установить эти стандарты, обучить им команду и иметь повседневный процесс для их соблюдения и изменения по мере необходимости.

Паттерн, которому следует команда, чтобы определить, как решать, что строить и как выполняется работа, называется рабочим процессом (workflow). Пять самых популярных паттернов рабочего процесса:

- Agile
- SCRUM
- Kanban
- Waterfall (каскадная модель)
- Shape Up

Об этих паттернах написаны целые книги, и моя любимая — *Scrum: The Art of Doing Twice the Work in Half the Time* Джеффа Сазерленда (Jeff Sutherland).

У этих подходов есть некоторые фундаментальные сильные и слабые стороны, которые я обсужу в этой главе; однако в реальном мире различия между процессами меркнут по сравнению с влиянием того, насколько хорошо руководитель реализует выбранный процесс. Ваша задача как технического лидера — выбрать процесс и обеспечить, чтобы он был хорошо реализован и итеративно улучшался.

Хороший процесс разработки уважает следующие прописные истины о разработке ПО:

- Никто не может идеально предсказать, сколько времени займёт выполнение той или иной инженерной задачи.
- Инженерия редко идёт по прямой линии; создание функции X может потребовать времени на решение проблемы Y, прежде чем X можно будет построить.
- Не существует такого понятия, как идеальная спецификация; всегда есть пробелы и вещи, которые предстоит обнаружить по ходу создания технологии.

В целом цель процесса рабочего потока — обеспечить, чтобы команда была хорошо организована и поставляла результат в приемлемом темпе. Окольным путём некоторые процессы рабочего потока даже пытаются количественно оценить скорость инженерной команды (velocity), позволяя отчитываться нетехническим стейкхолдерам о том, как скорость меняется с течением времени.

Waterfall (каскадная модель)

Самый старый процесс рабочего потока, восходящий к 1950-м годам, — это waterfall, каскадная модель (см. cto.hb.com/waterfall). Каскадная модель разбивает деятельность по проекту на последовательные шаги, где каждый шаг зависит от предыдущего и начинается после его завершения. В разработке ПО это выглядит примерно так: сначала формируется видение продукта, затем выполняется концептирование продукта, затем дизайн продукта, затем разработка ПО и, наконец, тестирование, развёртывание и сопровождение. Самая распространённая критика waterfall в том, что эта структура жёсткая, негибкая и не способствует итеративной разработке.

Agile/Scrum

Процесс Agile и SCRUM — более тонкая и предписывающая методология, чем waterfall. Существует множество отличных ресурсов, которые подробно раскрывают эти нюансы, в том числе *Scrum* Сазерленда, *Agile Estimating and Planning* Майка Кона (Mike Cohn), *The Art of Agile Development* Джеймса Шора (James Shore) и Шейна Уордена (Shane Warden).

Главное, что нужно осознать об этих процессах, — это то, что они являются руководствами, а не писанием. Чтобы получить максимум от вашей инженерной команды, начните с какого-то процесса и посмотрите, насколько хорошо он работает для вашей конкретной группы людей с вашим конкретным типом технических задач. У некоторых команд работа гораздо лучше поддаётся оценке и присвоению story points, тогда как у других гораздо более неоднозначные brownfield-проекты, где оценка почти невозможна. Обращайте внимание на то, действительно ли какая-либо конкретная церемония из процесса добавляет ценность инженерной команде, или же это просто длительная встреча, которой все боятся.

Не стесняйтесь пропускать церемонии, которые не являются очевидной пользой для команды. Например, я считаю предписание SCRUM по planning poker неэффективным для большинства команд.

Shape Up

Shape Up — это методология, формализованная компанией Basecamp и опубликованная в электронной книге, доступной на basecamp.com/shapeup. Основной цикл в Shape Up длится шесть недель — гораздо более длинный спринт, чем проповедует SCRUM. Этот цикл использует фиксированное время и переменный объём работ (score). Идея в том, что более длительный период времени даёт больше пространства для подготовки чётких питчей (спецификаций) и хорошей работы над проектом. Shape Up уделяет значительно меньше внимания оценке, чем другие модели, что, как я вскоре обсужу, для инженерных команд является благом.

Инженерные оценки

Согласно Google, техническое определение точности (accuracy) — это степень, в которой результат измерения, вычисления или спецификации соответствует правильному значению или эталону. То есть точность (accuracy) — это показатель общей корректности. Когда вы бросаете дротики в мишень, целясь в яблочко, точный (accurate) набор бросков — это набор бросков, который тяготеет к центру.

Прецизионность (precision), напротив, технически определяется как степень детализации в измерении, вычислении или спецификации, особенно выраженная количеством приведённых цифр. Иными словами, прецизионность указывает на уровень точности воспроизведения. При бросании дротиков, если все ваши броски, независимо от их цели, плотно сгруппированы вместе, это можно назвать прецизионной (точно воспроизводимой) группировкой.

Как должно помочь вам визуализировать это описание, нечто может быть точным (accurate) без прецизионности (широкая группировка дротиков вокруг или рядом с яблочком, но не попадающая в него), прецизионным без точности (плотная группировка дротиков, которая промахивается мимо яблочка) и, конечно же, одновременно точным и прецизионным (плотная группировка дротиков, которая попадает в яблочко).

Вам следует ожидать от своей команды и спрашивать с неё точные (accurate), но не обязательно прецизионные оценки выполнения задач разработки ПО. Если сегодня первое число месяца, разумным ориентиром от вашей команды будет: «Мы выпустим функцию в этом месяце». Если команда говорит: «Мы выпустим функцию 23-го», она с большей вероятностью пропустит этот срок.

Нет необходимости пытаться оценивать часы или дни на тикет, если вы планируете работу/распределение ресурсов по неделям, месяцам или кварталам. Со временем обращайте внимание на то, действительно ли ваши оценки дают вам ту способность планировать, на которую вы надеетесь. Если нет — не наказывайте команду продолжением процесса или, что хуже, использованием этого как фактора в performance review. Вместо этого корректируйте оценки так, чтобы они помогали, а не вредили вам. Меняйте свои ожидания и вместо того, чтобы реагировать на пропущенные оценки, реагируйте на трудности, с которыми сталкивается команда, пытайтесь уложиться в оценки.

Последнее замечание об оценках: не путайте пропущенные оценки с низким общим выходом результата/скоростью. Некоторые команды будут очень эффективны, иметь высокий выход результата и всё равно промахиваться по оценкам. Скорость (velocity) — более важная метрика, и команду с высоким выходом, но неидеальными оценками не следует наказывать. И наоборот, команда, которая регулярно промахивается по оценкам и испытывает трудности с поставкой новой ценности, недорабатывает и должна измениться.

Диаграммы сгорания (Burndown Charts)

Диаграммы сгорания (burndown charts) в SCRUM показывают прогресс команды относительно оценок и могут быть отличным инструментом для измерения продуктивности спринта. Однако оценки несовершенны, и по разным причинам диаграмма сгорания может показывать ровную линию или даже «сгорать вверх». Это может быть связано с тем, что команда действительно не делает прогресса, либо может быть артефактом проблем с оценкой или плохого сбора данных.

Диаграмма сгорания, которая стабильно «сгорает вверх», несмотря на то что команды выпускают результат и делают хорошую работу, деморализует и не достигает задуманной пользы. Если есть простые корректировки, которые помогут вам лучше собирать данные и исправить диаграмму, внесите это изменение. Но если вы обнаруживаете, что конкретный способ измерения выхода результата всё равно не работает, просто избавьтесь от него. Это нормально — признать, что ваш метод оценки этих конкретных типов историй с этой командой недостаточно прецизионен, и перейти к другим методам мониторинга и улучшения производительности.

По моему опыту, лишь небольшой процент команд добивается успеха с диаграммами сгорания, так что не унывайте, если именно эта техника не помогает вашей инженерной команде.

Выбор рабочего процесса

Я утверждаю, что выбор рабочего процесса не будет значимым фактором конечного успеха и скорости вашей инженерной команды. Ключевой фактор в том, что вы уделяете внимание рабочему процессу команды и непрерывно итеративно улучшаете сам рабочий процесс, чтобы убедиться, что ваши паттерны добавляют ценность и хорошо подходят как вашей команде, так и типам проблем, с которыми она сталкивается.

При этом вот грубая модель для размышлений о том, какой тип рабочего процесса, скорее всего, будет лучшей отправной точкой: хорошо понятная работа (то есть задачи конкретные, greenfield и легко объяснимые) проще в управлении и принесёт больше пользы при более тонком или предписывающем процессе планирования. Иными словами, если ваша работа неоднозначна и плохо поддаётся оценке, ею, скорее всего, лучше управлять с помощью Kanban, чем SCRUM.

Хорошо понятные истории, которые обычно хорошо работают со SCRUM, это:

- Greenfield, то есть новый код, который не зависит, возможно, от легаси- или сложных в работе внешних модулей
- Не зависят от новых паттернов/инструментов/технологий, опираясь вместо этого на существующий (скучный) технологический стек
- Легко разбиваются из историй на более мелкие задачи. Знакомы команде по предыдущей работе

И наоборот, вам, возможно, лучше подойдёт Kanban, если ваша работа:

- Brownfield или сильно затруднена техническим долгом с неясными путями погашения/рефакторинга
- Регулярно меняется или обременена непредсказуемыми приоритетами
- Зависит от внедрения новых и непривычных инструментов и паттернов, которые могут привести неожиданные издержки в первых нескольких реализациях
- Поручена совершенно новой команде, у которой нет истории совместной работы или работы над такими типами проектов

Спринты остывания / инноваций

При использовании регулярной каденции, такой как спринты, главная опасность, в которой оказываются команды, — это ожидание того, что спринт завершится, функции будут выпущены, и команда сможет немедленно переключиться на следующий набор функций. Из-за накопления долга и необходимости итераций над функциями продукта поддерживать такое попросту невозможно. Должно быть выделено либо непрерывное, либо периодическое время на оплату долга.

Распространённая практика периодического погашения долга — понятие спринта остывания (cooldown). Иногда его называют спринтом технического долга или инновационным спринтом, но идея одна и та же: дать команде время навести порядок в своём цифровом рабочем пространстве, провести «уборку» кода и убедиться, что они и код находятся в хорошем состоянии для высокоскоростной работы в дальнейшем. Как обсуждалось в разделе «Стратегии погашения долга», стр. 150, разумно выделять от 5 до 20 процентов всего времени разработки на работу по остыванию.

Если вы делаете двухнедельные спринты, это может означать, что один из четырёх или пяти спринтов посвящён остыванию.

Техническое планирование и спецификации

Когда я обсуждаю с инженерами процесс написания технических спецификаций, меня часто спрашивают: «Как вы находите время писать спецификации?» Обычно я парирую: «Как вы находите время *не* писать технические спецификации?» В моём ответе подразумевается, что время, потраченное на продумывание того, что вы создаёте, прежде чем это создавать, в итоге экономит время.

Разработка ПО по своей природе является творческим процессом, то есть мы не делаем одно и то же каждый раз, и существует более одного способа реализовать любую конкретную историю. Хороший процесс планирования признаёт, что есть ценность в продумывании истории заранее, но уравнивает этот акцент на предварительном планировании пониманием того, что единственный реальный способ по-настоящему узнать всё о функции — это действительно её построить.

Хороший процесс технического планирования может достичь нескольких целей:

- Уменьшить объём переделок в функции
- Выявить способы сделать меньше работы для достижения той же функциональности
- Снизить вероятность того, что будут забыты важные, но не видимые бизнесу соображения, такие как обработка ошибок / негативные сценарии, тестирование, логирование, мониторинг, аналитика,

безопасность, масштабируемость, планирование запуска и погашение технического долга

- Повысить вероятность того, что работа нескольких людей/команд будет выполнена совместимым образом
- Предоставить ценную документацию о том, как и почему функция была построена определённым образом, для будущего сопровождения, улучшения или расширения
- Держать команду вдумчивой и согласованной по необратимым/дорогим техническим соображениям (например, инструментарий и архитектура), а также по ключевым деталям, которые маловероятно забыть
- Оказаться достаточно легковесным, чтобы его можно было завершить в разумные сроки и чтобы он не вынуждал принимать решения по второстепенным деталям, которые либо не имеют значения, либо по которым недостаточно информации заранее

Ведущие технической спецификации

Я рекомендую назначать ведущего (lead) для любого проекта, который нуждается в планировании: одного человека, ответственного за создание технической спецификации и проведение этой спецификации через ваш процесс согласования.

Это не значит, что он единственный участник. Напротив, если другие участники команды доступны в окне планирования и обладают полезными знаниями, они могут и должны вносить вклад.

Планирование может быть синхронным (то есть все находятся в одной комнате в течение всего периода времени) или асинхронным. Я рекомендую асинхронное планирование, так как большая часть работы при планировании будет включать исследование (например, чтение продуктовой документации, чтение кода, прототипирование / proof of concept, оценку инструментов и API и т. д.), которое прекрасно можно делать самостоятельно.

Время на планирование

Ваш процесс технического планирования должен экономить вам время на начальной реализации. Он также должен экономить время в будущем за счёт минимизации технического долга и оставления после себя документации, которая может ускорить будущие улучшения.

Неправильный объём времени, вкладываемый в планирование, не отвечает этим целям — либо потому что он слишком короткий и не экономит вам время / не создаёт хорошей документации, либо настолько длинный, что не окупается экономией.

Универсальной формулы для правильного объёма времени не существует, но я приведу эмпирическое правило: выделяйте один день на техническое планирование на каждую неделю работы, которую, по вашей оценке, займёт проект. В общем случае это приведёт к окнам планирования от полудня до трёх дней. Если ваш проект требует менее двух дней работы, ему, скорее всего, нужны очень минимальные усилия на планирование и у него низкий риск. И наоборот, если вы смотрите на проект, который, как ожидается, займёт более трёх полных недель усилий на разработку, вы, возможно, не сможете эффективно спланировать что-то столь крупное всё сразу и вам стоит подумать о его декомпозиции.

Если ваша команда отказывается вкладывать время в планирование, вы, вероятно, слишком сильно давите на результаты в ущерб процессу. Способ обеспечить, чтобы инженерный процесс давал хорошие результаты для бизнеса, — не «щёлкать кнутом» сильнее, а выстроить здоровый процесс, который позволяет достигать хороших результатов. Вы не ускорите команду инженеров-строителей, проектирующую мост, заставив их работать дольше. Вы убедились бы, что у них есть лучшие доступные инструменты проектирования мостов с наилучшей возможной информацией о пролёте, который нужно перекрыть. Разработка ПО ничем не отличается. Только вместо CAD-софта или реальных измерений грунта/породы у нас есть продуктовые спецификации, процесс дизайна и программные инструменты.

И наоборот, чрезмерно длительный процесс планирования, при котором участники команды настаивают на получении каждой мельчайшей детали заранее, может быть индикатором серьёзной культурной проблемы, когда участники команды парализованы страхом совершить ошибку. Эффективное планирование не устранил риск, но продумывание важных, высокоуровневых решений заранее может его минимизировать. Команда, одержимая деталями, может бояться совершать ошибки или не желать итерировать свою работу — оба симптома чрезмерно ориентированного на результат управления. Людей не следует наказывать за разумные ошибки или упущения в планировании. Это нормально, если техническая спецификация не идеальна с самого начала; ожидайте, что ваша

команда найдёт ошибки или пробелы во время реализации, и обновляйте спецификацию, когда эти проблемы обнаруживаются.

Прототипирование как часть написания спецификации!

Нередко при написании технической спецификации у вас есть несколько вариантов достижения цели, и ни один из них на бумаге не выглядит убедительно лучше остальных. Или вы обнаруживаете неизвестные факторы, влияющие на эффективность конкретного варианта, из-за чего решение становится неоднозначным. Если есть возможность — особенно если это можно сделать быстро, — я призываю вас давать своим инженерным командам пространство для прототипирования одного или нескольких из этих решений, чтобы получить данные и принимать более качественные решения на этапе планирования. Полдня, потраченные на создание игрушечного образца с новым инструментом, чтобы заранее убедиться, что инструмент даст желаемый результат, — это полдня, потраченные с пользой.

Содержание технической спецификации

Наличие шаблона, который ваша команда использует, приступая к написанию технической спецификации, — отличный способ ускорить процесс и гарантировать, что важные темы не будут упущены. Я рекомендую, чтобы ваш шаблон представлял собой прежде всего последовательность заголовков с тематическими разделами, которые нужно раскрыть, возможно, с небольшими инструкциями или напоминаниями для авторов спецификаций. Я приложил пример технической спецификации на (cto.hb.com/templates) [<https://cto.hb.com/templates>].

Небольшое отступление, прежде чем перейти к содержанию: технические документы могут быть несколько сухими и серьёзными. Если это вписывается в вашу культуру, я призываю вас добавлять немного лёгкости там, где это уместно и не отвлекает. Хороший пример — остроумный мем в начале документа, отсылающий к теме спецификации. По моему опыту, достаточно, чтобы менеджер/руководитель один раз вставил мем в спецификацию, чтобы воодушевить остальных членов команды (читай: открыть шлюзы) добавлять свои.

Несколько рекомендуемых компонентов для включения в шаблон:

- Напоминание о том, что документ на самом деле является шаблоном, и авторам следует сделать копию, прежде чем приступать к написанию (эту ошибку легко допустить!)
- Указания о том, как следует подходить к спецификации / ссылка на корпоративные рекомендации по составлению спецификаций и процессы их утверждения
- Раздел с предысторией, объясняющий бизнес-обоснование проекта
- Любые особенно выделяющиеся области технического риска данного проекта (например, он затрагивает чувствительные персональные данные (PII) или предполагает использование ранее не применявшихся инструментов/архитектуры)
- Глоссарий/определения любых неочевидных терминов
- Любые явные бизнес-цели, которые спецификация стремится достичь / корреляция с ранее заявленными целями (т. е. квартальными KPI или OKR)
- Обзор архитектуры решения (основная часть документа)
- Технический долг — отдельно обсудите, почему вы будете или не будете устранять любой необходимый/смежный долг
- Моделирование данных, включая необходимые изменения в базе данных или конвейерах данных
- Требования к внутренней и внешней отчётности или аналитике и измерениям
- Тестирование
- Развёртывание
- Переключатели/флаги функциональности (feature toggles/flags)
- Влияние на общую надёжность системы или аварийное восстановление. Безопасность и приватность
- Контрольные точки поставки результатов

Утверждение технической спецификации

Чтобы достичь цели обеспечения согласованности и единого направления между проектами и членами команды, вы должны обеспечить, чтобы члены команды читали процессы планирования друг друга и вносили в них вклад. Чтобы добиться этого, я рекомендую внедрить облегчённый процесс утверждения спецификации, прежде чем её можно будет считать завершённой.

Цели ревью

Цели ревью вашей технической спецификации должны включать следующее:

- Обеспечить, чтобы все команды/проекты придерживались единого технического направления и строили систему согласованно.
- Провести ревью важных концепций данных/технических контрактов и обучить им команду.
- Обеспечить всеобщее и единое понимание решаемой задачи.
- Минимизировать вероятность упустить важные граничные случаи или другие требования, не видимые с точки зрения бизнеса.

Процесс ревью

Для облегчённого процесса ревью я рекомендую асинхронное обсуждение в документе с последующей синхронной встречей по разрешению разногласий (подробнее о встречах по разрешению разногласий см. «Встречи и управление временем», с. 28). Автор технической спецификации, как только он добился определённого прогресса по ключевым элементам проекта, должен разослать документ другим инженерам, обладающим достаточным контекстом. Идея в том, чтобы остальные прочитали документ и оставили комментарии и вопросы в удобное для них время. Многие из этих вопросов могут быть быстро и асинхронно решены ведущим автором, но некоторые могут быть спорными или крайне нюансированными, требуя более интенсивного общения.

Чтобы завершить процесс, автор должен назначить встречу, участниками которой будут только те, кто прочитал документ и заранее внёс свой вклад. Цель встречи — рассмотреть открытые вопросы и разногласия и прийти к разрешению. Цель встречи — не в том, чтобы автор просто зачитал спецификацию вслух перед скучающей или незаинтересованной аудиторией. Если есть открытые вопросы, требующие дополнительной проработки для изучения и разрешения, то сделайте это офлайн и затем обсудите результаты только с заинтересованными сторонами.

Как только все открытые вопросы разрешены, задокументируйте, кто внёс вклад в спецификацию (чтобы будущий читатель знал, кому адресовать дальнейшие вопросы), и считайте документ утверждённым.

Руководители на встречах по ревью спецификаций

Технический руководитель или менеджер не обязан быть тем, кто утверждает все технические документы. Я призываю вас выстраивать культуру, в которой команда в целом чувствует себя в безопасности, внося вклад, и не полагается на вас как на источник технических ограничений или поддержки. На раннем этапе, когда отдел относительно небольшой, вам следует активно участвовать в большинстве или во всех спецификациях, но такой подход не масштабируется. Как только вы наймёте других старших индивидуальных контрибьюторов (IC), архитекторов или менеджеров, наделите их полномочиями быть ведущими ревьюерами и доверяйте им, позволяя выполнять ту работу, ради которой вы их наняли. Если вы замечаете, что старший сотрудник плохо направляет команду на этих ревью, не топчите его на публичной площадке. Обсудите ситуацию и скорректируйте курс наедине.

Технические спецификации как документация

Ваша техническая команда теперь тратит время на создание продуманных документов, описывающих, как вы проектируете свой продукт, и команда в результате должна совершать меньше ошибок. Последний способ, которым технические спецификации помогают вам, — это предоставление полезных ресурсов для будущих инженеров, которым нужно дополнить или изменить уже проделанную работу. Я рекомендую создать хорошо организованный каталог с возможностью поиска (например, внутренняя вики, такая как Confluence или Notion, или хранилище документов, такое как Google Drive), и чтобы ваша команда добросовестно следила за тем, чтобы все документы со спецификациями попадали в этот каталог. Также может быть полезно ссылаться на спецификацию в комментариях к коду, чтобы объяснить, почему что-то реализовано именно так.

Developer Experience (DX)

Компания Harness (harness.io), занимающаяся инструментами DevOps, определяет Developer Experience (DX) как совокупность взаимодействий и ощущений, которые разработчик испытывает, работая над достижением цели. Это

определение похоже на определение User Experience (UX), за исключением того, что в данном случае основным пользователем является инженер-программист.

Качество developer experience не всегда можно измерить на дашборде, но когда оно спроектировано плохо, команда это чувствует и может громко на это жаловаться. Плохой developer experience способен выбить инженера из колеи на целый день: например, попытка запустить микросервис для его тестирования выдаёт загадочную трассировку стека (traceback), а сопровождающий сервис разработчик в отпуске, поэтому инженер среднего уровня часами буксует на месте, просто пытаясь добраться до надёжного цикла «сборка — выполнение — тестирование».

Умножьте эту неэффективность на всех инженеров вашей команды и на все разнообразные типы репозиторий, сервисов и проектов, существующих в вашей компании, и это может быстро вылиться в потерю человеко-месяцев продуктивности прямого времени. Добавьте дополнительное время на переключение контекста, потраченное на привлечение других людей для помощи в решении проблем, и плохой DX быстро поднимется в верхнюю часть списка областей, которые, оставаясь без внимания, могут потопить в остальном высокоэффективную инженерную команду.

Существует две предпосылки для отличного developer experience:

1. Инструменты, которые упрощают создание высоконадёжных и воспроизводимых окружений и цепочек зависимостей
2. Документация и единообразие практик в том, как делаются те или иные вещи

К счастью, сегодня множество легкодоступных инструментов и экосистем помогают с пунктом №1. У большинства языков программирования есть экосистема со стандартизированными инструментами для управления зависимостями и воспроизводимых окружений. Вам остаётся лишь выявить их и использовать (например, npm, pipfile и т. д.). Многие из этих систем создают файл, называемый lock-файлом (lock file).

Lock-файл предназначен не для управления конкурентным доступом во избежание взаимоблокировок (deadlocks); он создан для того, чтобы зафиксировать конкретный экземпляр графа зависимостей. Вам следует коммитить эти lock-файлы и следить за тем, чтобы другие разработчики и любые системы сборки их использовали. Lock-файл помогает гарантировать, что все в команде установили идентичный набор зависимостей.

Если выбранный вами язык программирования не предоставляет таких инструментов, то обеспечение воспроизводимости ложится на вас — возможно, с помощью docker-контейнеров, makefile-ов и тому подобного.

Зачастую разница между хорошим DX и плохим DX — это двадцать или тридцать минут предварительных усилий со стороны кого-то, кто знаком с кодовой базой.

Не нужно много времени, чтобы убедиться, что базовые команды сборки работают на чистой установке и что эти команды задокументированы в локальном README.

Одна из возможностей для вас как СТО облегчить это — обеспечить, чтобы команды сборки, используемые в разных репозиториях и кодовых базах вашей компании, были единообразными. Может быть, это всегда docker compose up или всегда yarn run. Что бы это ни было, любой разработчик должен иметь возможность сделать git clone любого репозитория, и тогда первая же команда, которая приходит на ум для сборки и запуска ПО, должна сработать.

Приоритизация developer experience

Всё, что не входит в продуктовую дорожную карту (roadmap), может быть сложно приоритизировать.

К счастью, DX редко требует достаточно крупных вложений времени, чтобы его нужно было сортировать по приоритетам в дорожной карте. На раннем этапе развития компании я предпочитаю следовать «правилу бойскаута» — оставляя кодовую базу (или developer experience) лучше, чем ты её застал. Каждый раз, когда разработчик сталкивается с проблемой при сборке, запуске или тестировании чего-либо, его обязанность — исправить, задокументировать или иным образом обеспечить, чтобы тому, кто придёт к этому коду следующим, было легче.

По мере роста систем приведение всего в рабочее состояние локально для совместного тестирования функциональности может становиться всё более масштабной рутинной задачей. На этом этапе, возможно, стоит более формально инвестировать в DX в рамках дорожной карты или даже выделить отдельную штатную единицу, чтобы обеспечить работоспособность инструментов и чтобы разработчики не теряли большие куски времени на борьбу с системой вместо написания продуктивного кода.

Лёгкие победы в developer experience

Вот несколько лёгких побед для улучшения DX по всей вашей команде разработки ПО:

- Заведите файл README с инструкциями по запуску кодовой базы — в идеале одну команду для установки зависимостей, а затем сборки и запуска кода.
- Требуйте, чтобы весь код проходил линтинг по строгому набору правил линтинга, единому для всех случаев использования этого языка в вашей компании. Заваливайте сборки, если линтинг не проходит. Если у всех разработчиков IDE настроена на автоматический линтинг, сборки должны редко падать из-за проблем с линтингом.
- Обеспечьте, чтобы конфигурация линтинга по возможности была помещена в систему контроля версий (т. е. вложив усилия в настройку чего-то вроде файла settings.json в VSCode, который можно найти на [cto.hb.com/vscode](https://code.visualstudio.com/docs/editor/settings)).
- Вложите время в обеспечение того, чтобы локальные тестовые данные можно было настроить в локальных базах данных с нуля. Зачастую быстрый генератор данных или скрипт для заполнения начальными данными (seed data) может избавить от множества головных болей разработчиков. Ещё лучше, если начальные данные можно легко дополнять, добавляя новые граничные случаи/сценарии использования по мере развития системы, чтобы базовый набор тестовых данных был как можно более полным/репрезентативным.
- Разработайте план, как при необходимости либо мокать, либо реально поднимать зависимые сервисы локально для тестирования взаимодействий между несколькими сервисами. В идеале, при хороших контрактах и предметно-ориентированном проектировании (domain-driven design), потребность в этом будет редкой, хотя это всё равно должно быть несложно, когда это необходимо.

Смена инструментов ради developer experience

В 2022 году Stripe, финтех-декакорн (т. е. компания, оценённая более чем в 10 миллиардов долларов), решила, что Flow, использовавшийся на тот момент язык программирования, стал слишком дорогим в использовании. Он потреблял слишком много памяти, подвешивал ноутбуки и плохо интегрировался с IDE разработчиков.

TypeScript, как и Flow, — это язык с опциональной типизацией, построенный поверх JavaScript. TypeScript получил гораздо более широкое распространение, чем Flow, и тем самым решил многие проблемы, с которыми команды Stripe сталкивались при работе с Flow, с которым со временем становилось всё болезненнее работать. Становилось всё очевиднее, что TypeScript предлагает значительное улучшение DX по сравнению с Flow. Единственная проблема: как конвертировать миллионы строк кода с одного языка на другой?

Ответом, как выяснилось, стал восемнадцатимесячный проект команды инженеров по подготовке к единственному, гигантскому merge-коммиту, чтобы обновить всю кодовую базу разом. В воскресенье, 6 марта 2022 года, масштабное изменение Stripe приземлилось, а в понедельник, 7 марта, команда вернулась к работе и начала использовать новый язык программирования. Один из разработчиков описал это изменение как самый большой прирост продуктивности разработчиков за всё его время в Stripe.

Урок здесь в том, что если боль от плохого developer experience достаточно сильна, то почти никакая цена не будет слишком высокой и никакой проект не окажется недостижимым, чтобы добиться улучшений. Ваша команда почти наверняка меньше, чем у Stripe, и вы вряд ли имеете дело с миллионами строк кода, но применима та же логика: если ваша команда сталкивается с трением в DX, которое её замедляет, вы должны вложить необходимое время и усилия разработчиков, чтобы улучшить его и вернуть эту эффективность.

Ещё одна проблема, с которой часто сталкиваются команды, — слишком частая смена инструментов. В определённых технических экосистемах (особенно в мире JavaScript) кажется, что каждый месяц выходит что-то новое и блестящее, что могло бы дать прирост продуктивности вашей команде. Я призываю вас быть дисциплинированными в отношении внедрения новых инструментов, убедиться, что вы потратили время на то, чтобы действительно понять существующую боль, тщательно изучить новый инструмент, проверить, отвечает ли он всем вашим требованиям, а не только блестящему заголовку, и принимать решения соответственно. Подробнее о рекомендуемом мной процессе см. «Внедрение внутреннего технологического радара», с. 204.

Техническая архитектура

Одна из ваших ключевых обязанностей как технического лидера — принимать удачные решения относительно архитектуры и инструментов. Хорошая архитектура согласует сильные стороны выбранных вами инструментов и паттернов с потребностями вашей организации сейчас и в обозримом будущем. Это требует понимания сильных сторон, слабостей и компромиссов, присущих каждому выбору. Моя цель в этом разделе книги — в целом ознакомить вас с ландшафтом вариантов в различных областях и помочь вам распознать общие компромиссы, которые влекут за собой разные стратегии.

Одну вещь стоит держать в голове при обсуждении инструментов и их выбора с вашей командой: инженеры могут эмоционально относиться к выбору инструментов. Инструменты оцениваются как хорошие и плохие, у людей есть личные симпатии, антипатии и предубеждения. Как лидер и тот, кто принимает решения, я настоятельно предостерегаю вас от принятия такой манеры выражаться при обсуждении инструментов. Это не только потенциально способно оттолкнуть членов команды, если вы пренебрежительно отзываетесь об их личном любимом инструменте; это ещё и непродуктивно и может отвлекать от цели — найти хорошее решение вашей задачи.

Некоторые отдельные инструменты действительно плохо спроектированы и затмеваются более совершенными альтернативами. Однако чаще всего более нюансированная оценка покажет, что данный инструмент не плох по своей сути, а скорее подходит или не подходит для конкретной компании или проекта. Не позволяйте одному неудачному прошлому опыту попытки использовать инструмент, неподходящий для решения одной задачи, помешать вам или вашей команде использовать его в другой раз, когда он может оказаться более подходящим.

Архитектура

Существует множество прекрасных ресурсов, которые глубоко исследуют различные архитектурные паттерны; один из моих любимых — книга Мартина Фаулера (Martin Fowler) *Patterns of Enterprise Application Architecture*. В этой главе я приведу краткий обзор некоторых ключевых терминов, которые вам встретятся, чтобы у вас был контекст при углублённом изучении этих тем в других источниках.

Предметно-ориентированное проектирование (Domain-Driven Design)

Предметно-ориентированное проектирование (domain-driven design, DDD) — это подход к разработке ПО, который фокусируется на понимании и моделировании предметной области задачи, чтобы проектировать более качественные программные решения.

Основные понятия DDD включают:

Доменная модель (Domain model): представление бизнес-концепций в виде объектов в вашей технической системе;

Единый язык (Ubiquitous language): общий, согласованный словарь и язык, используемый по всей компании, чтобы минимизировать путаницу;

Ограниченный контекст (Bounded context): граница, в пределах которой применяется доменная модель и используется единый язык.

Высокоуровневые паттерны

Когда кто-то употребляет фразу «техническая архитектура», обычно имеется в виду, как выполняется код и как информация перемещается внутри системы. В большинстве описаний архитектуры встречаются термины «сервисы», «монолиты» или «транспорты сообщений». Это контрастирует с паттернами написания кода (coding patterns), в которых часто фигурируют такие термины, как «объектно-ориентированное программирование», «функциональное программирование» или «внедрение зависимостей» (dependency injection). Паттерны написания кода иногда называют архитектурой кода, и они обсуждаются в разделе «Паттерны написания кода», с. 188.

Решение с наибольшим влиянием в технической архитектуре — будет ли код выполняться как монолит или как набор сервисов (обычно называемых микросервисами). Я начну с описания того, как выглядит каждый паттерн, а затем дам некоторые рекомендации по компромиссам между ними.

Монолитная архитектура

Паттерн монолитной архитектуры — это такой паттерн, при котором весь код выполняется как единый процесс, а информация перемещается между частями вашей системы полностью в памяти, моделируясь как простые вызовы функций. Если вы когда-нибудь сажались и собирали простое приложение за один день, велика вероятность, что оно относилось бы к категории монолита. Монолиты бывают всех форм и размеров — от очень маленьких до огромных, многомиллионных по строкам проектов.

Ключ к построению успешного монолита — тщательно проектировать потоки данных внутри приложения, используя предметно-ориентированное проектирование. Это довольно легко измерить: вы хотите обеспечить, чтобы, когда разработчик собирается изменить функциональность приложения, было очевидно, в каком месте монолита ему следует работать. Ему должно понадобиться менять код только в очевидной и чётко определённой или ограниченной области для достижения своей цели. Каждая дополнительная область кодовой базы, которую нужно менять для выполнения функционального требования, добавляет дополнительную сложность или возможность ошибки и в целом замедляет разработку.

Ключевые особенности монолита:

- Код развёртывается как единое целое.
- Код управляется в едином репозитории исходного кода.
- Развёрнутый код масштабируется как единое целое вверх и вниз.
- Информация перемещается между частями системы в памяти, обычно с помощью вызовов функций.
- Предметно-ориентированное проектирование и чёткое проектирование потоков информации не навязываются системой, оставляя задачу хорошего проектирования на усмотрение инженеров.

Сервис-ориентированная архитектура (SOA)/микросервисы

Фраза «сервис-ориентированная архитектура» (service-oriented architecture, SOA) возникла в 1990-х и используется для обозначения некоторых довольно конкретных технологических решений. В наши дни эта фраза используется более широко — для описания системы, в которой информация перемещается между частями системы по сети. Основным компромисс SOA в том, что по сравнению с монолитом её может быть очень сложно осмыслить, и она требует от команды значительной настройки и продуманного проектирования, чтобы по-настоящему гарантировать, что выгоды перевешивают добавленную сложность.

Микросервисы — это подмножество сервис-ориентированной архитектуры, где каждый сервис, как следует из названия, очень мал. Существуют реализации систем с тысячами микросервисов, каждый из которых состоит всего из нескольких строк кода. При этом вам не нужно иметь тысячи микросервисов, чтобы ощутить преимущества сервис-ориентированной архитектуры. Даже разбиение системы на четыре-пять более мелких сервисов в правильных обстоятельствах может дать значительное улучшение здоровья кода.

Возможно, вы слышали, что микросервисы — единственный хороший архитектурный паттерн; это неправда. Такое восприятие проистекает из того, что многие монолиты плохо спроектированы или не получили внимания и инвестиций в технический долг, необходимых для раскрытия продуктивности. Идея о том, что все микросервисные архитектуры — сплошное удовольствие для работы, тоже неверна. Существует множество реализаций микросервисов, которые по той или иной причине также не реализуют ожидаемых преимуществ.

Ключевые особенности системы SOA или микросервисов:

- Разные сервисы могут независимо развёртываться и масштабироваться.
- Код управляется либо одним репозиторием исходного кода, либо множеством репозиториях кода.

- Информация перемещается между частями системы по сети, часто через HTTP, RPC (Remote Procedure Call) или системы очередей.
- Контракты данных должны быть намеренно спроектированы и хорошо продуманы, поскольку контракты реализуются как API и передаются по сети.

Выбор между сервис-ориентированной архитектурой и монолитом

В целом монолит проще настроить, чем SOA, и он требует значительно меньше технической организационной возни в управлении. По этой причине монолит — правильный ответ в первый же день для подавляющего большинства задач. Если команда очень дисциплинирована и вдумчива в проектировании монолита, он может масштабироваться вместе с командой бесконечно. Однако так будет не для всех. Для многих команд/проектов отсутствие у монолита навязанных контрактов, неспособность масштабировать отдельные компоненты по отдельности и отсутствие навязанного разделения ответственности станут барьером для продуктивности.

Если вы всё же обнаружите, что боретесь с неуправляемым монолитом, это не значит, что ваши инженеры плохо справляются со своей работой. Природа разработки ПО такова, что требования меняются, а системы развиваются. Сопровождение монолита может означать, что временами приходится вкладывать значительные ресурсы в обновление дизайна системы, чтобы он тоже развивался, и именно когда команда не делает этих вложений, сложность монолита становится барьером для продуктивности.

Есть несколько обстоятельств, при которых переход на сервис-ориентированную архитектуру явно является правильным выбором:

В вашем сервисе есть элементы, которые нужно масштабировать независимо. Например, одна функция потребляет много ресурсов CPU, и вы не хотите, чтобы это мешало другим функциям, или вы предпочитаете не платить за масштабирование всех функций, когда экономически выгоднее масштабировать этот один кусок независимо.

Вы работаете над функциональностью, которой нужно предоставлять собственный независимый API и которая имеет собственную исключительную предметную область данных отдельно от основной системы. Особенно если этот API предназначен для обслуживания внешних клиентов, то размещение этой функциональности в виде отдельного сервиса — очевидно хороший выбор.

По какой-то причине вам нужно использовать ещё один язык программирования как часть вашего приложения. Хорошим примером может быть то, что для решения определённого рода задачи существует надёжный и качественный фреймворк на Python, а остальная часть вашего приложения написана на Java. Связать эти два языка в памяти возможно, но неуклюже. Более простой вариант — связать их через API, оставив их естественным образом размещёнными как отдельные сервисы.

Развёртывание вашего монолита чрезмерно дорого, медленно или рискованно. В этом случае вы можете повысить продуктивность и сократить время до развёртывания, развёртывая новый код как независимый сервис. Только убедитесь, что новый сервис работает независимо от монолита и что вы не создаёте новых зависимостей при развёртывании.

Контроль версий для сервис-ориентированных архитектур: `monorepo` и `manurepo`

Управление исходным кодом для монолита довольно просто, потому что он живёт в едином репозитории с единой системой сборки. Как только вы начинаете разбивать код на разные пакеты, проекты и сервисы, перед вами встаёт выбор: управлять несколькими сервисами в едином репозитории кода или создавать несколько репозиторий? Этот компромисс называют `monorepo` против `manurepo`.

Если вы решите управлять несколькими сервисами как `monorepo`, вам, скорее всего, понадобится поискать решение для управления рабочими пространствами (`workspace`) (например, `yarn workspaces` для экосистемы JavaScript), чтобы управлять сборкой проектов по отдельности. Вот некоторые основные различия между подходами `monorepo` и `manurepo`:

****Плюсы и минусы `monorepo` TODO: Put the chart here**

легко обеспечить, чтобы каждая зависимость сервиса или пакета была актуальной с последней версией.

Многие CI-системы изначально не поддерживают несколько пакетов в одном репозитории, поэтому вам приходится вручную создавать обвязку для поддержки этого.

Наличие всего кода в едином репозитории улучшает обнаруживаемость, упрощая разработчикам поиск нужного им модуля или ссылки. IDE имеют надёжную поддержку такого рода поиска.

Манугеро, напротив

Требует использования центрального менеджера пакетов с контролем версий. Это не обязательно плохо, но может приводить к значительным накладным расходам при одновременной работе над проектом и его зависимостями.

Чисто интегрируется с системами CI/CD-конвейеров (Bitbucket pipelines, GitHub actions и т. д.).

Мой общий совет — держать всё простым. Для малых и средних проектов моногеро будет проще настроить и сопровождать. Переход на манугеро означает готовность вложиться в инструментарий, чтобы манугеро гладко работал для ваших разработчиков; это значительная цена. Для маленького стартапа эта цена, скорее всего, того не стоит. С другой стороны, если вы быстро растёте или преодолеваете отметку в пятьдесят с лишним разработчиков, моногеро становится неповоротливым, и у вас есть выделенная внутренняя платформенная или DevOps-команда, способная взять на себя тяжёлую работу по облегчению использования манугеро, тогда переход на паттерн манугеро может оказаться правильным выбором.

Распределённый монолит

Распределённый монолит — это система, развёрнутая как несколько сервисов, которые не спроектированы с достаточной независимостью или изоляцией и потому не могут развёртываться независимо. Если говорить начистоту, это худшее из двух миров. Вместо того чтобы позволить разработчику обратиться к любому сервису и работать над ним изолированно, не думая ни о каком другом сервисе, такая конструкция требует, чтобы разработчик рассуждал о том, как этот сервис влияет на другие сервисы. Мало того, ему приходится затем вносить изменения, потенциально в несколько сервисов, и координировать развёртывания в определённом порядке между сервисами, чтобы обеспечить совместимость во время релизов. Эта сложность разработки и развёртывания сводит на нет ключевые преимущества микросервисной системы.

Если вы замечаете, что ваша команда скатывается в эти паттерны или жалуется на координацию релизов между сервисами, это должно стать для вас красным флагом, чтобы присмотреться внимательнее и подумать о погашении части технического долга, чтобы вернуться к независимо развёртываемым сервисам. Этот технический долг обычно находится в ваших контрактах, дизайне ваших API и в том, как обрабатываются данные в вашей системе.

Написание читаемого, хорошего кода

В профессиональной среде главная аудитория любой отдельной строки кода — не компьютер, а разработчик, которому придётся прочитать этот код в какой-то момент в будущем для дальнейшей разработки. Это золотое правило программирования: инженеры должны писать код с тем же уровнем читаемости, какой они ожидают от кода любого другого.

Выбор языка и экосистемы

Согласно золотому правилу программирования, ваш выбор языка должен позволять вашей команде писать код, который высоко читаем и сопровождается. В целом хороший инженер может делать это на любом языке; однако некоторые языки облегчают делать это последовательно сильнее, чем другие. Несколько других соображений относительно того, какой язык или экосистему выбрать:

- Насколько велик пул талантов, знакомых с этим языком, и, более конкретно, знакомых с этим окружением и при этом заинтересованных в стартапах, подобных вашему?
- Существуют ли готовые реализации, которые вы можете использовать как отправную точку?
- Есть ли у вас особые требования к производительности или масштабированию? Некоторые языки для определённых типов задач намного быстрее других. Haskell печально известен своей неэффективностью в работе со строками, а C славится высокой скоростью в большинстве задач, хотя есть и другие языки, которые для определённых задач приближаются к скорости C или превосходят её, при этом предоставляя более простое и дружелюбное окружение для написания кода.

- Есть ли конкретный фреймворк, который может быть хорошей отправной точкой на определённом языке? Например, React Native — это мощный кросс-платформенный мобильный язык, требующий JavaScript или TypeScript.

В корпоративной среде я рекомендую языки со статическими системами типов, такие как Golang, TypeScript, Rust и т. д. При статической системе типов компилятор берёт на себя больше работы по обеспечению корректности кода. Вам следует стремиться к локальному окружению разработки, в котором инструменты находят ошибки до того, как ваш код будет выполнен. Исправление проблемы на этапе компиляции в целом гораздо быстрее и дешевле, чем исправление проблемы во время выполнения.

Стиль кода и форматирование

В любом широко используемом языке существует либо опубликованный стандарт того, как должен форматироваться код (например, PEP8 в Python), либо настраиваемый инструмент, который может навязывать определённый стиль кода и форматирование (например, ESLint или Prettier в JavaScript или ReSharper в C#). Большинство этих инструментов очень хорошо обеспечивают, чтобы код, независимо от того, кто его написал, был стилистически идентичен. В духе обеспечения читаемости вашей кодовой базы нет оправдания тому, чтобы не использовать один из этих инструментов и не обеспечить, чтобы 100 процентов вашей кодовой базы были отформатированы по одним и тем же правилам. Какие правила использовать — целиком предпочтение ваше и вашей команды, просто убедитесь, что оно последовательно и даёт читаемый результат.

Я рекомендую иметь набор опций конфигурации или инструкций для интегрированных сред разработки (IDE), которые используют ваши разработчики, о том, как автоматически форматировать код при сохранении файла. Затем вам следует в своей системе непрерывной интеграции обеспечить, чтобы весь новый код был отформатирован правильно.

Будьте осторожны: навязывание стиля в вашей системе непрерывной интеграции без автоматического форматирования очень раздражает инженеров, поэтому обязательно обучите всех правильной настройке их IDE в первый же день, чтобы избежать постоянных сюрпризов и потраченного впустую времени цикла из-за падений линтинга в CI.

Статический анализ кода

Современный статический анализ кода способен выявлять и оповещать о широком спектре распространённых проблем в коде — от прорех в безопасности до явных багов и стилистических несоответствий. Эти инструменты довольно недороги и аккуратно интегрируются с широким спектром распространённых систем непрерывной интеграции и IDE разработчиков. По опыту использования этих инструментов в ряде проектов и языков программирования соотношение сигнал/шум очень хорошее, а результат — чистый прирост продуктивности и качества ПО. Относительно рано в жизни вашего программного проекта вам следует интегрировать статический анализ кода. Я призываю вас присмотреться к инструментам, специфичным как для вашего выбранного языка программирования — например, ESLint для JavaScript, — так и к универсальным платформам анализа, таким как SonarCloud, Codebeat, Scrutinizer-CI, Code Climate или Cloudacuity.

Greenfield против Brownfield

Разработка ПО «с нуля» (greenfield) относится к работе по разработке в новом окружении с минимальным количеством уже существующего унаследованного кода и свободой выбора инструментов, паттернов и архитектуры. Это даёт очевидное преимущество — возможность вдумчиво выбрать правильную архитектуру и инструментарий для задачи и отсутствие отвлечения на существующий технический долг. Неочевидный минус в том, что при таком большом выборе и столь малом числе ограничений риски принятия неудачных решений выше. Кроме того, обычно у новых проектов есть значительная стоимость начальной раскрутки, которую недооценивают, — такие вещи, как настройка тестирования, систем сборки, статического анализа кода и т. д.

Разработка ПО на «обжитом поле» (brownfield) относится к противоположности greenfield — работе с существующими унаследованными системами. Компромиссы по сути инвертированы: к лучшему или к худшему, вы застряли с высокоуровневыми решениями, принятыми теми, кто был до вас.

Наибольший риск в brownfield-разработке — синдром «изобретено не здесь» (not invented here). «Изобретено не здесь» — это склонность людей избегать ответственности за то, что они не создавали сами, или не уделять этому

достаточного внимания. В brownfield-разработке ПО это может приводить к систематическому недоинвестированию в понимание существующей работы, вызывая фрустрацию и неэффективность при дополнении или изменении существующих систем. Я настоятельно призываю менеджеров явно выделять команде пространство для чтения и понимания существующей системы, прежде чем просить её хоть как-то её модифицировать. Время, потраченное на осмысление заранее, окупится меньшим числом сюрпризов и более высокой скоростью в дальнейшем.

Паттерны написания кода

Тема о том, в каком стиле писать код, для многих программистов становится предметом почти религиозных споров. В этой главе я ставлю целью кратко описать, что означают наиболее распространённые понятия из мира паттернов кода, и направить вас к более подробным ресурсам по каждой практике.

Если вы оказались втянуты в разговор на эту тему, который кажется вам эмоциональным, помните: существует множество успешных компаний, использующих каждый из этих паттернов. Всё есть компромисс. Плохой программист устроит беспорядок любым инструментом, и наоборот — отличный программист найдёт способ написать читаемое решение даже на неоптимальных инструментах.

Объектно-ориентированное программирование (ООП)

Объектно-ориентированное программирование (ООП) — это методология проектирования кода так, чтобы он отражал существительные и глаголы реального мира. Типичный пример: смоделировать взаимодействие двух людей как два объекта Person, и любые действия людей, например разговор, будут функциями этих объектов. Многие языки изначально объектно-ориентированы, например Python, Ruby и C#. Некоторые, такие как JavaScript или C++, объектно-опциональны (в той или иной мере поддерживают как объектно-ориентированный, так и функциональный стиль), а другие представляют собой нечто совершенно иное.

Чистота

Код, который является чистым (pure), не имеет внешних зависимостей или побочных эффектов. Иначе говоря, при одних и тех же входных данных чистый фрагмент кода всегда выдаёт один и тот же результат. Преимущество чистого кода в том, что он легко тестируется и не требует никакой внешней подготовки или мокинга. Чистый код также легче читать и понимать, поскольку для понимания того, что он делает, не нужно читать какой-либо дополнительный код. Простой пример чистого кода — функция, складывающая два числа: при любых двух входных числах функция суммирования всегда выдаёт один и тот же результат.

Некоторый код по своей природе нечист — например, код, взаимодействующий с внешним миром, таким как файловая система, сеть или база данных. В большинстве же других сценариев бизнес-логику можно смоделировать чистым образом. Там, где это возможно, я призываю вас и вашу команду писать чистый код.

Функциональное программирование

Если продолжать модель с частями речи для описания паттернов кода, то функциональное программирование рассматривает глаголы (функции) как полноправную часть системы. Большинство функционального программирования начинается с очень мелких кусочков функциональности и компонует их вместе, чтобы создавать более сложные и изощрённые системы. Когда это делается хорошо, преимущество функционального кода в том, что он, как правило, более чист и поэтому легче читается, понимается и тестируется в изоляции. Существуют даже академические примеры функционального кода, который можно формально проанализировать, то есть можно построить математическое доказательство того, что код работает корректно.

Функциональное программирование, выполненное плохо, может породить очень многословный и трудночитаемый код. Например, при компоновке нескольких функций важно учитывать, сколько функций компонуется и насколько очевидно поведение каждой функции в цепочке композиции.

Худший сценарий: представьте цепочку из десяти функций подряд, каждая с именами, которые для вас ничего не значат (например, `a(b(c(d(e(f(g(h(i(j(input))))))))))`). Хуже было бы только в том случае, если бы определения этих «алфавитных» функций находились в десяти разных файлах в разных местах кодовой базы, или, что ещё хуже, приходили из разных импортируемых библиотек.

Экстремальное программирование и разработка через тестирование (TDD)

Экстремальное программирование — это методология разработки, родственная Agile или SCRUM. Этот термин может использоваться для обозначения формальной методологии, описанной в книге Кента Бека (Kent Beck) *Extreme Programming Explained*, или более неформально — для обозначения некоторых практик написания кода, которых придерживается эта методология. Неформальное употребление этой фразы описывает практики тестирования в методологии, в частности идею разработки через тестирование.

Разработка через тестирование (TDD) — это процесс, при котором тесты пишутся раньше функционального ПО, в противоположность написанию сначала функционального кода, а затем тестов. Разработка через поведение (BDD) и разработка через приёмочные тесты (ATDD) — похожие практики.

Внедрение зависимостей

Внедрение зависимостей (dependency injection) — это паттерн, при котором сервисные зависимости конкретного объекта, модуля или блока кода передаются извне, а не инстанцируются внутри. Например, объект данных может сам создать собственное подключение к базе данных, отыскав строку подключения в файле конфигурации и создав клиент базы данных. Альтернативно, вызывающий родительский блок кода может создать сервис базы данных, а затем передать единственный экземпляр этого сервиса в каждый экземпляр объекта данных.

Главное преимущество внедрения зависимостей в том, что оно снижает связанность (coupling) между сервисом и его зависимостями, фактически добавляя между ними документированный интерфейс. Этот интерфейс позволяет, например, использовать другие реализации интерфейса, такие как мок-сервис в контексте тестирования.

В чистой реализации внедрения зависимостей есть свои тонкости. Я рекомендую вам перенять фреймворки или паттерны, которые широко используются и хорошо продуманы для вашего языка программирования.

Предметно-ориентированное проектирование

Термин «предметно-ориентированное проектирование» (Domain-Driven Design) пришёл из книги Эрика Эванса (Eric Evans) *Domain-Driven Design*, опубликованной в 2003 году. Основная идея — создать модель (будь то для объектов в объектно-ориентированном проектировании или для схемы вашей базы данных), которая моделирует существенные вашей бизнес-области. Это может показаться простым и интуитивным; однако в сложных бизнес-областях коду легко либо непоследовательно моделировать предметную область, либо моделировать её так, что это затрудняет понимание командой. Особенно с более крупными и сложными задачами я всегда настаиваю на том, чтобы команда села и договорилась о едином способе моделирования задачи, используя согласованную терминологию для обозначения одних и тех же понятий во всей системе.

Контракты API

Программный интерфейс приложения (API) во многом подобен юридическому договору.

Он проектируется, дорабатывается и согласовывается заранее, до реализации, и обе стороны ожидают, что другая будет соответствовать договору для достижения желаемого результата. Когда вы проектируете и реализуете API, вы берёте на себя обязательство перед потребителями вашего API, что он будет работать определённым образом. Как и в случае юридического договора, у вас может быть конкретное представление о том, как будет функционировать ваш API, но если нюансы будут истолкованы другой стороной иначе, вы можете оказаться неспособны достичь своей цели. Детали API действительно важны, и как технический руководитель вы отвечаете за то, чтобы ваша команда проектировала и строила API единообразным, эффективным образом.

При всём этом построение высококачественного API — на удивление сложная задача. Она требует учёта множества вещей: проектирования интерфейса, реализации кода, обрабатывающего логику/данные, тестирования функциональности, создания документации, решения вопросов версионирования/управления изменениями, поддержания документации в актуальном состоянии по мере изменения API и упрощения взаимодействия разработчиков с API. Хорошее выполнение этих задач может означать разницу между API, который разработчики любят, и API, который тормозит реализацию и замедляет вывод важных проектов. В вашем распоряжении как у руководителя есть два главных рычага, чтобы убедиться, что вы делаете всё это хорошо: управление (governance) и архитектура.

Управление проектированием API

В создание каждого элемента API входит бесчисленное множество решений. Хорошие API отличаются от плохих согласованностью, предсказуемостью и корректностью этих решений. Как на технического руководителя на вас ложится задача убедиться, что во всей вашей организации существует структура, помогающая разработчикам строить API, которые согласованы друг с другом, предсказуемы в том смысле, что используют общие паттерны, подходящие для решаемой задачи, и корректны.

Достижение этих целей требует той или иной формы системы управления (governance). Она может варьироваться от набора чётко задокументированных руководств и стандартов до группы людей, отвечающих за регулярное рецензирование и утверждение всех API. Чем больше ваша команда, тем больше времени и усилий вам придётся вкладывать в процессы и управление, чтобы поддерживать высокий стандарт.

Архитектура API

В реальном мире вы, скорее всего, столкнётесь с двумя основными типами API: основанными на HTTP и не основанными на HTTP. Как и любой инструмент, HTTP имеет свои компромиссы и не идеален для любой задачи, поэтому если ваши бизнес-требования диктуют сверхнизкую задержку, или сверхвысокую пропускную способность/низкие накладные расходы, или приложения с потоковой передачей в реальном времени, вам, вероятно, понадобится что-то помимо HTTP. Ниже я обсуждаю несколько типов HTTP API, а затем кратко рассматриваю некоторые не-HTTP API, с которыми вы, скорее всего, столкнётесь.

API на основе HTTP

Если вы создаёте веб- или мобильное приложение или даже большинство системных бэкендов, шансы очень высоки, что вы в основном будете иметь дело с HTTP API.

XML- и SOAP-API

В начале 2000-х наиболее распространённым паттерном API был основанный на XML протокол Simple Object Access Protocol (SOAP). SOAP и другие основанные на XML стили API в 2020-х по-настоящему вышли из моды у стартапов, но они всё ещё распространены в legacy-системах, особенно у крупных компаний в технологически медленно меняющихся отраслях. Вам не следует строить новые API на основе SOAP или XML.

REST

REST (Representational State Transfer) — это общий термин, описывающий использование JSON поверх HTTP в качестве API. REST иногда дополняется паттерном под названием HATEOAS, который придаёт более формальный набор стандартов содержимому/полезной нагрузке REST API. Без HATEOAS (который не так уж распространён) REST не включает формальных или фирменных рекомендаций о том, как моделируются данные JSON. REST API обычно моделируют одно существительное как эндпоинт и используют HTTP-глаголы (GET, PUT, POST, DELETE и т. д.) для определения действий над существительными. Например, GET `/users` выведет список пользователей, POST `/users` создаст нового пользователя, а DELETE `/users/123` удалит пользователя с ID 123.

REST, скорее всего, наиболее распространённая форма API, с которой вы столкнётесь. У REST широкая и надёжная экосистема инструментов, и почти каждый инженер с ним знаком.

GraphQL

GraphQL похож на REST в том, что это JSON поверх HTTP; однако он не опирается на HTTP-глаголы. Почти всё в GraphQL — это POST, и он использует структурированную схему запросов (queries) и мутаций (mutations).

Мне нравится думать о GraphQL как о REST с типами и самодокументируемой схемой. В результате GraphQL API, как правило, поставляются с автоматически генерируемой документацией и обозревателями схем. Благодаря своей системе схем GraphQL также позволяет компоновать несколько схем из нескольких сервисов, формируя более крупный, мощный и сложный граф данных, иногда называемый федеративной схемой (federated schema). Компания Apollo предоставляет изощрённые решения для управления графом и его масштабирования.

Многое можно сказать о преимуществах построения графа для моделирования данных вашей компании и о хороших привычках, которые возникают, когда вы вынуждены проектировать схему. При этом ни одна система не

обходится без компромиссов. Поскольку GraphQL отказывается от стандартных HTTP-глаголов, он не очень хорошо уживается с некоторыми элементами веб-стека. Кэширование GET-запросов и инструментарий для разработчиков всё ещё догоняют, чтобы аккуратно работать с GraphQL-запросами. Если эти недостатки не являются существенной заботой для вашего бизнеса, я настоятельно рекомендую вам заглянуть на apollographql.com и рассмотреть использование GraphQL — особенно для внутренних сценариев применения ваших API.

Не-HTTP API

В целом для традиционных синхронных API в стиле «запрос/ответ» (то есть удалённого вызова процедур, или RPC) вы захотите использовать HTTP API из-за его повсеместного распространения. Однако существует несколько паттернов API — особенно для асинхронных операций, — которые не отображаются на HTTP аккуратно и имеют широко используемые альтернативные реализации.

Системы очередей

Система очередей поддерживает входящий ящик (или набор ящиков) для приёма сообщений и интерфейс, через который потребитель читает сообщения с определёнными гарантиями.

Типичная система очередей может гарантировать порядок сообщений (либо «первым пришёл — первым ушёл», FIFO; либо «последним пришёл — первым ушёл», LIFO), а также доставку «хотя бы один раз» или «не более одного раза». В большинстве облачных платформ есть размещённые реализации очередей, такие как AWS Simple Queue Service (SQS) или Google Cloud Task queues.

Системы очередей часто имеют понятие *явного* вызова, то есть когда издатель (publisher) создаёт сообщение, он явно указывает, как именно следует обработать или выполнить запрос. В противоположность этому, большинство систем «издатель — подписчик» поддерживают *неявное* выполнение. Это означает, что издатели не обязательно знают заранее, какая система обработает сообщение, — лишь то, что система pub/sub его доставит.

Паттерн «издатель — подписчик» (pub/sub)

Паттерн «издатель — подписчик», сокращённо pub/sub, позволяет проектировать систему, в которой сообщения создаются потенциально несколькими источниками и доставляются по различным схемам потенциально нескольким подписчикам. Отношения «издатель — подписчик» моделируются как «один к одному» (прямое), «один ко многим» (fan-out), «многие к одному» (fan-in) и «многие ко многим». Различные реализации pub/sub могут давать гарантии того, что сообщения доставлены всем подписчикам, хотя бы одному подписчику, хотя бы один раз и т. д. Подобно очередям, существуют готовые решения, такие как RabbitMQ, а также легко масштабируемые облачные варианты, такие как Amazon Simple Notification Service (SNS) или Google Cloud Pub/Sub.

Паттерн pub/sub и предоставляемые им гарантии чрезвычайно мощны. Однако компромисс в том, что реализации требуют определённой аккуратности и внимания к деталям, чтобы реализовать заявленные гарантии. Реализация подписчика, например, требует пристального внимания к семантике подтверждения сообщений (acknowledgement) и тщательного управления подписками на топики, чтобы нужные сообщения попадали в нужное место.

Если вы разрываетесь между реализацией решения на очередях, pub/sub или HTTP API, моя общая рекомендация — придерживаться простоты и выбрать синхронный HTTP API. Сам факт того, что вы разрываетесь между реализациями, указывает на то, что гарантии, предлагаемые асинхронными системами, не критичны для вашей реализации, а значит, добавленная сложность, скорее всего, не стоит того для вашего стартап-проекта.

Системы заданий

Задания (jobs), или cron-задания, — это тип бэкенд-API, который редко запускается издателем или клиентом, а вместо этого срабатывает по той или иной форме таймера. Распространённые примеры включают ночные задачи очистки данных или отправку еженедельных сводок/уведомлений по электронной почте. Несколько лучших практик для заданий:

- Используйте систему заданий, поддерживаемую кем-то другим, не стройте её самостоятельно.
- При выборе системы заданий (или её построении самостоятельно, если уж приходится) убедитесь, что она:
 - ведёт логирование каждого выполнения задания;
 - позволяет настраивать повторный запуск (retry) заданий, завершившихся с ошибкой;

- о уведомляет о сбоях заданий. Очень распространённая ситуация: инженер настраивает запланированное задание, наблюдает за его работой в первый день, а на пятнадцатый оно падает, и никто не замечает этого до тридцатого дня;
- о предоставляет интерфейс для просмотра заданий и их статуса;
- о позволяет хранить конфигурацию заданий в виде кода или конфигурации в системе контроля версий;
- о позволяет запускать задания внутри вашего окружения/частных сетей/групп безопасности по мере необходимости для доступа к другим внутренним системным API/ресурсам;
- о интегрируется с вашей системой управления секретами.
- о позволяет легко настраивать задания локально в средах разработки и эксплуатации, а также легко тестировать их в каждой из этих сред.

Документация

Наличие исчерпывающей, ясной и актуальной документации для вашего API столь же критично, как и то, как вы его строите и поддерживаете. Несколько ключевых характеристик отличной документации API:

- Всегда соответствует реализации
- Документирует все возможные входные данные и их типы. Документирует все возможные ошибки
- Легко читается и навигируется другими инженерами

Всегда хорошая идея — строить свой API с помощью системы, включающей генерацию документации API. Иначе достичь всех этих целей на постоянной основе будет практически невозможно. Если вы строите REST API, я настоятельно рекомендую проектировать ваш API с использованием OpenAPI (документ в формате YAML или JSON, описывающий ваш API). В большинстве языков есть SDK для потребления спецификации OpenAPI и автоматической генерации контроллеров/маршрутов, соответствующих спецификации, и/или генерации тестового каркаса, обеспечивающего соответствие реализованного API спецификации. Кроме того, существуют онлайн-инструменты, такие как stoplight.io и readme.com, которые могут потреблять документы OpenAPI и генерировать эстетически приятную и удобную для навигации документацию.

Если вы используете GraphQL, GraphQL Playground или обозреватель Apollo Studio могут служить разумной заменой обширной документации по типам. Я всё же рекомендую вам построить отдельную страницу документации API — либо с помощью инструмента вроде readme.com, либо создав что-то вручную — чтобы она выступала в роли вводного руководства или руководства по началу работы. Во встроенной документации GraphQL отсутствует описание того, как работает аутентификация, и она также плохо справляется с задачей предоставить место для объяснения связей между данными в вашем API.

Это пробелы, которые вам нужно восполнить в другом месте.

Ещё одно преимущество использования OpenAPI или GraphQL в том, что результирующая спецификация API переносима не только в генераторы документации и тестовые фреймворки, но и в IDE для разработчиков, такие как [Insomnia](https://insomnia.rest) или [Postman](https://www.postman.com). Эти IDE позволяют разработчикам быстро взаимодействовать с API для проверки функциональности без написания кода. Формальные спецификации также можно использовать с инструментами генерации кода, чтобы обеспечить согласованность типизации в коде.

Идемпотентность

Запрос к API называется идемпотентным, когда выполнение одного и того же запроса несколько раз даёт тот же эффект, что и выполнение его один раз. Идемпотентность — важное понятие в построении надёжных систем и предотвращении повреждения данных. Как и всё, идемпотентность даёт вам полезные гарантии о системе, но за неё приходится платить: реализация идемпотентности добавляет сложности бэкенд-системам.

В REST API широко предполагается, что каждый HTTP-глагол, кроме POST, должен быть идемпотентным. GET-запросы, например, по определению всегда должны возвращать один и тот же результат для одних и тех же входных данных (если, конечно, не изменились лежащие в основе данные). В целом PUT-запросы изменяют существующие объекты и должны естественным образом быть идемпотентными. Несколько вызовов POST-запроса, однако, в большинстве систем сигнализируют о намерении создать несколько объектов.

Ключи идемпотентности

Для HTTP POST-запросов в REST и для API мутаций GraphQL идемпотентность не обеспечивается стандартом/спецификацией. Если вы хотите, чтобы клиент мог повторять такие запросы с идемпотентным поведением, вам следует реализовать паттерн ключа идемпотентности (idempotency key). Ключ идемпотентности — это произвольная строка, предоставляемая клиентом (либо в виде HTTP-заголовка, либо, возможно, в GraphQL как входная переменная), которую бэкенды используют для дедупликации входящих запросов. Это требует, чтобы бэкенд хранил ключ идемпотентности, а также хранил ответ на запрос с этим ключом, чтобы позднее предоставить его клиентам.

Обратите внимание, что реализация ключа идемпотентности нетривиальна, так как потребует дополнительных записей в базу данных, логики вокруг захвата ответов на запросы и обработки вопросов конкурентности/блокировок для дублирующихся запросов, приходящих одновременно. Если идемпотентность важна в вашем приложении — скажем, если вы имеете дело с финансовыми транзакциями, — я рекомендую вам перенять реализацию бэкенд-API, предоставляющую надёжную систему идемпотентности «из коробки», а не строить её самостоятельно с нуля.

Данные и аналитика

У большинства стартапов есть как минимум три различных вида данных, которые они используют в рамках своего бизнеса:

- Транзакционные данные
- Аналитические данные бизнес-аналитики (business intelligence)
- Поведенческие данные

Каждый из этих типов данных будет поступать в разных объёмах, иметь разные паттерны чтения/записи и требовать разных инструментов для визуализации и извлечения инсайтов.

Небольшое замечание о фразе «большие данные» (big data). Как у стартапа, шансы очень высоки, что у вас *нет* больших данных в том смысле, что их нужно проектировать с прицелом на бесконечный масштаб (или веб-масштаб). Типичные готовые базы данных с разумным количеством оборудования и более-менее приличным проектированием модели данных вполне способны обрабатывать десятки миллионов строк и сотни гигабайт данных с приемлемой производительностью. Большинство решений для больших данных, таких как конвейеры данных (data pipelines) или хранилища данных (data warehouse appliances), сопряжены со значительной дополнительной сложностью настройки, задержкой и стоимостью, и они, скорее всего, избыточны для вашего стартапа. Ради простоты решения для больших данных следует рассматривать только в том случае, если вы можете привести убедительный аргумент, что обычная база данных (например, PostgreSQL) не справится с задачей. Иначе говоря, не оптимизируйте архитектуру вашей базы данных преждевременно.

Транзакционные данные

Транзакционные данные — это данные, которые питают само ваше приложение, как правило, ваша основная база данных NoSQL или SQL. Транзакционные данные требуют очень низкой задержки и высокой доступности и невелики по общему размеру по сравнению с другими формами данных. Моя рекомендация — выбрать готовое SQL- или NoSQL-решение, предпочтительно что-то размещённое для вас, например MongoDB Atlas или Google Cloud SQL. Несколько желательных функций в вашей продакшен-базе данных:

- Восстановление на момент времени (point-in-time restore) в один клик
- Регулярные резервные копии с восстановлением в один клик
- Реплики только для чтения для сброса нагрузки (load shedding)
- Многозональная репликация и размещение для доступности
- Аудит-логирование на основе событий
- Автоматическое расширение/сжатие диска
- Безопасность на уровне подключения/IP

- Мониторинг и оповещение по ресурсам (CPU, RAM, диск, сеть). Масштабирование вверх/вниз по CPU/RAM в один клик
- Мониторинг медленных запросов

Аналитические данные бизнес-аналитики

Бизнес-аналитика (business intelligence, BI) — это данные, которые используются для получения инсайтов о поведении ваших пользователей, обычно извлекаемые из ваших транзакционных данных. На раннем этапе вы зачастую можете обойтись выполнением запросов бизнес-аналитики напрямую на вашей транзакционной базе данных. По мере роста объёма данных и сложности запросов это становится более проблематичным, так как добавляет дополнительную нагрузку на систему, требующую высокой доступности и низкой задержки. Естественным решением тогда становится либо запрашивать реплику вашей транзакционной базы данных только для чтения, либо копировать/преобразовывать данные в другую систему хранения данных через конвейер данных (data pipeline).

Построение конвейеров данных и хранилищ данных — это целая отдельная книга, и передовые подходы постоянно развиваются. У меня есть лишь несколько высокоуровневых советов:

Рассмотрите корпоративные решения для данных, такие как Snowflake, Databricks или Google BigQuery, для вашего основного хранилища данных бизнес-аналитики. Эти инструменты меняют правила игры. Бессерверные хранилища, в частности (BigQuery, Aurora), тривиально настраиваются, имеют довольно постоянную задержку независимо от размера данных и весьма экономически эффективны для стартапов ранней/средней стадии.

В наши дни стартапу не нужно строить и хостить изощрённые архитектуры конвейеров данных. Инструменты ELT (Extract, Load, Transform) и ETL (Extract, Transform, Load) теперь могут работать полностью внутри корпоративного озера/хранилища данных, а такие инструменты, как dbt, обеспечивают воспроизводимость, тестируемость и возможности «конвейер как код» (pipeline-as-code), делая запуск конвейеров данных гораздо более управляемым.

Рассмотрите использование размещённых или облачно-нативных решений для визуализации данных, таких как Looker, Domo или Preset.

Убедитесь, что ваши инженерные и продуктовые команды тесно сотрудничают с тем членом команды, который владеет данными и бизнес-аналитикой. Привлечение перспективы данных на раннем этапе продуктового процесса избавит от множества головной боли в будущем по принципу «семь раз отмерь — один раз создай схему данных».

Поведенческие данные

Поведенческие данные — также называемые событиями поведенческой аналитики — описывают, как пользователи использовали ваше приложение. Поведенческие данные часто имеют довольно большой объём при несколько ограниченной схеме и лучше всего используются в сочетании с мощным программным обеспечением для визуализации.

В целом вы захотите, чтобы поведенческие данные из вашего приложения отправлялись в несколько источников. Это создаёт определённую проблему маршрутизации: у вас есть единственный источник данных (ваше приложение), но вы хотите, чтобы события попадали во множество мест. Почти повсеместно принятое решение этой проблемы — платформа Segment от Twilio, хотя есть и набирающие силу альтернативы, называемые платформами клиентских данных (Customer Data Platforms, CDP), такие как RudderStack. CDP может принимать данные из вашего приложения, а затем отправлять их в ваше хранилище данных и в любое количество других SaaS-платформ по вашему желанию.

Одно важное различие между поведенческими данными, генерируемыми вашим приложением, и транзакционными данными — их точность. Большинство поведенческих данных подвержены потерям (lossy): у пользователей есть блокировщики рекламы, запросы теряются, или мешают файрволы. Есть много причин, по которым события могут не дойти от клиентского устройства до вашей CDP. Это не означает, что поведенческие данные бесполезны, но осознание их потерь должно сформировать ваши ожидания относительно этих данных и ограничить сценарии использования при их запросах. Если вам нужны точные числа, ожидайте, что вы будете выводить их из вашей BI-платформы и ваших транзакционных данных.

Общие советы и лучшие практики проектирования архитектуры

Завершим этот раздел несколькими общими рекомендациями по проектированию вашей архитектуры.

Размещайте бизнес-логику в бэкенде

По мере построения вашего приложения вы часто сталкиваетесь с выбором, где должна жить логика: на клиенте (например, в веб-браузере, на мобильном устройстве, физическом оборудовании) или на той или иной форме бэкенд-сервера. Для определённых типов логики — таких как всё, что связано с аутентификацией, расчётами значений, механизмами защиты от читерства/подделки, — это твёрдое требование. Для большинства же другой логики это всё равно хорошая идея по следующим причинам:

Бэкенды зачастую легче тестировать, чем клиенты, поэтому вы можете с большей уверенностью подтвердить корректность бизнес-логики на бэкенде.

Больше логики на бэкенде означает более «тонких» клиентов, а также означает, что вы можете производить клиентов для нескольких платформ, которые могут опираться на единый источник логики, сокращая дублирование кода.

Логику на бэкенде нельзя подделать или изменить со стороны клиента.

Делайте сервисы пригодными для экстернализации

API от вашего бэкенда к другим бэкендам или от вашего бэкенда к фронтендам следует рассматривать как API общего назначения, которые могли бы потребляться третьими сторонами. Это вынуждает вас поддерживать несколько хороших проектных привычек, включая обеспечение того, чтобы интерфейсы были понятны сами по себе (предметно-ориентированное проектирование), а также использование разумных механизмов аутентификации и подходящих высокоуровневых абстракций владения при проектировании данных. И на тот маловероятный случай, если однажды вы действительно захотите экстернализовать сервис, путь к этому будет гораздо короче.

Используйте как можно меньше языков

С каждым языком программирования приходит связанная с ним система сборки, система управления зависимостями, лучшие практики программирования и интерфейсы. Ваша команда должна вкладывать значительные усилия, чтобы ваш основной язык и экосистема были хорошо интегрированы и хорошо работали для локальных разработчиков, тестовых сред и продакшен-сред.

Для каждого дополнительного языка, который вы добавляете в свой стек, вам придётся воспроизвести все эти усилия, и вы будете страдать от невозможности делиться кодом между средами выполнения. Прежде чем допускать дополнительный язык в свой стек, вы должны быть в состоянии построить надёжный и неопровержимый аргумент, что преимущества нового языка многократно перевешивают операционное и эксплуатационное бремя, которое этот новый язык добавляет. Иначе вам лучше обойтись без него.

Инструменты

Экосистема инструментов и паттерны разработки ПО постоянно развиваются и меняются. Вы неизбежно будете испытывать соблазн — либо самостоятельно, либо по инициативе членов вашей команды — изменить что-то в том, как вы занимаетесь инженерией, например перенять новую библиотеку, фреймворк, язык или паттерн. Быстрое принятие каждого из этих изменений быстро приводит к «лоскутному одеялу» из плохо продуманной архитектуры. И наоборот, игнорирование всех изменений оставляет вас с устаревшей кодовой базой, которая со временем будет менее эффективной и более трудной для работы вновь нанятых талантов. Правильный подход — формализовать процесс

изменения вашего технологического стека и предоставить некоторые ограничители (guardrails), мотивирующие вашу команду быть любознательной и вдумчивой в отношении изменений инструментария.

Внедрение внутреннего технологического радара

Thoughtworks, ведущая консалтинговая компания по разработке ПО, базирующаяся в Сан-Франциско, публикует инструмент под названием Technology Radar (cto.hb.com/radar), который оценивает сотни проектов, с которыми Thoughtworks сталкивается каждый год. Они помещают новые инструменты, техники, паттерны и языки (которые они называют «блипами», blips) в одну из четырёх категорий в зависимости от того, насколько они эффективны в реальном мире.

Эти категории — hold (придержать), assess (оценить), trial (испытать) и adopt (принять).

Если вы никогда не читали Thoughtworks Radar, я настоятельно рекомендую его как общее введение в то, что происходит, а также как источник вдохновения для процесса вашей собственной команды.

Мой предпочтительный способ сбалансировать задачу поддержания мотивации инженеров и актуальности кодовой базы с текучкой инструментов — последовать примеру Thoughtworks и внедрить внутренний технологический радар. Вместо того чтобы взвешивать что-то новое на предмет его универсальной привлекательности, как делает Thoughtworks, мой подход оценивает блипы на предмет их соответствия и эффективности для нашей организации, используя те же четыре уровня. Если конкретнее:

1. Кто-то предлагает использовать новый инструмент, технику, платформу или язык (блип). Это предложение поначалу относится к категории «assess». Предлагающий должен в техническом документе обосновать, что новый блип даст материальную выгоду проекту, уже выбранному бизнесом (или в качестве эксперимента в инновационном спринте — см. «Cooldown/Инновационные спринты», стр. 163). Затем, если оно одобрено, оно переходит в trial.
2. Новый блип используется разработчиком в проекте — либо выбранном бизнесом, либо в его окне инновационного спринта. В конце проекта автор создаёт последующий письменный документ, описывающий его опыт работы с блипом, включая плюсы и минусы и то, насколько хорошо блип уживается с остальной экосистемой инструментария в компании.
3. На основе результатов trial команда в целом перейдёт либо к принятию (adopt) блипа, разблокировав его для использования остальной командой без дальнейших церемоний, либо переместит его в hold, что потребует нового trial и новой оценки для его повторного использования. Если trial проваливается в бизнес-проекте, команде рекомендуется тщательно подумать о том, удалять ли блип из этой реализации, чтобы избежать будущих проблем с сопровождением.

В большинстве случаев я обнаруживаю, что когда trial блипа проваливается, он проваливается относительно рано, и инженер, ведущий проект, не включает блип в финальную поставленную реализацию.

Скучные технологии

«Скучные технологии» (Boring Technology) — выражение, придуманное Дэном Маккинли (Dan McKinley) и изложенное на cto.hb.com/boring. Ключевая идея в том, что задача вашей команды — поставлять функциональность для поддержки бизнеса, и в большинстве случаев это не зависит от использования модных новых инструментов. На самом деле использование чего-то не-скучного часто несёт множество скрытых издержек, и только если ваша команда полностью осознаёт эти издержки и верит, что выгоды больше, новый инструмент стоит принимать. Как описывает это «Скучные технологии», совокупная стоимость = стоимость сопровождения – выгода в скорости. Некоторые скрытые издержки, которые стоит учитывать:

- Неполная, неточная или незрелая документация
- Не до конца развитые экосистемы вокруг инструмента/технологии, включая SDK и интеграции с другими инструментами
- Более высокая вероятность столкнуться с дефектами или отсутствующей функциональностью/возможностями
- Дополнительные затраты на обучение членов вашей команды для освоения нового инструмента
- Бремя поддержания этого инструмента или пакета в актуальном состоянии, закрытия дыр в безопасности и т. п.

Стоимость инструментов

Это жизненный факт: современные стартапы будут тратить много денег на программное обеспечение как услугу (Software As A Service, SaaS). Ваша компания, скорее всего, не исключение, поэтому не удивляйтесь, когда

обнаружите, что тратите целую штатную единицу или больше на инфраструктуру и инструменты ещё до раунда Series B.

Бюджет

Существует несколько опубликованных бенчмарков расходов на SaaS и инструменты на разных стадиях развития компании — в виде процента либо от выручки компании, либо от совокупных расходов.

Единого точного бенчмарка нет, но похоже, что типичная себестоимость SaaS (Cost of Goods Sold, COGS) находится где-то между 10 и 30 процентами выручки.

Знайте свои расходы и следите за их ростом. Очень легко случайно оставить пару машин работающими в AWS и добавить \$10 000 к годовым счетам за облачный хостинг. У большинства облачных платформ есть встроенные функции бюджетирования, так что нет оправдания тому, чтобы ими не пользоваться. Если вы используете инфраструктуру как код, легко настроить модуль, который для каждой новой развёрнутой облачной системы будет автоматически применять облачный бюджет, отслеживающий и оповещающий о расходах по этой конкретной системе.

Для затрат на SaaS типично расти со временем — будь то из-за роста вашей инфраструктуры или потому, что вы находите нового SaaS-вендора, который может сэкономить время вашей команды. Я рекомендую не использовать стоимость как причину отказа от внедрения типичного SaaS-инструмента (со стоимостью в диапазоне сотен долларов в месяц). Вместо этого я бы советовал закладывать регулярный рост в прогнозы расходов на SaaS.

Отслеживание

Вам следует отслеживать, сколько ваша организация тратит на инженерные инструменты, включая IDE, SaaS и инфраструктуру (облачные платформы). Делать это можно вручную в таблице или с помощью платформы управления SaaS (SaaS Management Platform, SMP). Доступные от различных вендоров, таких как BetterCloud, Zluri и Vendr, эти решения связываются с вашей кредитной картой или банком и автоматически категоризируют денежные расходы.

DevOps

Wikipedia описывает DevOps как набор практик, который объединяет разработку программного обеспечения и ИТ-операции. Его цель — сократить жизненный цикл разработки систем и обеспечить непрерывную поставку с высоким качеством ПО.

Для меня ключевая часть этого определения в том, что DevOps стремится сократить жизненный цикл разработки ПО, иными словами — DevOps является катализатором продуктивности команды в широком смысле. Если вы ещё не думаете специально о DevOps, вы, скорее всего, в той или иной степени недооцениваете DevOps или недоинвестируете в него. Это не только моё мнение; в технологических отраслях всё шире принимается тот факт, что высококачественный DevOps — ключевой драйвер общей скорости разработки.

Четыре ключевые метрики (DORA)

Самым высоко оценённым «всплеском» (blip) в выпуске Technology Radar от Thoughtworks за 2022 год (cto.hbr.org/techradar) стали «Четыре ключевые метрики». Эти метрики описаны командой внутри Google Cloud под названием DORA (DevOps Research and Assessment), и эта система метрик появилась в результате более чем семилетней исследовательской программы, подтверждающей результаты и их влияние на технологии, процессы, культуру и количественные показатели. Вот эти четыре метрики:

Время до выпуска (Lead Time): сколько времени проходит от коммита кода до его работы в продакшене

Частота развёртываний (Deployment Frequency): как часто код выпускается в продакшен / для конечных пользователей

Среднее время восстановления (Mean Time to Recovery, MTTR): сколько времени требуется на восстановление сервиса после инцидента/дефекта

Доля неудачных изменений (Change Fail Percentage): какой процент продакшен-релизов требует хотфикса, отката, патча и т. п.

Вместе эти метрики количественно выражают идею того, насколько уверенно ваша команда может развёртывать ПО. Высокие показатели по всем четырём метрикам требуют инвестиций в автоматизацию, DevOps, тестирование и культуру. Как быстро отмечает Thoughtworks, извлечение пользы из этих метрик не обязательно требует крайне детализированной инструментовки, метрик или дашбордов. DORA публикует быстрый опрос-проверку (csohb.com/dora), который ваша команда может пройти, чтобы отслеживать свой прогресс на крупнозернистом уровне. Существует также множество инструментов с довольно низким порогом входа, которые дадут качество данных более чем достаточное для того, чтобы судить о вашем прогрессе, например LinearB или Code Climate.

В следующих подразделах о DevOps представлены концепции, дисциплины и области фокуса, которые тем или иным образом способствуют улучшению этих метрик.

Воспроизводимость

Развёртывание кода — это крайне тонкая деятельность, требующая чрезвычайной точности. Один неправильно поставленный символ в конфигурационном файле может привести к тому, что сервис не запустится. Хуже того, отладка DevOps-проблем для большинства инженеров медленна и мучительна, и обнаружение и исправление этой единственной ошибки в один символ может означать потенциальную потерю часов времени на отладку и исправление. Все мы люди; такие ошибки неизбежны. Поскольку в DevOps они столь дороги, крайне важно внедрить системы, минимизирующие возможность человеческой ошибки. Ключевой компонент минимизации частоты и влияния человеческой ошибки в DevOps — это концепция воспроизводимости.

Воспроизводимость подразумевает, что у нас есть возможность сделать что-то во второй раз так, чтобы это было одновременно дёшево и гарантированно идентично тому, как это было сделано в первый раз. Воспроизводимость в DevOps требует автоматизации и инструментария. Пожалуй, самый важный инструмент в арсенале DevOps для повышения воспроизводимости и ускорения времени разработки — это контейнеризация. На втором месте, с небольшим отрывом, — идея инфраструктуры как кода (Infrastructure as Code, IaC). Поскольку эти техники столь фундаментальны, я уделю немного времени, чтобы представить каждую из них здесь.

Контейнеризация

Самый распространённый способ объяснить роль контейнеров в контексте DevOps — рассмотреть, откуда возникло название: от транспортных контейнеров. До стандартизации транспортных контейнеров, если вы хотели перевезти товары через океан, вы упаковывали груз в самых разных формах — от размещения на паллетах, хранения в коробках или бочках до простого заворачивания в ткань. Погрузка и разгрузка товаров, прибывших упакованными всеми этими разными способами, была неэффективной и подверженной ошибкам, главным образом потому, что не существовало единого вида крана или тачки, которые могли бы эффективно перемещать весь груз.

Сравните этот хаотичный подход с использованием стандартизированного транспортного контейнера, где судно и портовый оператор могут работать с единым форм-фактором, используя стандартизированное оборудование и грузоотправителей и единую гибкую форму упаковки для всех своих товаров. Исторически использование стандартизированных транспортных контейнеров запустило смену парадигмы, которая снизила стоимость глобальных перевозок на порядки. Упаковка ПО в стандартизированный контейнер, который можно запускать на любой системе одинаковым образом, обеспечивает аналогичный скачок в возможностях и эффективности.

Чаще всего вы будете взаимодействовать с контейнерами через программную систему под названием Docker. Docker предоставляет декларативный язык программирования, который позволяет описать в файле под названием Dockerfile, как вы хотите настроить систему, — то есть какие программы нужно установить, какие файлы куда поместить, какие зависимости должны существовать. Затем вы собираете этот файл в образ контейнера (container image), который представляет всю файловую систему, описанную вашим Dockerfile. Этот образ затем можно переместить на любую другую машину с Docker-совместимой средой выполнения контейнеров и запустить с гарантией, что каждый раз он будет стартовать в изолированном окружении с точно теми же файлами и данными.

Лучшие практики управления контейнерами

Проектируйте контейнеры по принципу «собери один раз / запускай где угодно»

Соберите контейнер один раз (скажем, в CI), чтобы его можно было запускать в ваших различных окружениях — продакшене, разработке и т. д. Используя единый образ, вы гарантируете, что в точности тот же код с в точности теми же настройками перейдёт в неизменном виде из разработки в продакшен.

Чтобы добиться принципа «запускай где угодно» с вашими контейнерами, вынесите любые различия между окружениями в переменные окружения контейнера, задаваемые во время выполнения (runtime).

Это секреты и конфигурации, такие как строки подключения или имена хостов. Как вариант, вы можете реализовать в своём образе скрипт точки входа (entrypoint), который перед вызовом вашего приложения скачивает необходимую конфигурацию и секреты из централизованного хранилища секретов (например, Amazon или Google Secret Manager, HashiCorp Vault и т. п.).

Дополнительное преимущество стратегии скачивания секретов/конфигурации во время выполнения в том, что она переиспользуется для локальной разработки, избавляя разработчиков от необходимости когда-либо вручную доставать секреты или просить другого разработчика прислать им файл с секретами.

Собирайте образы в CI

В духе воспроизводимости я призываю вас собирать образы с помощью автоматизации, желательно как часть непрерывной интеграции. Это гарантирует, что сами образы собираются повторяемым способом.

Используйте размещённый реестр

Как только вы начнёте собирать образы контейнеров и перемещать их, вам сразу же захочется навести порядок в управлении самими собранными образами. Я рекомендую помечать (tag) каждый образ уникальным значением, производным от системы контроля версий, возможно, ещё и с временной меткой (например, git-хешем коммита, на котором был собран образ), и хранить образ в реестре образов. У Dockerhub есть продукт с приватным реестром, и все основные облачные платформы также предлагают размещённые реестры образов.

Многие размещённые реестры также предоставляют сканирование уязвимостей и другие функции безопасности, привязанные к их реестру образов.

Держите размеры образов как можно меньше

Образы Docker меньшего размера быстрее загружаются из CI, быстрее скачиваются на серверы приложений и быстрее стартуют. Разница между загрузкой образа в 50 МБ и в 5 ГБ с операционной точки зрения может означать разницу между пятью секундами на запуск нового сервера приложений и пятью минутами. Это ещё пять минут, добавленных к вашему времени развёртывания, среднему времени восстановления/отката и т. п. Это может показаться несущественным, но особенно в сценарии хотфикса или когда вы управляете сотнями серверов приложений, эти задержки складываются и оказывают реальное влияние на бизнес.

Лучшие практики Dockerfile

Каждая строка или команда в Dockerfile порождает то, что называется слоем (layer), — по сути, снимок всего жёсткого диска образа. Последующие слои хранят дельты между слоями. Образ контейнера — это набор и композиция таких слоёв.

Отсюда следует, что вы можете минимизировать суммарный размер образа вашего контейнера, поддерживая отдельные слои небольшими, а минимизировать слой можно, гарантируя, что каждая команда очищает любые ненужные данные перед переходом к следующей команде.

Ещё одна техника для удержания размера образа на низком уровне — использование многоэтапных сборок (multi-stage builds). Многоэтапные сборки слишком сложны, чтобы описывать их здесь, но вы можете ознакомиться с собственной статьёй Docker об этом на [ctoib.com/docker](https://docs.docker.com/build/multi-stage/).

Оркестрация контейнеров

Теперь, когда у вас есть воспроизводимые образы разумно небольшого размера, управляемые в размещённом реестре, вам нужно запускать их и управлять ими в продакшене. Управление включает:

- Скачивание и запуск контейнеров на машинах
- Настройку безопасной сети между контейнерами/машинами и другими сервисами
- Конфигурацию обнаружения сервисов / DNS

- Управление конфигурацией и секретами для контейнеров
- Автоматическое масштабирование сервисов вверх и вниз с нагрузкой
- Существует два общих подхода к управлению контейнерами: размещённый (hosted) и самоуправляемый (self-managing).

Размещённое управление контейнерами

Если только ваши требования не уникальны или ваш масштаб не очень велик, вы получите наивысший ROI, выбрав размещённое решение, которое выполняет основную часть работы по управлению продакшен-контейнерами за вас. Распространённая и справедливая критика этих решений в том, что они, как правило, значительно дороже самоуправляемых вариантов и предоставляют меньше возможностей и больше ограничений. Взамен вы получаете кардинально меньше накладных расходов и меньше сложности, что для большинства стартапов — компромисс, на который определённо стоит пойти. У большинства небольших команд нет экспертизы для эффективного самостоятельного хостинга, и поэтому самостоятельный хостинг в итоге либо требует существенных временных вложений от существующих членов команды, либо вынуждает вас рано нанимать дорогостоящего DevOps-специалиста. Дополнительная трата \$1000 в месяц, чтобы избежать любой из этих проблем, скорее всего, принесёт очень хороший ROI.

Некоторые распространённые размещённые контейнерные платформы — Heroku, Google App Engine, Elastic Beanstalk, Google Cloud Run. Vercel — ещё одно популярное размещённое бэкенд-решение, хотя оно не запускает контейнеры в том виде, как описано здесь.

Самоуправляемый / Kubernetes

Самое популярное решение для самостоятельного управления контейнерами называется Kubernetes, часто сокращается до K8s. Kubernetes — чрезвычайно мощная и гибкая, а потому и сложная система. Кривая обучения крутая, но выгоды и ROI оправданы, если вы дошли до момента, когда нужно самостоятельно управлять своими контейнерами.

Если вы рассматриваете этот путь, я настоятельно не советую осваивать Kubernetes по ходу работы. Особенно для руководителя команды это слишком много, чтобы взяться и хорошо делать на лету. Вместо этого я рекомендую купить книгу о Kubernetes и выделить неделю или две на её чтение и настройку собственной песочницы, чтобы войти в курс дела перед тем, как погружаться в профессиональный проект. Также хорошая идея — найти советника или ментора, который хорошо понимает Kubernetes, чтобы он выступил ускорителем вашего освоения этого инструмента.

ClickOps против IaC

ClickOps означает процесс настройки вашей облачной инфраструктуры через пользовательский интерфейс, в противоположность предоставляемым API. По мере роста вашей инфраструктуры количество нюансов и деталей в вашей системе быстро превысит вашу способность воспроизводить её с помощью ClickOps. ClickOps хорош для прототипов или проверок концепции, но когда приходит время действительно строить продакшен-окружение и зеркальные окружения разработки, использование ClickOps быстро приведёт к значительной фрустрации и затратам, а также к ограниченным возможностям. Альтернатива ClickOps известна как инфраструктура как код (Infrastructure as Code, IaC).

Существует несколько инструментов и фреймворков, позволяющих определять IaC. Ведущий из них — Terraform от HashiCorp. Terraform использует HashiCorp Config Language (HCL), декларативный синтаксис конфигурации, позволяющий инженерам определять, какие ресурсы им нужны и как они должны быть сконфигурированы. Код Terraform затем может и должен управляться как любой другой код — с использованием системы контроля версий и практик ревью коллегами. После одобрения Terraform может генерировать планы изменений инфраструктуры и применять эти планы за вас у выбранного облачного провайдера (или провайдеров). Я не могу достаточно подчеркнуть, насколько Terraform прост в использовании, мощен и удобен в сопровождении, и какой большой ROI вы, вероятно, получите, мигрировав с ClickOps на IaC.

Непрерывная интеграция

Непрерывная интеграция (Continuous Integration) — это процесс автоматизации включения нового кода в проект. Это может включать запуск статического анализа нового кода, запуск тестов, сборку кода и генерацию любых требуемых артефактов сборки (таких как образы контейнеров). Большинство стартапов используют размещённые

CI-платформы, такие как GitLab runners, GitHub actions, Bitbucket pipelines, Jenkins или CircleCI для выполнения операций непрерывной интеграции.

Некоторые лучшие практики для CI:

- Убедитесь, что команда понимает CI-систему и комфортно чувствует себя, дополняя её, обновляя под новые требования и устраняя неполадки, когда что-то неизбежно ломает сборку.
- Обеспечьте, чтобы сборки были согласованными и детерминированными. Ненадёжные или «флакающие» (flaky) сборки — чрезвычайно мощный пожиратель продуктивности и времени.
- Старайтесь держать время сборки низким. Для большинства команд хорошая цель — чтобы CI занимал менее пятнадцати минут.
- Изучите возможности вашего CI-инструмента, помогающие поддерживать скорость сборок, включая кеши сборки, артефакты сборки и параллельный запуск задач.
- Сборки могут стать сложными. Старайтесь держать ваш код для CI *DRY*. Переиспользуйте код между конвейерами сборки там, где это возможно.
- Будьте последовательны в том, как ваши сборки получают доступ к секретам. Либо полагайтесь на облачный менеджер секретов, либо на секреты окружения сборки там, где это необходимо, и старайтесь их не смешивать. Должен быть один очевидный и согласованный способ работы с конфигурацией и секретами.

Непрерывное развёртывание

На ранних стадиях проекта развёртывать новый код сравнительно просто. В этот момент кода и архитектурной сложности немного. Однако вскоре необходимость управлять зависимостями, зависимыми сервисами, CDN, файрволами, артефактами сборки, конфигурацией сборки, секретами и многим другим приводит к сложному процессу развёртывания. По мере накопления этих требований легко пренебречь автоматизацией и просто положиться, например, на выделенного и пользующегося высоким доверием человека в роли релиз-менеджера. Существует бесчисленное множество команд, следующих этому шаблону, и я гарантирую вам, что большинство из этих релиз-менеджеров заранее обходят даты релизов в своих календарях и страшатся стресса, долгих часов и фрустрации, которые эти дни неизбежно влекут за собой.

К счастью, есть лекарство от подверженной ошибкам концентрации стресса, которой является ежемесячный (или ещё более редкий!) день релиза. Оно состоит в том, чтобы выпускать каждый день, или, чёрт возьми, каждый час! Из одного из «Десяти столпов технологической культуры» (см. стр. 138), «Частота снижает сложность», логически следует, что более частые релизы вынудят вашего релиз-менеджера и команду автоматизировать сложные части развёртываний. При достаточном количестве итераций релизы могут стать полностью автоматизированными, и при достаточном тестировании, дающем вам уверенность в новых изменениях, вы можете дойти до точки запуска новых релизов на каждый набор изменений кода, что называется непрерывным развёртыванием (Continuous Deployment).

Помимо устранения сложности процесса релиза через автоматизацию, более частый выпуск означает, что каждый релиз содержит меньший объём кода. Меньшие изменения кода легче ревьюить другим разработчикам. Меньшие изменения, просто в силу своего меньшего размера, дают меньше возможностей для дефектов.

Автоматизированный процесс релиза также, как правило, подразумевает улучшенную способность откатывать изменения или восстанавливаться после проблемы в продакшене. Это также измеряется как среднее время восстановления (Mean Time to Recovery, MTTR).

Подводя итог: автоматизация релизов означает, что код выходит быстрее (снижая время до выпуска*), означает, что вы можете развёртывать чаще (увеличивая частоту развёртываний), и улучшает MTTR. Это три из четырёх ключевых метрик DORA (см. стр. 208) с помощью одной инициативы!

В компаниях, с которыми я работал, напрямую или в роли советника, я видел как минимум дюжину команд, вложивших усилия в частичный или полный переход к непрерывному развёртыванию. Это не всегда прямолинейный путь, он не случается за одну ночь, и часто есть хорошо обоснованные возражения. Тем не менее в каждом случае, когда команда оглядывается на вложенное время — будь то три недели, три месяца или два года спустя, — разница оказывается не чем иным, как трансформационной для культуры и общего результата и скорости команды по каждой метрике.

Окружения для веток фич

Окружение для ветки фичи (feature branch environment) — это размещённое окружение и набор инфраструктуры, доступные внутри вашей компании и запускающие код из конкретной ветки. Окружения для веток фич невероятно полезны для проверки того, что код работает в продакшен-подобной среде, и для того, чтобы члены команды в вашей компании могли использовать или тестировать изменения, не собирая и не запуская код самостоятельно. Не недооценивайте человеческую лень; каждый лишний шаг, который кому-то нужно сделать, чтобы протестировать и проверить ваш код, означает, что они будут делать это тестирование тем реже. Получить код, установить зависимости и запустить серверы — значительно больше работы, чем перейти по автоматически сгенерированному URL ветки фичи, и поэтому окружение для ветки фичи будет использоваться гораздо чаще.

Окружения для веток фич также решают проблему конкуренции, с которой сталкиваются команды, имеющие лишь единственное staging-окружение. Я выступаю за то, чтобы каждой ветке давать собственное staging-окружение.

Некоторые соображения по окружениям для веток фич:

- Автоматизация здесь ключевая; настройка окружений для веток фич вручную почти всегда непрактична.
- Где возможно, используйте систему вроде Vercel или Firebase, которая включает окружения для веток фич как первоклассную функцию, чтобы минимизировать ваши затраты на настройку и сопровождение.
- Тщательно продумайте, как обращаться с данными в окружениях для веток фич. Вам нужно будет ответить на следующие вопросы:
 - Получает ли каждое бэкенд-окружение ветки фичи новую базу данных? Моя рекомендация — да.
 - Использует ли окружение ветки фичи продакшен-данные? Моя рекомендация — нет. Вместо этого используйте seed-скрипт, который генерирует объёмы данных, схожие с продакшеном. Стоит иметь процесс, который при необходимости для отладки копирует, обезличивает и восстанавливает продакшен-данные для разработчиков.
 - Как обрабатываются изменения/миграции схемы базы данных в окружениях для веток фич?
 - Как работает обнаружение сервисов в окружениях для веток фич для сервис-ориентированных архитектур? Часто можно обойтись общим набором сервисов и развёртывать версии веток фич только для тестируемого сервиса. Я рекомендую иметь окружение для каждого сервиса, которое всегда работает, под названием integration. Пусть все ветки фич ссылаются на integration-окружение других сервисов. Каждое обновление integration-окружения должно быть продакшен-обновлением, но вы также должны позволять разработчикам временно развёртывать код ветки фичи в integration-окружениях для межсервисного интеграционного тестирования.

Просмотрев этот список забот, вы можете быть обескуражены бременем настройки и сопровождения веток фич. Действительно, правильная настройка веток фич недёшева, но ценность, которую она предлагает, существенна — в улучшении вашей способности тестировать и в снижении логистических накладных расходов, требуемых для проверки разных видов изменений в ПО.

Управление DNS

Знание того, как работает система доменных имён (Domain Name System, DNS), и умение управлять DNS и его последствиями для безопасности вашей компании — критически важная работа, которая часто ложится на СТО стартапа. Если вы ещё не знакомы с основами того, как работает DNS, и с разными типами записей, я рекомендую вам потратить сейчас несколько минут на просмотр Wikipedia, чтобы получить базовое понимание предмета.

Вам также следует знать, как DNS используется для электронной почты в вашей организации. В частности, ознакомьтесь с записями Sender Policy Framework (SPF) и DKIM/DMARC.

Вам также следует настраивать ваши DNS-записи с помощью инфраструктуры как кода (Infrastructure as Code, IaC). Я видел бесчисленное количество компаний, где DNS управляется исключительно одним руководителем, чья двухфакторная аутентификация — единственная, которой разрешено обновлять запись зоны, и когда этот человек в отпуске, нет запасного механизма для управления сайтом.

Лучшее решение — настроить DNS с помощью Terraform (у которого есть интеграции со всеми основными DNS-провайдерами) и затем управлять DNS-записями через систему контроля версий, наделяя отдельных разработчиков возможностью добавлять новые записи ответственным образом, не завязанным на каком-либо одном человеке.

Разделение выкатки кода и выкатки фич (фича-тогглы)

На высоком уровне фича-тоггл (feature toggle) — это переключатель, который позволяет менять поведение системы без изменения самого кода. Я настоятельно выступаю за использование фича-тогглов, в частности потому, что они позволяют вашей команде иметь отдельные процессы и таймлайны для выкатки кода и выкатки фич. У Пита Ходжсона (Pete Hodgson) есть замечательное, подробное объяснение фича-тогглов на cto.hb.com/hodgson.

Четыре ключевые метрики выступают за то, чтобы выкатывать код как можно чаще. Это приносит много замечательных положительных выгод для здоровья вашего инженерного процесса. Однако естественное опасение состоит в том, что ваш бизнес может быть не готов к запуску конкретной фичи в ту самую секунду, когда код готов. Существует множество причин, по которым ваши графики разработки и релизов могут таким образом рассинхронизироваться, например необходимость согласовать тайминг с маркетинговой активацией, создание документации поддержки клиентов, ожидание регуляторных одобрений, пауза для внутренних коммуникаций и т. п. Фича-тогглы позволяют вашей инженерной команде сосредоточиться на том, чтобы выкатывать как можно быстрее и надёжнее, и делегировать задачу согласования момента включения фич внешнему (out-of-band) процессу, которым, вероятно, владеют другие команды.

Мониторинг систем: APM и RUM

Мониторинг производительности приложений (Application Performance Monitoring, APM) и мониторинг реальных пользователей (Real User Monitoring, RUM) — это два типа инструментов, которые помогают командам понимать, как приложение работает в продакшене, и выявлять или предотвращать сбои, затрагивающие пользователей.

Инструмент APM обычно располагается внутри или рядом с вашим приложением в продакшен-окружении и предоставляет аналитику и инсайты об использовании ресурсов, задержке запросов и пропускной способности с точки зрения вашего бэкенда. RUM — это внешний инструмент, который притворяется пользователем и предоставляет аналитику о задержке с точки зрения фронтенда, или с точки зрения реального пользователя.

Если вам пришлось бы выбирать один (а эти инструменты часто очень дороги, так что вы можете быть вынуждены выбрать один или другой), выбирайте тот, что закрывает слепое пятно, более вероятно проблемное для вашего приложения. Если у вас тонны пользователей и вас заваливают жалобами в реальном времени на каждый мелкий баг или граничный случай, то APM, мониторящий нагрузку на бэкенд, может оказаться ценнее, чем RUM, выдающий избыточные оповещения от ваших пользователей. Для большинства стартапов на стадии seed или роста, однако, у вас, скорее всего, будет непостоянное использование приложения, особенно покрывающее ваши граничные случаи, и в этом случае RUM может быть ценнее, чем APM на по большей части простаивающем бэкенде.

Некоторые распространённые инструменты в этой области — New Relic, Datadog и Akamai mPulse.

Тестирование

Представьте, что вместо тестирования программного кода ваша работа — быть инспектором муниципального моста. Мост построен, и теперь ваша работа — осмотреть его и решить, разрешать ли открывать его для публики. Будете ли вы осматривать каждый болт и каждую заклёпку на мосту? Это, вероятно, дало бы вам высокую уверенность, что мост безопасен, но это также заняло бы много времени и сорвало бы планы мэра открыть мост на следующей неделе.

Другой разумной стратегией было бы точно решить, какие именно элементы моста существенны для достижения заданного запаса прочности, и протестировать/осмотреть их. Может быть, это только каждая вторая заклёпка плюс все тросы и конструкционный бетон.

Аналогично, в тестировании в программной инженерии цель — не покрытие ради покрытия, а покрытие, которое даёт уверенность в том, что ПО делает то, для чего оно предназначено.

Эффективное тестирование ПО не всегда о 100-процентном покрытии кода. Планка для хорошего набора тестов ПО в том, что он даёт вашей команде уверенность: когда сборка зелёная и все тесты проходят, ПО готово к выпуску для конечных пользователей. Это может означать 100-процентное покрытие кода, а может означать 30-процентное покрытие кода. Точное число — за вами, его определять и отслеживать, и этот объём усилий может меняться со временем, если вы обнаружите, что тесты не дают той же уверенности, что раньше (и, наоборот, если вы обнаружите, что переинвестируете, — то есть тратите много ресурсов на набор тестов, а баги всё равно слишком часто прорываются наружу).

Команды тестирования / обеспечения качества

В зависимости от размера вашей команды у вас будет либо вообще никакой выделенной команды тестирования, либо одна команда тестирования, работающая над одним или многими типами тестов, либо много команд тестирования, работающих над различными видами тестирования ПО. Неважно, кто проводит тест, важно осознавать, что тестирование ПО — сложный процесс, и нюансы имеют значение. Чтобы эффективно тестировать ПО, тестировщик — написал ли он код сам или ему «перебросили через забор» — должен глубоко понимать, что код/ПО должно делать. Ваша роль — выстроить ваши команды так, чтобы они могли соперничать друг с другом. Чтобы это сделать, обеспечьте, чтобы команды разделяли цели/KPI, чтобы в вашем процессе была надёжная и непрерывная коммуникация между разработчиком и тестировщиком, и следите за тем, чтобы у команд были здоровые, продуктивные отношения.

Качество тестов

Прежде чем углубляться в детали разных парадигм тестирования ПО, стоит подумать о том, в чём цель тестирования ПО, и, следовательно, что делает тест хорошим, и наоборот, что делает тест плохим.

Определить плохие тесты просто. Плохие тесты имеют более высокую стоимость, чем выгоду для вашей команды. Некоторые общие характеристики плохих тестов:

- На сопровождение и починку тестов при легитимных изменениях кода уходит больше времени, чем боли, которую вы экономите за счёт найденных багов.
- Тест имеет высокую долю ложных срабатываний или непоследователен, что замедляет CI и вызывает у разработчиков фрустрацию и затраты на переключение контекста, чтобы перезапустить ложные провалы.
- Тест плохо продуман и буквально проверяет, что код делает неправильную вещь.
- Набор проверяет, что код делает правильную вещь, и имеет низкий уровень ложных срабатываний, но он настолько запутан и труден для понимания, что только человек, написавший тест, может добавить новый, а у любого другого инженера, который смотрит на него, начинается мигрень.
- Тест не вселяет уверенности в том, что оцениваемый код готов к выпуску для конечных пользователей.

Держа эту картину в голове, относительно легко увидеть по контрасту, какими атрибутами должны обладать хорошие тесты. Хорошие тесты:

- Легко и быстро запускаются как локально, так и в общем CI-окружении
- Способны выполняться надёжно и стабильно выдавать один и тот же результат, легко дополняются
- Легко рефакторятся или обновляются при изменениях в нижележащей логике
- Уместно связаны с нижележащими паттернами кода, тестируют на правильной «высоте» (так сказать), чтобы улавливать поломки, которые имеют значение
- Легко понятны и удобны в работе для любого инженера

Один из лучших способов оценить ваш подход к тестированию — также и самый очевидный: спросить, как ваша команда относится к тестам, с помощью простого анализа настроений. Результаты, как правило, очень бинарные: либо тесты — источник защищённости, на который команды полагаются, и они естественно их дополняют, потому

что те очевидно являются чистой добавленной ценностью; либо команды пассивно или даже активно ненавидят свои тесты, потому что те истощают продуктивность без достаточной очевидной ценности.

Что тестировать

Ваша команда должна добавлять тесты так, чтобы повышать уверенность в корректной работе системы, минимизируя при этом дополнительную нагрузку по сопровождению этих тестов в долгосрочной перспективе по мере органического роста системы. Общий приём для снижения этой боли — тестировать публичный интерфейс. Публичные интерфейсы должны быть хорошо продуманы и сравнительно стабильны во времени.

Кроме того, это «счастливый путь» (happy path), с которым реально столкнутся потребители вашего ПО.

Поэтому тесты публичных интерфейсов должны меняться со временем мало и при этом давать уверенность в том, что важные части кода работают корректно.

Публичный интерфейс вашего ПО может различаться от проекта к проекту. Для многих проектов это будет настоящий HTTP-API; для некоторых — набор функций/классов в библиотеке или внутреннем сервисе. Для других проектов это может быть пользовательский интерфейс.

Сравнение типов тестирования

Тестирование ПО можно разбить на следующие категории/парадигмы:

- Модульное тестирование (unit testing)
- Интеграционное тестирование (integration testing)
- Сквозное тестирование (end-to-end testing)
- Ручное тестирование (manual testing)
- Полуавтоматизированное тестирование (semi-automated testing)

Атрибуты, которые отличают эти типы тестирования, таковы:

Планирование кода: Насколько заранее нужно продумывать написание самого кода, чтобы он был тестируемым в рамках данной парадигмы.

Охват теста: Какой объём кода каждый тест способен проверить за раз.

Гранулярность обнаружения изменений: Какой размер или тип изменения кода тест с высокой вероятностью обнаружит и приведёт к падению теста.

Стоимость запуска: Насколько быстро или насколько дорого запускать тесты — по времени или в деньгах.

Усилия на добавление: Сколько усилий требуется, чтобы добавить дополнительное покрытие.

Усилия на настройку: Сколько усилий нужно, чтобы настроить эффективный набор тестов.

Следующая таблица обобщает, как пять парадигм соотносятся с этими метриками. Заметьте, что идеальной парадигмы тестирования не существует; у каждой свои компромиссы, и я призываю вас тщательно подумать, какие компромиссы имеют смысл для вашей компании и кодовой базы. Правильный ответ обычно — сочетание разных парадигм тестирования, при котором каждый тип тестов используется там, где он приносит наибольшую ценность в вашем приложении.

TODO: Insert the table from the book

Модульные тесты

Модульное тестирование (unit testing) часто первым приходит на ум, когда кто-то говорит о тестировании ПО. Именно его в основном преподают в учебных заведениях и описывают в учебниках, и обычно именно ему уделяется больше всего усилий в реальном мире. Учитывая всё это, можно подумать, что модульные тесты — лучший тип тестов, хотя я бы возразил, что это не всегда так. Давайте начнём с чёткого определения модульных тестов, чтобы отличать их от других парадигм тестирования.

Модульные тесты выполняются целиком в памяти на одной машине, в общем адресном пространстве с проверяемым кодом, не требуя сетевого подключения или зависимости от внешних сервисов. Большинство модульных тестов выполняются очень быстро, проверяют за раз очень небольшой объём кода и относительно просты для начала работы. Модульные тесты обычно также тесно связаны с контрактами вашего кода и часто требуют нового кода в виде моков (mocks), чтобы тестируемый код мог выполняться без обычно доступных внешних зависимостей.

Ключевой компромисс модульных тестов и их главный недостаток в том, что они тесно связаны с тестируемым кодом. Слишком уж легко получить модульные тесты, глубоко переплетённые с фактическими вызовами функций и внутренними объектами данных. Эта глубокая зависимость означает, что любой рефакторинг кода — даже простые и безобидные изменения — потребует значительного обновления и модульных тестов. Создание моков и тестовых фикстур данных также часто требует существенного объёма кода для создания и сопровождения, чтобы модульные тесты могли выполняться, что добавляет неожиданную стоимость фреймворку модульного тестирования.

Интеграционные тесты

Интеграционный тест ослабляет ограничения модульных тестов, связанные с выполнением в памяти и отсутствием зависимостей. В результате интеграционные тесты, как правило, выполняются медленнее и интегрируются с тестируемым кодом на более высоком уровне. Когда они выполняются в отдельном процессе от тестируемого кода, они обычно задействуют внешне предоставляемый контракт, например API. Это означает, что внутренние рефакторинги, не меняющие поведение API, как правило, остаются незамеченными интеграционным тестом, что делает такие тесты в целом менее хрупкими, но также с меньшей вероятностью обнаруживающими более мелкие побочные эффекты.

Кроме того, интеграционные тесты требуют создания меньшего числа моков либо вовсе обходятся без них и в результате могут требовать меньше кода для реализации.

Сквозные тесты

Сквозной тест (end-to-end, e2e) задействует код тем же образом, что и конечный пользователь. Для бэкенд-кода сквозные и интеграционные тесты могут работать одинаково, выполняя код через внешний контракт. Для клиентского фронтенд-кода сквозной тест обычно подразумевает автоматизацию клиентского интерфейса, часто веб-браузера или мобильного телефона. Я бы не советовал ни одному техническому руководителю писать собственный код автоматизации клиента — это особенно неприятная задача, которую могут снять с ваших плеч загружаемые инструменты вроде Selenium, Cypress и Puppeteer. Для мобильных устройств есть инструменты вроде HeadSpin и Detox.

Ключевой компромисс сквозных тестов в реальном мире — надёжность. По крайней мере на момент написания надёжные веб-сквозные тесты всё ещё несколько труднодостижимы; сама природа того, как браузер выполняет рендеринг, означает, что состояния гонки (race conditions) очень легко возникают случайно. Построение надёжного набора веб-сквозных тестов требует значительной осторожности, внимания к деталям и сопровождения.

Однако отдача от них немалая, так как с помощью наборов сквозных тестов можно создать очень высокое покрытие и очень высокую уверенность в работоспособности пользовательских сценариев. Есть несколько компаний, включая testim.io и rainforestqa.com, которые исследуют применение AI и ML для решения этой проблемы. Эти решения, например, используют нечёткое визуальное тестирование вместо опоры на наличие или отсутствие CSS-селекторов, пытаясь повысить надёжность тестов. Будем надеяться, что к моменту, когда вы это читаете, состояние дел в этой области продвинется чуть дальше, и ценностное предложение сквозных тестов окажется ещё сильнее, чем на момент написания этого текста.

Визуальное регрессионное тестирование

Визуальное регрессионное тестирование — относительно новая парадигма, которая стремится обнаруживать дефекты в клиентских приложениях, вычисляя различия (дельты) в отрисованных визуальных элементах. Существуют фреймворки, которые делают это на разных уровнях гранулярности — от захвата скриншотов целых страниц до рендеринга отдельных компонентов — и формируют дельты для обнаружения дефектов. Очевидный недостаток в том, что каждое преднамеренное изменение любого визуально тестируемого компонента потребует изменения теста.

К счастью, эти тестовые фреймворки часто делают простым и безболезненным воспроизведение набора корректных визуальных эталонов для сравнения, что открывает ещё одну ловушку: при удобном инструментарии для перезаписи тестового эталона становится очень легко получать ложноотрицательные результаты, случайно принимая визуальную дельту, которая на самом деле ошибочна.

Ручное тестирование

Ручное тестирование, как следует из названия, выполняется людьми, а не машинным кодом. Тестирование кода людьми можно дополнительно подразделить на специализированных и неспециализированных тестировщиков.

Специализированное ручное тестирование

Специализированное ручное тестирование — это собственная команда ручного тестирования. Помимо традиционных преимуществ собственной команды, таких как согласованность стимулов и долгосрочные рабочие отношения, ценность того, что команда внутренняя, в том, что она может выстроить экспертизу в вашем продукте и глубоко понять ваших пользователей. Это позволяет команде выявлять дефекты, которые неподготовленный или незнакомый тестировщик пропустил бы. Высококласная команда тестирования может не только служить ресурсом для отлова дефектов ПО, но и давать ценную обратную связь на продуктовом уровне, выявляя несоответствия в дизайне и задавая провокационные вопросы о том, как и почему что-то работает так, как работает.

Отличная собственная команда ручного тестирования может обеспечить колоссальное улучшение общего качества ПО и продукта.

Специализированные команды ручного тестирования должны создавать подробные планы тестирования функциональности продукта и хранить эти планы в легкодоступном/повторяемом виде, в идеале с помощью инструмента вроде TestRail, который позволяет создавать полноценные наборы ручных тест-планов, которые команда может перезапускать вручную по требованию. Ещё одно преимущество такого инструмента — интеграция с другими инструментами разработчиков и продукта, например связывание прогона в TestRail с эпиком в Jira, чтобы показать, сколько ручных и регрессионных тестов было выполнено к релизу той или иной фичи. Это ценно не только как пункт чек-листа запуска, но и помогает в ретроспективах по любым выпущенным дефектам, позволяя пересмотреть, какие ручные тесты были выполнены перед релизом той или иной фичи, и добавить дополнительные ручные тесты для отлова любых дефектов, которые проскочили.

Неспециализированное ручное тестирование

Неспециализированное ручное тестирование часто называют краудсорсинговым тестированием (crowdsourced testing). Существует несколько проприетарных платформ для привлечения тестировщиков, таких как Rainforest QA, Pay4Bugs, 99Tests и Testlio. Модель ценообразования этих платформ обычно зависит от числа подтверждённых найденных багов. В зависимости от характера вашего продукта и типов дефектов, которые вы стремитесь оптимизировать, краудсорсинговое тестирование может быть очень экономически эффективным и малозатратным способом повысить качество продукта.

Полуавтоматизированное тестирование

Полуавтоматизированное тестирование — относительно новая категория тестирования ПО. Эти тесты создаются нетехническим персоналом — возможно, вашей специализированной командой ручного тестирования — и затем выполняются в полностью автоматическом или контролируемом окружении.

Главная ловушка этих тестов — надёжность. Поскольку они создаются нетехническим персоналом, они могут быть не столь точны, как полностью автоматизированные тесты, что делает их более подверженными ложноположительным и ложноотрицательным результатам. Тем не менее эта область быстро развивается: каждый год появляются новые компании и инструменты, такие как Rainforest QA и Testim, со стратегиями повышения надёжности и снижения общей стоимости.

Контроль версий

Отраслевым стандартом, используемым подавляющим большинством компаний для управления контролем версий, является Git. Чтобы вашей организации использовать что-то иное, нужна была бы очень веская причина;

большинство ваших нынешних и будущих членов команды придут, уже зная основы Git. Не используя его, вы, скорее всего, навязаете им ненужную кривую обучения, заставляя осваивать выбранную вами альтернативу.

Существует три основные облачные платформы хостинга Git: GitLab, GitHub и Bitbucket. Эти три занимают львиную долю рынка, и, опять же, вам стоит хорошо подумать, прежде чем отклоняться от этих стандартов.

У Git интересной формы кривая обучения. Большинство людей выходят на плато, где работают с рудиментарным пониманием, которого хватает в большинстве сценариев «счастливого пути». Однако иногда что-то идёт не так, и разработчик теряет коммит или устраивает беспорядок при слиянии. Когда это происходит, неспособность преодолеть оставшуюся часть кривой обучения Git вызывает фрустрацию и замедление. Я призываю вас, как тимлида, вложить усилия, чтобы преодолеть вторую половину кривой. Используйте Git исключительно в командной строке, чтобы освоиться с тем, что на самом деле происходит. Изучите reflog, интерактивные ребейзы, bisect и различные встроенные стратегии слияния. Вооружившись этими знаниями, вы сможете обойти целый класс проблем, истощающих продуктивность, и обучить свою команду тоже стать экспертами в Git.

Ревью коллег

В целом эксперты рекомендуют внедрять надёжный процесс ревью коллег (peer review) для всех изменений кода. (Пока я пишу это в начале 2023 года, набирает силу движение, оспаривающее эту рекомендацию или, по крайней мере, добавляющее в неё нюансы, что я обсужу в следующем разделе.) Большинство ревью коллег делается с помощью того, что — в зависимости от вашего решения для хостинга кода — называется pull request, code review или merge request. Вот несколько рекомендаций, как сохранять ревью кода продуктивным и эффективным:

- Делайте ревью небольшими! Установите максимальный размер для ревью кода — что-то вроде десяти файлов и 200 строк. Всё, что больше, следует разбивать на несколько составных/инкрементальных ревью. (Составное ревью — это ревью кода, основанное на предыдущем ревью или зависящее от него. По завершении ревью сливаются последовательно, складываясь в полное изменение.)
- Заранее установите цели ревью кода вместе с командой истройте их в свою культуру. Ревью кода — не про стиль или мелочную семантику; для этого есть ваш автоформаттер/линтер кода и статический анализ. Цель ревью кода — обеспечить ясность, выявить архитектурные опасения, отметить дефекты и отклонения от паттернов, указать на граничные случаи и гарантировать соответствие бизнес-правилам.
- Требуйте от автора облегчать работу ревьюеру. Авторы должны включать описание изменения, ссылку на соответствующие требования и тикеты, а также видеозапись (с помощью инструмента вроде loom.com) кода и того, как код работает как задумано.
- Поощряйте автора любого ревью кода делать самопроверку перед тем, как просить других о ревью. Несколько метко расставленных комментариев от автора, направляющих читателей, могут сэкономить массу времени.
- Выделяйте специальные временные интервалы/окна для ревью, чтобы минимизировать отвлечения.

Ship, Show, Ask

Расхожая мудрость гласит, что каждое изменение кода должно быть просмотрено двумя людьми, прежде чем попасть к клиентам. Как и во всём, здесь есть компромиссы. Ручное ревью кода не бесплатно и не гарантирует качества ПО. Учитывая, что ручное ревью кода имеет свою цену, стоит задуматься о том, когда эта цена даёт наибольшую отдачу, и использовать ревью кода как инструмент для сценариев с наивысшим ROI. Эту общую идею популяризировал пост в блоге Руана Уилсенаха (Rouan Wilsenach) 2021 года под названием Ship/Show/Ask (cto.hb.com/ssa).

Давайте рассмотрим стоимость ревью кода. Ревью кода требует от двух людей — назовём их Автором и Ревьюером — пережить ряд переключений контекста. Распространённый асинхронный паттерн работы с кодом может выглядеть так:

Переключение контекста №1: Автор прекращает писать код по Проекту 1, готовит ревью кода и тега Ревьюера. Автор начинает работу над Проектом 2.

Переключение контекста №2: Ревьюер получает уведомление, прекращает работу над Проектом 3 и начинает ревью Проекта 1. Ревьюер оставляет обратную связь Автору, возобновляет работу над Проектом 3.

Переключение контекста №3: Автор получает уведомление об обратной связи по Проекту 1, прекращает работу над Проектом 2 и обрабатывает комментарии Ревьюера. Затем Автор возобновляет работу над Проектом 2.

Переключение контекста №4: Ревьюер прекращает работу над Проектом 3 и в лучшем случае теперь удовлетворён изменениями в Проекте 1 и одобряет ревью кода. Ревьюер возобновляет работу над Проектом 3. В худшем случае Автору и Ревьюеру приходится повторить Переключения контекста №3 и №4 несколько раз.

Переключение контекста №5: Автор получает уведомление об одобрении, прекращает работу над Проектом 2, сливает Проект 1, затем возобновляет работу над Проектом 2.

Есть способы минимизировать эти переключения контекста, но они тоже сопряжены с компромиссами. Распространённая альтернатива — делать все ревью кода как синхронное парное программирование; однако эта стратегия меняет переключения контекста на синхронное время встреч, что по-прежнему тормозит продуктивность. Как ни крути, человеческое ревью кода обходится дорого.

Моя предлагаемая альтернатива — классифицировать типы работы по их уровню риска и ожидаемой пользе от ревью кода. Пример системы классификации:

Тривиальные изменения — одобрение не требуется

- Обновления текстов/переводов
- Незначительные изменения UI, в идеале представленные с визуальным подтверждением изменения
- Изменения только в тестах
- Новый код, который явно ещё не используется или отключён за фичефлагом (feature toggle)
- Код, к которому не имеет доступа ни один клиент или пользователь (например, скрытые страницы)

Незначительные изменения — минимальное ревью или ревью постфактум

- Изменения кода, которые сопровождаются тестами и подразумевают расширение существующих паттернов и функциональности
- Код с ограниченным реальным использованием или без него (например, неразвёрнутый продукт)
- Рефакторинги, корректность которых можно доказать с помощью надёжного тестирования

Крупные изменения — тщательное ревью заранее

- Всё, что связано с новыми инструментами, фреймворками, паттернами или архитектурой
- Значительные новые фичи
- Всё, что связано с чувствительными данными или PII либо с потенциальным влиянием на состояние безопасности

Хотя я считаю, что эта система повышает общую эффективность команды, признаю, что она подойдёт не всем. Многие режимы комплаенса (такие как PCI или SOC 2) требуют политики 100-процентного человеческого ревью кода. Лучшее, что вы можете сделать в этом сценарии, — соблюдать требования и, возможно, выделить продукты или функциональные области, не подпадающие под рамки комплаенса, чтобы экспериментировать с более нюансированным и эффективным процессом.

Модели ветвления

Существует множество способов работы с ветвлением в контроле версий, хотя отрасль в целом набирает обороты вокруг концепции trunk-based development (разработка на основе ствола). Поскольку на данный момент это, по-видимому, наиболее эффективный и часто используемый паттерн, именно его мы здесь и обсудим. Если вы всерьёз рассматриваете другой паттерн, в интернете вы найдёте множество ресурсов, обсуждающих методологии и лучшие практики альтернативных подходов.

Есть множество постов в блогах с полезной графикой, описывающих модели ветвления git, например вот этот от Reviewpad: cto.hb.com/branching. Если следующее описание окажется вам непонятным, я настоятельно рекомендую обратиться к любому из этих постов и сопутствующим им визуальным материалам.

В традиционной модели ветвления — иногда называемой GitFlow — есть две долгоживущие ветки: `main` и `develop`. Работа ведётся на основе `develop` в фиче-ветках, а затем часто ответвляется в отдельную `release`-ветку для каждого релиза, после чего, наконец, сливается обратно в `main`. Хотфиксы затем делаются от `main`, тогда как дальнейшая разработка ведётся от `develop`. Эта система задействует не менее четырёх веток для каждого изменения, чтобы добраться до продакшена, и предполагает одновременное сопровождение множества веток. По этим и другим причинам GitFlow в значительной мере вышел из моды и больше не считается лучшей практикой.

Trunk-based development и его чуть более изощрённый родственник, GitHub Flow, — это модели управления исходным кодом, нацеленные на минимизацию числа и продолжительности существования веток. Конкретная реализация GitHub Flow и trunk-based development будет различаться, но общее у них то, что есть единственная ветка, имя которой варьируется и не особо важно. Здесь мы назовём её `production`. `Production` всегда пригодна к развёртыванию. Более того, я рекомендую настроить автоматизацию так, чтобы каждый коммит в `production` действительно можно было задеплоить в продакшен. Работа затем может вестись в фиче-ветках, ответвлённых от `production`, проверяться в фиче-ветке и сливаться, когда готова. Вот и всё — одна долгоживущая ветка и множество короткоживущих (и в идеале небольших) фиче-веток.

Чтобы эта модель работала хорошо, вам нужно несколько предпосылок:

- Continuous Integration, прогоняющая надёжный набор тестов, чтобы убедиться, что фиче-ветки безопасно сливать.
- Культура и реализация использования фичефлагов (feature toggles), чтобы ветки можно было быстро сливать, а затем деплоить/включать фичи позднее, когда это имеет смысл для бизнеса.
- Надёжный мониторинг продакшена для обнаружения изменений.
- Возможность быстро, без простоя, разворачивать изменения кода в продакшен-окружении. Аналогично — возможность быстро откатывать отдельные изменения в ответ на инцидент.
- Культура, дисциплинированная в отношении небольших и короткоживущих фиче-веток. Модель GitHub Flow теряет свою эффективность и простоту, если фиче-ветки становятся большими, долгоживущими и громоздкими. Как обсуждалось в разделе 3.3.5 «Окружения фиче-веток», небольшие коммиты, небольшие ветки и небольшие pull request'ы — ключевой драйвер продуктивности.

Долгоживущие против короткоживущих веток

Ключ к поддержанию гладкой системы веток и слияний в вашей команде — держать ветки короткоживущими. Почти все проблемы, связанные со слиянием кода, проистекают из того, что ветки открыты слишком долго или содержат слишком большой diff (ctohb.com/diffs). В общем случае короткоживущая ветка должна быть открыта всего несколько дней, максимум — две недели.

Имейте в виду, что фичу не обязательно реализовывать в одной ветке. Например, у вас может быть начальная ветка только с тестами, которая проверяется и сливается, а затем за ней следует ветка с реализацией. В качестве альтернативы вы можете построить реализацию, не подключённую к основному приложению, провести её ревью и слияние, а затем построить подключение и тесты в последующей ветке.

При небольшом размышлении и практике большинство реализаций можно разбить на независимо сливаемые части. Это навык, который команды под вашим руководством могут развить со временем.

Преимущества короткоживущих веток:

- Ограничивает время, в течение которого новый код из других веток может быть слит в ствол, тем самым ограничивая возможности для конфликтов кода. У меньших веток также по своей природе меньше «площадь поверхности» для конфликтов.
- Поддерживает код фиче-ветки относительно небольшим, тем самым облегчая ревьюерам чтение и ограничивая возможности для поломок.
- Поощряет более быструю обратную связь в ревью и позволяет раньше вносить корректировки курса в процессе реализации фич.

- Поощряет вашу команду иметь надёжную систему Continuous Integration. Частые слияния высвечивают недостатки в вашем окружении сборки/тестирования, делая их болезненными, если системы ненадёжны, и мотивируя улучшать эти системы.

Эскалации в продакшене

Эскалатор — это инструмент, который принимает инциденты и управляет ротацией дежурств (on-call), вызывая дежурного инженера, а затем эскалируя другим, если вызовы остаются без подтверждения. PagerDuty, вероятно, самый популярный из таких инструментов.

Внедрение эскалаторов

Прежде чем внедрять эскалатор и настраивать ротацию, убедитесь, что инженеры вашей команды согласились участвовать в ротации и что все знают и понимают ожидания по созданию исключений (например, обмен окном дежурства с кем-то другим во время отпуска).

Вы также захотите убедиться, что у вас есть адекватная документация и что все понимают стандартные процедуры действий при получении вызова. Некоторые соображения по установлению этих процедур:

- Укажите, где получатель должен публиковать подтверждение получения вызова (возможно, в самом инструменте-эскалаторе или в общем групповом чате, выделенном специально для обработки эскалаций).
- Обеспечьте лёгкий доступ к плейбукам, которые используются для помощи в диагностике конкретных видов проблем.
- Определите, нужно ли и где настраивать какое-либо уведомление о недоступности сайта (например, нужно ли обновлять страницу статуса компании).
- Решите, где и как часто публиковать обновления о статусе расследования, оценке влияния и оценке времени восстановления.
- Определите, что делать после закрытия инцидента: запланировать упражнение по анализу первопричин и обеспечить, чтобы конкретный инцидент не повторился.

Упражнения по анализу первопричин (RCA)

Каждый раз, когда возникает проблема системы с измеримым влиянием на пользователей, ваша команда должна провести некоторый уровень анализа первопричин (root cause analysis, RCA). Цель

RCA — понять, где в ваших системах произошёл сбой, позволивший значимому дефекту попасть в продакшен и к конечным пользователям.

Чтобы было предельно ясно: анализ первопричин *не должен* быть про выявление вины или назначение виновных. Это должно быть верно в каждой части процесса RCA и встроено в культуру вашей команды. RCA атакует системные проблемы (а не человеческие ошибки) в вашей системе, которые позволяют сбою произойти.

Без такой безопасности и готовности членов команды быть откровенными в своей обратной связи и документации вы упустите ключевые возможности улучшить систему.

Документы RCA

Ваша команда должна готовить документацию в той или иной форме для каждого RCA. В зависимости от того, как часто возникают проблемы в вашей системе и каков их характер, вы можете захотеть создать систему классификации RCA, при которой малозначимые инциденты получают более облегчённый процесс RCA, чем высоковливающие инциденты. Следует признать, что тщательный RCA по высоковливающему инциденту — дорогостоящее усилие, требующее значительного времени и вдумчивости, и что он может оказаться чересчур тяжеловесным для тривиальных дефектов.

При этом для большинства компаний лучше ошибаться в сторону перерасхода в этой области и обеспечения большей надёжности. Вам следует начать с тщательного RCA по всему и перейти к стратифицированной системе RCA, как только вы получите хорошее понимание ландшафта и влияния тех видов проблем, с которыми столкнётся ваша команда.

Для проблем, заслуживающих полного, вдумчивого анализа, вот шаблон, который поможет вам начать и задавать команде правильные вопросы: cto.hb.com/rca. Хорошей практикой — а на деле требованием для большинства фреймворков комплаенса — является создание нового документа подобного рода для каждого инцидента и организация их во внутреннем хранилище документов компании для последующего обращения.

Встречи RCA и временная шкала

Как только это станет практически осуществимым после того, как вы разрешили инцидент, назначьте подходящего человека ведущим (лидом) RCA. Лид должен клонировать шаблон и начать заполнять соответствующие данные об инциденте, а также приступить к исследованию «Пяти почему» (cto.hb.com/5whys) для инцидента.

Он должен подготовить первоначальный черновик RCA и разослать его соответствующим коллегам, прежде чем назначать групповое время для изучения и попытки улучшить анализ и будущие шаги предотвращения.

Участники встречи должны прочитать черновик RCA заранее и прийти подготовленными к разбору сути инцидента и генерации идей по будущим шагам предотвращения.

Выбор ведущего автора RCA

Лидом RCA не обязательно должен быть человек, который реагировал на инцидент. Идеальный лид RCA — это тот, кто очень хорошо знаком с задействованными системами и способен задавать пронизательные вопросы о том, где подвели инструменты и процессы, а также генерировать идеи для улучшения.

Заметьте, что мы не подставляем под удар того, кто совершил человеческую ошибку. Этот человек может быть лидом RCA, если он соответствует предыдущим критериям, но его ошибка сама по себе не делает его правильным человеком для руководства RCA. Он, безусловно, должен внести вклад и воспользоваться возможностью научиться в процессе. Но, опять же, его не наказывают за его ошибку в рамках процесса. Авторство RCA — не наказание; это важная ответственность и элемент сопровождения системы.

Планирование работ по устранению по итогам RCA

Хороший процесс RCA часто выявляет множество рабочих задач для команды по улучшению системы и снижению вероятности будущих инцидентов. Естественный следующий вопрос: делаем ли мы их сейчас? Для вовлечённых инженеров ответ, скорее всего, да; для менеджера, обеспокоенного соблюдением сроков и дорожной карты, ответ будет менее очевиден.

Единственно правильного ответа на этот вопрос нет, но вот некоторые общие рекомендации:

Никогда не давайте хорошему кризису пропасть зря. Мотивация устранять проблемы будет на пике вокруг инцидента и встречи RCA, а высокомотивированные инженеры часто наиболее эффективны. Также легко недооценить общую стоимость для вашей команды проблем с надёжностью системы и тем самым недооценить приоритет улучшений надёжности. Сам факт, что произошёл инцидент в продакшене, должен напомнить вам и вашей команде, что эти инвестиции критичны для ограничения отвлечений и для того, чтобы команды могли сосредоточиться на продуктивной работе над фичами и стабильно высокой скорости.

Уровень усилий по многим задачам устранения, скорее всего, будет сильно варьироваться. Некоторые типичные тикеты могут звучать как «добавить больше логирования» или «изменить настройку у нашего CI-провайдера, чтобы PR'ы с падающими сборками нельзя было слить». Такого рода тривиальные тикеты стоят дороже в сопровождении и груминге в бэклоге, чем заняло бы просто сделать их прямо сейчас, поэтому просто делайте их. Шансы, что они окажутся неправильным решением, довольно низки, а если возникнут негативные последствия, их легко обратить.

Для высокозатратных шагов устранения я призываю вас провести их через триаж и пропустить через ваш обычный процесс планирования. Часто высокозатратные шаги устранения можно упростить, имея в распоряжении время и планирование. Иначе говоря, выявленный в первый день «правильный способ» решить проблему может не быть идеальным решением, и только пропустив проблему через обычные этапы технического разбора, можно прийти к лучшему, возможно менее затратному, решению.

Здесь под IT (information technology, информационные технологии) я подразумеваю внутренние корпоративные инструменты и технологии, которые используются для ведения бизнеса изо дня в день. Это противоположность технологиям, которые ваша компания создаёт для своего клиентского продукта.

IT обычно включает такие инструменты, как корпоративное оборудование (стационарные компьютеры, ноутбуки и телефоны), VPN, электронную почту, антивирусное и мониторинговое программное обеспечение и т. д. Будучи стартапом в современном мире — независимо от того, работаете ли вы офисной или удалённой командой, — если вы примете несколько разумных решений, вам не придётся тратить много времени или капитала на IT.

Несколько ключевых решений, которые помогут минимизировать затраты на IT в большинстве небольших технологических компаний:

Используйте облачную систему для корпоративной электронной почты, данных и документов. Большинство стартапов используют Google Workspace, но если вашим сотрудникам (и потенциальным будущим кандидатам на найм) комфортнее с какой-то альтернативой — выбирайте её. На этом этапе нет никакого смысла разворачивать собственный почтовый сервер, хранилище документов, систему доступа к данным, сетевую инфраструктуру и т. д.

На раннем этапе, если этого не требует какая-либо система комплаенса (соответствия требованиям), не обязывайте сотрудников использовать корпоративное оборудование. При малых масштабах закупка (особенно до достижения соответствия продукту рынку), выдача и управление корпоративным оборудованием — это нетривиальная задача (и затраты!), которая приносит лишь незначительную или редкую реальную пользу.

Возможно, признавать это неприятно, но надлежащая защита вашего продукта и IT-системы — это серьёзная задача, и для молодого стартапа нереалистично выполнять её исчерпывающе на раннем этапе. Я призываю вас быть прагматичными и сосредоточиться на защите системы от наиболее вероятных источников взлома или кражи данных: человеческих ошибок ваших сотрудников. Гораздо вероятнее, что ваша инженерная команда забудет поставить аутентификацию перед API, или кто-то оставит ноутбук разблокированным в кофейне, чем что злоумышленник сумеет провести атаку «человек посередине» (man in the middle) с вашими данными или взломать вашу облачную инфраструктуру через эксплойт.

Даже следуя лучшим практикам для минимизации усилий на IT, у вас всё равно останутся некоторые неизбежные IT-задачи — в основном связанные с активацией и деактивацией учётных записей пользователей и восстановлением паролей для сотрудников. Я призываю вас задокументировать и обучить выполнению этих задач других коллег, например в HR, чтобы они не отвлекали вас или инженерную команду на регулярной основе.

Безопасность и комплаенс

В этом разделе я дам краткий обзор темы безопасности (security) и комплаенса для стартапов. Вы можете и должны приложить усилия, чтобы найти более глубокие источники по этим темам помимо этой книги.

Терминология безопасности аутентификации

Особенно в области безопасности важно быть точным и аккуратным в формулировках. Несколько определений часто неправильно употребляемых терминов:

Аутентификация, или AuthN: проверка того, что пользователь или клиент является тем, за кого себя выдаёт. Ваша система входа выполняет аутентификацию пользователей.

Авторизация, или AuthZ: проверка того, что пользователь или клиент имеет право делать то, что он пытается сделать. Ваша система ролевого управления доступом (RBAC) или система прав выполняет авторизацию.

2FA или MFA: двухфакторная аутентификация (two-factor authentication) и многофакторная аутентификация (multi-factor authentication) — это процесс аутентификации в сервисе с использованием более чем одного типа учётных данных. Обычно это делается с помощью пароля (первый фактор) и какого-либо доказательства владения, например одноразового пароля, отправленного на email (доказывает, что вы владеете почтой), SMS (доказывает, что вы владеете номером телефона) или одноразового пароля с привязкой ко времени (TOTP) (доказывает, что вы владеете устройством/passkey). Обратите внимание, что из-за распространённости атак на перенос SIM-карты (SIM Porting), при которых злоумышленник получает возможность перехватывать или перенаправлять SMS, использование SMS в качестве второго фактора в целом не рекомендуется.

Безопасность в стартапах

Стартапы часто характеризуются степенью ограниченности их ресурсов. В результате уровень безопасности и комплаенс нередко оказываются первыми пунктами, которым понижают приоритет в списке дел, поскольку они с меньшей вероятностью представляют экзистенциальную угрозу для бизнеса, чем другие неотложные задачи. Если у вас нет пользователей или выручки, что хакеру вообще красть?

Учёт безопасности также может стать обузой для продуктивности или дорогостоящей задачей, особенно если ваша миссия — защитить уже существующую систему. Но если вы начинаете с первого дня, у вас есть возможность принять правильные решения в самом начале, которые создадут надёжный уровень безопасности с минимальными дополнительными затратами.

Несколько способов внедрить безопасность в вашем стартапе, которые обойдутся вам недорого:

- Закрепите безопасность как приоритет в мышлении вашей команды через материалы для онбординга и обучения.
- Запишите всех инженеров на вводное и регулярное базовое обучение по безопасности — например, по OWASP Top Ten или различным геймифицированным курсам по безопасности, которые занимают несколько минут в месяц и помогают держать безопасность в фокусе внимания.
- Полагайтесь на проверенные и хорошо поддерживаемые инструменты для всего, что связано с аутентификацией или авторизацией.
- Не тратьте время на самостоятельное создание страницы входа; в 2023 году для этого действительно нет причин. Такие инструменты, как Auth0, SuperTokens и AWS Cognito, обеспечивают безопасную регистрацию пользователей, вход, вход через соцсети, управление забытыми паролями, аутентификацию по email, двухфакторную аутентификацию и управление сессиями. Некоторые из этих инструментов также предлагают надёжные системы авторизации. Работа с аутентификацией — это серьёзный проект; он очень сложен, а ошибки обходятся дорого. Нет никаких причин вашему стартапу решать эту проблему.
- Не ленитесь в вопросах IT-безопасности. Независимо от того, используете ли вы Dropbox, Box, Google Drive, SharePoint и т. д., потратьте несколько минут и настройте политики, которые помогут избежать человеческих ошибок, например выставьте права на доступ по умолчанию только для внутренних пользователей. Настройте регулярные отчёты о совместном доступе к данным и назначьте сотрудника, который будет ежеквартально проверять настройки прав доступа к любым особо чувствительным документам или таблицам.
- Используйте корпоративное решение для управления паролями, например 1Password, и убедитесь, что все сотрудники применяют надёжные пароли для важных инструментов. Аналогично, как можно чаще используйте единый вход (Single Sign-On, SSO) и убедитесь, что ваш SSO-провайдер настроен с высоким уровнем безопасности (как минимум с обязательной многофакторной аутентификацией).
- Не храните секреты в коммитах своей кодовой базы. Используйте защищённый менеджер секретов, такой как Google Cloud Secret Manager или AWS Secret Manager, и фиксируйте в коде имя/местоположение секрета, а затем разрешайте это имя в значении в production — либо на этапе запуска с помощью инструмента вроде Berglas или Whisper, либо во время выполнения напрямую через API менеджера секретов.

Комплаенс

Будь то из-за отрасли, в которой вы работаете, размера вашего бизнеса или характера ваших клиентов, большинству стартапов необходимо соответствовать как минимум одному формальному фреймворку комплаенса. Если ваши пользователи находятся в Европе, вам нужно соблюдать GDPR. Если вы собираете пользовательские данные, разумно разобраться в CCPA. Если вы работаете с корпоративными клиентами, у вас попросят сертификат SOC 2 или ISO 27001. В здравоохранении есть HIPAA, а если вы работаете в сфере платежей, вы, вероятно, слышали о PCI DSS.

Для стартапа поддержание соответствия любому или всем этим фреймворкам может быть неприемлемо дорогим. Вот несколько советов по сохранению соответствия и прогнозированию затрат:

- Не пытайтесь получить сертификат соответствия в последний момент. Подготовка и проведение аудита, например по PCI DSS или SOC 2, от начала до конца — это длительный процесс, занимающий у большинства стартапов от шести до двенадцати месяцев. Начать заранее и поддерживать соответствие дешевле, чем начать поздно и переделывать.
- Используйте как можно больше автоматизации для обеспечения соответствия или предоставления доказательств соответствия. Существует процветающий сектор SaaS-компаний, специализирующихся на автоматизации этих фреймворков комплаенса; такие компании, как Vanta, Tugboat Logic, Secureframe, Laika и Drata, предлагают решения, которые значительно сократят время до сертификации и общие затраты.
- Если вам повезло иметь штатного специалиста или отдел по комплаенсу, опирайтесь на эти отношения. Чем проактивнее вы будете делиться планами с отделом комплаенса и чем раньше учтёте их обратную связь, тем менее затратным и раздражающим будет поддержание соответствия.

Заключение: измерение успеха

Вы выстроили отличный процесс найма, команда довольна, вы проводите спринты как профессионал, а ваша архитектура выдерживает растущие требования бизнеса. Это приятно, но как понять, достаточно ли этого? Как нам измерить собственный успех и результативность в роли технического лидера или СТО?

Один из способов определить выдающиеся результаты в этой роли — взглянуть с точки зрения CFO (финансового директора): насколько эффективно СТО способен распорядиться бюджетом на R&D и превратить его в инженерную и продуктовую отдачу?

Или, возможно, на это можно посмотреть с точки зрения CEO: как быстро команда, которую возглавляет СТО, может выполнить определённые бизнес-цели?

Или, учитывая, насколько важно для преуспевания в этой роли лидерство в работе с людьми, мы можем взглянуть на это через гуманистическую призму: делает ли ваша команда свою лучшую работу? Ведь миссия отличного СТО — построить организационную культуру, которая позволяет отдельным инженерам делать свою лучшую работу и достигать невозможного с помощью технологий.

Или, вместо того чтобы пытаться определить единственный критерий, возможно, лучшее определение выдающегося СТО — это сумма всех навыков, которые СТО может применять изо дня в день. Возможно, «выдающийся» — это когда вы берёте сумму архитектуры, управления эффективностью, управления поставщиками, исполнительного лидерства, вклада в культуру, публичной евангелизации, наставничества и DevOps, пропускаете через формулу и получаете число больше 42.

Как бы мы ни старались, похоже, что выдающееся лидерство — даже выдающееся *техническое* лидерство — это нечто, что мы не можем точно количественно измерить или оценить. Умные люди будут с трудом приходить к согласию относительно общего описания величия, но мы все согласимся, что эта роль многогранна и постоянно меняется, требуя непрерывного обучения и адаптации.

В инженерном лидерстве мало универсальных истин, но одна из них в том, что становление хорошим инженерным лидером — это бесконечный путь самосовершенствования, открытий и роста. Чтобы идти по этому пути, нужны смирение, готовность совершать ошибки и, прежде всего, любопытство и желание учиться.

Я надеюсь, что этот справочник стал для вас полезным путеводителем по вызовам, с которыми вы сталкиваетесь на своём пути лидерства. Справочник охватывает многие из вызовов, с которыми сталкивался я сам на протяжении многих лет, а также те, с которыми сталкивались многие замечательные лидеры, с которыми мне посчастливилось взаимодействовать.

Я сделал всё возможное, чтобы дать некоторую структуру для преодоления этих вызовов, хотя каждая ситуация уникальна, и в конечном счёте путь, который вы выберете, — ваш собственный, и результаты тоже принадлежат вам.

В какой-то момент жизни каждого спрашивают: какой совет вы дали бы более молодой версии себя? Этот справочник — мой ответ на этот вопрос.

Я надеюсь, что он поможет вам на вашем пути к созданию мощных технологий, мотивированных и наделённых полномочиями команд и успешных компаний, и, самое главное, получать удовольствие и приносить пользу миру.

Список книг

Getting Things Done — David Allen

Extreme Programming Explained — Kent Beck

Work Rules! — Laszlo Bock

How to Win Friends and Influence People — Dale Carnegie

Agile Estimating and Planning — Mike Cohn

Good to Great — Jim Collins

The 7 Habits of Highly Effective People — Stephen Covey

Domain-Driven Design — Eric Evans

Patterns of Enterprise Application Architecture — Martin Fowler *High Output Management* — Andy Grove

Scaling Up — Verne Harnish

The Hard Thing About Hard Things — Ben Horowitz

Immunity to Change — Robert Kegan

The Phoenix Project — Gene Kim

The Five Dysfunctions of a Team — Patrick Lencioni

Managing Humans — Michael Lopp

Team of Teams — Stanley McChrystal

Escaping the Build Trap — Melissa Perri

Good Authority — Jonathan Raymond

Good Strategy/Bad Strategy — Richard Rumelt

Radical Candor — Kim Scott

The Art of Agile Development — James Shore and Shane Warden

Scrum: The Art of Doing Twice the Work in Half the Time — Jeff Sutherland

Extreme Ownership — Jocko Willink and Leif Babin

Цифровые источники

VSCode's Setting Files (cto hb.com/vscode): <https://code.visualstudio.com/docs/getstarted/settings>

Key to Gmail (cto hb.com/keytogmail): <https://techcrunch.com/2010/03/14/key-to-gmail>

Shit Umbrella (cto hb.com/umbrella): <https://medium.com/@rajsarkar/word-of-the-month-shit-umbrella-33e3182a0f1b>

Liar's Paradox (cto hb.com/liarsparadox): [https://en.wikipedia.org/wiki/Liar's paradox](https://en.wikipedia.org/wiki/Liar%27s_paradox)

Gitlab Compensation Calculator (ctohb.com/gitlabcompcalc): <https://about.gitlab.com/handbook/total-rewards/compensation/>

Five Whys (ctohb.com/5whys): https://en.wikipedia.org/wiki/Five_%20whys

Take the DORA DevOps Quick Check (ctohb.com/dora): <https://www.devops-research.com/quickcheck.html#questions>

Netflix Keeper Test (ctohb.com/keeper): <https://empowerment.ee/wp-content/uploads/2021/05/NETFLIX-%E2%80%93-THE-KEEPER-TEST.pdf>

Why GitLab Pays Local Rates (ctohb.com/local): <https://about.gitlab.com/blog/2019/02/28/why-we-pay-local-rates/>

What is the topgrading interview process? (ctohb.com/interview): <https://www.greenhouse.io/blog/what-is-the-topgrading-interview-process>

Topgrading (ctohb.com/topgrading): <https://en.wikipedia.org/wiki/Topgrading>

Painting the Bridge (ctohb.com/painting): <https://www.goldengate.org/bridge/bridge-maintenance/painting-the-bridge/>

Dare to Lead: e BRAVING Inventory (ctohb.com/braving): <https://brenebrown.com/resources/the-braving-inventory/>

Root Cause Analysis Template (ctohb.com/rca): https://docs.google.com/document/d/1GuRZgDpMVg_%20Qf3sR7r8tZqRx6Re0oBrwlcnj-ekgJ60/edit#

Ship Small Diffs (ctohb.com/diffs): <https://blog.skyliner.io/ship-small-diffs-741308bec0d1>

GitHub Flow, Trunk-Based Development, and Code Reviews(ctohb.com/branching): <https://reviewpad.com/blog/github-flow-trunk-based-development-and-code-reviews>

Ship/Show/Ask (ctohb.com/ssa): <https://martinfowler.com/articles/ship-show-ask.html>

Thoughtworks Technology Radar 27 (ctohb.com/techradar): <https://www.thoughtworks.com/en-us/radar>

Feature Toggles (aka Feature Flags) (ctohb.com/hodgson): <https://martinfowler.com/articles/feature-toggles.html>

Multi-stage Builds (ctohb.com/docker): <https://docs.docker.com/build/building/multi-stage/>

Choose Boring Technology (ctohb.com/boring): <https://boringtechnology.club/>

Technology Radar (ctohb.com/radar): <https://www.thoughtworks.com/en-us/radar>

e Waterfall Model (ctohb.com/waterfall): https://en.wikipedia.org/wiki/Waterfall_%20model#History

e Pragmatic Programmer (ctohb.com/tpp): https://en.wikipedia.org/wiki/e_Pragmatic_Programmer

Rubber Duck Debugging (ctohb.com/rdd): https://en.wikipedia.org/wiki/Rubber_%20duck_%20debugging

FrequencyReducesDifficulty (ctohb.com/fowler): <https://martinfowler.com/bliki/FrequencyReducesDifficulty.html>

Figma Material Design (ctohb.com/figma): <https://www.figma.com/@materialdesign>

How to Design with the Atlassian Design System (ctohb.com/design): <https://atlassian.design/get-started/design>

Building Productive Teams (ctohb.com/teams): <https://docs.microsoft.com/en-us/DevOps/plan/building-productive-teams>

GitHub Compensation Calculator (ctohb.com/calc): <https://about.gitlab.com/handbook/total-rewards/compensation/>

Codecademy Engineering Competencies (ctohb.com/competencies): <https://github.com/Codecademy/engineering-competencies>

e Multitasking Myth (ctohb.com/myth): <https://blog.codinghorror.com/the-multi-tasking-myth>

Acronyms Seriously Suck: Elon Musk (ctohb.com/acronyms): <https://gist.github.com/klaaspieter/12cd68f54bb71a3940eae5cdd4ea1764>

How We Work Without Meetings at Levels (ctohb.com/async): <https://medium.com/levelshealth/how-we-work-without-meetings-at-levels-a6a525e21aa5>

Collaborate with kindness: Consider these etiquette tips in Slack (ctohb.com/slack): <https://slack.com/blog/collaboration/etiquette-tips-in-slack>

How to Use Skip-Level Meetings Effectively (ctohb.com/skip): <https://www.managementcenter.org/resources/skip-level-meeting-toolkit/>

From Founder to CTO (ctohb.com/founder2cto): <https://calv.info/founder-cto>

How to Prioritize the Developer Experience and Improve Output (ctohb.com/dx): <https://www.harness.io/blog/developer-experience>

Глоссарий

Agile-церемония (Agile ceremony): Agile-церемонии — это встречи, на которых команда разработки собирается на различных этапах процесса разработки для обсуждения планирования будущей работы, информирования о текущей работе или анализа и рефлексии над прошлой работой.

Система отслеживания кандидатов (Applicant Tracking System, ATS): система отслеживания кандидатов (ATS) — это программное обеспечение для рекрутеров и работодателей, позволяющее отслеживать кандидатов на протяжении всего процесса рекрутинга и найма.

Правило бойскаута (Boy Scout Rule): оставляй после себя лучше, чем было до тебя. Применительно к технической команде это означает: всякий раз, когда вы работаете с каким-либо участком кода, всегда вносите хотя бы небольшое улучшение — возможно, в тесты или документацию — либо иным образом улучшайте ясность, читаемость или сопровождаемость.

Brownfield-разработка (Brownfield development): противоположность greenfield-разработки: работа с существующими унаследованными (legacy) системами, часто сильно обременёнными техническим долгом. Вы вынуждены мириться с высокоуровневыми решениями, принятыми в прошлом, и располагаете ограниченной гибкостью для крупных изменений.

Бизнес-аналитика (Business Intelligence, BI): бизнес-аналитика (BI) — это программное обеспечение, которое получает бизнес-данные и представляет их в удобных для пользователя видах, таких как отчёты, дашборды, диаграммы и графики.

ClickOps: ClickOps — это подверженный ошибкам и трудоёмкий процесс, при котором люди вручную кликают по различным пунктам меню на сайтах облачных провайдеров, чтобы выбрать и настроить нужную автоматизированную вычислительную инфраструктуру.

Контейнеризация (Containerization): контейнеризация подразумевает упаковку программного обеспечения, содержащего все необходимые для запуска приложения элементы, в контейнерную среду. Это позволяет организациям запускать приложения откуда угодно — в частном дата-центре, публичном облаке или даже на личном ноутбуке.

Переключение контекста (Context switching): переход от одной задачи к другой. Для инженеров это означает отложить решаемую проблему в сторону и начать работать над другой. Сам акт переключения, как правило, отнимает время и менее эффективен, чем работа над одной проблемой за раз.

Прямой подчинённый (Direct report): прямые подчинённые — это сотрудники, которые подчиняются напрямую кому-то, кто находится выше них в организационной структуре, часто менеджеру, супервайзеру или лидеру команды.

Engineering Product and Design (EPD): идея объединения в единый отдел того, что традиционно представляло собой три отдельных департамента: Design (дизайн), Product (продукт) и Engineering (инженерия). Порядок букв в акрониме часто меняется: EDP, PDE.

Greenfield-решение (Greenfield solution): greenfield-разработка ПО относится к работе в новой среде с минимальным количеством уже существующего унаследованного кода и свободой выбора инструментов, паттернов и архитектуры.

Горизонт один/два/три (Horizon One/Two/Three): каждый горизонт представляет собой свой временной масштаб. Горизонт один обычно краткосрочный (дни/недели), два — среднесрочный (месяцы), а три — долгосрочный (годы). Часто используется как инструмент планирования, чтобы убедиться, что вы учитываете каждый горизонт.

Идемпотентность (Idempotent): техническая операция идемпотентна, если последующие выполнения операции не изменяют результат.

Кайдзен (Kaizen): кайдзен — это философия непрерывного улучшения всех процессов в организации.

Ключевой показатель эффективности (Key Performance Indicator, KPI): KPI — это критически важные измеримые показатели прогресса в достижении желаемого результата. Иногда их называют входными метриками.

Цели и ключевые результаты (Objectives and Key Results, OKR): «цели и ключевые результаты» — это фреймворк постановки целей, возникший в Intel в 1970-х годах, который используется отдельными людьми, командами и организациями для определения измеримых целей и отслеживания их результатов.

Принцип Дирихле (Pigeonhole principle): описывает ситуацию, когда есть фиксированное число исходов и большее число испытаний. Если вы подбросите монету три раза, то по крайней мере один из исходов — орёл или решка — должен выпасть дважды.

Документ с требованиями к продукту (Product Requirements Document, PRD): PRD — это документ, который собирает в одном месте предысторию, ссылки, обоснование и формулировку требований к продукту.

Воспроизводимость (Reproducibility): способность воспроизвести заданный результат по запросу. В разработке ПО это обычно принимает форму: пользователь нажимает кнопку X, после чего приложение падает. Если нажатие кнопки является полным и надёжным описанием того, как вызвать падение приложения, то это воспроизводимое падение с понятными шагами воспроизведения.

Запрос на комментарии (Request for Comment, RFC): документ, излагающий идею, философию, предложение или методологию, который предназначен для сбора обратной связи и в конечном счёте становится долгоживущим справочным материалом.

Анализ первопричин (Root Cause Analysis, RCA): подход, и, как правило, документ, который пытается копнуть глубже поверхностного, чтобы по-настоящему понять, почему произошло событие. Обычно используется как инструмент расследования, а затем как справочная документация о том, почему произошёл инцидент в программном проекте.

Платформа управления SaaS (SaaS Management Platform, SMP): SMP предоставляет единое место для просмотра, управления, оптимизации и контроля SaaS-инструментов, используемых в рамках всей организации.

Сервис-ориентированная архитектура (Service-Oriented Architecture, SOA): словосочетание «сервис-ориентированная архитектура» (SOA) возникло в 1990-х годах и используется для обозначения некоторых довольно конкретных технологических решений. В наше время эта фраза используется в более широком смысле для описания системы, в которой информация перемещается между частями системы по сети.

Standup-встреча (Standup meeting): (она же Daily Scrum) регулярная встреча в рамках scrum/agile-церемоний. Обычно предполагается, что она короткая — менее 30 минут, — чтобы облегчить коммуникацию, обновления, разрешение конфликтов и принятие решений внутри команды.

Straw Man Model (модель «соломенного человека»): первый набросок предложения, который можно быстро собрать на основе неполных данных. Часто используется как стартовое предложение для команды, чтобы ускорить процесс сбора обратной связи и выработки решения.

Синхронная/асинхронная культура рабочего места (Synchronous/Asynchronous Workplace Culture): идея асинхронной культуры рабочего места заключается в том, что потоки коммуникации поощряются к асинхронности, то есть без необходимости одновременного участия обеих сторон коммуникации. Например, письменный

документ обеспечивает асинхронную коммуникацию: автору не нужно присутствовать, когда читатели знакомятся с документом. Асинхронные культуры снижают значимость встреч и делают упор на письменную или записанную аудио-/видеодокументацию.

Об авторе

Zach Goldberg окончил Университет Пенсильвании с отличием (*magna cum laude*), получив степень в области компьютерных наук и инженерии. Он был СТО шести стартапов, включая WiFast, Sticks and Brains, AutoLotto, Trellis Technologies, GrowFlow (приобретена Dama Financial, 2022) и Towards Equilibrium Inc, а также предпринимателем-в-резиденции (Entrepreneur-in-Residence) в Tencent и младшим продакт-менеджером (Associate Product Manager) в Google. После того как Dama приобрела GrowFlow в 2022 году, Zach сел и вложил весь свой опыт в эту книгу. Узнайте больше о работе Zach на сайте zachgoldberg.com.

Об издателе

Основанная в 2021 году Bryna Haynes, WorldChangers Media — это бутиковое издательство, ориентированное на «Идеи ради влияния» (Ideas for Impact). Мы знаем, что великие книги меняют жизни, разрушают устаревшие парадигмы и создают движения. Наша задача — выпускать высококачественную трансформационную нон-фикшн литературу, созданную следующим поколением лидеров мысли и для него.

Готовы написать и опубликовать с нами свою книгу о лидерстве мысли? Узнайте больше на www.WorldChangers.Media.